



EmulatR User and Developer Reference

© 2026 Timothy Peer - eNVy Systems Inc.

Table of Contents

Foreword	0
Part I 1. Introduction	6
1 What is EmulatR	6
2 Release Notes	7
3 License	14
4 Goals & Objectives	15
5 Supported Systems	17
6 Current Platform Support	17
7 Feature Matrix	18
8 Current Development Status	21
9 Known Gaps	23
10 EmulatR Version Numbering and Release Stages	39
Part II 2. Installation	42
1 Installing with the Windows Installer	42
2 Required Tools & Frameworks	44
3 Firmware Requirements	44
4 Building from Source	45
./tools/build_emulatr_diag.bat	47
5 Dependencies	48
6 First Startup	48
Part III 3. Quick Start	50
1 Launching EmulatR	50
2 Loading Firmware	51
3 Running from the Shell	51
4 Running from a Run Directory	56
Run-Directory Contents	57
5 Connecting to the Console	60
6 Reaching the SRM Prompt	60
7 Clean Shutdown	61
Part IV 4. Operating EmulatR	62
1 4.1 Configuring & Running	62
Configuring a System with the Launcher	63
Configuring	69
Platform Identity & Memory Geometry	71
Executing	74
Executing Two Instances	75

Example - Performance	75
Command Line Options	77
2 4.2 Console Connectivity	79
Console Support	80
TCP Console Overview	80
Terminal Clients (PuTTY / Plink / TeraTerm / MobaXterm / Netcat)	81
3 4.3 SRM Console	82
SRM Overview	83
Successful Boot to SRM (per platform)	83
Common Commands	89
show config	89
show device	89
show memory	90
boot	90
set	90
SRM Environment Variables	91
>>> Evaluate Address	91
4 4.4 Storage	92
ATAPI Devices	92
IDE Devices	93
SCSI Devices	94
ISO-9660 PFD (CD001)	94
SDI / DSSI / ST506 Devices	95
Storage Interconnect and Fabric	96
Create Guest Disk Storage	96
System Disk Configuration	99
Virtual CDROM (dqa1)	104
Virtual Disk Images	105
Virtual Floppy (dva0)	105
5 4.5 Snapshots & Boot Acceleration	106
Boot Acceleration - Entry Snapshot	107
6 4.6 Firmware Update Utility (LFU)	109
LFU Commands (List / Display / Update)	109
7 4.7 Guest Boot	110
8 4.8 Logging & Diagnostics	112
SRM Performance Logging	113
Diagnostics & Debugging	113
Bring-Up Probes (EMULATR_BRINGUP_PROBES)	114
Logging Subsystems	116

Part V 5. Architecture & Internals 118

1 Overview	118
2 5.1 Execution, Decode & Internals	122
Execution Flight Plan	122
Execution / Boot Internals	123
grainFactoryLib	125
DispatchTables	127
Floating-Point Considerations	128
CMake Macro Guards	130
3 5.2 Memory, Addressing & Deep Dives	131

Memory	132
Addressing	133
Memory Map & IPRs	134
Addressing & TLB	136
4 5.3 Chipset & Devices	139
Tsunami \ Typhoon	140
Tsunami Error Chain (PERROR / No Devsel)	142
DECchip 21274 (Titan)	144
South Bridge	145
ALi M1543C South-Bridge Interface	147
Cypress	148
CY82C693	149
82077 Floppy Controller	150
DE500 Ethernet (21143 Tulip)	153
5 5.4 Firmware Decode & Decompression	157
Host Decompressor	158
Decompressor Internals	160
Post Flash	161
6 5.5 PALcode, HWRPB & SMP	162
HWRPB Key/Value Offset Map	170
SMP (MultiProcessor) Design	175
ES40 TIG Module Reset (Experimental)	177
7 5.6 Platform Execution Modes & Clock	178
Platform Execution Mode: ISP vs Silicon	181
Reset into Secondary Boot (CSERVE START Handoff)	185
Virtualization Methods	194
Platform Execution	195
Clock Frequency and the MHz Reading	198
WARP Variants and Cycle-Accounting	200
Model Badging (IIC Discriminator)	203
Part VI 6. Reference	206
1 Windows Registry Settings	206
2 Windows Defender Firewall	209
3 Execution Matrix (Models x Modes)	210
4 Per-CPU Halt Codes	212
5 Environment Variables (EMULATR_*)	216
6 Development Conventions	223
7 Glossary	225
Part VII 7. Troubleshooting	227
1 EmulatR will not start	227
2 Firmware Hangs / Slow Boot / Missing Devices	228
3 Console Connection Issues	229
4 Halt Button is IN, BOOT NOT POSSIBLE	229
5 Example (Console) execution	230

Part VIII	8. Appendices - Developer Reference	232
1	8.1 Platform Appendices	232
	Appendix - DS10	233
	Env Vars, Shutdown & Known Limitations	234
	SRM Boot, Commands & SuperIO	235
	Task Matrix & TimeOut Loops	236
	Appendix DS20 / ES40 / ES45	237
	Platform Memory Reference	238
	SRM Boot Call-Stack	241
2	8.2 Tools	244
	Ghidra	245
	Help & Manual	245
	Software Verify	246
	Qt Creator	246
	script	247
	alpha_disasm	248
	analyze_retire_trace	249
	build_test_run	249
	decompile_ds10	249
	diag_halt_irq4	250
	launch_vms_boot	250
	mkdisk	251
	phase1 dispatch gate	251
	profile_resolve	252
	run_fw.sh	252
	snapmap	253
	trace_halt	253
	Tools Script Reference	254
	Platform Editor	259
3	8.3 Source Cross Reference	262
4	Testing (draft)	264
	Supported Test Cases	265
	Index	274

1 1. Introduction

EmulatR is a software emulator of the Digital (DEC) Alpha AXP family. It models the 21264 / 21264A (EV6 / EV67) processor and the 21272 "Tsunami / Typhoon" core-logic chipset, and is complete enough to load and run original, unmodified DEC SRM console firmware to the interactive >>> prompt.

This guide serves two audiences. Operators will find everything needed to build EmulatR, load firmware, connect a terminal, drive the SRM console, and shut down cleanly. Developers will find the architecture and subsystem internals in the later sections. Where a fact is not yet confirmed against the current build, the text flags it with a [TODO] marker rather than guessing.

In this section

What is EmulatR -- the emulated machine at a glance.

Supported Systems and Current Platform Support -- the DS10 target and the ES40 / ES45 placeholders.

Feature Matrix -- per-subsystem status (working, in progress, planned).

Current Development Status -- where the project stands right now and what is next.

Where to go next

New here? Start with Quick Start for the fastest path from a built binary to the >>> prompt. To build EmulatR first, see Installation. Once at the prompt, the SRM Console section covers the commands; Troubleshooting covers what to check when something does not start.

See Also.

[What is EmulatR](#) → 6

[Supported Systems](#) → 17

[Current Platform Support](#) → 17

[Feature Matrix](#) → 18

[Current Development Status](#) → 21

1.1 What is EmulatR

EmulatR is a software emulator of the Digital (DEC) Alpha AXP family. It models the 21264 / 21264A (EV6 / EV67) processor and the 21272 "Tsunami / Typhoon" core-logic chipset, targeting AlphaServer DS10 / ES40 / ES45-class machines.

Its goal is faithful, cycle-aware emulation -- complete enough to load and execute original, unmodified DEC SRM console firmware and reach the interactive >>> prompt. The core is

written in modern C++ (C++20) and is kept free of any GUI-framework dependency; the console subsystem layers on top of it.

At a glance

CPU: Alpha 21264A (EV6 / EV67), with OpenVMS and Tru64 UNIX PALcode.

Core logic: 21272 Tsunami / Typhoon -- Cchip (21272-CA), Dchip (21272-DA), Pchip (21272-EA).

Southbridge: Cypress CY82C693 -- PCI-to-ISA bridge, IDE/ATAPI, and SuperIO (FDC, COM1/COM2, LPT1).

Firmware: runs a real SRM image, for example DS10 SRM V7.3 (ds10_v7_3.exe).

Console: a RAW TCP server, reached with any terminal client (PuTTY, plink, and similar).

See Also.

[Supported Systems](#) ¹⁷

[Current Platform Support](#) ¹⁷

[Feature Matrix](#) ¹⁸

[Introduction](#) ⁶

1.2 Release Notes

EmulatR brings the real DEC/Compaq Alpha SRM console firmware to life on emulated Tsunami/Typhoon-class hardware. Each release records what landed, what was fixed, and the current known limitations. Entries are newest first.

v1.6 (Beta) — 2026-08-01

Two firsts this cycle: EmulatR produces a guest crash dump on disk, and the reason OpenVMS stops short of the login prompt is now identified rather than merely observed. The operator crash path, which previously stopped after the register dump, now runs to completion and writes a dump that the OpenVMS System Dump Analyzer reads. That dump is what located the remaining boot blocker.

Highlights this cycle

- **Crash dumps reach the disk.** *The sequence now runs from Ctrl/P through the bugcheck banner, the register dump and the loaded-image table to a written dump — Memory dump complete, 28809 blocks used of 167418 blocks in dump file. The dump loads cleanly in SDA, which makes guest-side post-mortem analysis a working tool rather than a goal.*
- **Execution accelerations removed for fidelity.** *The delay-loop and idle-spin "warp" levers are gone from the execution path. They advanced the guest clock, and one of them wrote a guest register directly to force a countdown loop to exit. Runs are now cycle-honest end to*

end: the cycle accounting reports `warp_cyc=0` with `MHz_real == MHz_eff`, meaning every cycle was executed and none fabricated. Boot is slower as a result. That is the intended trade.

- **An undocumented processor register no longer faults.** *The SRM writes an EV6 internal register absent from the published 21264 register table. EmulatR raised an illegal-instruction fault on it, which aborted part of firmware reset initialization and sent secondary boot back into console startup. Accepting the write is required to boot a guest operating system — see [Conditions](#).*

Conditions — what this result requires

The crash-dump result was produced under the configuration below. Departing from it, in particular from `EMULATR_2D_NOOP`, will not reproduce it.

Setting	Value and why
Platform	DS20, firmware <code>ds20_v7_3.exe</code> , guest media <code>dka0</code> , OpenVMS Alpha V8.3.
Launcher	<code>tools/run_ds20_bplus.sh</code> — it carries the secondary-boot stack. The plain console launcher <code>tools/run_ds20_putty.sh</code> does not, and falls back into console initialization instead of entering the bootstrap.
<code>EMULATR_2D_NOOP</code>	Must be 1. Without it the undocumented register write faults, firmware reset initialization does not finish, and the boot re-enters console startup. Set automatically by <code>run_ds20_bplus.sh</code> .
<code>SRM_STARTUP_P1</code>	Left blank, i.e. a full startup. A minimum startup may not bring up enough of the system to reach <code>SYS\$SYSTEM:SYSDUMP.DMP</code> , and a dump can then fail quietly.
<code>SRM_DUMPSTYLE</code>	11 (selective + full console output + compressed). Selective preserves system space and the current process; the console-output bit puts the crash-time registers and stacks in the console log where they are readable without opening the dump.
Console	PuTTY over the TCP console port. Use a port no other instance holds — a bind collision suppresses the PuTTY auto-launch silently.

Boot command variants

All three forms are accepted. The console echoes the normalized form it will act on.

Command	Effect
b dka0	Normal boot. Runs into the operating system without stopping.
b -fl 0,1 dka0	Conversational boot. Stops at <code>SYSBOOT></code> so system parameters can be set before the OS starts. This is the form used to set <code>DUMPSTYLE</code> above.
b dka0 -flags 0,1	Equivalent long form. The console echoes <code>(boot dka0.0.0.8.0 -flags 0,1)</code> for either spelling.

The crash sequence, end to end

This uses the firmware's own `controlp` machinery. There are no emulator hooks in the path.

1. At `SYSBOOT>`, set `dumpstyle 11`, then `c` to continue.
2. With OpenVMS running, press `Ctrl/P`. The console reports `halted CPU 0, halt code = 1, operator initiated halt`.
3. At `P00>>>`, type `crash`. The console reports `CPU 0 restarting`.
4. OpenVMS prints the BUGCHECK banner with code `0000064C: OPERCRASH`, the register dump, the stack and the loaded-image table.
5. The dump is written to the system disk, then analysed with `ANALYZE/CRASH_DUMP SYS$SYSTEM:SYSDUMP.DMP`.

Restored: the dump now reaches the disk

Everything from `Accessing system disk` onward is new in this release. The previous cycle stopped after the register dump with no banner and no dump file.

```
**** Accessing system disk via original boot path
**** Starting compressed selective memory dump...
** Dumping PT space...
** Dumping S0/S1 space...
** Dumping S2 space...
**** Memory dump complete, 28809 blocks used of 167418 blocks in dump
file...
```

Storage: why OpenVMS stops short of the login prompt

This limitation is unchanged in effect but is no longer unexplained. The crash dump identified it precisely, and the finding is recorded here because it is the single blocker between the current release and an interactive guest.

OpenVMS boots into the executive and prints its banner. The swapper then waits for the system disk to be marked valid, and that never happens. The chain, measured rather than inferred:

1. The SCSI driver issues an `INQUIRY` asking for up to 255 bytes. The disk answers with the standard 36.

2. On real hardware the target then changes bus phase, the adapter detects the mismatch, records the unsent byte count and raises an interrupt. EmulatR instead padded the transfer with zeroes and reported it complete, so no interrupt was ever raised.
3. The port driver waits on that interrupt to finish initialization, so the SCSI port never comes online and reports zero completed operations.
4. The disk behind it therefore never leaves initialization and never reports a device type, so it is never marked valid.
5. The swapper's wait for that valid flag never ends. Because no timeout routine is registered on either device, nothing recovers it.

The phase-mismatch behaviour has been implemented against DEC's own NCR 810 register definitions and the SRM's own driver for the same chip. **Verification is in progress and this release does not yet claim the guest reaches a login prompt.** Note that the adapter identity itself is correct: OpenVMS classifies the emulated controller exactly as it classifies the real part.

Known limitations (what to expect in this beta)

- **OpenVMS does not yet reach the login prompt.** *Cause identified — see Storage above.*
- **The cause of the restored dump path is not yet isolated.** *Three things changed together this cycle: the accelerations were removed, DUMPSTYLE moved, and the startup went from minimum to full. Any of the three could carry the result. Recorded as an observed result, not an explained one.*
- **Boot is slower than the previous release.** *Deliberate. Firmware settling delays and idle spins now run at their real cost. Cold boot to P00>>> runs at roughly 3.7 to 4.1 million emulated cycles per second on the reference host, and a full crash dump takes several minutes to write.*
- **Guest clock date.** *The guest boot time and dump timestamps do not track the host clock. The TOY/RTC date path is a known open item, and because driver timeouts are driven from that clock it may also be why a stalled device is never recovered.*
- **Single-CPU only.** *Unchanged from the previous release.*

v1.5 (Beta) — 2026-07-29

Breakthrough: OpenVMS makes it off the firmware and into the operating system.

OpenVMS Alpha V8.3 now boots from `dka0` past SYSBOOT into the executive — banner, exec init, and a running scheduler — on DS20. Two CPU-faithfulness defects that each ended the boot in an `INVEXCEPTN` bugcheck were root-caused with the full oracle loop (EmulatR crash dump, SDA analysis, Alpha Architecture Manual confirmation, fix, in-boot verification) and eliminated. The previous release reached the SRM `>>>` prompt only; this cycle the guest OS runs.

Highlights this cycle

- **OpenVMS boots into the executive.** *The boot now runs an order of magnitude past both former crash sites with a live scheduler, currently progressing through early system initialization.*
- **The crash-dump workflow is a proven tool.** *Operator halt (Ctrl/P) from a running OS to >>>, conversational boot (b -flags 0,1 to SYSBOOT>), and the halt / crash / dump / SDA loop all work against EmulatR guests — the firmware's own controlp machinery, no emulator hooks needed.*
- **The TOY clock behaves like real hardware.** *Dead-battery lifecycle, guest SET TIME honored, and the CMOS persists across runs (see Added below).*

Fixed

- **Stale firmware bytes in the instruction stream.** *The SRM loader's instruction-fetch override served frozen firmware-image bytes for any fetch in the firmware load range for the life of the run; once OpenVMS loaded executive code into that physical range, the CPU executed stale firmware content while data reads saw real memory (INVEXCEPTN, garbage instruction stream). The override now retires at the decompressor hand-off — the only era it legitimately models — verified at the bus level.*
- **Loads to R31/F31 could fault.** *Per the Alpha architecture, a load whose destination is R31/F31 is a prefetch and never signals an exception (LDQ_U R31 is the canonical UNOP). EmulatR executed them as real accesses, and SWAPPER's scheduler epilogue padding — LDQ_U R31, (SP) at a stack boundary — took an architecturally impossible ACCVIO. Such loads now perform no memory access and raise no fault.*

Added

- **TOY/RTC time-source modes** via EMULATR_TOY_MODE: unset (default) is the faithful dead-battery lifecycle — the clock reads invalid until the guest sets the time, then stays set; fixed is a pure function of guest cycles (required for differential/oracle gate runs); host seeds from the host wall clock (best usability, breaks bit-reproducibility).
- **Guest SET TIME honored and persisted.** *The MC146818 SET-protocol write is decoded and latched as the new time origin (previously it was silently discarded), and the full CMOS — clock plus the NVRAM bytes the console really uses — persists to nvram/toy_cmos.bin in the run directory (EMULATR_TOY_NVRAM overrides). Corrupt or mode-mismatched stores are refused loudly.*
- **21 new unit tests** covering the instruction-stream regressions and the full TOY lifecycle (write protocol, persistence round-trip, corrupt store, gate rule).

Known limitations (what to expect in this beta)

- **OpenVMS does not yet reach the login prompt.** *After the banner the guest idles between bootstrap-driver disk reads; whether this is emulation pacing or a wait condition is the active investigation. The boot-time clock also logs 17-NOV-1858 (VMS time-zero), implicating the OS-side RTC access path — both are next-cycle leads.*
- **Single-CPU only.** *The HWRPB correctly advertises the second DS20 CPU slot as empty (verified live); SMP execution remains future work.*

Operator note — the LFU prompts during init

- LFU> `update srm` The running console is V7.3-2 (Feb 27 2007) — the same `ds20_v7_3.exe` image EmulatR loads fresh on every launch. There's nothing newer to update to, and the loaded firmware always comes from that file, not from what an update would write. For now: LFU>`exit`
 - The LFU write path is untested territory in EmulatR. An update attempt exercises the firmware-update utility's flash-programming sequence against the emulated TIG flash, whose backing file (`ds20_v7_3.rom` beside the firmware) is where SRM environment variables live —
 - `auto_action`, `boot_osflags`, `controlp`, the `wwid` entries, everything you saw in `show *`. A partial or misbehaving update could corrupt that store, and then every subsequent boot inherits scrambled env state.
 - The LFU prompts you see during init are noise, not a request. This firmware auto-enters the LFU during startup (sometimes twice, per the run-emulatr gotchas — "Option firmware files were not found on CD or floppy..."). The correct response is to `exit/decline` back to `P00>>>` — the scripted console driver does exactly that. It's asking whether you want to load option firmware, not telling you an update is needed. If the prompt in front of you is the LFU asking for a device, just get back to `P00>>>` and proceed with the boot.

v1.4.4 (Beta) — 2026-07-23

This beta reaches the SRM `>>>` prompt on DS10, DS20, and ES40. Guest-operating-system boot is not yet available (see Known limitations below).

Highlights this cycle

- **A second classic Alpha comes to life.** *The DS20 now cold-boots to the SRM `>>>` prompt and badges correctly as AlphaServer DS20 — joining DS10, with ES40 (Clipper) bring-up also reaching the console.*
- **Platform manifest system.** *Each model's device tree is described by a runtime-loaded `<firmware>_platform.json`, editable through the Platform Editor desktop application (`editors/platedit_qt`) with live validation and format-preserving save.*

- **Deterministic execution substrate + fast resume.** *A multi-agent scheduler (agent + dispatcher, Sequential==Threaded verified) and Level-1 snapshots (auto-save / autoload-newest, plus an entry snapshot that skips the firmware decompressor).*
- **New device models.** *Floppy FDC, keyboard + VGA console, and IDE/ATAPI storage (Cypress func1 on DS10/DS20; ALi M5229 on ES40) over a common block-media seam.*
- **Cross-platform.** *macOS/Linux build hardening alongside the Windows build.*
- **Packaging & docs.** *Versioned redistributable packaging, the full user guide shipped inside the package, and this release-notes track.*

Fixed

- **DS20 model badging** — *root-caused to an IIC completion-interrupt gap and fixed; the console now reports the correct system identity.*
- **ES40 memtest ACV** — *resolved (an AAR ASIZ decode-width mismatch); a spurious CSERVE 0x66 get_time behavior was identified as a regression and removed (the 0x66 no-op is the faithful behavior).*
- **ES40 console** — *SCB base-mismatch halt fixed and the COM2 console backend landed.*
- **MMU / TLB** — *the DTB/ITB are now modeled as truly 128-entry fully-associative, fixing a conflict-eviction thrash.*
- **Memory** — *a page-crossing store overrun in the sparse pager was fixed.*
- **Build portability** — *added the explicit <new> / <cstdlib> includes that the Release configuration and the clang (macOS) build require.*

Known limitations (what to expect in this beta)

- **No guest OS boot yet.** *The SRM-to-OS hand-off at VA 0x20000000 is the active frontier: the VMB loads and builds the HWRPB/page tables but the transfer to the OS entry does not yet fire.*
- **OpenVMS/Tru64 install is gated on floating point.** *The FP unit is an IEEE-T proof-of-concept; VAX G/F float (which VMS relies on) is not yet implemented.*
- **Partial PCI enumeration.** *EmulatR currently enumerates about one on-board device where silicon shows about nine; the full device list and SYSBOOT> path depend on the PCI bus walk still in progress.*
- **No Ethernet yet.** *The on-board DE500 / 21143 ("Tulip") NIC is designed but not implemented.*
- **Single-CPU only.** *SMP is scaffolded but boots run cpuCount=1 deliberately.*

- **DS25 / ES45 (Titan chipset) are experimental** — *model-only, not yet booting.*
- **Cosmetic mismatches.** *The clock-MHz banner and a handful of SRM defaults differ from real hardware.*

Beta test focus

We would most value reports on:

- *Cold-boot to >>> on DS10 / DS20 / ES40 (via `run_fw.sh` or the `run_*` helper scripts).*
- *Console over PuTTY/plink on TCP 10023 — `show config`, `show device`, `show memory`, `environment variables`.*
- *Storage — attaching a virtual disk / CD-ROM and `show dev enumeration`.*
- *The Platform Editor — opening and editing a `<model>_platform.json`.*

Because EmulatR treats itself as the primary correctness oracle, anything that diverges from real-hardware behavior is worth reporting.

Getting started

Two build variants are provided: `EmulatR_v1.4.4_relwithdebinfo.zip` (diagnostic logging and trace facilities compiled in — preferred for beta, since it lets you capture the logs/traces that make a bug report actionable) and `EmulatR_v1.4.4_release.zip` (optimized, diagnostics compiled out; for performance and clean runs). Firmware is not included — DEC/Compaq/HPE SRM images are licensed, so supply your own `<model>_v7_3.exe` in `./firmware/`. Full documentation ships in `documentation/EmulatRGuide.pdf`.

Reporting issues

Please include the model, the build variant, the run command, and — on a `RelWithDebInfo` build — the run log and any `.trc` from `logs/` and `traces/`.

See Also

[Current Development Status](#) ^{↗ 21}

[Known Gaps](#) ^{↗ 23}

[Running from a Run Directory](#) ^{↗ 56}

1.3 License

ASA-EMulatR

Copyright © 2025–2026 Timothy Peer (eNVy Systems, Inc.)
<https://envysys.com>

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Commercial Licensing

This project is also available under a separate commercial license. If you wish to use ASA-EMulatR in a proprietary product, closed-source environment, or in a manner incompatible with the GNU General Public License, you must obtain a commercial license from:

Timothy Peer
eNVy Systems, Inc.
peert_at_envysys_dot_com
<https://envysys.com>

Commercial licensing terms are available upon request.

For the full license terms and commercial-licensing inquiries, contact Timothy Peer / eNVy Systems, Inc. at peert_at_envysys_dot_com.

See Also.

[Attributions & Credits](#) ¹⁶⁰

1.4 Goals & Objectives

EmulatR is a full-system, cycle-based, high-fidelity emulator of the DEC Alpha AXP architecture. Its purpose is not to approximate an Alpha but to behave like one -- faithfully enough that the most demanding original software cannot tell the difference. This page sets out the single standard the project is held to and the priority-ordered goals every design decision serves.

The fidelity standard

EmulatR is governed by one overriding standard: it must behave like real Alpha hardware in every architecturally visible way, even when doing so complicates the implementation. This is applied uniformly across all subsystems -- the instruction set, memory translation, cache and coherency behavior, the PAL boundary, interrupt and exception delivery, device and

DMA semantics, and multiprocessor ordering. A subsystem is correct only when no architecturally visible difference from hardware can be observed through it.

The test of fidelity is black-box and adversarial. The judges are not internal assertions but the most demanding guests available: the SRM console and PALcode exercise the privileged architecture directly, a booting operating system exercises the memory model, trap machinery, and device fabric under real workloads, and differential validation against a reference emulator checks behavior at the instruction and subsystem level. Fidelity here means architectural-semantic accuracy, not the reproduction of a particular chip's timing: EmulatR reproduces exactly what the architecture makes visible -- the cycle-based run loop, the precise point at which results and exceptions commit, the ordering guarantees and barriers, and the privileged state transitions.

Primary goals (in strict priority order)

EmulatR is designed around five primary goals. Every design decision is evaluated against them in this order; when two goals conflict, the higher-priority goal wins.

1. **Architectural correctness first.** The emulator models Alpha's architectural semantics, not its performance characteristics. Correctness is always prioritized over speed.
2. **Deterministic and debuggable execution.** Execution must be understandable, reproducible, and inspectable. One iteration of the run loop corresponds to one hardware clock cycle, enabling deterministic replay and debugging.
3. **Explicit serialization.** Ordering and synchronization are never implicit. All serialization is driven by architectural mechanisms -- the MB, WMB, EXCB, and TRAPB barriers, PAL entry, and exceptions -- and exceptions are delivered precisely at the architectural commit point, the Writeback stage.
4. **SMP as a first-class concern.** Multiprocessor behavior is fundamental to the design, not layered on afterward. Cross-CPU coordination is modeled with explicit barriers, reservation tracking, and IPI delivery.
5. **Separation of concerns.** Instruction decode, execution, memory access, coherency, privilege, and devices are isolated into well-defined functional domains (the "Boxes") with one-way dependency flow. No domain reaches into another's internal state.

Strategy: one set of objectives, four implementations

These objectives have been invariant across the entire history of the project. Four distinct implementations have been written; each changed the mechanism, never the goals. The current design (V4) is the first to satisfy all five at once -- it keeps a flat, shallow dispatch path for determinism and debuggability while resolving each instruction to a faithful, fully-architected execution leaf. The strategy throughout has been to let the implementation evolve toward the fixed standard, never to relax the standard to fit an implementation.

See Also.

[What is EmulatR](#) ⁶

[Architecture Overview](#) ¹¹⁸

[Current Development Status](#) ²¹

1.5 Supported Systems

EmulatR targets the Tsunami / Typhoon AlphaServer line. The emulated hardware reported by the SRM console (`show config`) reflects a DS10-class machine:

- *Processor: Alpha EV6 pass 1, 268 MHz (a calibrated figure -- see [Clock Frequency and the MHz Reading](#) ¹⁹⁸), Bcache disabled.*
- *Core logic: DECchip 21272-CA (Cchip) Rev 2, 21272-DA (Dchip) Rev 1, 21272-EA (Pchip 0) Rev 1; TIG / Arbiter Rev 7.31.*
- *Memory: 1024 MB default array (configurable via [System] memorySize in the ini or --mem), base 0x000000000.*
- *I/O: PCI Hose 00 with the Cypress 82C693 bridge / IDE at bus 0, slot 5; ISA devices MOUSE, KBD, COM1, COM2, LPT1, FLOPPY.*

Platform selection is driven by a per-model bus-configuration manifest (see Configuring). DS10 is the actively developed and validated platform; ES40 and ES45 manifests are scaffolded (see Current Platform Support).

See Also.

[What is EmulatR](#) ⁶

[Current Platform Support](#) ¹⁷

[Configuring](#) ⁶⁹

1.6 Current Platform Support

A platform manifest describes the machine's device tree. It is named after the FIRMWARE IMAGE that actually loaded, not after the model name: `firmware/ds20_v7_3.exe` selects `ds20_v7_3_platform.json`. Keying the manifest to the firmware means the model and its firmware cannot drift apart, and a firmware variant can carry its own device set. The file is resolved next to the executable, not in the directory you launched from.

The table below summarizes current status. "Boots to >>>" means the platform has been driven to the interactive SRM prompt in EmulatR; DS10, DS20 and ES40 additionally form a standing three-platform gate that must stay green before any shared change is committed.

Platform	Manifest	CPU / Chipset	Status
DS10	ds10_v7_3_platform.json	EV6 / 21272	Active -- boots to >>>

Platform	Manifest	CPU / Chipset	Status
DS20	ds20_v7_3_platform.json	EV6 / 21272	Active -- boots to >>>
ES40	es40_v7_3_platform.json	EV6 / 21272 (Typhoon)	Active -- boots to >>>; init does not yet complete
DS25	ds25_v7_3_platform.json	EV67 / 21274 (Titan)	Manifest present; not yet driven to the prompt
ES45	es45_v7_3_platform.json	EV67 / 21274 (Titan)	Firmware decompresses; platform not yet driven

The selection order is: the `EMULATR_PLATFORM_CONFIG` environment variable if set, otherwise the name derived from the firmware image stem as described above.

A missing or invalid manifest does NOT stop the run. It falls back to the compiled-in default DS10 device tree, and the reason is printed, for example: `PlatformConfig: using built-in default DS10 manifest -- "unsupported manifest_version 999 (expected 1)"`. This is worth knowing because a typo in the manifest name or a schema-version mismatch will not announce itself as a failure - the machine simply comes up as a DS10. If a platform behaves like the wrong model, check that line first.

See Also.

[Supported Systems](#) ¹⁷

[Feature Matrix](#) ¹⁸

[Configuring](#) ⁶⁹

1.7 Feature Matrix

Subsystem status as of August 2026. This reflects the DS10 / DS20 target, with ES40 in active bring-up; status evolves and should be reconfirmed against the current build.

Status vocabulary. **Working:** exercised by real firmware or a real guest and covered by tests. **In progress:** functional on the measured path, with named gaps still open. **Planned:** not implemented. A subsystem marked Working is faithful for the paths that have been exercised - it is not a claim of complete device coverage.

Subsystem	Status	Notes
CPU pipeline (EV6) + TLB	Working	128 ITB + 128 DTB; PALcode relocation in place.
SRM firmware boot to >>>	Working	Reaches the interactive console prompt.
TCP console server	Working	RAW TCP; PuTTY auto-launch.
Snapshot save / restore	Working	.axpsnap; autoload-newest, marker capture, plus the EMULATR_FAST_DECOMPRESS entry-snapshot lever.
SuperIO (FDC / COM / LPT)	Working	CY82C693 SuperIO word-fold confirmed.
IDE / ATAPI virtual CD-ROM	In progress	dqa0 fixed disk enumerates and reads; ATA/ATAPI reset signatures and IDENTIFY are faithful. READ-ONLY and POLLED by design so far: there is no interrupt path (IRQ 14/15 are never driven), no write path (writes abort), and no DMA. Adequate for enumeration and console-era reads; not yet usable as a guest-OS system disk or install target.
IIC (PCF8584)	In progress	Sufficient for console bring-up on DS10 and DS20, including the discriminator device that yields the correct DS20 system badge. The bus runs POLLED (the completion interrupt is never enabled by firmware). The ES40 IIC decode base is unresolved and is likely an ALi M1543C SMBus rather than a fixed PCF8584 row.
Flash / NVRAM persistence	Working	Persists to <firmware>.rom / ds10_flash.rom; set and update srm survive across boots.

Subsystem	Status	Notes
SCSI host adapter (NCR 53C810)	Working	The <code>pka</code> adapter is modeled at register level, including its SCRIPTS I/O processor: selection, bus-phase sequencing, message in / message out, DMA through the chipset address windows, and interrupt delivery. Register layout and the phase-mismatch recovery contract are verified against the DEC SRM console's own SCSI driver. Disconnect / reselection and tagged command queuing are deliberately not modeled yet.
SCSI disk and CD-ROM (<code>dka</code>)	Working	SCSI-2 target command set audited against ANSI X3.131-1994: INQUIRY, TEST UNIT READY, REQUEST SENSE with the UNIT ATTENTION state machine, READ CAPACITY, MODE SENSE / MODE SELECT with rigid-disk geometry pages, and READ / WRITE with range and write-protect checking. Genuine DEC drive identities (RZ28L / RZ29L / RZ40) carry documented geometry and capacity. Available on DS20 and ES40; DS10 is an IDE-only machine and has no SCSI bus.
boot / OS load	In progress	An explicit <code>boot dka0</code> loads the bootstrap from the SCSI disk and transfers control to it. <code>auto_action</code> HALT remains the fully wired automatic action; automatic BOOT and <code>boot_osflags</code> handling are not yet validated.

Subsystem	Status	Notes
Guest operating system (OpenVMS)	In progress	OpenVMS Alpha V8.3 boots from the SCSI disk to its banner reliably and its device drivers initialize against the emulated adapter and drive. The system disk is not yet mounted, so a login prompt is not yet reached - this is the active bring-up front. Tru64 is untried.

Basis for these claims

"Faithful" and "verified" on this page mean checked against a primary source, not against expectation. The sources behind the storage and chipset rows are:

- *ANSI X3.131-1994 (SCSI-2) - the target command set, sense data, and mode pages.*
- *NCR / Symbios 53C895 Data Manual, and the DEC SRM console's own SCSI driver sources - the adapter registers, its SCRIPTS processor, and the driver contract the adapter has to satisfy.*
- *EK-KZPBA-UG.B01 (KZPBA UltraSCSI adapter guide) - how a real DEC console renders drive identities.*
- *EK-SM2DR-PN.B01 (StorageWorks drive reference) - RZ28L / RZ29L / RZ40 geometry and capacity.*
- *The 21272 and 21264 hardware reference manuals - chipset registers and CPU behavior.*

Per-claim verification, down to the clause or table and the line of code it was checked against, is recorded in the engineering audit records rather than here; this page is the summary they roll up to. Where a value could not be traced to a source it is called out as provisional rather than presented as fact.

See Also.

[Current Development Status](#) ²¹

[Current Platform Support](#) ¹⁷

1.8 Current Development Status

This page is a plain-language snapshot of where EmulatR stands as of August 2026 -- what works end to end, the milestone the project is working toward next, and the known rough edges. It is deliberately a moving target; the authoritative day-to-day record lives in the engineering journals and the dated CHECKPOINT notes. For a per-subsystem breakdown see the Feature Matrix.

What works today

A clean cold boot runs the original, unmodified DEC SRM console firmware all the way to the interactive `>>>` prompt, fully usable over a serial terminal. The commands `show config`, `show memory`, `show pal` and `show version` all return correct output. Firmware is a real SRM 7.3-1 image carried in emulated flash; the LFU `update srm` path works, and machine-state snapshots have been captured for both the firmware-update (`UPD>`) and console (`>>>`) states. Snapshots also back the entry-snapshot boot-acceleration lever (`EMULATR_FAST_DECOMPRESS`), which reuses a saved post-decompression state to skip the self-decompression on later runs (see Boot Acceleration - Entry Snapshot). ES40 (Typhoon) silicon-mode bring-up is in progress alongside the DS10 / DS20 line.

The current frontier: mounting the system disk

The frontier has moved. Both gaps that previously stood between the console prompt and an OS boot are CLOSED: on-board PCI devices are now walked and enumerated (the SCSI adapter and the Ethernet controller are modeled and appear in `show config`), and a bootable SCSI disk is enumerated - `show device` lists a `dka` drive carrying a genuine DEC identity, and `boot dka0` loads and transfers to the operating system bootstrap.

OpenVMS Alpha V8.3 now boots from that disk to its startup banner reliably, and its device drivers initialize against the emulated adapter and drive: the adapter is identified correctly as a SCSI port, the drive's identity strings and capacity reach the guest byte-exact, and the power-on attention handshake completes.

The single remaining blocker is the next step after that: the system disk never transitions to VALID, so the volume is never mounted and the boot does not reach a login prompt. The drive geometry reported through MODE SENSE is the leading suspect and is under active measurement. Everything upstream of it has been measured working rather than assumed.

Known cosmetic issues

Two harmless display quirks are still open. Neither affects operation.

Garbled SROM revision, and a stray "PuTTY" at the prompt. EmulatR does not populate the SROM revision field, so when the console prints it the firmware formats an uninitialized buffer and emits a short run of garbage characters. That garbage happens to contain an ENQ control byte (`0x05`), and PuTTY answers ENQ by transmitting its answerback string - which is literally the word `PuTTY` - back to the host, where the console echoes it. That is why `show config` can leave you looking at `P00>>>PuTTY`. It is a round trip through your terminal emulator, not the emulator inventing text, and it is specific to clients that implement answerback. WORKAROUND: clear PuTTY's "Answerback to ^E" setting. The real fix is to populate the field, or to strip control bytes on the console transmit path so every client benefits.

Line-doubling. `show device` can print each line twice, also seen at the firmware-update `UPD>` prompt. The leading hypothesis is that this is not a per-command bug but the console transport echoing input that the guest firmware already echoes; the test is to default terminal echo off and see whether the doubling clears. That test has not been run yet, so the cause is not confirmed.

What is planned next

- *Get the system volume mounted, then carry the OpenVMS boot through to a login prompt.*
- *Complete the SCSI CD-ROM target so an OS can be installed from CD media, not only booted from a prepared disk.*
- *Give the IDE channel an interrupt path, a write path and DMA, so an IDE disk can serve as a system disk on DS10.*
- *Finish ES40 `init`, and drive the Titan machines (DS25, ES45) to the console prompt.*

Completed since the previous revision of this page: SRM environment persistence (`set` now survives reset and cold boot via emulated flash), PCI bus enumeration with on-board Ethernet and SCSI device models, and a bootable SCSI disk with real DEC drive identities.

Snapshot date: August 2026. Because status changes frequently, treat this page as orientation and confirm specifics against the current build and the latest CHECKPOINT journal.

See Also.

[Boot Acceleration - Entry Snapshot](#) ¹⁰⁷

[Feature Matrix](#) ¹⁸

[Architecture Overview](#) ¹¹⁸

[Halt Button is IN, BOOT NOT POSSIBLE](#) ²²⁹

1.9 Known Gaps

Release checkpoint = DS10 + DS20 + ES40 certified to the same boot milestone. All three reach `P00>>>`, and that three-platform console gate is now a standing pre-commit requirement for any shared core, chipset, or device change; ES40 does not yet complete `init`. Delete a row when its gap is closed. IDs are stable handles (a closed row's ID is not reused).

Beyond the console, the current front is the guest operating system: OpenVMS Alpha V8.3 boots from the SCSI disk to its banner reliably on DS20 and its drivers initialize, but the system disk has not yet gone valid, so the volume is not mounted and no login prompt is reached. Rows G-27 through G-30 carry that front.

Severity legend. BLOCKER: on the ES40 boot/console path, gates the checkpoint. OPEN: unresolved contradiction to reconcile. GAP: fidelity or downstream, not blocking the prompt. REF: blocked on an absent primary source. DEFER: intentional deferral (silicon-mode / V5 / post-prompt). PROV: a value is _PROVISIONAL.

ID	Sev	Area	Gap - what is missing / current state	Blocks
G-01	BLOCKER	Console	ES40 SRM console commands almost entirely untested (show dev/config/mem, set/show env, examine/deposit, init, boot); no ES40 iteration history vs DS10/DS20. Beta: execute each platform model, SRM>>> set display/persist (requires a clean shutdown)	Release parity with DS10/DS20
G-02	BLOCKER	Console	init not wired for ES40; P00>>>init faults with a non-existent-memory error via vector 0x660 (see G-10).	ES40 checkpoint
G-03	BLOCKER	Console	boot + boot_osflags not wired/validated on DS10/DS20/ES40; cmdBoot resolves bootdef_dev /boot_osflags	Any guest-OS boot

ID	Sev	Area	Gap - what is missing / current state	Blocks
			but the load path is unverified. Silicon mode is now a live surface for the warp validation.	
G-04	BLOCKER	IIC/FRU	IIC controller base missing (kIicBaseByMode1[ES40]); build_power_hw_fopen("iic_system0"/"iic_system1") cannot resolve, fread fails, !status aborts config.	build_config on ES40
G-05	GAP	IIC/FRU	IIC 0x70/0x72 return 0x00 (not faithful; SRM sees "nothing present"); FRU EEPROM reads 0xA2/0xA4 in build_fru_root are the likely next blocker.	Fidelity; next wall
G-06	DEFER	Exec mode	ISP-mode default => no real device discovery; dq_initialize returns msg_failure before IDENTIFY (confirmed mode-behavior (and consider that its real home is the execution-modes topic, as	Device datapath / OS boot

ID	Sev	Area	Gap - what is missing / current state	Blocks
			the worked example).	
G-07	OPEN	Exec mode	DS20 enumerates IDE under the ISP default even though the ISP gate is upstream of the Cypress special case; unreconciled against the <code>ds20</code> (<code>pc264</code>) binary. Silicon-mode runs make the DS20-under-ISP anomaly sharper.	Trust in the ISP model
G-08	DEFER	Exec mode	Silicon-mode delays busy-wait on RSCC; <pre>krn\$_micro_delay y returns immediately only in ISP mode, so silicon grinds. EmulatR models RSCC = cycleCount/5594 ; safe warp advances cpu.cycleCount coherently so RSCC crosses limit and the spin exits (as EMULATR_IDLEWAR P does). Do NOT re-enable the quarantined EMULATR_RSCCWAR P (jumped ticks,</pre>	Silicon-mode completion

ID	Sev	Area	Gap - what is missing / current state	Blocks
			rewrote 0x3c970 out-of-band = confirmed boot corruption). Gated EMULATR_PLATFORM=silicon, discuss-before-code. Silicon mode is now a live surface for the warp validation.	
G-09	GAP	South bridge	REFERENCE BLOCK CLEARED - the ALi M1543C datasheet is now in the reference library, and PCI identities and legacy port decode are verified against it. What remains is behavioral: south-bridge configuration space above the identity header is stored but inert (IDE enable/disable, IDSEL relocation, interrupt routing and PIRQ steering have no effect), which is the prerequisite for wiring the IDE	Faithful bridge init; IDE interrupts

ID	Sev	Area	Gap - what is missing / current state	Blocks
			interrupt path (G-30).	
G-10	BLOCKER	Pchip	PCHIP 1 CSR window undecoded (all regs read 0x00000000FFFFFFF) -> NXM on init (HYPOTHESIS); needs a compile-gated PA probe. Pchip1 base 0x801_C000_0000_PROV (Pchip0 0x801_8000_0000, stride 0x4 0000_0000).	init completion (G-02)
G-11	PROV	Memory	build_hwrpb MEMDSC 4GiB layout reached-but-unaudited; must lay four 1GB arrays (AAR0..3), not one region, or a downstream memory walker underflows.	Latent walker underflow
G-12	PROV	ISA/IPR	HW_MTPR to reserved IPR index 0x2d; confirm the ignore-write is now faithful on ES40 and not merely masked (all 4 platforms fault; ES40 cascaded to DtbMissDouble).	Confirm not masked

ID	Sev	Area	Gap - what is missing / current state	Blocks
G-13	GAP	SCSI	REFERENCE BLOCK CLEARED and the adapter is BUILT - this row no longer describes a missing file. The reference set is closed (ANSI X3.131-1994, the 53C895 data manual, the DEC SRM SCSI driver source, and the KZPBA guide), and an NCR 53C810 adapter with its SCRIPTS processor plus SCSI-2 disk and CD-ROM targets are modeled and audited. Residual fidelity items are individually ledgered: bus-device-reset and abort messages are accepted but not acted on, adapter identity/revision nibbles read zero, and the selection timeout is instantaneous rather than timed.	Error-recovery fidelity
G-15	REF	PAL/console	ES40 platform PAL/console source absent;	ES40-specific init facts

ID	Sev	Area	Gap - what is missing / current state	Blocks
			exact <code>C\$ERVE</code> semantics (<code>0x44/0x45/0x46/0x65/0x66</code>), <code>get_sysvar/platform-detect</code> path, and <code>CSC</code> strap value unresolved.	
G-16	PROV	Topology	ES40 board BDF assignments <code>_PROVISIONAL</code> (DS10-derived); resolvable from an ES40 service manual or a <code>PCICFG</code> trace.	Faithful device map
G-17	GAP	FP/FBOX	Only 4 of 20 FP leaves do real arithmetic; Tier 0 <code>FPCR</code> / rounding-mode infrastructure required first.	OS boot (post-console)
G-18	DEFER	DMA	Scatter-gather translation is now IMPLEMENTED and verified against the chipset manual (page-table walk, address composition, window-miss behavior) with unit tests, and device DMA is routed through it. It remains UNEXERCISED by real firmware: the	OS install (first live witness)

ID	Sev	Area	Gap - what is missing / current state	Blocks
			console and the guest so far use direct-mapped windows, so the gather path has no live witness yet. Expect the OS-installer DMA-mapped path to be its first real consumer.	
G-19	GAP	SMP/IPI	ipcr modeled storage-only; cross-CPU IPI delivery and secondary-CPU start not functionally modeled (ES40/ES45 are multi-CPU).	OS SMP bring-up
G-20	PROV	TIG	arb_rev=0 _PROVISIONAL (needs a real DS10 config dump); TIG snapshot is an assert-guarded defer; EMULATR_TIG_TRACE guards the next unmodeled TIG register.	Fidelity
G-21	DEFER	JIT	JIT compiler is the V5 variant; V4 is fidelity-first with JIT deferred until a guest OS boots (no perf numbers	(by design)

ID	Sev	Area	Gap - what is missing / current state	Blocks
			quoted publicly until then).	
G-22	DEFER	OS boot	End-to-end OpenVMS / Tru64 boot. PARTIALLY ADVANCED: OpenVMS Alpha V8.3 now boots from the SCSI disk to its banner reliably and its drivers initialize, so this is no longer a distant horizon - it is the active front, carried by G-27. Reaching a login prompt still depends on G-17 (floating point) and, for the multi-CPU machines, G-19 (SMP). Tru64 remains untried.	Milestone
G-23	BLOCKER	Boot	RUN/STOP handling on the boot path; mechanism _PROVISIONAL - CPU RUN/STOP (console halt/continue, HWRPB BIP/RC handoff) and/or device RUN/STOP (controller start/stop bit in the boot driver). Future blocker	boot handoff (G-03)

ID	Sev	Area	Gap - what is missing / current state	Blocks
			once <code>boot</code> is wired; trace to pin the mechanism before scoping.	
G-24	OPEN	Silicon	ES40 does not yet initialize in silicon mode (<code>EMULATR_PLATFORM=silicon</code>) -- outstanding diagnostic, active surface (not a deferral). Under warp the RSCC micro-delay spin now exits (G-08), but ES40 <code>init</code> still does not complete on the silicon path; entangled with the ISP-path init blockers (G-02 / G-10) and the exec-mode reconciliation (G-06 / G-07). Needs an evidence-first trace of where silicon-mode init diverges before scoping a fix.	ES40 silicon-mode bring-up
G-25	DEFER	Config/CLI	Unified configuration precedence not yet implemented. TODAY: built-in defaults < <code>EmulatrV4.ini</code> < CLI (CLI wins) for	Unified config override (intended)

ID	Sev	Area	Gap - what is missing / current state	Blocks
			shared tunables (firmware, memory). EMULATR_* env vars are ORTHOGONAL -- most are ENV-only diagnostic/trace toggles; the two that overlap a CLI flag (--no-autoload / EMULATR_NO_AUTOLOAD, --autosnapshot / EMULATR_AUTOSNAPSHOT) are OR/AND-merged, not ranked. INTENDED: a single order <code>ini < CLI < ENV</code> (ENV overrides both), by threading env reads through the same settings resolver as ini/CLI.	
G-26	DEFER	Config/INI	Default settings file is the versioned <code>EmulatrV4.ini</code>; INTENDED name is <code>EmulatR.ini</code> (release versioning handled by the directory tree / hive, not the	Filename cleanup (intended)

ID	Sev	Area	Gap - what is missing / current state	Blocks
			filename). Rename spans the loader search paths (IniLoader::defaultSearchPaths), the shipped config/EmulatrV4.ini, the tools/scripts that reference it, and six doc topics. Keep EmulatrV4.ini as a back-compat fallback in the search order during the transition.	
G-27	BLOCKER	SCSI / OS	The SCSI system disk never goes VALID under OpenVMS, so the volume is never mounted and the boot cannot proceed to a login prompt. The adapter, the drive identity, capacity and the sense/attention path are all confirmed reaching the guest correctly; what is not yet confirmed is the drive GEOMETRY binding through	Volume mount; any guest-OS use

ID	Sev	Area	Gap - what is missing / current state	Blocks
			MODE SENSE. This is the single active blocker on the guest-OS front and the successor to the earlier storm-class defects, all of which are now measured closed.	
G-28	DEFER	SCSI	No disconnect / reselection and no tagged command queuing. The adapter runs every command to completion without releasing the bus, which is self-consistent and adequate for a single initiator with one outstanding request, but it is not how the real chip behaves under a loaded multi-drive system. INTERLOCK, learned the hard way: the drive must not advertise the queuing capability until the adapter can actually honour it	Concurrency; multi-drive load

ID	Sev	Area	Gap - what is missing / current state	Blocks
			- claiming it while tag messages are silently discarded wedged the mount path for tens of thousands of operations. Advertise only when both halves land.	
G-29	GAP	SCSI / CD	The virtual SCSI CD-ROM lags its disk sibling on protocol behaviour shared by both: it has no UNIT ATTENTION condition at all (so a media change is invisible to a running guest, and swapping an image under a mounted volume goes unannounced), pending sense is not cleared by an intervening successful command, an unsupported command does not record why it failed, and MODE SELECT is absent - the same command whose absence	Bootimg or installing from CD media

ID	Sev	Area	Gap - what is missing / current state	Blocks
			previously broke the disk boot path. Enumeration and console-era use are unaffected.	
G-30	GAP	IDE	The IDE / ATAPI channel is a polled, read-only enumeration surface: no interrupt path (IRQ 14/15 are never driven), no write path (every write aborts), and no DMA. The no-DMA story is at least internally honest - the drive advertises no DMA capability, so a conforming driver stays in programmed I/O. Harmless while the system disk is SCSI; FATAL for any IDE-resident OS disk or install target, and the three pieces are interlocked and must land together. Blocked behind G-09.	IDE as a system disk (DS10)

Notes

Release-critical set (must clear for the DS10/DS20/ES40 console checkpoint): G-01, G-02, G-03, G-04, G-10, plus the G-07 reconciliation. Guest-OS-critical set (the separate, now-active front): G-27, with G-17 and G-19 behind it.

Reference blocks cleared since the previous revision: G-09 (Ali M1543C datasheet) and G-13 (SCSI datasheets plus the DEC SRM driver source). Both rows were rewritten rather than deleted because a behavioural gap survives the reference arriving - the source was the blocker, not the whole gap. No ID has been reused.

Next probe (evidence-before-scope): a compile-gated NXM-address probe at the raise site to confirm or refute G-10 before any Pchip-decode edit lands.

Unifying failure mode across G-05/G-10/G-14/G-20: an unmodeled region returning a plausible-looking value (0x00 or 0xFFFF...) the firmware branches on, so the symptom surfaces far downstream from the defect. Every CSR the console reads must be decoded and faithful; INERT-TRUTHFUL store-through is allowed only where no consumer reads a side-effect.

Second failure mode, learned on the mount path and now standing doctrine (see G-28): a CAPABILITY CLAIM is not a label - it is an assertion the other side acts on. A wrong geometry value is inert until something reads it, but a wrong capability bit changes what the guest DOES. A device datasheet's feature list describes the silicon, not the model; advertise a capability only when the model can honour it end to end.

1.10 EmulatR Version Numbering and Release Stages

EmulatR uses a single version identity across the console badge, the documentation, and the distribution kit. The format follows the Digital (DEC) convention familiar from SRM firmware version strings:

`<stage><major>.<variant>-<build>`

Example: X1.5-23
Fields

Field	Example	Meaning
Stage	X	Release stage letter (see below)
Major	1	Release series
Variant	5	EmulatR application generation (AppUniV5)

Build	23	Incrementing build number, set manually per release
-------	----	---

Release stage letters (DEC convention)

Letter	Meaning
X	Experimental (pre-release development build)
T	Field test
V	Released

This is the same grammar a real Alpha console user knows from firmware strings such as V7.3-1, so a version shown by EmulatR reads exactly like one shown by genuine SRM firmware.

Where the version appears

The version identity is emitted on the console banner when a terminal client attaches, above the emulated DEC firmware banner. That banner is EmulatR's own voice, not emulated firmware output:

```
ASA EmulatR -- Alpha AXP (EV6 / 21264) Emulator
Alpha Emulator Console X1.5-12
(c) 2026 Timothy Peer / eNVy Systems, Inc.
```

The same identity is carried by the documentation set and by the distribution kit, so a support question can always be tied to one exact build.

How the version is set (maintainers)

There is exactly one source of truth. Both the numbers and the stage letter are declared in the project build file, and every other place the version appears is generated from it - the version is never typed in twice:

```
CMakeLists.txt
project(Emulatr VERSION 1.5.12 ...)  <- major . variant . build
set(EMULATR_VERSION_STAGE "X")      <- stage letter X | T | V
```

At build time those values are rendered into a generated header, which the console banner consumes; a build-time check confirms the badge and the header agree, so the two cannot drift apart. The build also mirrors the three numbers into this help project, so the manual and the emulator always report the same build.

To raise the build number, edit the one version line and rebuild. To change release stage, flip the single stage letter: **x** during development, **⌘** for a field-test kit, **v** at release.

2 2. Installation

This chapter covers the two ways to get EmulatR onto a machine -- the Windows installer (the route for beta testers) and building from source -- along with supplying an SRM firmware image and the first launch. When upgrading an installed copy to a new build, uninstall the existing installation via Add/Remove Programs first; see Installing with the Windows Installer. Build specifics are marked with [TODO] where they still need to be confirmed against the current project files.

In this section

- *Installing with the Windows Installer -- the MSI route, and the uninstall-first upgrade procedure.*
- *Required Tools and Frameworks -- the compiler, build system, and libraries.*
- *Building from Source -- configuring and building the binary.*
- *Firmware Requirements -- supplying the SRM console image.*
- *First Startup -- the first launch through to the prompt.*

See Also.

[Installing with the Windows Installer](#)  42

[Required Tools and Frameworks](#)  44

[Building from Source](#)  45

[Firmware Requirements](#)  44

[First Startup](#)  48

[Quick Start](#)  50

2.1 Installing with the Windows Installer

Beta and release builds of EmulatR are distributed as a Windows installer (.msi). The installer places a complete, ready-to-run installation under:

```
C:\Program Files\eNVy Systems, Inc\ASA-EmulatR
```

No source tree, compiler, or Qt installation is required -- the installer carries everything the applications need. Building from source remains available for developers and is described in [Building from Source](#) .

Upgrading or reinstalling: uninstall first

Before installing a new build over an existing installation, uninstall the existing one first: open Windows Settings, then Apps, then Installed apps (on older Windows: Control Panel, Add/Remove Programs), locate ASA EmulatR, and choose Uninstall. Then run the new installer.

This is the recommended upgrade path for every new build. Installing on top of an existing installation can leave files from the previous version behind, and Windows loads DLLs from the application directory before anywhere else -- a stale library alongside a new executable can cause startup failures that look like a broken build. A clean uninstall/install guarantees the installation directory contains exactly one consistent version.

Uninstalling does not touch your data. Named-system definitions, run directories, disk images, and logs all live outside the installation directory and are preserved across uninstall/reinstall cycles. The one exception to check: if you copied SRM firmware images into the installation's `firmware\` folder, keep a copy elsewhere before uninstalling.

What the installer delivers

Component	Purpose
<code>Emulatr.exe</code>	The emulator itself, with its <code>config\</code> data and the platform manifests.
<code>EmulatrLaunch.exe</code>	The launcher -- the recommended front door. It creates and manages named systems, run directories, and disk images, and starts the emulator for you.
<code>editors\platedit_qt.exe</code>	The Platform Editor for authoring and validating platform manifests.
<code>firmware\</code>	Intentionally empty. SRM console images are licensed material and are not distributed; you supply your own. See Firmware Requirements ⁴⁴ .
<code>tools\</code>	A small set of helper utilities (disk image creation, console driver, disassembler). The Python scripts require a Python 3 installation.

After installing

1. Copy your SRM firmware image(s) into the installation's `firmware\` folder.
2. Start `EmulatrLaunch.exe` and create a named system -- the launcher builds the run directory, disk images, and configuration for you.
3. Boot, and connect to the console. See [First Startup](#) ⁴⁸.

See Also.[Firmware Requirements](#) → 44[First Startup](#) → 48[Building from Source](#) → 45[Quick Start](#) → 50

2.2 Required Tools & Frameworks

Tooling and libraries used by EmulatR. Versions below are those the project currently builds against.

- *Compiler: MSVC (Visual Studio 2022) with C++20 (required, extensions off). Qt Creator is the usual IDE / CMake environment.*
- *Build system: CMake 3.16 or newer (3.20 for the schedLib subproject); Ninja generator.*
- *Qt 6 (developed against 6.10.2, msvc2022_64): Core / Network / Concurrent only -- no GUI / Widgets; the emulator core is deliberately Qt-free.*
- *spdlog: logging infrastructure (dedicated EMULATR_CONSOLE_TRACE channel); header-only, pulled from vcpkg (SPDLOG_HEADER_ONLY).*
- *A terminal client for the console: PuTTY / plink (auto-launched), or any RAW-TCP client.*
- *Optional: Ghidra, for SRM firmware reverse-engineering work.*

See Also.[Building from Source](#) → 45[Dependencies](#) → 48[Ghidra](#) → 245

2.3 Firmware Requirements

EmulatR requires an SRM console firmware image to execute. The DS10 reference image is an SRM V7.3 build (ds10_v7_3.exe).

The image is selected in precedence order:

--firmware <path> on the command line (overrides the ini for a single run); or

the [ROM]/firmwareImage ini key (default: ds10_v7_3.exe).

Path gotcha: the ini value is the bare file name (no `firmware/` prefix) and resolves against the current working directory. Launch from the EmulatR root, or pass an explicit `--firmware firmware/ds10_v7_3.exe`.

Optional integrity check: set `[ROM]/firmwareSha256` to verify the image at load. When empty, the integrity check is skipped.

[TODO: document firmware provenance, licensing, and the set of supported SRM versions.]

See Also.

[First Startup](#) ⁴⁸

[Loading Firmware](#) ⁵¹

[Building from Source](#) ⁴⁵

2.4 Building from Source

EmulatR is a CMake project built with MSVC (C++20). The build tree produces `Emulatr.exe` under `out/build/<config>/`, with the two configurations in active use being `release` and `relwithdebinfo` (the latter is the day-to-day debug / profiling build).

Configure the MSVC environment in bash:

```
base $ cd /d/EmulatR/EmulatRAppUniV4/Emulatr/out/build/release
bash $ cd /d/EmulatR/EmulatRAppUniV4/Emulatr
```

```
cd /d/EmulatR/EmulatRAppUniV4/Emulatr/out/build/release
../../tools/run_fw.sh ds20 cold          # or ./run_fw.sh ds20 cold if deployed
# look for: platform latched: model=DS20 manifest=DS20 usedDefault=0 ocp40=Y ocp42=Y

source tools/vsenv.sh
[vsenv] loading MSVC x64 env from: C:\Program Files\Microsoft Visual Studio\2022\Commu
[vsenv] cmake : /c/Program Files/Microsoft Visual Studio/2022/Community/Common7/IDE/C
[vsenv] ninja : /c/Users/tim/AppData/Local/Programs/Python/Python313/Scripts/ninja
[vsenv] cl     : present (MSVC x64)
```

```
cmake --build out/build/release -j          # build everything (Ninja, parallel)
```

```
cmake --build out/build/release -j --target Emulatr
cmake --build out/build/release -j --target Emulatr_tests
```

To build just the main target or the tests:

```
# resulting binary
./out/build/relwithdebinfo/Emulatr.exe
```

Generator: Ninja, supplied by the Visual Studio 2022 CMake and surfaced by `tools/vsenv.sh`. Dependencies: Qt and spdlog -- provision them before configuring; the CMake configure step resolves them. The canonical build is: `source tools/vsenv.sh`, then `./tools/build_emulatr.sh relwithdebinfo` (or `cmake --build out/build/relwithdebinfo -j`), then run `Emulatr_tests.exe`.

Note: the diagnostic build flag `EMULATR_MEMDIAG` is a CMake option, not a runtime variable, and must stay OFF for normal use -- its per-instruction logging imposes roughly a 900x slowdown.

Advanced / diagnostic build recipes

The recipes below flip sticky CMake cache options (bring-up probes, IRQ diagnostics) or run ES40 hook / VPTB trace builds. They are for debugging only -- keep these OFF for a release build.

```
source ./tools/vsenv.sh
cmake -DEMULATR_IRQDIAG=ON .
grep -i irqdiag CMakeCache.txt          # want: EMULATR_IRQDIAG:BOOL=ON
./tools/build_emulatr.sh relwithdebinfo
```

```
cd /d/EmulatR/EmulatRAppUniv4/Emulatr && \
  TARGET=Emulatr_tests ./tools/build_emulatr.sh relwithdebinfo && \
  ./out/build/relwithdebinfo/Emulatr_tests.exe
```

```
cd /d/EmulatR/EmulatRAppUniv4/Emulatr && source tools/vsenv.sh && cmake -DEMULATR_BRINGUP_PROBES=ON .
```

```
cd /d/EmulatR/EmulatRAppUniv4/Emulatr && ./tools/build_emulatr.sh relwithdebinfo
```

```
cd /d/EmulatR/EmulatRAppUniv4/Emulatr/out/build/relwithdebinfo && \
EMULATR_HOOKA_VA=0x600 \
EMULATR_2D_NOOP=1 EMULATR_SPINSKIP=1 \
./Emulatr.exe --firmware firmware/es40_v7_3.exe --mem 4294967296 \
--no-autoload --max-cycles 0x50000000 2> es40_hooka.log
```

the format for cmake on windows bash

```
cmake -DEMULATR_BRINGUP_PROBES=ON -DEMULATR_IRQDIAG=OFF .      # flip the sticky cache
cmake --build . -j          # or your build_emulatr.sh
cd /d/EmulatR/EmulatRAppUniv4/Emulatr/out/build/relwithdebinfo
EMULATR_VPTB_DIAG=1 EMULATR_HOOKA_CYC_FLOOR=280000000 \
./Emulatr.exe --firmware firmware/es40_v7_3.exe --mem 4294967296 \
--no-autoload --max-cycles 0x50000000 2> es40_vptb2.log
grep -a "VPTB-DIAG\|ACVPROBE HOOKA" es40_vptb2.log | head -60
```

```
cd /d/EmulatR/EmulatRAppUniv4/Emulatr/
cmake -DEMULATR_BRINGUP_PROBES=ON -DEMULATR_IRQDIAG=OFF .
grep EMULATR_BRINGUP_PROBES CMakeCache.txt          # expect :BOOL=ON
cmake --build . -j          # or your build_emulatr.sh
strings Emulatr.exe | grep -c "ACVPROBE HOOKB"      # expect > 0 (probe is in the bin)
cd /d/EmulatR/EmulatRAppUniv4/Emulatr/out/build/relwithdebinfo
./Emulatr.exe --firmware firmware/es40_v7_3.exe --mem 4294967296 \
--no-autoload --max-cycles 0x50000000 2> es40_hookb.log
grep -a "ACVPROBE HOOKB" es40_hookb.log
```

1. probes OFF for release (they ballooned the log to 5.2 GB)

```
cd /d/EmulatR/EmulatRAppUniv4/Emulatr && cmake -DEMULATR_BRINGUP_PROBES=OFF
.&&\
```

2. tests green

TARGET=Emulatr_tests ./tools/build_emulatr.sh relwithdebinfo &&

./out/build/relwithdebinfo/Emulatr_tests.exe

3. boot gate: DS10/DS20 still reach >>> (align ini model per firmware); ES40 = "SCB halt cleared, no regression"

See Also.

[Required Tools & Frameworks](#) ⁴⁴

[Firmware Requirements](#) ⁴⁴

[First Startup](#) ⁴⁸

[./tools/build_emulatr_diag.bat](#) ⁴⁷

2.4.1 ./tools/build_emulatr_diag.bat

```
@echo off
REM =====
REM build_emulatr_diag.bat -- MSVC x64 build of EmulatR with bring-up probes ON
REM (so the RetireProfiler dumps its histogram at run end).
REM
REM Run from a VS Developer prompt, or from Git Bash via:
REM cmd //c "D:\EmulatR\EmulatRAppUniV4\Emulatr\tools\build_emulatr_diag.bat"
REM
REM For a clean/perf build, set PROBES=OFF below (or use a normal build).
REM =====
setlocal
set "VCVARS=C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Auxiliary\Build\
set "BUILD=D:\EmulatR\EmulatRAppUniV4\Emulatr\out\build\release"
set "PROBES=OFF"

if not exist "%VCVARS%" ( echo FATAL: vcvars64.bat not found at "%VCVARS%" & exit /b 1 )
call "%VCVARS%"
if errorlevel 1 ( echo FATAL: vcvars64.bat failed & exit /b 1 )

cd /d "%BUILD%" || ( echo FATAL: build dir not found: %BUILD% & exit /b 1 )

echo === reconfigure (EMULATR_BRINGUP_PROBES=%PROBES%) ===
cmake -DEMULATR_BRINGUP_PROBES=%PROBES% . || exit /b 1

echo === build ===
cmake --build . --config Release || exit /b 1

echo === done -- Emulatr.exe rebuilt with profiler dump enabled ===
endlocal
```

2.5 Dependencies

What EmulatR is built with and what it relies on at build and run time. Versions are those the project currently builds against.

Component	Role	Notes
C++20 / MSVC	implementation language and compiler	Visual Studio 2022; C++20 required (extensions off)
CMake	build system	3.16 minimum (3.20 for schedLib); Ninja generator
Visual Studio 2022	host toolchain / IDE	Windows host
Qt 6	console / UI periphery only	6.10.2 (Core / Network / Concurrent); the emulator core is deliberately Qt-free
spdlog	logging infrastructure	header-only via vcpkg; level macros and channels
Berkeley SoftFloat	IEEE / VAX floating-point backend	selectable vs a native backend
Python 3	tooling (mkdisk.py, codegen)	build / utility scripts
PuTTY / plink	terminal client for the console	any RAW-TCP client also works
Ghidra	firmware reverse-engineering	optional, developer-side

EmulatR also depends on reference inputs it does not reimplement: the **Alpha Architecture Reference Manual** and the original **DEC SRM console / PALcode sources** (the authoritative specification and the firmware image it loads), and the `AXPBox` emulator used as a non-authoritative differential cross-check.

See Also.

[Required Tools & Frameworks](#) ⁴⁴

[Attributions & Credits](#) ¹⁶⁰

2.6 First Startup

A first run, launched from the EmulatR root so relative paths resolve correctly:

```
cd /d/EmulatR/EmulatRAppUniv4/EmulatR
./out/build/relwithdebinfo/EmulatR.exe --firmware firmware/ds10_v7_3.exe
```

On launch the TCP console server comes up (default port 10023) and, unless `EMULATR_NO_PUTTY` is set, a PuTTY session is auto-launched against it. The firmware then runs through its power-up sequence to the SRM >>> prompt. For the step-by-step walkthrough, see Quick Start; to stop cleanly, see Clean Shutdown.

See Also.[Quick Start](#) → 50[Launching EmulatR](#) → 50[Clean Shutdown](#) → 61[Platform Execution Mode: ISP \(Engineering-Virtual\) vs Silicon \(REAL_HW\)](#) → 51

3. Quick Start

The fastest path from a built binary to an interactive SRM prompt, and back down cleanly. Each step has its own topic below; follow them in order.

In this section

- *Launching EmulatR -- start the emulator from the Emulatr root.*
- *Connecting to the Console -- attach a terminal to the TCP console.*
- *Loading Firmware -- point the run at the SRM image.*
- *Reaching the SRM Prompt -- what the power-up sequence shows.*
- *Clean Shutdown -- stop the emulator so NVRAM is flushed.*

See Also.

[Launching EmulatR](#) ⁵⁰

[Connecting to the Console](#) ⁶⁰

[Loading Firmware](#) ⁵¹

[Reaching the SRM Prompt](#) ⁶⁰

[Clean Shutdown](#) ⁶¹

3.1 Launching EmulatR

Launch from the Emulatr root. A genuine cold boot (no snapshot restore), quiet, is:

```
cd /d/EmulatR/EmulatRAppUniv4/Emulatr
EMULATR_NO_AUTOLOAD=1 ./out/build/relwithdebinfo/Emulatr.exe \
  --firmware firmware/dsl0_v7_3.exe --autosnapshot off 2> run.log
```

Omit `EMULATR_NO_AUTOLOAD` to restore the newest snapshot instead of cold-booting. Diagnostics are written to the redirected stderr (`run.log`).

Common launch modes

Autoload to the prompt in seconds -- drop `--no-autoload` so the newest >>> snapshot is restored:

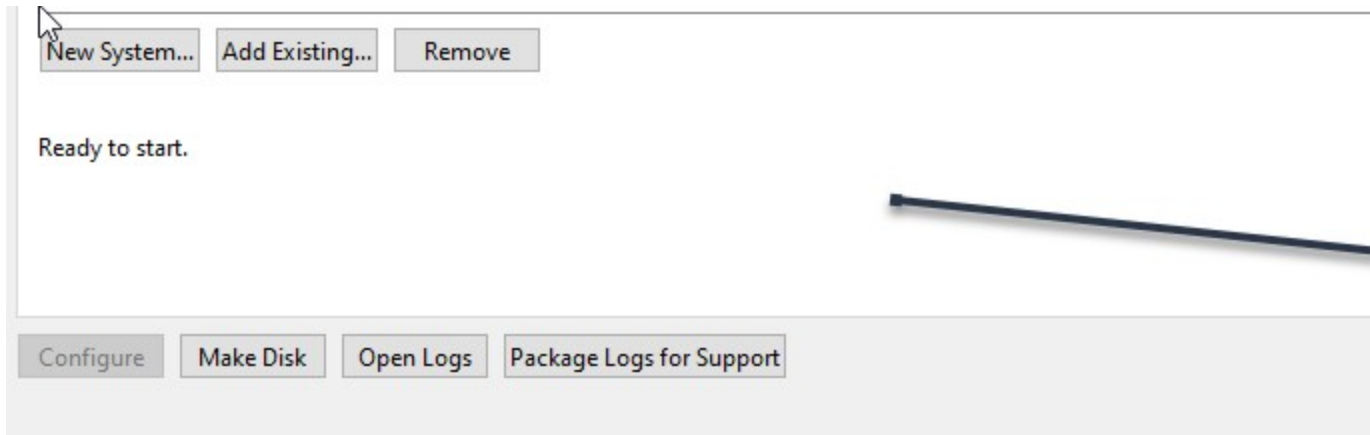
```
./out/build/relwithdebinfo/Emulatr.exe --firmware firmware/dsl0_v7_3.exe --autosnapsh
```

Bounded headless cold boot -- determinism / step-correctness over a few minutes (covers decompression and early init):

```
EMULATR_NO_PUTTY=1 ./out/build/relwithdebinfo/Emulatr.exe \
  --firmware firmware/dsl0_v7_3.exe --no-autoload --autosnapshot off \
  --max-cycles 2000000000 2> run_cold.log
```

See the Environment Variables reference for the full set of launch gates.

EmulatRLauncher is Supported ([Configuring a System with the Launcher](#)⁶³)



See Also.

[Connecting to the Console](#)⁶⁰

[Environment Variables](#)²¹⁶

[First Startup](#)⁴⁸

[Command Line Options \(EmulatR\)](#)⁷⁷

3.2 Loading Firmware

Pass the SRM image explicitly so the path resolves regardless of the ini default:

```
--firmware firmware/ds10_v7_3.exe
```

Relying on the ini fallback resolves the bare name `ds10_v7_3.exe` against the current directory, so an explicit `--firmware` is the reliable form. See Firmware Requirements for selection precedence and the optional SHA-256 integrity check.

See Also.

[Firmware Requirements](#)⁴⁴

[Reaching the SRM Prompt](#)⁶⁰

3.3 Running from the Shell

EmulatR is a console application driven from a command line. The standard execution shell is `bash` -- the project's own launch and trace scripts are `bash`, and the examples here

assume it (on Windows, a bash shell such as Git Bash / MSYS, where the disk appears as `/d/EmulatR`). Everything below is run from the run directory.

The run directory

The run directory is the folder that holds the built executable. CMake's post-build step lays it out so the program and its inputs sit together:

```
<run>/ Emulatr.exe the emulator
firmware/ SRM images (e.g. ds10_v7_3.exe)
tools/ deployed bash helpers (trace_halt.sh, ...)
EmulatrV4.ini settings (memorySize, paths, ...)
ds10_flash.rom persisted NVRAM / flash
snapshots/ saved machine states (.axpsnap)
```

Run EmulatR through a model launch script in `tools/` -- `run_<model>.sh` (plain boot), `run_<model>_trace.sh` (full trace), or `run_<model>_gate.sh` (gated retire window). Each self-locates the run directory from its own path, `cd`'s there, sets the firmware, model, memory and console for you, and writes output under the run-dir `logs/` and `traces/`. Do NOT launch the executable directly -- the scripts are the supported entry point; pass any extra emulator flags after a bare `--` (for example `./tools/run_ds10.sh -- --max-cycles 0x100000`).

Basic invocation

```
cd out/build/<config>
./tools/run_ds10.sh
```

With no `--firmware`, the image falls back to `[Rom] firmwareImage` in the ini; with no `--mem`, memory falls back to `[System] memorySize` in the ini (1024 MB by default), and only then to the built-in 64 MiB default. The command line always wins over the ini. See Command Line Options for the full configuration-precedence model (defaults `< ini < CLI`) and how the `EMULATR_*` env vars fit alongside it.

Command-line switches

Switch	Meaning
<code>--firmware <path></code>	Firmware image to load (resolved against the run directory).

Switch	Meaning
<code>--firmware-format auto raw srm</code>	Override image-format detection (default auto -- SRM if the <code>.exe</code> signature scan succeeds, else raw).
<code>--mem <bytes></code>	Guest memory size in bytes (e.g. 1073741824 for 1 GiB).
<code>--load-pa <addr> / --start-pa <addr></code>	Physical load address and execution start address (hex or decimal; start-pa low bit sets PAL mode).
<code>--max-cycles <N></code>	Cap the run loop at N cycles (default unbounded).
<code>--pal-mode</code>	Force PAL mode at reset.
<code>--no-autoload</code>	Suppress autoload of the newest snapshot -- forces a genuine cold boot.

Switch	Meaning
<code>--autosnapshot on off</code>	Toggle periodic <code>auto_*.axp</code> <code>snap</code> saves (off avoids the disk cost on a long cold boot).
<code>--snapshot-on-pc <pc[,pc...]></code>	Capture a one-shot snapshot when the CPU retires at the given PC(s).
<code>--dump-disasm <pa>:<count></code>	Disassemble count instructions at a PA after load, before the run starts.
<code>--trace <dec,machine></code>	Install the trace sink writing the decode and machine listing logs.
<code>--log-disable / --log-only / --log-verbose / --log-file</code>	Tune the log subsystem (see Logging below).
<code>--help</code>	Print the usage text and exit.

Logging

The simplest log is the console transcript, captured with `tee` so it is both shown and saved:

```
./tools/run_ds10.sh 2>&1 | tee logs/run_$(date +%Y%m%d_%H%M%S).console.log
```

The log subsystem is filtered per channel. Show only certain subsystems, silence noisy ones, or send a channel to its own file:

```
./tools/run_ds10.sh -- --log-only ide,iic
./tools/run_ds10.sh -- --log-disable chipset
./tools/run_ds10.sh -- --log-file ide=logs/ide.log
```

Retire / instruction traces are written to the directory named by `EMULATR_TRACE_DIR`; `EMULATR_TRACE_WINDOW=1` arms a windowed sink so a cold boot does not emit a multi-gigabyte stream. A typical traced cold boot:

```
export EMULATR_TRACE_DIR=/d/EmulatR/traces; mkdir -p "$EMULATR_TRACE_DIR"
export EMULATR_TRACE_WINDOW=1
./tools/run_ds10_trace.sh 2>&1 | tee logs/cold_$(date +%H%M%S).log
```

The full list of `EMULATR_*` runtime variables (console port, flash path, idle warp, trace controls) is on the Environment Variables page.

Running as a forked / background process

Because the console is a RAW TCP server (reached by a terminal client over `EMULATR_CONSOLE_PORT`), the emulator does not need the controlling terminal and can run detached. Launch it in the background, capture its output to a file, and note the process id:

```
HEADLESS=1 ./tools/run_ds10.sh > run.log 2>&1 &
echo "emulatR pid = $!"
```

To survive the shell closing, use `nohup` and `disown`, then connect from a separate terminal client:

```
nohup env HEADLESS=1 ./tools/run_ds10.sh > run.log 2>&1 & disown
# then, from anywhere: putty -raw 127.0.0.1 -P 10023 (or any RAW-TCP client)
```

Stop a detached instance cleanly by ending its console session or signalling the captured pid; avoid a hard kill during a flash write, which can corrupt `ds10_flash.rom`. To run two instances from one directory, give each its own flash and console port so they do not collide:

```
PORT=10024 EMULATR_FLASH_ROM=flash_b.rom HEADLESS=1 \
./tools/run_ds10.sh > run_b.log 2>&1 &
```

See Also.

[Command Line Options \(EmulatR\)](#) ⁷⁷

[Environment Variables \(EMULATR_*\)](#) ²¹⁶

[Executing Two Instances](#) ⁷⁵

[Configuring](#) ⁶⁹

3.4 Running from a Run Directory

The run directory is the folder that holds the built executable -- `out/build/<config>/`, where `<config>` is `relwithdebinfo`, `debug`, or `release`. It is the runtime root: firmware, config, logs, traces, snapshots, the platform manifests, and the deployed helper scripts all live there. The project's `tools/` folder is the source of truth for those scripts; every build copies them into the run directory, and you run from that copy.

The model

`<project>/tools/` is the committed repository of run and diagnostic shell scripts, covering the execution permutations (models `ds10` / `ds20` / `ds25` / `es40` / `es45`, across configs and trace modes). On every build, CMake copies `tools/*.sh` and `tools/*.py` into `out/build/<config>/tools/`. The copy is GLOB-driven with `CONFIGURE_DEPENDS`, so a newly added script deploys on the next build with no hand-editing. You then run from the run directory's copy:

```
out/build/<config>/tools/<script>.sh
```

Why it runs from anywhere

Every runtime script sources `tools/_emulatr_runenv.sh`, which locates the run directory from the script's own path (via `BASH_SOURCE`) -- never a hardcoded or source-tree path. It then `cds` into the run directory and pre-creates `logs/` `traces/` `snapshots/`. Because the emulator also writes relative to its current directory, all output lands under the run directory automatically. The current working directory does not matter -- these are equivalent:

```
cd out/build/relwithdebinfo && ./tools/run_ds10_trace.sh
/any/where $ /full/path/out/build/relwithdebinfo/tools/run_ds10_trace.sh
```

Distributing a build (download, unzip, run)

A distributed artifact is the run directory itself: `zip out/build/<config>/` (Emulatr.exe, the Qt DLLs, firmware/, config/, the platform manifests, and tools/). The end user unzips it anywhere, opens `tools/`, and runs a script -- for example:

```
./tools/run_ds10_trace.sh
```

No source tree is required. The scripts use only run-dir-relative paths; firmware and manifests are the copies already shipped in the run directory.

Where output lands

Relative to the run directory (`out/build/<config>/`):

Directory	Contents
<code>logs/</code>	Run and console logs, named <code>purpose_YYYYMMDD_HHMMSS.log</code> .

Directory	Contents
traces/	Retire / dec / machine listings and .trc traces.
snapshots/	Saved machine states (.axpsnap checkpoints).
firmware/	The v7_3 SRM images the run boots.
config/	EmulatrV4.ini (model set per run, restored on exit).

Environment knobs

Variable	Effect
RUN_DIR=<dir>	Force the run directory, overriding self-location.
CONFIG=relwithdebinfo debug release	Steers only the developer fallback used when a script is run from a source checkout instead of a shipped run directory; ignored once a shipped run directory is found.

Adding a new script

1. Drop the .sh in <project>/tools/.
2. Open it so it self-locates the run directory:

```
set -euo pipefail
SELF_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$SELF_DIR/_emulatr_runenv.sh" # sets RUN_DIR, BIN, HOST; cd's in
```

3. Use only run-dir-relative paths for output (logs/, traces/, ...).
4. Rebuild -- CMake auto-deploys it to out/build/<config>/tools/.

See Also.

[Running from the Shell](#) ⁵¹

[Building from Source](#) ⁴⁵

[Command Line Options](#) ⁷⁷

[Environment Variables \(EMULATR_*\)](#) ²¹⁶

3.4.1 Run-Directory Contents

An inventory of the run directory, out/build/<config>/ (for the beta the config is relwithdebinfo). This directory is the runtime root: Emulatr.exe and every relative path it

resolves `-- config/`, `firmware/`, the platform manifests, `tools/`, and the `trace/log/snapshot` output dirs -- live here. The CMake `POST_BUILD` step creates the subdirectory layout and stages the shipped files on every build.

Beta zip = run directory minus firmware. The firmware images are licensed vendor blobs; the tester drops their own `<model>_v7_3.exe` into `./firmware/`. The developer-only artifacts listed at the bottom are stripped before packaging.

Core runtime files (shipped)

File	What it is
<code>Emulatr.exe</code>	The emulator executable. The run directory is its working directory; all relative paths resolve from here.
<code>run_fw.sh</code>	The firmware launcher, staged to the run-dir root (beside the exe) so it can anchor to its own directory and launch the binary directly.
<code>Qt6Core.dll</code>	Qt 6 core runtime, deployed by <code>windeployqt</code> . Emulatr is a console app using QtCore + QtNetwork (no GUI module).
<code>Qt6Network.dll</code>	Qt 6 network runtime (the TCP SRM console listener).
<code>icuuc.dll</code>	ICU Unicode library that Qt6Core depends on; deployed alongside the Qt DLLs.
<code><model>_v7_3_platform.json</code>	Platform device manifests (<code>ds10</code> / <code>ds20</code> / <code>ds25</code> / <code>es40</code> / <code>es45</code>). Host-agnostic description of the guest device tree; Machine resolves the one matching the firmware stem beside the exe. Edit with the Platform Editor.

Subdirectories

Directory	What it holds
<code>config/</code>	Runtime configuration: <code>Emulatr.ini</code> (the operator settings), <code>disk_types.json</code> , and <code>dec_disk_media_types.tsv</code> (storage reference data). Copied from source <code>config/</code> by CMake.
<code>firmware/</code>	Firmware blobs the emulator loads at runtime (<code><model>_v7_3.exe</code>). NOT shipped in the beta zip -- the tester supplies their own image here.
<code>editors/</code>	The Platform Editor: <code>platedit_qt[.exe]</code> (Qt GUI; Qt DLLs deployed beside it on

Directory	What it holds
	Windows), <code>platedit_tui</code> , its <code>schema/</code> + <code>catalog/</code> <code>data</code> , and the <code>webui/</code> prototype. See Platform Editor.
<code>tools/</code>	The deployed tool scripts (see Tools Script Reference).
<code>documentation/</code>	The rendered Help & Manual user guide, <code>EmulatRGuide.pdf</code> , staged by CMake <code>POST_BUILD</code> so it ships with the build and the <code>redist</code> package.
<code>logs/</code>	<code>spdlog</code> output, separate from the CPU trace. Created at build; populated at runtime.
<code>traces/</code>	Retire / breakpoint / profiler <code>.trc</code> captures and the per-run trace logs, written when a trace is armed.
<code>trace/</code>	Legacy per-run <code>cpu_trace.log</code> location (created for backward compatibility).
<code>snapshots/</code>	Saved-state files (<code>.axpsnap</code>): auto snapshots and the <code>predig_*</code> fast-forward anchors.
<code>networkinformation/</code>	Qt network-information plugin backends, deployed by <code>windeployqt</code> .
<code>tls/</code>	Qt TLS backend plugins, deployed by <code>windeployqt</code> .

Generated at runtime (not part of the shipped set)

File	What it is
<code><model>_diag_flash.rom</code>	Per-run diagnostic flash / NVRAM image. A fresh one gives a clean cold boot with the OEM string cleared; regenerated as needed.
<code>flash_backup_<ts>.rom</code>	Timestamped backup of flash/NVRAM taken before a destructive flash operation.
<code>putty_console_p<port>_<ts>.log</code>	PuTTY console session capture from an attached terminal.
<code>run_*_<ts>.log</code> / <code>fw_*_<ts>.out</code>	Timestamped console/stderr tee logs written by the run scripts.

Developer-only artifacts (strip before packaging)

File	What it is
<code>Emulatr.pdb</code>	MSVC debug symbols for the <code>RelWithDebInfo</code> build. Useful for local crash analysis; large, and not required to run.

File	What it is
*.svlPV_functionAndLineHooks2	Software Verify (Performance Validator) instrumentation sidecars. Profiling artifacts, not runtime files.
dbgHelp.dll, symsrv.dll, BorDebug.dll	Debug/symbol-server helper DLLs used by the profiler and crash tooling; not needed for a plain run.

See Also
[Tools Script Reference](#) → 254
[Platform Editor](#) → 259
[Running from a Run Directory](#) → 56

3.5 Connecting to the Console

The emulator exposes a RAW TCP console (default `localhost:10023`). Unless `EMULATR_NO_PUTTY` is set, PuTTY is launched for you. To attach a client manually:

```
plink -raw localhost -P 10023
```

Full client coverage (PuTTY, plink, Tera Term, MobaXterm, netcat, telnet) is in the Console Connectivity chapter.

See Also.
[Loading Firmware](#) → 51
[TCP Console Overview](#) → 80
[Terminal Clients](#) → 81

3.6 Reaching the SRM Prompt

After launch the firmware proceeds through power-up to the console. The visible sequence is:

- *Banner and memory sizing (e.g., "1024 Meg of system memory").*
- *PCI / ISA probe; GCT / FRU initialization at 3f32000.*
- *Option-firmware scan of the CD / floppy devices.*
- *The Loadable Firmware Update (LFU) utility -- the UPD> prompt. From here, exit returns to the console, or update srm refreshes firmware.*
- *System / disk testing, then the interactive >>> prompt.*

A full annotated transcript is in Example - Performance. By default the console halts at >>> (auto_action = HALT).

See Also.

[Clean Shutdown](#) ^{↳ 61}

[SRM Console](#) ^{↳ 82}

[Firmware Update Utility \(LFU\)](#) ^{↳ 109}

3.7 Clean Shutdown

There are two graceful paths and one hard kill.

1. Stop sentinel (flushes NVRAM via ~Machine)

```
touch /d/EmulatR/EmulatRAppUniV4/Emulatr/EMULATR_STOP
```

```
bash: EmulatR(run directory)-- touch EMULATR_STOP
```

The run loop polls the sentinel even mid-stall, exits within a moment, and writes the flash cleanly. The resolved sentinel path is logged at startup (grep graceful-stop run.log).

2. CALL_PAL HALT at the prompt

Halting at >>> exits through ~Machine with a clean flush -- no sentinel touch needed.

3. Hard kill (instant, no flush)

```
taskkill //F //IM Emulatr.exe
taskkill //F //IM PuTTY.exe
```

See Also.

[Launching EmulatR](#) ^{↳ 50}

[Kill the EmulatR \(appendix\)](#) ^{↳ 234}

4 4. Operating EmulatR

Day-to-day tasks once EmulatR is built and runs: configuring the machine, attaching a console, driving the SRM console, managing storage and snapshots, updating firmware, and turning on logging and diagnostics. Each task has its own topic.

In this section

- *Configuring & Running -- the platform manifest and the ways to launch a run.*
- *Console Connectivity -- the TCP console and the terminal clients that attach to it.*
- *SRM Console -- the commands available at the prompt.*
- *Storage -- disks, CD-ROM, and the storage devices.*
- *Snapshots -- saving and restoring machine state.*
- *Firmware Update Utility (LFU) -- updating the SRM image in flash.*
- *Guest Boot -- booting an installed guest.*
- *Logging & Diagnostics -- the trace and diagnostic facilities.*

See Also.

[Configuring & Running](#) ^{↳ 62}

[Console Connectivity](#) ^{↳ 79}

[SRM Console](#) ^{↳ 82}

[Logging & Diagnostics](#) ^{↳ 112}

4.1 4.1 Configuring & Running

How the emulated machine is configured by its platform manifest, and the ways to launch a run -- a quiet cold boot, a traced boot, a long unattended run, or two instances side by side.

In this section

- *Configuring -- the per-model platform manifest and its key settings.*
- *Executing -- run scenarios beyond the basic launch: traced boots, unattended runs, performance harvesting.*
- *Executing Two Instances -- running two instances safely from one directory.*
- *Example - Performance -- an annotated boot transcript.*

See Also.

[Configuring](#) ^{↳ 69}

[Executing](#) ▷ 74[Launching EmulatR](#) ▷ 50

4.1.1 Configuring a System with the Launcher

This topic walks through configuring an emulated machine from a fresh installation to a booted guest. Work through the steps in order; each one builds on the last.

Terms you need first

The product is **EmulatR**. The **launcher** (`EmulatrLaunch.exe`) and the **Platform Editor** (`editors\platedit_qt.exe`) are subsystems of it.

The **installation** lives at `C:\Program Files\eNVy Systems, Inc\ASA-EmulatR` and is read-only. A kit upgrade replaces it wholesale, so nothing you customize is ever stored there.

A **named system** is the launcher's unit of configuration: one platform (bound at creation) plus one **profile directory**, also called the run directory. The profile directory holds everything that system owns -- `Emulatr.ini`, the system's own copy of the platform manifest, `firmware\`, `disks\`, `logs\` and `traces\`.

The launcher's catalog of systems lives in the per-user registry hive

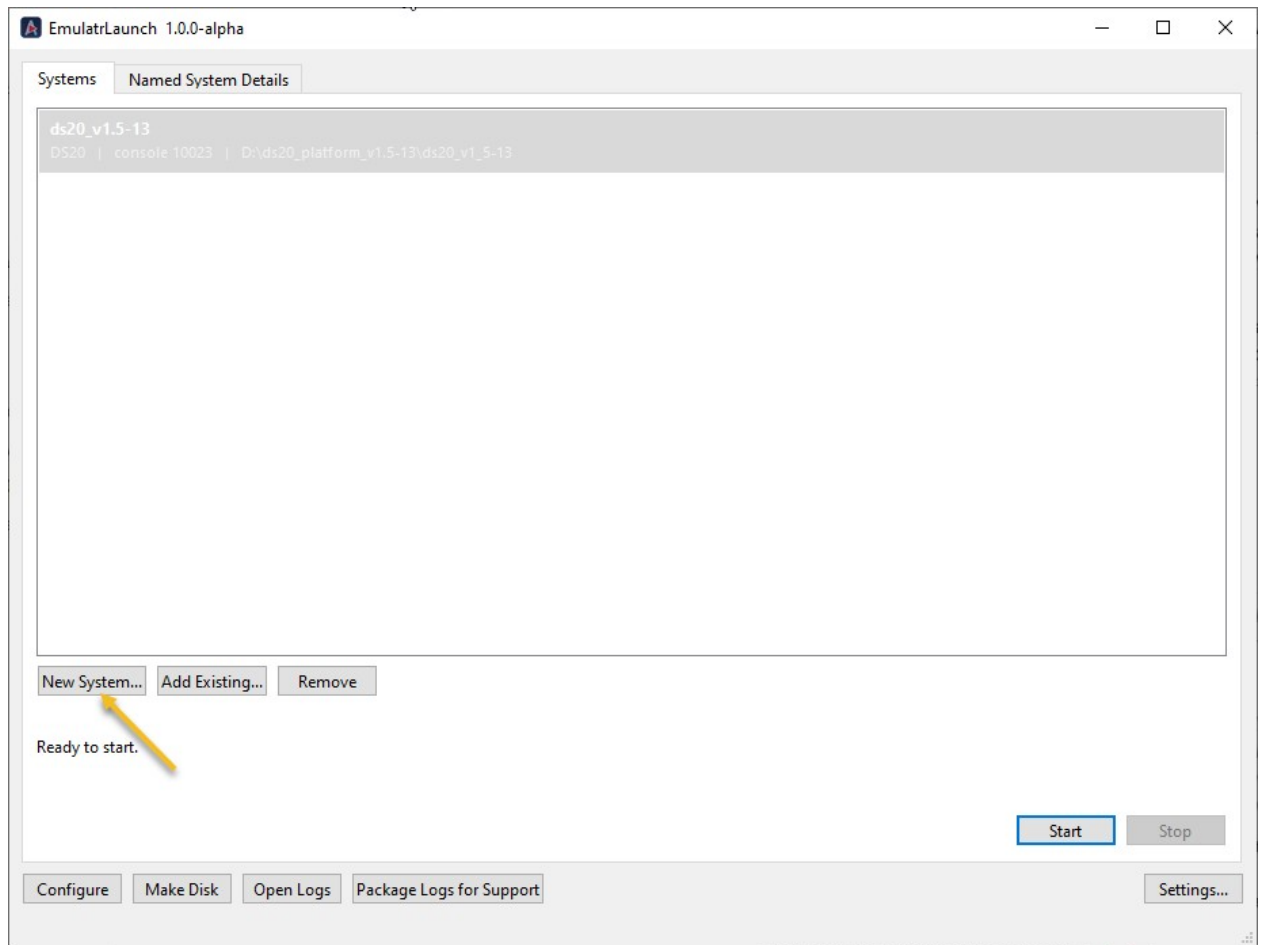
`HKEY_CURRENT_USER\Software\eNVy Systems\EmulatR`, where every launcher key carries a `launch_` prefix. The registry is the CATALOG, not the substance: it stores names, pointers and small per-system knobs, while everything load-bearing lives in the profile directory on disk. That split is why both survive a kit uninstall and reinstall, and why a lost hive costs you nothing but the re-registration.

Step 1 -- Install and start the launcher

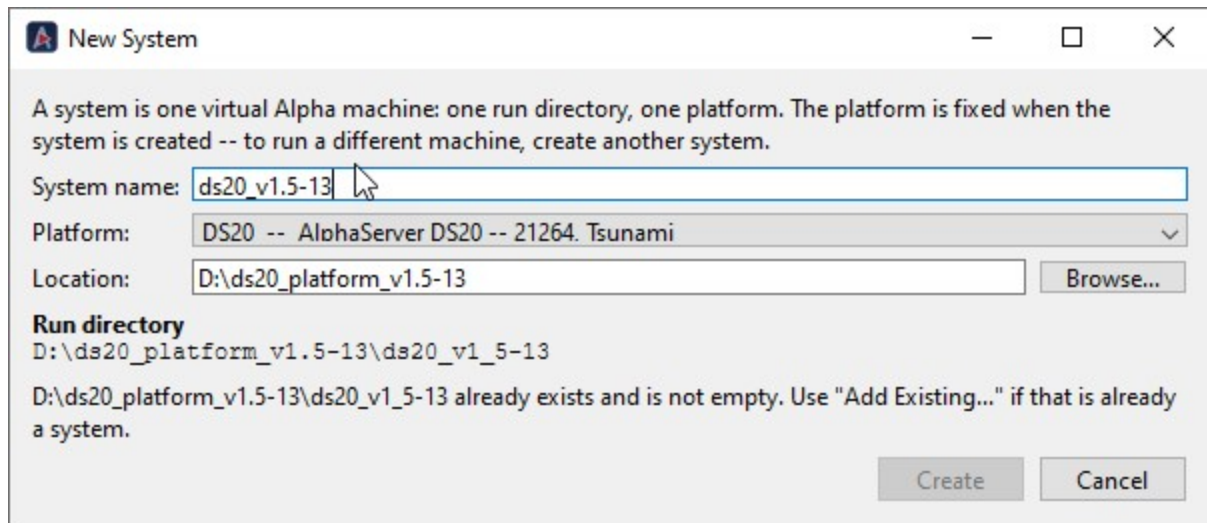
Run the installer, uninstalling any existing kit first, then start `EmulatrLaunch.exe` from the installation directory. No console window or shell is involved: the launcher starts the emulator directly as a child process with an explicit working directory and environment.

Step 2 -- Create a named system

On the Systems tab, choose **New System...** and supply:



- **Name** -- any display name, unique ignoring case.
- **Platform** -- DS10, DS20 or ES40. This is immutable after creation.
- **Profile (run) directory** -- anywhere you own, for example under Documents. Locations under Program Files are REFUSED: Windows silently redirects writes there and the emulator's flash state would be lost.



On OK the launcher builds the profile:

created	purpose
Emulatr.ini	seeded from a template with the model and console port filled in
firmware\	where your SRM image goes, with a readme; firmware is never shipped
disks\	default resolution root for relative disk-image names
logs\, traces\	run output
<stem>_platform.json	this system's OWN copy of the platform manifest, copied from the installation and made writable

The manifest copy at creation is the key design point: the profile is self-contained from birth. Editing the manifest affects only this system, and kit upgrades never touch it. Systems created by older launcher builds are seeded on their first **Open in PlatEd** instead.

Step 3 -- Supply the SRM firmware image

EmulatR does not distribute firmware, for licensing reasons. Copy your firmware image -- for example `ds20_v7_3.exe` -- into the profile's `firmware\` folder. Back in the launcher, the Details tab's **Firmware -> Image** control lists every file it finds there; pick one. A platform-mismatched image is flagged as a warning, never refused.

The selection is written to `Emulatr.ini`, and the image's basename -- the **stem** -- names the manifest the system uses: `ds20_v7_3.exe` selects `ds20_v7_3_platform.json`.

Step 4 -- Configure devices and storage

Details tab -> **Devices** -> **Open in PlatEd** opens this system's manifest copy. The note under the button shows the exact path and confirms it is `(system-local)`. The manifest declares the PCI devices and, under each storage controller, the disk and CD rows. On a DS20 the rows that matter to a first boot are:

- `pka_53c810` (the NCR SCSI HBA): SCSI id 0 becomes console device `dka0`, id 1 becomes `dka100`, and so on. Each row's `media` names its backing image file.
- `cypress_ide`: the ATAPI CD, `dqa1`. Its `media` names an ISO; empty means no disc.

How a media path is resolved

One rule, two cases:

- An **ABSOLUTE** path (`D:/Disks/dka0.vdisk`) is used as-is. This is the simplest way to attach a disk you already have.
- A **RELATIVE** path (`Alpha/dka0.vdisk`) resolves against `[Storage] diskDir` in the profile's `Emulatr.ini`. New profiles are seeded with `diskDir = disks`, so that example means `<profile>\disks\Alpha\dka0.vdisk`.

If a row's file does not exist at launch, the emulator leaves that unit empty -- the SRM then reports `device dka0 is invalid` when you try to boot it -- and writes a `Storage: ... not attached` line to the mirror log naming the exact path it tried. That log line is the first place to look when a disk is missing.

New blank disks come from the launcher's **Make Disk** tool, whose drive geometries are read from the installation's verified `disk_types.json`, or from `tools\mkdisk.py` for scripted provisioning.

Step 5 -- The per-system knobs

All of these live on the Details tab, persist per system, and take effect at the next Start.

control	choices	effect
Emulator binary	Discovered (recommended) / Release / RelWithDebInfo / Debug / Custom	which <code>Emulatr.exe</code> this system launches. Discovered finds the installed one.
Snapshots	Safety net (default) / Diagnostic / Off	machine-state snapshot cadence. Safety net saves rarely (50B cycles); Diagnostic saves densely (1B cycles) and each save pauses the guest.

control	choices	effect
Execution mode	ISP (default) / Silicon	ISP is the firmware's pre-silicon simulator path and boots to the SRM prompt. Silicon is the faithful real-hardware path -- experimental, and does not yet reach the console.
Console -> Port	TCP port, defaulted per system	the telnet console port. Open Console connects a terminal, using PuTTY if one is found.
Runtime environment variables	per-variable Enable plus values	see Step 6

Step 6 -- Runtime environment variables

The environment panel lists exactly what the SELECTED emulator binary declares -- each binary carries its own registry of variable names beside it. The installed release binary declares NONE, because the diagnostic layer is compiled out of release builds, and the panel says so rather than offering knobs that would do nothing. Pinning the system to a RelWithDebInfo or Debug binary lights up the full curated registry of 130-plus variables with descriptions.

An unchecked variable is ABSENT from the child environment, not merely empty. Quarantined variables are stripped even when they are set in your own shell.

Step 7 -- Start, console, stop

Start launches `Emulatr.exe` directly, with no shell: the working directory is the profile, the firmware comes from the Image selection, and the environment is composed from the knob overlays plus the variables you checked. When the profile owns a manifest copy, the launch points the emulator at it explicitly, so the installed defaults are never consulted.

Every run writes a **mirror log** to `<profile>\logs\run_launch_*.log`, headed by the effective diagnostic environment for that run.

Open Console connects to `127.0.0.1:<port>`. At the `P00>>>` prompt, verify the configuration took -- `show dev` should list the disks you configured in Step 4 -- then boot with `b dka0`.

Stop requests a clean shutdown by writing a stop sentinel into the profile; the emulator exits after persisting its state. If it has not exited after ten seconds the launcher OFFERS escalation -- it never kills on its own. A forced kill loses unpersisted flash state, and is logged as such.

What survives what

event	registry catalog	profile directories
Launcher restart	kept	kept
Kit uninstall, reinstall or upgrade	kept -- HKCU is not touched	kept, including customized manifests
Registry hive deleted	lost -- but Add Existing... re-registers any profile directory	kept
Profile directory deleted	stale entry flagged broken by preflight	lost -- it IS the system

Because a profile directory is self-contained, the recovery story is simple: point **Add Existing...** at the directory and the system is back.

Troubleshooting

symptom	cause	fix
device dka0 is invalid at the SRM prompt	the storage row's media file was not found	check the <code>Storage:</code> line in the mirror log, then fix the row's <code>media</code> -- an absolute path is simplest -- or put the file where <code>diskDir</code> plus the relative path points
Devices note says (installed default ...)	the profile predates manifest seeding	click Open in PlatEd once; it copies the manifest into the profile
Environment panel says there are no variables	a release binary is selected	expected by design -- pin a <code>RelWithDebInfo</code> or <code>Debug</code> binary for diagnostics
memtest: No such command, net: No such command, keyboard notice during boot	normal ISP-path console chatter	none needed
New System refused under Program Files	deliberate guard against write redirection	choose a directory you own

See Also.

[System Disk Configuration](#) ⁹⁹

[Firmware Requirements](#) ⁴⁴

[Platform Editor](#) ²⁵⁹

[Environment Variables \(EMULATR *\)](#) ²¹⁶

[Connecting to the Console](#) ⁶⁰

[Guest Boot](#) ¹¹⁰

4.1.2 Configuring

Configuration precedence (lowest to highest): compiled-in struct defaults < `Emulatr.ini` < command-line qualifiers. The committed ini drives a no-flag launch; a CLI qualifier overrides the ini for a single run. `IniLoader` populates `EmulatorSettings`, which is the single source of truth threaded to every consumer. Two settings resolve outside this ladder: when `--firmware` is given the machine model is derived from the firmware stem rather than the ini, and the requested memory size is capped at the model's silicon maximum -- see [Platform Identity & Memory Geometry](#)⁷¹.

Settings

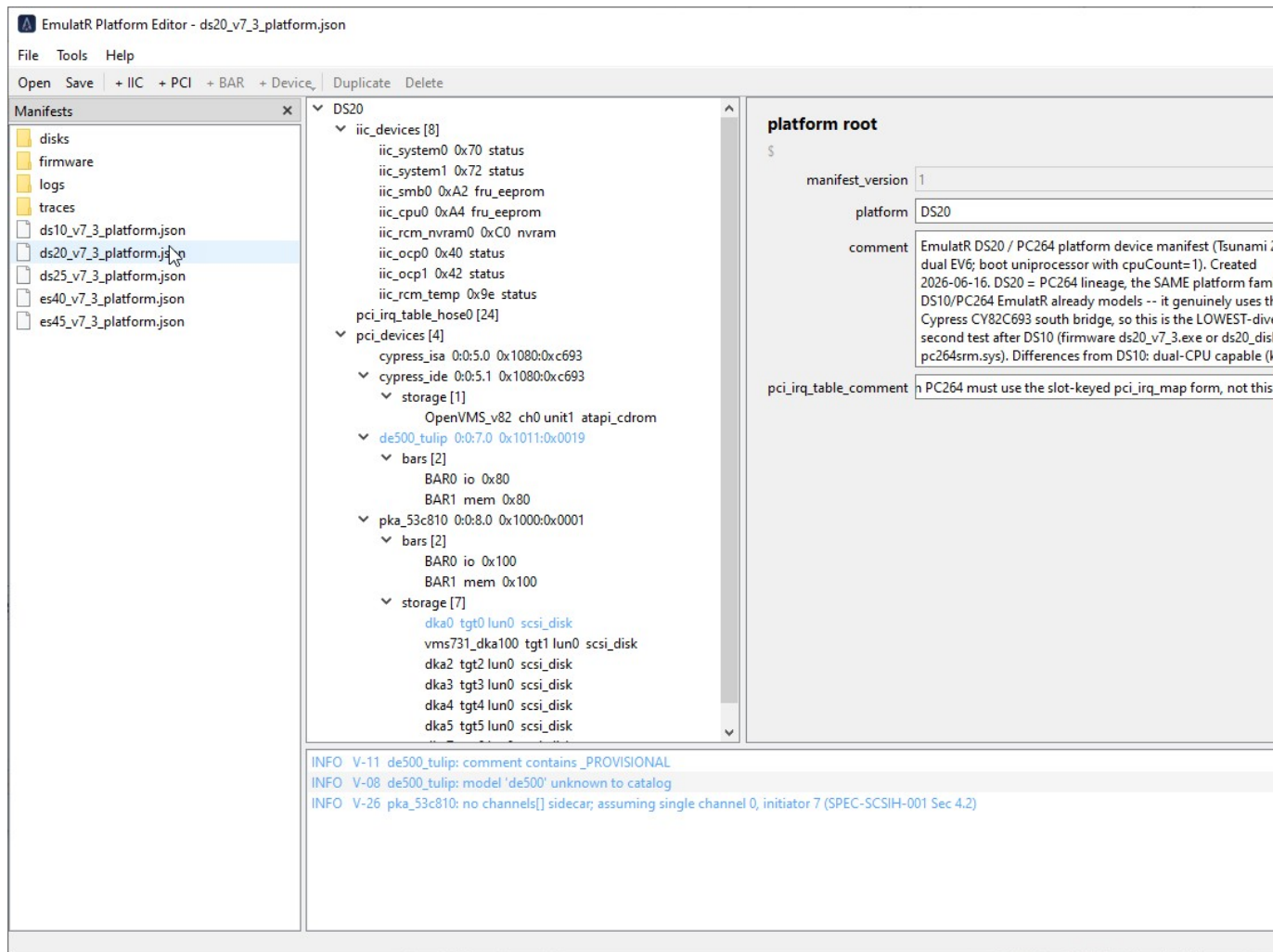
Field	ini Key	Default	Example	CLI / Env override
model	[System]/model	DS10	ds10	
cpuCount	[System]/cpuCount	1		
port	[SRMConsole]/port	10023		EMULATR_CONSOLE_PORT
puttyPath	[SRMConsole]/puttyPath	putty.exe	C:\Program Files\PuTTY\putty.exe	
firmwareImage	[ROM]/firmwareImage	ds10_v7_3.exe		--firmware
firmwareSha256	[ROM]/firmwareSha256	(empty -- integrity check skipped)		

Command-line qualifiers override the ini for a single run. Live today: --firmware (firmware image path) and --mem (memory size). Planned (locked): --console-port, --platform-config, --flash-path; the EMULATR_CONSOLE_PORT / EMULATR_FLASH_ROM / EMULATR_PLATFORM_CONFIG environment variables remain as documented aliases.

Debug-only environment variables (EMULATR_IDE_TRACE, EMULATR_IIC_TRACE, EMULATR_FLASH_TRACE, EMULATR_NO_PUTTY) are diagnostic toggles, not configuration: they have no ini key, no CLI qualifier, and no precedence interaction.

Bus configuration (platform manifest)

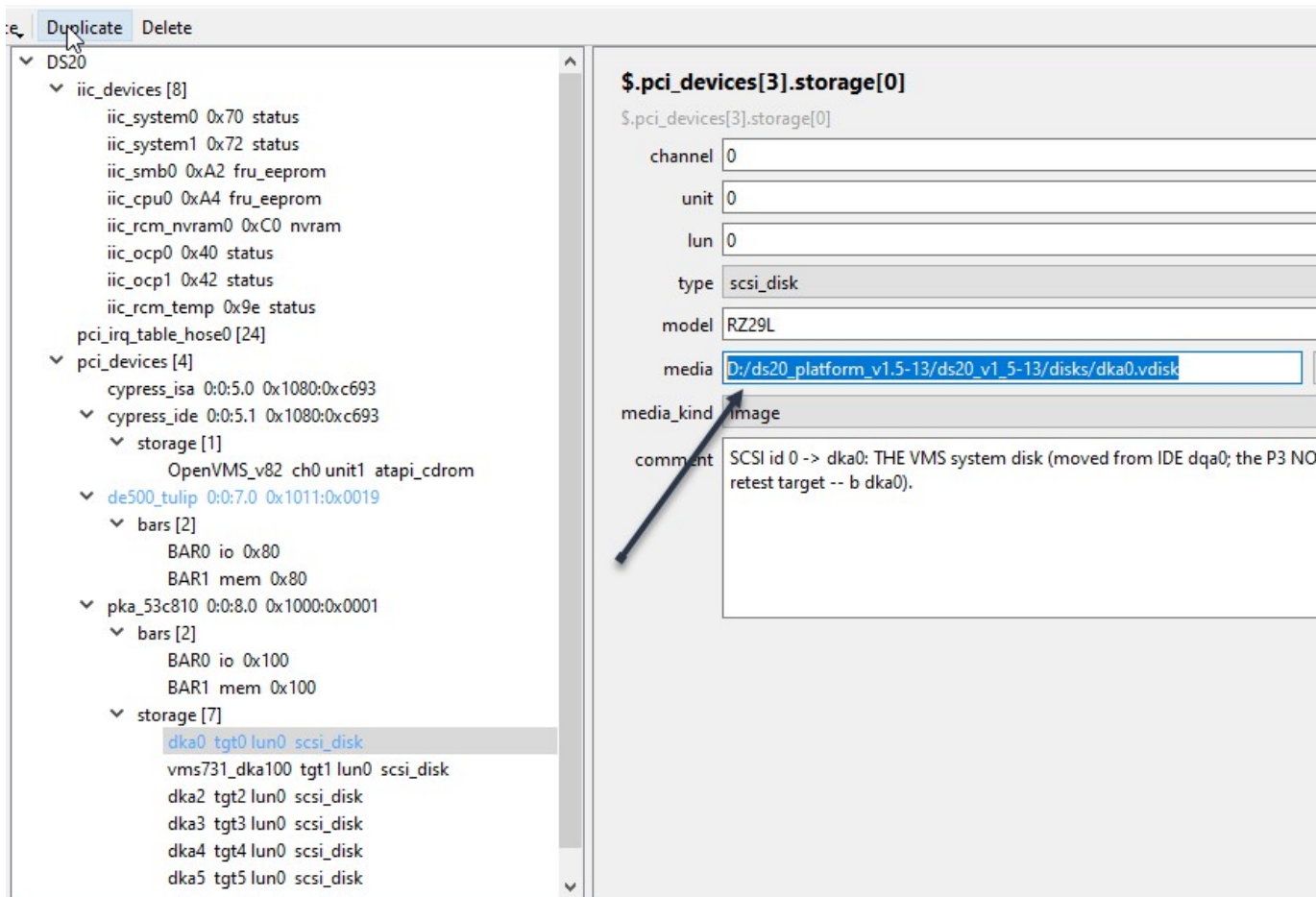
The bus configuration is loaded from a platform manifest. The manifest is named after the FIRMWARE IMAGE stem, not the model -- `firmware/ds20_v7_3.exe` selects `ds20_v7_3_platform.json` -- and it is resolved next to the executable rather than in the launch directory. Keying it to the firmware is what stops the model and its device set from drifting apart, and it lets a firmware variant carry its own devices. The manifest is host-agnostic: the device tree it describes belongs to the guest, not to the host. Selection order is the `EMULATR_PLATFORM_CONFIG` environment variable if set, otherwise the name derived from the firmware stem. A missing or invalid manifest does not stop the run -- it falls back to the compiled-in default DS10 device tree and prints the reason, so a mistyped manifest name will not announce itself as a failure.



Configure SCSI LUNS

It is strongly recommended that you modify the platform manifest files in the platform's profile directory.

You could create a single disk and use host-OS copy (cp) and latch them to the platform's manifest.



See Also.

[Current Platform Support](#) ¹⁷

[Platform Identity & Memory Geometry](#) ⁷¹

[Executing](#) ⁷⁴

[Environment Variables](#) ²¹⁶

4.1.3 Platform Identity & Memory Geometry

Two rules settle what machine EmulatR is about to be, and both run at argument resolution -- before the machine is constructed. The firmware image names the model, and the requested memory size is capped at what that model's silicon could physically hold.

The firmware stem names the model

When `--firmware` is given, the model is derived from the firmware stem's leading token -- the text before the first underscore, upcased. `ds10_v7_3.exe` gives DS10. A recognized token overrides the `[System] model` line in the ini, and the override is announced on stderr when the two disagree.

The recognized tokens are DS10, DS20, DS25, ES40 and ES45. With no `--firmware`, or a leading token that names no known model, the ini model stands -- no guess is made. A vendor firmware revision inside the stem is not part of the identity, so `ds20_v7_3_b.rom` still resolves to DS20.

Before this rule the platform had two identity channels. The manifest was already keyed to the firmware stem, but the model rode the ini -- so an ini reading `model=DS20` launched with `--firmware ds10_v7_3.exe` booted a DS10 device bus under a DS20 model. South-bridge selection, the capability latch and the memory-geometry expectations all followed the wrong machine, while the guest firmware badged itself from the DS10 IIC bus it actually probed. The platform latch reports that split as `PLATFORM MISMATCH`; it stays in place as the second line of defense for firmware whose stem is unrecognized.

Memory is clamped to the silicon maximum

A requested memory size larger than the modeled machine could physically hold is reset to that maximum, and the change is announced on stderr. The policy is clamp-and-continue rather than fail: the ini carries a single `memorySize` shared by every model, and a DS10 run should not die because the ini was last tuned for an ES40.

Model	Maximum guest memory	Basis
DS10	2 GiB	Webbrick, 4 DIMM slots
DS15	4 GiB	Single EV68C, 1 GiB DIMMs (launcher-enforced only -- see below)
DS20	4 GiB	Tsunami 21272, four 1 GiB banks
DS25	16 GiB	Titan 21274, 16 DIMM slots
ES40	32 GiB	Model 2: 32 DIMMs at 8 per memory motherboard (Model 1: 16 DIMMs = 16 GiB)
ES45	32 GiB	Titan 21274, 32 DIMMs (MS620 kits of 4, 512 MiB to 4 GiB)

DS15 is in the table because the launcher enforces its ceiling, but the binary does not yet. EmulatR recognizes DS10, DS20, DS25, ES40 and ES45 in both its stem-token list and its clamp table, and DS15 is in neither, so a DS15 run takes its model from the ini and is not clamped inside the emulator at all -- the 4 GiB ceiling holds only because `tools/emulatr_launch.sh` applies it before launch. Adding DS15 to those two tables closes the gap.

These are BOX ceilings -- what the chassis holds in DIMMs -- not chipset addressing limits, and the two differ. Titan (21274), the core logic behind DS15, DS25 and ES45, addresses 32 GiB, so a DS25 clamped at 16 GiB is bounded by its 16 DIMM slots rather than by anything Titan cannot reach. The ES40 shows the same split from the other side: 32 GiB is the Model 2 figure, with 32 DIMMs at eight per memory motherboard, while Model 1 carries 16 DIMMs and tops out at 16 GiB. Where one model name covers both builds, the clamp table takes the larger figure, so it bounds the most capable machine of that name and leaves a smaller build of it unprotected.

Requests at or below the maximum pass through untouched -- the rule constrains the ceiling, not the floor. A size beneath the machine's minimum, or a geometry the firmware otherwise dislikes, remains the firmware's to reject, exactly as on real hardware.

Why an oversized request is not simply a bigger machine

An unsupported geometry does not run slowly and does not report an error. A DS10 -- real maximum 2 GiB -- handed 4 GiB churns its memory configuration indefinitely: past ten billion cycles with no console output after the early banner, alive but never converging, and landing in varied SRM init code rather than one identifiable loop.

At 2 GiB the same binary, the same firmware and the same boot reach the memory banner (2048 Meg of system memory), the PCI probe, LFU, the DS10 console banner and >>> within minutes. An unsupported geometry is not a bigger machine; it is an impossible one the firmware was never written to configure.

Resolution order

Each setting is resolved in this order, highest priority first:

Setting	Resolved from
firmware path	--firmware, else the ini [ROM] firmwareImage
model	the firmware stem, if its leading token is recognized; else the ini [System] model
memory requested	--mem, else the ini [System] memorySize, else 64 MiB
memory granted	the lesser of the request and the model's silicon maximum

All four are settled before the machine is constructed, so every downstream consumer -- the chipset and the Cchip address-array registers among them -- sees the clamped value rather than the request.

What these rules do not do

Neither rule overrides the platform manifest. `EMULATR_PLATFORM_CONFIG` still points the manifest wherever an experiment needs it. The model still follows the stem, and the latch still reports a hybrid combination rather than accepting one silently -- an experiment may want a hybrid, but it must not get one by accident.

The clamp is a ceiling, not geometry validation. DIMM interleave legality, array counts and alignment are not checked -- only the total size. Faithful geometry modelling, of the kind the ES40 banner reports, is future work.

Neither rule changes emulated behavior. Both act at argument resolution, before the machine exists, so there is no snapshot, oracle or determinism impact.

See Also.

[Configuring](#) ⁶⁹

[Current Platform Support](#) ¹⁷

[Platform Memory Reference](#) ²³⁸

[Command Line Options](#) ⁷⁷

[Memory](#) ¹³²

[show memory](#) ⁹⁰

4.1.4 Executing

Beyond the basic launch (see Launching EmulatR), a few run scenarios come up often. All are launched from the EmulatR root and write diagnostics to a redirected log.

Traced boot

Turn on the device trace channels and put the console on an alternate port (so it coexists with another instance). PuTTY follows `EMULATR_CONSOLE_PORT`.

```
EMULATR_NO_AUTOLOAD=1 EMULATR_CONSOLE_PORT=10024 \  
  EMULATR_IDE_TRACE=1 EMULATR_IIC_TRACE=1 EMULATR_FLASH_TRACE=1 \  
  ./out/build/release/Emulatr.exe \  
  --firmware firmware/ds10_v7_3.exe --autosnapshot off 2> run_ide.log
```

Unattended (overnight) run

Use `nohup` so the run survives logoff; leave autosnapshot on and mirror the console to the log.

```
nohup env EMULATR_NO_AUTOLOAD=1 EMULATR_CONSOLE_SNAPSHOT=1 \  
  EMULATR_CONSOLE_MIRROR=1 EMULATR_CONSOLE_PORT=10023 \  
  ./out/build/relwithdebinfo/Emulatr.exe --firmware firmware/ds10_v7_3.exe \  
  > run_overnight.log 2>&1 &
```

Harvest the slowest console phases

After a run with the console mirrored to the log, the slowest timestamped phases fall out of the log with:

```
grep -a '\[CON' boot.log \
| sed -E 's/.*\[CON [^+]*\+ *([0-9.]+) ms\] ?/\1\t/' \
| sort -rn | head -25
```

See Also.

[Launching EmulatR](#) ⁵⁰

[Executing Two Instances](#) ⁷⁵

[Environment Variables](#) ²¹⁶

[Running from the shell](#) ⁵¹

4.1.5 Executing Two Instances

Two instances launched from the same directory share `ds10_flash.rom` and the `snapshots/` directory, and concurrent NVRAM writes can corrupt the flash. Give each console its own port with `EMULATR_CONSOLE_PORT`, and if a run will write NVRAM (any `set` or `update` at the `>>>` prompt), isolate its flash first.

```
cp ds10_flash.rom ds10_flash_release.rom
# then prepend on the launch line: EMULATR_FLASH_ROM=ds10_flash_release.rom
```

With a separate `EMULATR_FLASH_ROM` per instance, NVRAM writes no longer collide. Confirm the second instance came up on its own port by checking its log for "Listening on TCP port 10024".

See Also.

[Executing](#) ⁷⁴

[Clean Shutdown](#) ⁶¹

4.1.6 Example - Performance

A representative cold-boot transcript: power-up, the Loadable Firmware Update (LFU) utility, an `update srm`, and the interactive `>>>` prompt with a few `show / set` commands. Spacing is simplified for readability.

```
ASA EmulatR -- Alpha AXP (EV6 / 21264) Emulator
Alpha Emulator Console V1.4
(c) 2026 Timothy Peer / eNVy Systems, Inc.

*** keyboard not plugged in...
Flash ROM writes are disabled
1024 Meg of system memory
probing hose 0, PCI
probing PCI-to-ISA bridge, bus 1
bus 0, slot 5 -- dqa -- Cypress 82C693 IDE
```

```
initializing GCT/FRU at 3f32000
```

```
Checking dqa0.0.0.105.0 for the option firmware files. . .
dqa0.0.0.105.0 has no media present or is disabled via the RUN/STOP switch
Checking dva0.0.0.0.0 for the option firmware files. . .
Option firmware files were not found on CD or floppy.
... or just hit <return> to proceed with a standard console update:
```

```
***** Loadable Firmware Update Utility *****
Function Description
Display Displays the system's configuration table.
Exit Done exit LFU (reset).
List Lists the device, revision, firmware name, and update revision.
Update Replaces current firmware with loadable data image.
Verify Compares loadable and hardware images.
```

```
UPD> update srm
Confirm update on:
srm [Y/(N)]y
WARNING: updates may take several minutes to complete for each device.
DO NOT ABORT!
srm Updating to 7.3-1... Verifying 7.3-1... PASSED.
UPD> exit
```

```
Initializing....
Testing the System
Testing the Disks (read only)
snapshot Console V7.3-2, Feb 27 2007 13:25:53
Halt Button is IN, AUTO_ACTION ignored
```

```
>>>show config
SRM Console: V7.3-2
PALcode: OpenVMS PALcode V1.98-83, Tru64 UNIX PALcode V1.92-73
```

```
Processors
CPU 0 Alpha EV6 pass 1 268 MHz
SROM Revision: P<?>
Bcache is disabled
```

```
Core Logic
Cchip DECchip 21272-CA Rev 2
Dchip DECchip 21272-DA Rev 1
Pchip 0 DECchip 21272-EA Rev 1
TIG Rev 7.31
Arbiter Rev 7.31 (0xff)
```

```
MEMORY
Array # Size Base Addr
0 1024 MB 0000000000
Total Good Memory = 1024 MBytes
```

```
PCI Hose 00
Bus 00 Slot 05/0: Cypress 82C693
```

```

Bridge to Bus 1, ISA
Bus 00 Slot 05/1: Cypress 82C693 IDE
dqa0.0.0.105.0 EMULATR VIRTUAL CDROM

ISA (Slot/Device/Name/Type/Enabled/BaseAddr/IRQ/DMA)
0 MOUSE Embedded Yes 60 12
1 KBD Embedded Yes 60 1
2 COM1 Embedded Yes 3f8 4
3 COM2 Embedded Yes 2f8 3
4 LPT1 Embedded Yes 3bc 7
5 FLOPPY Embedded Yes 3f0 6 2

>>>show device
dqa0.0.0.105.0 DQA0 EMULATR VIRTUAL CDROM
dva0.0.0.0.0 DVA0
>>>show memory_test
memory_test full
>>>set memory_test none
>>>set full_powerup_diags off
>>>show memory_test
memory_test none
>>>

```

See Also.[Reaching the SRM Prompt](#) ⁶⁰[SRM Console](#) ⁸²[Current Development Status](#) ²¹

4.1.7 Command Line Options

The launch scripts (`tools/run_<model>.sh`) pass these command-line options (qualifiers) through to the emulator -- supply them after a bare --. Options are parsed left to right; a command-line value overrides the corresponding setting in `EmulatrV4.ini` (CLI > ini). Flags use a double dash and take the next argument as their value.

Usage:

```
./tools/run_<model>.sh -- [options]
```

The launch script selects its model's firmware image automatically; to override, pass --firmware <path> after the --, or rely on the [ROM] firmwareImage entry in `EmulatrV4.ini`.

Configuration precedence

Settings resolve in this order, later winning: built-in defaults, then `EmulatrV4.ini`, then the command line -- so CLI > ini. Environment variables (`EMULATR_*`) are a SEPARATE, orthogonal layer: most are ENV-only diagnostic / trace toggles with no CLI or ini equivalent (see Environment Variables). The two that overlap a CLI flag -- --no-autoload / `EMULATR_NO_AUTOLOAD` and --autosnapshot / `EMULATR_AUTOSNAP` -- are combined so either source can set them, rather than one overriding the other. A single unified order (ini < CLI < ENV, ENV winning) is planned but NOT yet in effect (Known Gaps G-25). For run

examples, background/detached execution, and the tools/ .sh helpers, see Running EmulatR from the Shell.

Command Line / Qualifier	Description
<code>--firmware <path></code>	Firmware image to load (raw binary or SRM .exe). Required; if omitted, falls back to [ROM] firmwareImage in EmulatV4.ini.
<code>--firmware-format <auto raw srm></code>	Loader contract. Default auto: picks srm for a .exe whose signature matches, raw otherwise.
<code>--load-pa <hex dec></code>	Physical address to load the image at. Default 0x0 for raw, 0x900000 for srm.
<code>--start-pa <hex dec></code>	Physical address to start execution at; the low bit is the PALmode bit. Default 0x0. Ignored for srm format (entry comes from the descriptor).
<code>--mem <bytes></code>	Guest memory size in bytes. Default 64 MiB. Overrides [System] memorySize in the ini.
<code>--max-cycles <N></code>	Cap on the run loop (hex or decimal). Default unlimited.
<code>--autosnapshot <on off></code>	Periodic auto_*.axpsnap saves. Default on. Off writes only the named --snapshot-on-pc file (use for cold-boot mint runs). Env equivalent: EMULATR_AUTOSNAP=off.
<code>--no-autoload</code>	Skip autoload-newest at startup, forcing a genuine cold boot instead of restoring the newest snapshot. Env equivalent: EMULATR_NO_AUTOLOAD=1.
<code>--pal-mode</code>	Force PALmode = true regardless of the --start-pa low bit.
<code>--trace <dec,machine></code>	Enable the DEC-listing and machine-parsable trace channels (PAL-window + retire-compact). Argument is the two output paths, comma-separated.
<code>--snapshot-on-pc <pc>[,<pc>...]</code>	One-shot snapshot trigger(s): the first retire at each PC writes a predig_*.axpsnap. A single PC also disables auto-save so the file stays mtime-newest for autoload; a comma-list mints several (pair with --autosnapshot off). predig_ files are never pruned.
<code>--snapshot-name-tag <s></code>	Base tag inserted after predig_ (e.g. gctbuild). With multiple PCs, _<pchex> is appended per file. Default: the hex PC.

Command Line / Qualifier	Description
<code>--inject-interrupt-at-cycle <N></code>	EXPERIMENTAL. Fires a one-shot synthetic INTERRUPT-class trap when the cycle count first reaches N. 0 = off (default).
<code>--dump-disasm <pa>:<count></code>	One-shot disassembly of <count> 4-byte instructions starting at physical address <pa>, dumped after firmware load and before run. Requires EMULATR_PA_DUMP=ON at build time.
<code>--log-disable <subsys></code>	Silence one logging subsystem entirely.
<code>--log-only <csv></code>	Silence every subsystem not in the comma-separated list.
<code>--log-verbose <subsys></code>	Unthrottled emit for one subsystem.
<code>--log-file <subsys>=<path></code>	Redirect one subsystem's emit stream to a file. Subsystems: Cbox, Unalign, IntervalTimer, Snapshot, PalRelocation, ChipsetCsr, StepD, Misc (case-insensitive). Requires EMULATR_DIAGNOSTIC_LOGGING=ON at build time.
<code>--help</code>	Print the usage/help text and exit.

Exit Codes

- 0 CPU halted cleanly (CALL_PAL HALT). 1 Any other stop (fault, max cycles, parse error).
2 Firmware load failure.

See Also.

[Running EmulatR from the Shell](#) ⁵¹

[Environment Variables \(EMULATR *\)](#) ²¹⁶

[Execution Matrix \(Models x Modes\)](#) ²¹⁰

[Known Gaps](#) ²³

4.2 4.2 Console Connectivity

EmulatR serves its console over a RAW TCP socket. This subsection covers what the console is, how the port is chosen, and how to attach each common terminal client.

In this section

- *Console Support* -- what the console is and which clients work.
- *TCP Console Overview* -- the port, the auto-launch, and running multiple instances.
- *Terminal Clients* -- step-by-step connection for PuTTY, Plink, Tera Term, MobaXterm, and netcat.

See Also.[Console Support](#) → 80[TCP Console Overview](#) → 80[Terminal Clients](#) → 81

4.2.1 Console Support

EmulatR exposes its console as a RAW TCP stream (default `localhost:10023`) -- there is no Telnet option negotiation, so a raw-capable client is preferred. Any of the following can attach:

- *PuTTY (Raw)*
- *Plink (-raw)*
- *Tera Term (TCP/IP)*
- *MobaXterm (TCP)*
- *netcat (nc) -- handy for scripting and automated testing*
- *telnet (may connect, but raw is preferred)*

If EmulatR runs on another machine, replace `localhost` with that host's name or IP. For development and debugging the recommended client is Plink with verbose logging:

```
plink -v -raw localhost -P 10023
```

See TCP Console Overview for the port and auto-launch behavior, and Terminal Clients for the step-by-step connection in each program.

See Also.[TCP Console Overview](#) → 80[Terminal Clients](#) → 81[Connecting to the Console](#) → 60

4.2.2 TCP Console Overview

EmulatR serves the console over a RAW TCP socket -- no Telnet option negotiation. Defaults:

```
Host: localhost
Port: 10023
Protocol: RAW TCP
```

Override the port with `EMULATR_CONSOLE_PORT` (the auto-launched PuTTY follows the port). Suppress the PuTTY auto-launch with `EMULATR_NO_PUTTY`; the listen socket still comes up for your own client. For a remote host, replace `localhost` with its hostname or IP.

Port 10023 is only the default -- the listener can bind any available TCP port. This is what makes running several instances at once practical: give each its own port (10023, 10024, 10025, and so on) so their consoles do not collide. Pick a free port above the privileged range (1024 and up); confirm it is free before binding:

```
netstat -ano | grep <port>
```

When connecting, point the client at the same port the instance was started with.

See Also.

[Terminal Clients](#) ⁸¹

[Executing Two Instances](#) ⁷⁵

[Configuring](#) ⁶⁹

4.2.3 Terminal Clients (PuTTY / Plink / TeraTerm / MobaXterm / Netcat)

Connection steps for the common clients. In every case the target is `localhost` port 10023 (or a remote host and the port the instance was started with), using a RAW connection.

Plink

```
plink -raw localhost -P 10023 # local
plink -raw 192.168.1.50 -P 10023 # remote
plink -v -raw localhost -P 10023 -log emulator.log # verbose + session log
```

Useful arguments: `-raw` (raw TCP stream), `-P <port>` (TCP port), `-v` (verbose diagnostics), `-log <file>` (save session log).

PuTTY

Launch PuTTY, set Host Name `localhost` and Port 10023, choose Connection type **Raw**, then press Open. For a remote instance use its IP in place of `localhost`.

Tera Term

New connection, select **TCP/IP**, Host `localhost`, TCP Port 10023, then connect.

MobaXterm

Session, then a Network session: Remote host `localhost`, Port 10023, Protocol TCP, then connect.

Netcat (Linux / macOS)

```
nc localhost 10023 # local
nc 192.168.1.50 10023 # remote
```

Useful for scripting and automated testing.

Telnet

Some telnet clients connect (`telnet localhost 10023`), but because EmulatR uses a raw socket and performs no Telnet option negotiation, a raw-capable client (PuTTY Raw, Plink – `raw`, Tera Term TCP/IP, MobaXterm TCP) is preferred.

Verifying the connection

After connecting you should see console output -- a banner, the `>>>` prompt, or an in-progress power-up sequence -- depending on the current emulator state.

Troubleshooting

Connection refused: verify EmulatR is running, the console server is enabled, and the port number is correct.

Connection timeout: verify firewall configuration and the host address, and that the listener is active.

No console output: verify the boot has progressed, console output is enabled, and you are on the right port.

See Also.

[TCP Console Overview](#) → 80

[Console Connection Issues](#) → 229

4.3 4.3 SRM Console

At the `>>>` prompt the console accepts `show` and `set` commands plus device and diagnostic commands; variable changes persist to the flash / NVRAM image. This subsection covers the console and its everyday commands.

In this section

- *SRM Overview -- what the console is and its reported versions.*
- *Common Commands -- the command set at a glance.*
- *show config -- the system configuration table.*
- *show device -- enumerated devices.*
- *show memory -- the memory array.*
- *boot -- bootstrap (planned; HALT today).*
- *set -- change and persist a console variable.*
- *SRM Environment Variables -- where the full variable list lives.*

See Also.

[SRM Overview](#) → 83

[Common Commands](#) ⁸⁹

[Example - Performance](#) ⁷⁵

4.3.1 SRM Overview

At >>> you interact with the console using `show` (display a variable), `set` (change a variable), and device / diagnostic commands. Settings persist to the flash / NVRAM image. The default power-up action is HALT, so the console waits at >>> for an operator command.

The reported console version is v7.3-2 (Feb 27 2007); PALcode is OpenVMS V1.98-83 / Tru64 UNIX V1.92-73.

See Also.

[Common Commands](#) ⁸⁹

[SRM Environment Variables](#) ⁹¹

4.3.2 Successful Boot to SRM (per platform)

Reference transcripts of a healthy boot to the SRM console prompt for each platform, so the testing teams have a known-good baseline to diff their own runs against. Two fields in the firmware banner (SROM Revision on Tsunami, ARC Console on ES40) emit uninitialized / binary bytes on a real boot; they are shown here as `[unset -- binary]` for ASCII fidelity, not literal console text.

DS10 (Tsunami, Virtual/ISP)

DS10 reaches the same `P00>>>` prompt reliably. A worked DS10 console session (examine / deposit) is in Example (Console) execution. A full DS10 `show config` capture will be dropped in below.

```
ASA EmulatR -- Alpha AXP (EV6 / 21264) Emulator
Alpha Emulator Console V1.4
(c) 2026 Timothy Peer / eNVy Systems, Inc.
```

```
*** keyboard not plugged in...
Running on the ISP model.
Flash ROM writes are disabled
2048 Meg of system memory
probing hose 0, PCI
probing PCI-to-ISA bridge, bus 1
```

```
ERROR: ISA table corrupt!
Initializing table to defaults
type >>>init to use these changes
bus 0, slot 5 -- dqa -- Cypress 82C693 IDE
*** system serial number not set. use set sys_serial_num command.
initializing GCT/FRU at 3ff32000
```

```

Checking dqal.1.0.105.0 for the option firmware files. . .
dqal.1.0.105.0 has no media present or is disabled via the RUN/STOP switch
Checking dva0.0.0.0.0 for the option firmware files. . .

```

```

Option firmware files were not found on CD or floppy.
If you want to load the options firmware,
please enter the device on which the files are located(ewa0),
or just hit <return> to proceed with a standard console update:

```

***** Loadable Firmware Update Utility *****

Function	Description
Display	Displays the system's configuration table.
Exit	Done exit LFU (reset).
List	Lists the device, revision, firmware name, and update revision.
Readme	Lists important release information.
Update	Replaces current firmware with loadable data image.
Verify	Compares loadable and hardware images.
? or Help	Scrolls this function table.

```
UPD> u srm
```

```
Confirm update on:
```

```
srm          [Y/(N)]y
```

```
WARNING: updates may take several minutes to complete for each device.
```

DO NOT ABORT!

```
srm          Updating to 7.3-1...  Verifying 7.3-1...  PASSED.
```

```
UPD> exit
```

```

  Initializing....
Testing the System
Testing the Disks (read only)
exer: No such command

```

```
AlphaServer DS10    5 MHz Console V7.3-2, Feb 27 2007 13:25:53
```

```
>>>show dev
```

```
dqal.1.0.105.0          DQA1          EMULATR VIRTUAL CDROM
```

```
dva0.0.0.0.0          DVA0
```

```
>>>show config
```

```
AlphaServer DS10    5 MHz
```

```
SRM Console:      V7.3-2
```

```
PALcode:          OpenVMS PALcode V1.98-83, Tru64 UNIX PALcode V1.92-73
```

```
Processors
```

```
CPU 0            Alpha EV6 pass 1    5 MHz
```

SROM Revision: P
Bcache is disabled

Core Logic

Cchip DECchip 21272-CA Rev 2
Dchip DECchip 21272-DA Rev 2
Pchip 0 DECchip 21272-EA Rev 1

TIG Rev 0.0
Arbiter Rev 0.0 (0x0)

MEMORY

Array #	Size	Base Addr
0	1024 MB	000000000
1	1024 MB	040000000

Total Bad Pages = 0
Total Good Memory = 2048 MBytes

PCI Hose 00

Bus 00 Slot 05/0: Cypress 82C693

Bridge to Bus 1, ISA

Bus 00 Slot 05/1: Cypress 82C693 IDE

dqa.0.0.105.0

dqa1.1.0.105.0

EMULATR VIRTUAL CDRO

ISA

Slot	Device	Name	Type	Enabled	BaseAddr	IRQ	DMA
0							
	0	MOUSE	Embedded	Yes	60	12	
	1	KBD	Embedded	Yes	60	1	
	2	COM1	Embedded	Yes	3f8	4	
	3	COM2	Embedded	Yes	2f8	3	
	4	LPT1	Embedded	Yes	3bc	7	
	5	FLOPPY	Embedded	Yes	3f0	6	2

>>>

DS20 (Tsunami, Virtual/ISP)

Boot to prompt, then `show dev` and `show config`. South bridge is the Cypress 82C693; four 1 GB arrays interleaved to 4 GB; both IDE units (dqa0 disk, dqa1 CDROM) enumerate.

```

ASA EmulatR -- Alpha AXP (EV6 / 21264) Emulator
Alpha Emulator Console V1.4
(c) 2026 Timothy Peer / eNVy Systems, Inc.

  Initializing....
Testing the System
Testing the Memory
memtest: No such command
System Temperature is 0 degrees C
AlphaServer DS20 5 MHz Console V7.3-2, Feb 27 2007 13:07:36
P00>>>show dev
dqa0.0.0.105.0 DQA0 EMULATR VIRTUAL DISK 1.0
dqa1.1.0.105.0 DQA1 EMULATR VIRTUAL CDROM
dva0.0.0.0.0 DVA0
P00>>>show config
  AlphaServer DS20 5 MHz
SRM Console: V7.3-2
PALcode: OpenVMS PALcode V1.98-79, Tru64 UNIX PALcode V1.92-74
Processors
CPU 0 Alpha EV6 pass 1 5 MHz SRM Revision: [unset -- binary]
  Bcache is disabled
Core Logic
Cchip DECchip 21272-CA Rev 2.1
Dchip DECchip 21272-DA Rev 2.0
Pchip 0 DECchip 21272-EA Rev 1
Pchip 1 DECchip 21272-EA Rev 4.2
TIG Rev 0.0
Arbiter Rev 0.0 (0x0)
MEMORY
Array # Size Base Addr
-----
  0 1024 MB 000000000
  1 1024 MB 040000000
  2 1024 MB 080000000
  3 1024 MB 0C0000000
Total Bad Pages = 0
Total Good Memory = 4096 MBytes
PCI Hose 00
  Bus 00 Slot 05/0: Cypress 82C693
  Bridge to Bus 1, ISA
  Bus 00 Slot 05/1: Cypress 82C693 IDE
  dqa.0.0.105.0
  dqa0.0.0.105.0 EMULATR VIRTUAL DISK
  dqa1.1.0.105.0 EMULATR VIRTUAL CDRO
PCI Hose 01
ISA
Slot Device Name Type Enabled BaseAddr IRQ DMA
0

```

```

0 MOUSE Embedded Yes 60 12
1 KBD Embedded Yes 60 1
2 COM1 Embedded Yes 3f8 4
3 COM2 Embedded Yes 2f8 3
4 LPT1 Embedded Yes 3bc 7
5 FLOPPY Embedded Yes 3f0 6 2
P00>>>

```

ES40 (Typhoon, Virtual/ISP)

A Virtual (ISP) boot. The (Silicon Mode) string in the console banner is the FIRMWARE build variant, NOT the EmulatR execution mode -- this run is Virtual/ISP. ES40 reaches P00>>> in Virtual mode; it does NOT yet reach the prompt under EmulatR Silicon Mode (see Known Gaps G-24). South bridge is the Acer Labs (ALi) M1543C at slot 5/0; the full firmware init log (heap / drivers / GCT-FRU / memory test) runs to the banner.

```

ASA EmulatR -- Alpha AXP (EV6 / 21264) Emulator
Alpha Emulator Console V1.4
(c) 2026 Timothy Peer / eNVy Systems, Inc.

```

```

starting console on CPU 0
initialized idle PCB
initializing semaphores
initializing heap
initial heap 240c0
memory low limit = 60e000 heap = 240c0, 17fc0
initializing driver structures
initializing idle process PID
initializing file system
initializing hardware
initializing timer data structures
lowering IPL
create dead_eater
create poll
create timer
create powerup
access NVRAM
Memory size 4096 MB
testing memory
...
probe I/O subsystem
probing hose 1, PCI
probing hose 0, PCI
probing PCI-to-ISA bridge, bus 1
bus 0, slot 5 -- dqa -- Cypress 82C693 IDE
starting drivers
entering idle loop
*** system serial number not set. use set sys_serial_num command.
Partition 0, Memory base: 000000000, size: 100000000
initializing GCT/FRU at 3fc30000
Memory Testing and Configuration Status
  Array Size Base Address Intlv Mode

```

```

-----
0 1024Mb 000000000000000000 4-Way
1 1024Mb 000000000400000000 4-Way
2 1024Mb 000000000800000000 4-Way
3 1024Mb 000000000c00000000 4-Way
4096 MB of System Memory
Testing the System
AlphaServer ES40 Console V7.3-2, built on Feb 27 2007 at 13:07:26 (ISP-Virtual Mode)
P00>>>show dev
dva0.0.0.1000.0 DVA0
P00>>>sh config
  hp AlphaServer ES40
Firmware
SRM Console: V7.3-2
ARC Console: [unset -- binary]
PALcode: OpenVMS PALcode V1.98-4, Tru64 UNIX PALcode V1.92-5
Serial ROM: V2.22G
RMC ROM: V1.0
RMC Flash ROM: V1.0
Processors
CPU 0 Alpha EV6 pass 1 5 MHz 8MB Bcache
Core Logic
Cchip DECchip 21272-CA Rev 9(C4)
Dchip DECchip 21272-DA Rev 2
Pchip 0 DECchip 21272-EA Rev 1
Pchip 1 DECchip 21272-EA Rev 4
TIG Rev 00
Memory
  Array Size Base Address Intlv Mode
-----
0 1024Mb 000000000000000000 4-Way
1 1024Mb 000000000400000000 4-Way
2 1024Mb 000000000800000000 4-Way
3 1024Mb 000000000c00000000 4-Way
4096 MB of System Memory
Slot Option Hose 0, Bus 0, PCI
5/0 Acer Labs M1543C Bridge to Bus 1, ISA
5/1 Cypress 82C693 IDE dqa0.0.0.105.0
Option Hose 0, Bus 1, ISA
Floppy dva0.0.0.1000.0
P00>>>

```

Note --

Separately, even in Virtual mode ES40 device enumeration is incomplete: `show dev` lists only `dva0` (floppy) with no `dqa` disk/CDROM, though `show config` shows the IDE bridge (see G-01 / G-06).

See Also.

[Execution Matrix \(Models x Modes\)](#) ²¹⁰

[Reaching the SRM Prompt](#) ⁶⁰

[Example \(Console\) execution](#) ²³⁰

[Known Gaps](#) ²³

4.3.3 Common Commands

The commands used most often at the >>> prompt. Each `show` command has its own page in this subsection.

- *show <var> -- display the stored and active value of a console variable.*
- *set <var> <value> -- change a variable (persisted to NVRAM).*
- *show config -- system configuration table (CPU, core logic, memory, PCI / ISA).*
- *show device -- enumerated storage / I/O devices.*
- *show memory -- memory array layout.*
- *boot -- bootstrap the OS (planned on EmulatR; HALT is the implemented action).*
- *help / ? -- console help.*

See Also.

[show config](#) ⁸⁹

[set](#) ⁹⁰

[SRM Environment Variables](#) ⁹¹

4.3.4 show config

`show config` prints the full configuration table: the SRM and PALcode versions; the processor (Alpha EV6 pass 1, 268 MHz); core logic (Cchip 21272-CA, Dchip 21272-DA, Pchip 21272-EA, TIG / Arbiter Rev 7.31); the memory array; the PCI hose (the Cypress 82C693 bridge and IDE at bus 0, slot 5); and the ISA device list (MOUSE, KBD, COM1, COM2, LPT1, FLOPPY with their base addresses and IRQs). A full sample is in Example - Performance.

See Also.

[show device](#) ⁸⁹

[show memory](#) ⁹⁰

[Example - Performance](#) ⁷⁵

[Clock Frequency and the MHz Reading](#) ¹⁹⁸

4.3.5 show device

`show device` lists enumerated devices. On the DS10 target:

```
dqa0.0.0.105.0 DQA0 EMULATR VIRTUAL CDROM
dva0.0.0.0.0 DVA0
```

`dqa0` is the virtual ATAPI CD-ROM on the CY82C693 IDE channel; `dva0` is the floppy device. The `dqa0` enumeration path is under active work; on the ES40 (silicon mode) the `dqa0` /

dqbb0 ALi M5229 channels are still being enumerated (see IDE Devices, Storage, and Current Development Status).

See Also.

[show config](#) ⁸⁹

[Storage](#) ⁹²

[Current Development Status](#) ²¹

4.3.6 show memory

The memory array is reported in the configuration table:

```
Array # Size Base Addr
  0 1024 MB 0000000000
Total Good Memory = 1024 MBytes, Total Bad Pages = 0
```

The default system memory is 1024 MB (set by [System] memorySize in the ini); the size can be overridden at launch with --mem.

See Also.

[show config](#) ⁸⁹

[Configuring](#) ⁶⁹

4.3.7 boot

On real SRM, `boot` bootstraps the operating system using `bootdef_dev` and `boot_osflags`. On EmulatR, OS bootstrap is not yet implemented -- the console reaches `>>>` and halts. The related `auto_action` variable supports HALT (the default and only fully implemented action); BOOT and RESTART are planned.

See Also.

[Current Development Status](#) ²¹

[Guest Boot](#) ¹¹⁰

4.3.8 set

`set` changes a console variable and persists it to the flash / NVRAM image; `show` reads it back. Example session:

```
>>>set
  snapshot
>>>set memory_test none
>>>set full_powerup_diags off
>>>show memory_test
memory_test none
```

Setting `memory_test none` and `full_powerup_diags off` shortens the power-up self-test -- useful for faster iteration (see Slow Boot).

See Also.

[SRM Environment Variables](#) ⁹¹

[Slow Boot](#) ²²⁸

4.3.9 SRM Environment Variables

The complete, annotated list of SRM console variables (`auto_action`, the `boot_*` set, `com1_*` / `com2_*`, the `d_*` diagnostics group, the N1 to N16 NVRAM slots, `wwid0` to `wwid7`, and so on) is maintained in the appendix:

- *-Appendix, DS10, "SRM Commands >>> Prompt".*

These SRM variables are distinct from the emulator's own `EMULATR_*` launch environment variables, which are documented under the Reference section's Environment Variables page.

See Also.

[set](#) ⁹⁰

[Environment Variables](#) ²¹⁶

4.3.10 >>> Evaluate Address

At the >>> prompt the SRM console can read and write memory directly -- useful for inspecting device CSRs and firmware state during debugging.

Examine (e)

`e <space>:<address>` reads a location and prints its value. The `pmem:` space addresses physical memory (including device / chipset MMIO):

```
>>> e pmem:80130000040
pmem: 80130000040 000000000000000000
```

Deposit (d)

`d <space>:<address> <value>` writes a value to a location:

```
>>> d pmem:80130000040 1
```

These are the commands behind the halt-gate diagnostics (for example the arming read of `pmem:80130000FF8`). Be precise with the address and space prefix; an address that is not in an executable region is rejected ("not executable"), and a malformed line produces a syntax error.

See Also.

[Example \(Console\) execution](#) ²³⁰

[Halt Button is IN, BOOT NOT POSSIBLE](#) ²²⁹

4.4 4.4 Storage

Storage on the DS-class targets is presented through the Cypress CY82C693 south-bridge IDE / ATAPI channels; the ES40 uses the ALi M1543C / M5229 (see IDE Devices). This subsection covers creating and attaching disk and optical images and the per-bus device models. SCSI and the legacy buses are noted where their status is still to be confirmed.

In this section

- *Create Guest Disk Storage -- provision a disk image (manifest or mkdisk).*
- *Virtual Disk Images -- the raw image format and how a disk attaches.*
- *Virtual CDROM -- the dqa0 optical device and ISO media.*
- *IDE Devices / ATAPI Devices (+ ISO-9660) -- the CY82C693 channels.*
- *SCSI Devices and the legacy buses (SDI / DSSI / ST506).*

See Also.

[Create Guest Disk Storage](#) ⁹⁶

[Virtual Disk Images](#) ¹⁰⁵

[IDE Devices](#) ⁹³

4.4.1 ATAPI Devices

The CD-ROM is presented as an ATAPI device on the IDE channel -- dqa0.0.0.105.0, "EMULATR VIRTUAL CDROM". ATAPI / IDE scaffolding for the **CY82C693** is in active development; the dqa0 enumeration gap (the firmware does not consistently see the device during the option-firmware scan) is a known open item.

Support notes

The master (CD) ATAPI handshake works end to end:

Alt-status reads 0x50 (DRDY | DSC) -- present and ready, not 0xFF, not 0x00.

The 0xAA scratch write to cyl-low reads back 0xAA -- the presence gate passes.

IDENTIFY DEVICE (0xEC) returns status 0x41 (DRDY | ERR), error 0x04 (ABRT), and cyl bytes 0x14 / 0xEB -- the ATAPI signature is correctly posted on abort.

IDENTIFY PACKET DEVICE (0xA1) runs, then two PACKET (0xA0) commands. A REQUEST SENSE CDB (03 00 00 00 12 00) returns sense key 0x02 (NOT READY) / ASC 0x3A (MEDIUM NOT PRESENT), then status 0x50, phase Complete.

Source References

- `deviceLib/Tsunami/AtaTaskfileEngine.h` -- *ATAPI handshake (alt-status 0x50, 0xAA scratch, PACKET).*
- `deviceLib/scsi/VirtualIsoDevice.h` -- *the EMULATR VIRTUAL CDROM media endpoint.*
- `deviceLib/Tsunami/Cy82C693Ide.h` -- *IDE channel carrying dqa0/dqa1.*
- `tests/deviceLib/test_cy82c693ide.cpp` -- *IDENTIFY / PACKET handshake coverage.*

See Also.

[Virtual CDROM](#) ¹⁰⁴

[ISO-9660 PFD \(CD001\)](#) ⁹⁴

[Current Development Status](#) ²¹

4.4.2 IDE Devices

On the DS10 / DS20 (Tsunami) the CY82C693 provides the IDE controller, exposed on PCI bus 0, slot 5, function 1 ("Cypress 82C693 IDE" in `show config`). The primary channel is the `dqa` device path, the secondary is `dqb`. IDE register / port behavior (the `0x1F0-0x1F7` primary and `0x170` secondary blocks, plus PCI config) and absent-slave probing are shared across the controllers.

ES40 -- ALi M1543C / M5229

The ES40 (Typhoon) uses the ALi M1543C south bridge, whose IDE function is the M5229 (device id `0x5229`). EmulatR wires the faithful ALi M5229 for ES40 / ES45 / DS25 via the SAME south-bridge lever (`m_southBridge`) that selects the ISA bridge, so bridge and IDE stay consistent by construction; DS10 / DS20 wire the Cypress instead. Both controllers wrap the shared ATA taskfile engine, which answers IDENTIFY on the `dqa0` and `dqb0` channels. Full `show dev` enumeration of `dqa0` / `dqb0` in silicon mode is active device-enumeration work (see Current Development Status).

Source References

- `deviceLib/Tsunami/Cy82C693Ide.h` -- *the DS10 / DS20 Cypress IDE controller.*
- `deviceLib/Tsunami/Alim5229Ide.h` -- *the ES40 ALi M5229 IDE (identity 0x522910B9).*
- `deviceLib/Tsunami/AtaTaskfileEngine.h` -- *the shared ATA taskfile / IDENTIFY engine.*
- `deviceLib/Tsunami/ITsunamiIde.h`, `PciConfigSpace.h` -- *the IDE interface and PCI config plumbing.*

See Also.

[ATAPI Devices](#) ⁹²

[Virtual Disk Images](#) ^{→ 105}

[Current Development Status](#) ^{→ 21}

4.4.3 SCSI Devices

The SRM console exposes SCSI-related variables (`scsi_poll`, `scsi_reset`), but EmulatR does not currently model a SCSI host adapter (no QLogic ISP or Symbios 53C8xx HBA is wired). The `VirtualScsiDevice` endpoint class exists in the device library for the eventual `dka` path.

For reference, the DEC RZ-series drives a DS10-class machine would address over SCSI (all 512-byte blocks):

Model	Capacity	Notes
RZ22	104 MB	SCSI drive
RZ23	209 MB	SCSI drive
RZ24	245 MB	SCSI drive
RZ25	426 MB	SCSI drive
RZ26	1.05 GB	in geometry catalog
RZ28	2.1 GB	in geometry catalog
RZ29	4.2 GB	in geometry catalog

Source References

- `deviceLib/scsi/VirtualScsiDevice.h` -- *dka endpoint class (planned; no HBA modeled yet).*
- `deviceLib/scsi/ScsiCommand.h` -- *SCSI command model (with `ScsiTypes.h`).*
- `config/dec_disk_media_types.tsv` -- *RZ-series geometry catalog.*

See Also.

[Create Guest Disk Storage](#) ^{→ 96}

[Current Development Status](#) ^{→ 21}

4.4.4 ISO-9660 PFD (CD001)

An ISO-9660 image begins with a 16-block (32 KiB) system area, after which LBA 16 holds the Primary Volume Descriptor (PVD). The firmware's first coherence check is the standard identifier "CD001" at offset 1 of that descriptor. `VirtualIsoDevice` serves the PVD so a `READ(10)` of LBA 16 returns it. The doctest that locks this behavior:

```
TEST_CASE("VirtualIsoDevice: serves the ISO-9660 PVD at LBA 16 (CD001)") {
    // System area = 16 zero blocks; LBA 16 = PVD.
    std::vector<uint8_t> img(17 * 2048, 0);
    uint8_t* pvd = img.data() + 16 * 2048;
    pvd[0] = 0x01; // descriptor type = PVD
```

```

std::memcpy(pvd + 1, "CD001", 5); // standard identifier
pvd[6] = 0x01; // version
// block size 2048 (both-endian) at offset 128
VirtualIsoDevice cd; REQUIRE(cd.loadMedia("pvd.iso"));
uint8_t buf[2048]; ScsiCommand c = read10(16, 1, buf);
cd.handleCommand(c);
CHECK(c.status == ScsiStatus::Good);
CHECK(std::memcmp(buf + 1, "CD001", 5) == 0);
}

```

Source References

- `deviceLib/scsi/VirtualIsoDevice.h` -- *serves PVD at LBA16 (CD001) on READ(10).*
- `deviceLib/scsi/FileBlockMedia.h` -- *backing block media (with IBlockMedia.h).*
- `tests/deviceLib/test_scsi_iso.cpp` -- *the CD001-PVD doctest quoted here.*

See Also.

[ATAPI Devices](#) 92

[Virtual CDROM](#) 104

4.4.5 SDI / DSSI / ST506 Devices

These are legacy DEC / PC disk interconnects, included here for reference. None is present on the DS10, which uses the CY82C693 IDE / ATAPI channel exclusively; they are not modeled by EmulatR.

- *-SDI (Standard Disk Interconnect) -- the serial interface DEC used between an HSC / KDM controller and RA-series disks on large VAX systems.*
- *DSSI (Digital Storage Systems Interconnect) -- a multi-host bus used to share RF-series disks across clustered VAX / Alpha nodes.*
- *ST506 -- the early MFM / RLL hard-disk interface of the PC / workstation era, predating IDE.*

Confirmed reference-only: no SDI, DSSI, or ST506 storage entry appears in any platform manifest (DS10, DS20, DS25, ES40, ES45) -- all modeled storage is `ata_disk` or `atapi_cdrom`.

Source References

- `deviceLib/scsi/BlockMediaFactory.h` -- *closest real seam; SDI/DSSI/ST506 are not modeled.*
- `config/dec_disk_media_types.tsv` -- *DEC media geometry catalog (reference).*

This topic is intentionally reference-only: no SDI, DSSI, or ST506 device is modeled in the tree.

See Also.[Storage Interconnect and Fabric](#) → 96[SCSI Devices](#) → 94

4.4.6 Storage Interconnect and Fabric

On the DS10 target the storage path is short and fixed: the EV6 reaches the PCI hose through the Pchip, the Cypress CY82C693 sits on PCI bus 0 (slot 5) as the PCI-to-ISA bridge and IDE controller, and the storage endpoints (the ATAPI CD-ROM and any ATA disk) hang off the CY82C693 IDE channel. There is no separate storage switch or expander -- the "fabric" is just the IDE channel and its master / slave units.

SCSI (the `dka` path) and the legacy DEC interconnects are reference material for other platforms; on the DS10 they are not part of the active fabric. See SCSI Devices and the legacy-bus page for those.

Source References

- `chipsetLib/TsunamiPchip.h` -- *EV6->PCI hose bridge (Pchip).*
- `chipsetLib/Cypress_CY82C693ISABridge.h` -- *CY82C693 on PCI bus0/slot5.*
- `deviceLib/Tsunami/Cy82C693Ide.h` -- *IDE channel (the fabric).*
- `deviceLib/Tsunami/AtaTaskfileEngine.h` -- *master/slave storage units.*

See Also.[IDE Devices](#) → 93[SCSI Devices](#) → 94[SDI / DSSI / ST506 Devices](#) → 95

4.4.7 Create Guest Disk Storage

EmulatR provisions disk images two ways: automatically at boot, driven by the platform manifest (`create_if_missing`), and manually with the `tools/mkdisk.py` generator. Both share one image-creation codepath and one geometry catalog, so a disk type's size is defined in exactly one place. A manifest references a disk **type**, not a raw byte count -- the size is resolved from the catalog at load.

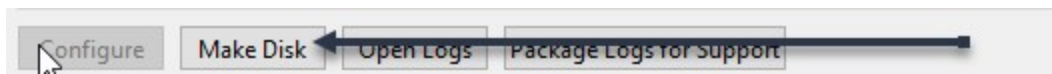
Geometry catalog (single source of truth)

The authoritative geometry source is `config/disk_types.json`, generated at build time from `dec_disk_media_types.tsv`. Never hand-maintain both: edit the TSV, regenerate the JSON. Each type names `total_lbn` and `block_bytes`; capacity is derived (`total_lbn * block_bytes`). A type is either a real DEC model (for example RZ28) or a sized blank target (for example blank-4G).

type	interface	total_lbn	block_bytes	capacity
blank-4G	ata	8388608	512	4 GiB
RZ26	scsi	2050860	512	1.05 GB
RZ28	scsi	4110480	512	2.1 GB
RZ29	scsi	8407200	512	4.2 GB

EmulatRLauncher Make Disk. ([Configuring a System with the Launcher](#)⁶³)

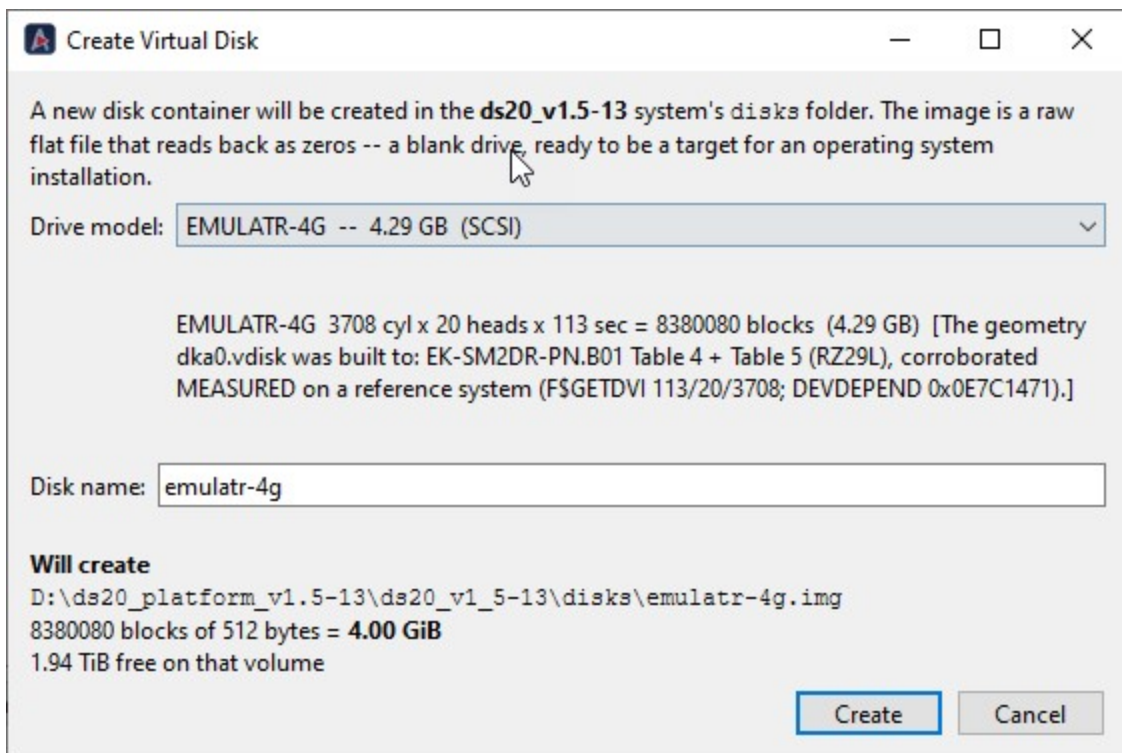
EmulatRLauncher - System's tab.



Make Disk button raises the following dialog.

We recommend that you select a drive model that is provided in the combo-selection-box.

Disk Name is the file name of the device.



Other Make Disk Options.

Automatic provisioning (create_if_missing)

When a manifest storage entry names a missing image and sets `create_if_missing`, EmulatR provisions a blank sparse image of the catalogued size on first boot. It never overwrites an existing image, and never provisions read-only media (an ISO is supplied, not created). Example manifest entry for the dqa0 ATA disk:

```
{ "channel": 0, "unit": 0, "type": "ata_disk",  
  "media": "dqa0.img", "disk_type": "blank-4G",  
  "create_if_missing": true }
```

On first cold boot the run log reports the provisioning and the attach:

```
Storage: created blank disk image 'dqa0.img' (4294967296 bytes)  
attached ATA disk 'dqa0.img' to IDE ch0 unit0
```

Manual creation (tools/mkdisk.py)

Use `mkdisk.py` to pre-make a typed image outside the boot path -- a specific DEC model, or a sized install target. It calls the same core as `create_if_missing`: looks up `total_lbn * block_bytes` from the catalog, writes a sparse zero-filled file, refuses to overwrite without `--force`, never creates for read-only media, and self-checks that the final size equals the catalog size.

```
python tools/mkdisk.py --type blank-4G --out dqa0.img  
python tools/mkdisk.py --type RZ28 --out dka0.img
```

The image written is BARE: exactly `total_lbn * block_bytes`, zero-filled, with no partition table and no DEC disklabel. The guest OS install writes those structures. For the install path (boot the installer, install onto a blank target, boot the installed disk) a bare image is what you want.

Cross-platform sparseness

Image creation uses a truncate-to-size, which is genuinely sparse on Linux ext4 (a 4 GiB image does not write 4 GiB physically). On Windows NTFS the file is lazily zero-filled and is not flagged sparse unless `FSCTL_SET_SPARSE` is issued; `mkdisk.py` sets it so behavior is identical on Windows and Linux. In practice the image costs nothing on disk until the guest writes to it.

See Also.

[Virtual Disk Images](#) ¹⁰⁵

[Guest Boot](#) ¹¹⁰

[Configuring a System with the Launcher](#) ⁶³

4.4.8 System Disk Configuration

This topic takes you end to end: create an EmulatR virtual disk, point the platform manifest at an OpenVMS distribution ISO and at the new disk, boot the ISO, install OpenVMS onto the virtual disk, and boot the installed system disk. The worked example is a DS20 (Tsunami/EV6) with firmware stem `ds20_v7_3`. DS25, ES40 and ES45 follow the same shape; DS10 differs in one respect, noted under Step 3.

The storage model in one paragraph

EmulatR enumerates EXACTLY the storage devices declared in the platform manifest (`<stem>_platform.json`, for example `ds20_v7_3_platform.json`). A storage row MEANS the device exists on the bus: an undeclared SCSI id selection-times-out, and a row whose `media` is empty enumerates as an empty drive. Each row names its media as a bare FILENAME, which resolves against the `[Storage] diskDir` setting in `Emulatr.ini` -- in a launcher-managed system that is the run directory's `disks` folder. Drive identity and geometry come from the row's `model` field, resolved against the storage catalog. So a manifest names a drive TYPE and a filename, never a byte count and never a host path.

Step 1 -- Create the virtual system disk

Use **Make Disk** in the EmulatR Launcher. It opens the Create Virtual Disk dialog, which writes the container into the SELECTED system's `disks` folder -- placement is fixed, there is no free path selection, and that is deliberate: the manifest can then name the file by bare filename and the whole run directory stays portable.

field	what it does
Drive model	28 verified DEC drive geometries (RZ, RA, RF, RC and ESE series), plus a Custom geometry entry. Selecting a model shows its exact geometry and pre-fills the name.
Custom geometry	Cylinders, heads, sectors per track and block size (512 for a fixed disk, 2048 for optical). Container size is the product of the four. Sanity-bounded, not policed.
Disk name	A file stem: ASCII, no path separators, no Windows-reserved names. The extension <code>.img</code> is added for you.
Will create	Live preview of the exact path, the block count and block size, the resulting size, and the free space on that volume -- all shown BEFORE anything is written.

Create stays disabled until the geometry and the name are both valid and there is room. The dialog refuses to overwrite an existing file, and refuses outright when the volume has less free space than the image needs, naming both numbers.

The container is a RAW FLAT FILE: exactly `total_lbn * block_bytes` zero bytes, sparse where the host filesystem supports it, with no header, no partition table, no disklabel and no filesystem. It reads back as zeros -- a blank drive. The OpenVMS installation writes every structure onto it. That is exactly what an install target should be.

Choosing a drive profile

The model list is read from the emulator's own storage catalog, `config/disk_types.json`, found beside the emulator the system is set to launch. The key you pick here IS the `model` value you will type into the manifest in Step 3 -- one name, one namespace, no translation. Retired profiles are never offered, so a name that reaches the dialog is a name the manifest accepts.

profile	blocks	use
RZ29L	8,380,080 -- 4.0 GB	recommended OpenVMS system disk
EMULATR-4G	8,380,080 -- 4.0 GB	synthetic equivalent
RZ28L	4,110,480 -- 2.1 GB	smaller system disk
RZ40	17,773,524 -- 9.1 GB	large data volume
EMULATR-512M	1,048,576 -- 512 MB	scratch or data disk

For an OpenVMS Alpha system disk, pick `RZ29L`. Custom geometry remains available for a size the catalog does not cover, but a custom container still has to be DECLARED under some catalog profile, and that profile must be no larger than the container: an image smaller than its declared profile is REFUSED at attach. Larger is accepted and simply wastes the difference.

If the dialog cannot locate an emulator -- and therefore cannot read the catalog -- it falls back to a bundled geometry list and says so. That list is for offline authoring; it may lag the catalog and can name profiles the manifest will not accept. Point the system at its emulator binary and reopen the dialog rather than creating from the fallback.

Step 2 -- Place the OpenVMS distribution ISO

Copy the distribution ISO into the same `disks` folder. An ISO is SUPPLIED, never created: the optical device is read-only, so no creation path applies to it and Make Disk has no part in this step.

Step 3 -- Declare both in the platform manifest

Open the **Platform Editor** from the launcher (the Configure button in the bottom bar, or Open in PlatEd on the system's detail tab). It is a schema-validated manifest authoring tool with the drive catalog built in, and its JSON round-trip preserves field order and `comment` fields, so your annotations survive an edit.

1. Select the SCSI HBA (`pka_53c810`). On the id-0 row, set `model` to `RZ29L` and `media` to the file from Step 1.
2. Select the IDE controller (`cypress_ide`). On the ATAPI row, set `media` to the ISO from Step 2.
3. Save. Validation flags illegal rows -- SCSI id 7, duplicate ids, missing media -- before they ever reach a boot.

Manifest reference

The manifest is strict JSON. Storage rows live in a `storage` array inside the controller's entry under `pci_devices`. SCSI disks hang off the NCR 53C810 HBA (console `pka0`, disks `dka*`); the CD-ROM is an ATAPI row on the Cypress 82C693 IDE function (console `dqa*`).

field	meaning
channel	SCSI channel; 0 on the 53C810
unit	SCSI target id 0..6. Id 7 is the HBA itself -- never declare it
lun	0
type	<code>scsi_disk</code> , or <code>atapi_cdrom</code> for the optical row
model	drive profile key from the runtime catalog -- NOT the launcher's model name
media	bare filename, resolved against [Storage] diskDir. An absolute path is used as-is
media_kind	image for a disk, iso for optical media

The system disk at SCSI id 0, and the distribution CD:

```
{ "channel": 0, "unit": 0, "lun": 0, "type": "scsi_disk",
  "model": "RZ29L", "media": "system_disk.img",
  "media_kind": "image",
  "comment": "SCSI id 0 -> dka0: the OpenVMS system disk." }

{ "channel": 0, "unit": 1, "lun": 0, "type": "atapi_cdrom",
  "model": "EMULATR VIRTUAL CDROM", "media": "OpenVMS_v82.iso",
  "media_kind": "iso",
  "comment": "primary slave = dqa1, ATAPI CD. media empty = no disc." }
```

SRM unit naming for SCSI is `dka<id*100+lun>`: id 0 is `dka0`, id 1 is `dka100`, id 6 is `dka600`. IDE names follow the real-hardware `100*func+slot` convention -- slot 5 func 1 gives 105, so

the CD is `dqa1.1.0.105.0`. Only IDE channel 0 is declared today; `dqb` is a known gap. DS10 is IDE-only and has no SCSI rows -- there the system disk is an IDE row and boots as `dqa0`.

What not to touch

Leave `iic_devices` (platform badge and FRU identity), `pci_irq_table_hose0` (board interrupt routing, transcribed verbatim from console sources) and the `vendor / device / class_code / bars` fields of existing devices alone. These are board data, not user configuration. Interrupt pins are silicon constants owned by the device models -- never add an `interrupt_pin` field.

Step 4 -- Boot the distribution CD and install

Start the system from the launcher and open its console. At the SRM prompt, confirm the CD and the blank target both enumerated, then boot the CD:

```
P00>>> show dev
P00>>> b dqa1
```

`show dev` lists exactly the enumerated devices and prints the first token of each drive model. From there the standard OpenVMS installation dialog runs: choose the install option, select `DKA0:` as the target device, give the volume a label, and let the installer initialize the disk and restore the distribution save sets. Nothing in the procedure is EmulatR-specific -- follow the OpenVMS installation guide for the version you are installing.

Beta status. Booting an ALREADY-INSTALLED system disk is the exercised path and is on the Ready-for-Testing list. A full install from distribution media onto a blank target is not yet on that list. If the installation dialog stalls, keep the console log and report it, and use the migration route below in the meantime.

Step 5 -- Boot the installed system disk

Once the install completes, boot the disk directly:

```
P00>>> b dka0
```

The recommended interactive entry path is a conversational boot, which reaches a live DCL prompt on the console:

```
P00>>> b dka0 -fl 0,1
SYSBOOT> SET/STARTUP=OPA0:
SYSBOOT> C
```

That yields the version banner, the date and time dialog, and a root `$` prompt. To run the full startup afterwards, use `@SYS$SYSTEM:STARTUP FULL "VD"`, which runs to completion with DCL verification echoed and leaves a transcript in `SYS$SYSTEM:STARTUP.LOG`. To

restore normal automatic startup permanently, return to `SYSBOOT>` and issue `SET/STARTUP SYS$SYSTEM:STARTUP.COM` together with `SET STARTUP_P1 ""`.

Alternative -- migrate an existing system disk

A system disk built elsewhere (another emulator product or real Alpha hardware) can be adopted directly, and this is the best-exercised route today.

1. Copy the disk image file into the system's `disks` folder.
2. Point the SCSI id-0 row's `media` at it, with `model` set to a profile no larger than the image -- `RZ29L` for a 4 GB image.
3. Boot with `b dka0`.
4. On first boot, check that `SHOW LOGICAL SYS$TIMEZONE_DIFFERENTIAL` matches the site's real offset -- imported state travels with the disk.

Never attach the same image file to two running emulators at once. Disk images are single-writer; concurrent attachment corrupts the volume.

Adding data disks

Make the disk, declare it at a free SCSI id, boot, then initialize and mount it from DCL:

```
$ INITIALIZE DKA200: MYDATA
$ MOUNT/SYSTEM DKA200: MYDATA
```

Add the `MOUNT` to `SYSSTARTUP_VMS.COM` so the volume returns across boots. This is ordinary OpenVMS administration; consult the VMS system manager documentation for the details.

Creating disks from a script

Make Disk is the supported path and needs nothing installed. For scripted or repeatable provisioning on a development machine, `tools/mkdisk.py` is the scriptable equivalent. It resolves its size from the runtime catalog rather than the launcher's table, so its `--type` takes the same key you put in the manifest:

```
> python tools/mkdisk.py --type RZ29L disks/system_disk.img
> python tools/mkdisk.py --type EMULATR-512M disks/mydata.img
```

`--type` is required and the size is DERIVED from it -- you never pass a size. It requires a Python interpreter, which is why it is not the tester path.

Caveats

- *Warm reboot after a halt (Ctrl/P to >>> then b) faults the second boot deterministically today (kernel-stack-not-valid; tracked as JRN-KSNV-001). Exit EmulatR and relaunch for a fresh boot instead.*

- *Periodic auto-snapshot makes iteration cheap: `--autosnapshot 1000000000` mints a save every 1B cycles, and a later launch without `--no-autoload` restores a mid-session guest to the prompt in seconds.*
- *The IDE secondary channel (`dqb`) is not yet declared.*
- *Tagged queuing and reselection on the SCSI bus are later-phase work; the target set is command-queue-free by design.*
- *Catalog geometry is checked against a +/-2 percent band, never an equality assert -- these are banded (ZBR) drives whose published sectors-per-track is a flattened average.*
- *Silicon memory ceilings are enforced per platform (DS10 2 GB, DS20 4 GB, ES40 32 GB); an oversized memory request is clamped.*

See Also.

[Create Guest Disk Storage](#) ⁹⁶

[Virtual CDRom \(dqa1\)](#) ¹⁰⁴

[Virtual Disk Images](#) ¹⁰⁵

[SCSI Devices](#) ⁹⁴

[Platform Editor](#) ²⁵⁹

[Guest Boot](#) ¹¹⁰

[mkdisk](#) ²⁵¹

4.4.9 Virtual CDRom (dqa1)

`dqa1` ("EMULATR VIRTUAL CDRom") is the virtual optical device the firmware probes for option firmware during power-up. It is the ATAPI unit on the CY82C693 IDE channel -- channel 0, unit 1; `dqa0` (channel 0, unit 0) is the ATA hard disk. When no media is present it reports "no media present or is disabled via the RUN/STOP switch".

An ISO image is supplied (not created) through a manifest storage entry: an `atapi_cdrom` type with the `media` set to the ISO file. The optical device is read-only, so `create_if_missing` never applies to it. ISO contents are read through the ISO-9660 path (see ISO-9660 PFD).

Attaching an ISO

The CDRom is a storage entry in the platform manifest (`<model>_v7_3_platform.json`, for example `ds20_v7_3_platform.json`). It ships with an empty `media` (no disc). Point `media` at an ISO file to insert a disc; leave it empty for an empty drive:

```
{ "channel": 0, "unit": 1, "lun": 0, "type": "atapi_cdrom",
  "model": "EMULATR VIRTUAL CDRom", "media": "install.iso" }
```


Source References

- `<model>_v7_3_platform.json` -- *storage manifest; the `atapi_cdrom` entry (channel 0, unit 1) whose `media` names the ISO.*
- `deviceLib/Tsunami/AtaTaskfileEngine.h` -- *the ATAPI unit answering `dqa1` ("EMULATR VIRTUAL CDROM").*
- `deviceLib/scsi/VirtualIsoDevice.h` -- *the ISO media endpoint, read via the ISO-9660 path.*

See Also.

[ATAPI Devices](#) ⁹²

[ISO-9660 PFD \(CD001\)](#) ⁹⁴

[Create Guest Disk Storage](#) ⁹⁶

4.4.10 Virtual Disk Images

An emulated fixed disk is backed by a plain raw, flat image file: the byte at offset `LBA × sector-size` is the first byte of that logical block. The ATA fixed disk (device `dqa0`) uses 512-byte sectors and is read-write, so an operating-system installer can write to it and the writes persist.

Images are created from the geometry catalog -- automatically at boot via `create_if_missing`, or manually with `tools/mkdisk.py` (see [Create Guest Disk Storage](#)). The file is bare: no partition table or DEC disklabel until the guest install writes them.

Attaching an image

A manifest storage entry names the channel, unit, type, and the `media` file. With `type = ata_disk` the image is opened as a 512-byte read-write block device on the CY82C693 IDE channel and appears to the firmware as `dqa0`. On launch the boot log confirms the attach:

```
Storage: attached ATA disk 'dqa0.img' to IDE ch0 unit0
```

Note: a blank image is not bootable on its own -- an operating system is installed onto it (booted from an ISO on the optical device). Enumeration of a bootable disk is in progress; see [Current Development Status](#).

See Also.

[Create Guest Disk Storage](#) ⁹⁶

[Virtual CDROM](#) ¹⁰⁴

[Current Development Status](#) ²¹

4.4.11 Virtual Floppy (dva0)

`dva0` ("EMULATR VIRTUAL FLOPPY") is not implemented.

4.5 4.5 Snapshots & Boot Acceleration

Automatic snapshots

Periodic `auto_*.axpsnap` saves are on by default. Disable them for clean cold-boot / profiling runs:

```
--autosnapshot off # or EMULATR_AUTOSNAP=off
```

On startup the newest snapshot is auto-loaded unless you force a cold boot:

```
--no-autoload # or EMULATR_NO_AUTOLOAD=1
```

Disabling autosnapshot on mint cold-boot runs avoids the per-save disk cost.

Creating snapshots

Two capture mechanisms beyond the automatic saves:

- - Console marker -- arm with `EMULATR_CONSOLE_SNAPSHOT=1`; when a console input line matches the marker (default "set oem_string snapshot"), a `snapshots/predig_oemsnap_cyc<N>.axpsnap` is written. Change the marker with `EMULATR_SNAPSHOT_MARKER` (leading / trailing whitespace trimmed; internal whitespace is NOT normalized).
- PC-triggered -- `--snapshot-on-pc <pc>[...]`, optionally named with `--snapshot-name-tag <S>`.

Restoring snapshots

By default, startup restores the newest snapshot automatically (autoload-newest). To ignore snapshots and force a genuine cold boot, launch with `--no-autoload` (`EMULATR_NO_AUTOLOAD=1`). Snapshot files are resolved in the CWD-relative `snapshots/` directory, so launch from the EmulatR root for the canonical set.

Pruning snapshots

Pruning of old snapshots is off by default -- without the gate, `pruneOldSnapshots` is a no-op and nothing is evicted. Enable it with:

```
EMULATR_SNAPSHOT_PRUNE=1
```

Boot-acceleration snapshot

A specialized use of this machinery mints a snapshot at the firmware's decompression handoff and reuses it to skip the decompressor on later runs, under the `EMULATR_FAST_DECOMPRESS` lever. It is documented separately -- see Boot Acceleration - Entry Snapshot.

Source References

- `systemLib/Snapshot.h`, `systemLib/Snapshot.cpp` -- *save / load / autoloadLatest; checksum + version + chipset-variant validation; pruneOldSnapshots.*

- `systemLib/Machine.cpp -- armSnapshotOnPc / addSnapshotOnPc` (PC triggers), console-marker + periodic saves in `systemTick`, `enableEntrySnapshot`.
- `main.cpp -- --snapshot-on-pc / --autosnapshot / --no-autoload` handling and the `autoload-newest` call.

See Also.

[Boot Acceleration - Entry Snapshot](#) ¹⁰⁷

[Launching EmulatR](#) ⁵⁰

[Executing Two Instances](#) ⁷⁵

[Environment Variables](#) ²¹⁶

4.5.1 Boot Acceleration - Entry Snapshot

EmulatR can skip re-running the guest self-decompression on a subsequent run by capturing a complete-machine snapshot at the firmware's own init-to-console handoff and restoring it. The handoff is the point where the decompressor stub transfers control into the decompressed console -- the entry PC the image declares, `entryPa = targetPalBase + finalPC`, the same point the decompressor internals describe. The feature is governed by the `EMULATR_FAST_DECOMPRESS` environment lever and is OFF by default: with it unset EmulatR always runs the faithful decompressor and is the Oracle.

The lever

Set `EMULATR_FAST_DECOMPRESS` before launching:

- *unset or faithful* -- the Oracle default: run the guest decompressor on every init.
- *snapshot (or 1)* -- on the first run, decompress faithfully and MINT a snapshot at `entryPa`; on a later run, REUSE that snapshot to skip the decompressor.

The convenience wrapper `tools/es40_fast_decompress.sh {snapshot|faithful}` sets the lever and runs the ES40 harness; the same lever works for any SRM firmware (see Model coverage below).

How it works

The mint fires once, from the same translated-physical-address handoff detection the console-relocation trigger uses (so it is immune to the PAL-mode bit that a raw PC compare would miss). At that instant the machine is quiescent -- the previous instruction has retired and its writeback is resolved into the CPU state -- so the snapshot is a complete, consistent image: the full CPU state (all registers and IPRs, PC, PAL_BASE, mode), the page-sparse guest memory at its true addresses, and the chipset state. The image the guest decompressor itself produced is captured as-is; EmulatR does not recompute the decompressed layout.

On a subsequent run the snapshot is validated and restored, and the boot re-enters at entryPa without re-running the decompression.

The snapshot file

The snapshot is written next to the firmware, named for it: `firmware/<stem>.axpsnap` (for example `firmware/es40_v7_3.axpsnap`). The name binds the machine state to the exact firmware image it was decoded from. It is a per-run performance artifact: it survives a normal exit, a clean shutdown, and an abort so it remains diagnostically available, and it is regenerated (overwritten) only when a run reaches entryPa and finds no VALID snapshot already in place.

Determinism and validation

A restored snapshot must match the run that consumes it, or determinism is lost. EmulatR enforces this with the existing snapshot validator before any restore: a stored checksum plus the CPU-state and chipset format versions plus the chipset VARIANT. Any mismatch -- a rebuild bumped a version, the file is truncated or corrupt, or it belongs to a different platform -- fails the load cleanly and the run falls back to the faithful decompressor and re-mints. A stale-code snapshot can therefore never be silently consumed. Determinism is broken only by a change the validator catches (a code rebuild) or by an input change; it is NOT necessary to delete the file between runs.

Restore-except-flash

The snapshot carries the TIG flash / NVRAM image, but a reuse re-points the flash to the CURRENT backing after restore. This means live environment changes -- a `set` at the `>>>` prompt, or an `update srm`, persisted to `<stem>.rom / ds10_flash.rom` -- are honored on the next run, not reverted to the snapshot's capture-time environment. Real flash is nonvolatile; the restore treats it that way.

Model coverage

The lever lives in the shared SRM load path, so it applies uniformly to DS10, DS20, and ES40. Two independent layers keep models isolated: each firmware keys its own `<stem>.axpsnap`, and the chipset-variant check in the validator blocks an ES40 (Typhoon) snapshot from ever restoring into a DS10 / DS20 (Tsunami) machine. Cross-model, wrong-build, or corrupt snapshots all resolve to a faithful decompress, never to silent corruption.

What it does and does not accelerate

The entry snapshot skips the self-decompression -- the roughly four-million-cycle spin at PC `0x60111c` -- and the boot up to entryPa. It does NOT by itself skip the post-entryPa silicon-mode initialization (the long real-hardware timing grind); that is addressed by the separate warp knobs. The two are complementary levers.

See Also.

[Host Decompressor](#) ¹⁵⁸

[Automatic / Creating / Restoring](#) ¹⁰⁶

[Env Vars Operation Ref](#) ²³⁴

4.6 4.6 Firmware Update Utility (LFU)

The Loadable Firmware Update (LFU) utility runs during power-up -- after the option-firmware scan finds nothing on CD or floppy, the firmware drops to the `UPD>` prompt. From there you can list and update the firmware images in flash. `exit` leaves LFU (a reset) and continues to the console. On the ES40 in silicon mode this exit reset drives the (experimental) TIG module reset -- see the ES40 TIG Module Reset topic.

In this section

- *-LFU Commands -- the menu, List, Display, and Update.*

Considerations.

- The LFU utility is a mandatory fork during SRM initial boot;
- The LFU utility, the SRM should be updated at least one time (LFU> update srm);
- The LFU is loaded from <firmware>.rom during subsequent SRM initialization boot.
- It is acceptable to enter "LFU> exit" during subsequent SRM initialization boot.
- The LFU ROM is updated during [Clean Shutdown](#) ⁶¹ only.

See Also.

[ES40 TIG Module Reset](#) ¹⁷⁷

[LFU Commands](#) ¹⁰⁹

[Reaching the SRM Prompt](#) ⁶⁰

[Example - Performance](#) ⁷⁵

[Clean Shutdown](#) ⁶¹

4.6.1 LFU Commands (List / Display / Update)

At the `UPD>` prompt the utility presents a function menu. `?` or `Help` scrolls it:

```
***** Loadable Firmware Update Utility *****
Function Description
Display Displays the system's configuration table.
Exit Done exit LFU (reset).
List Lists the device, revision, firmware name, and update revision.
Readme Lists important release information.
Update Replaces current firmware with loadable data image.
Verify Compares loadable and hardware images.
```

List

`list` shows each device with its current revision, firmware filename, and the revision available to update to:

```
UPD> list
Device Current Revision Filename Update Revision
nt Unknown nt_fw 5.71
srm 7.3-1 srm_fw 7.3-1
```

Display

`display` prints the system configuration table (the same PCI / ISA inventory shown by `show config` at the console). See `show config` and the Example - Performance transcript for the full layout.

Update

`update <device>` replaces the named device's firmware in flash. Updating `srm` refreshes the SRM console image; an unknown device name is rejected ("not supported under LFU").

```
UPD> update srm
Confirm update on:
srm [Y/(N)]y
WARNING: updates may take several minutes to complete for each device.
DO NOT ABORT!
srm Updating to 7.3-1... Verifying 7.3-1... PASSED.
```

When done, `exit` leaves LFU (a reset) and the firmware continues to the `>>>` prompt.

See Also.

[show config](#) ↗ 89

[Example - Performance](#) ↗ 75

[Firmware Requirements](#) ↗ 44

4.7 4.7 Guest Boot

From the SRM console prompt (`>>>`) you boot a device with `boot <device> [-fl root, flags]`. The device is a console device name; the optional boot flags select how the operating system bootstraps. Three SRM environment variables supply the defaults used when you type `boot` with no arguments.

Device names

`dqa0` is the IDE primary master (the ATA fixed disk); `dqa1` is the IDE primary slave (the ATAPI CD). `dka0` is the first SCSI disk; `ewa0` is the first Ethernet adapter (network boot). Booting the ATAPI CD (`dqa1`) is how an installer is started; booting `dqa0` boots the installed system.

SRM environment variables (the boot register settings)

Set with `set`, inspect with `show`. These persist in NVRAM across reboots.

variable	purpose	example
bootdef_dev	default boot device for a bare boot	set bootdef_dev dqa0
boot_osflags	default OS boot flags (root,flags for VMS)	set boot_osflags "0,0"
auto_action	action at power-on: HALT, BOOT, or RESTART	set auto_action HALT

Boot flags -- OpenVMS

OpenVMS flags take the form `root,flags`. `root` is the system root (`SYS0` .. `SYSn`, a single hex digit). `flags` is a hex bitmask; the load-bearing bit for interactive use is bit 0 (value 1), which requests a CONVERSATIONAL boot and stops at the `SYSBOOT>` prompt.

command	meaning
<code>boot dqa0 -fl 0,0</code>	normal boot from root <code>SYS0</code>
<code>boot dqa0 -fl 0,1</code>	conversational boot from <code>SYS0</code> -- stops at <code>SYSBOOT></code>
<code>boot dqa0 -fl n,1</code>	conversational boot from root <code>SYSn</code>

The flags field (the second value of `root,flags`) is a bitmask, combined by addition: 1 = conversational boot (stop in `SYSBOOT`), 2 = load XDELTA (debug), 4 = take the initial XDELTA breakpoint. Combinations follow: 3 = XDELTA + conversational, 6 = XDELTA + initial breakpoint, 7 = all three. On Alpha the full field is a 64-bit qualifier passed to the primary bootstrap (`APB.EXE`); for the complete platform set consult the OpenVMS Alpha Upgrade and Installation manual.

Conversational boot and SYSBOOT> (OpenVMS)

The bootstrap sequence: `VMB.EXE` (the primary bootstrap, on the boot disk) locates `SYS$SYSTEM:SYSBOOT.EXE` (the secondary bootstrap), loads it, and starts it. With the conversational flag set, the boot stops after loading `SYSBOOT.EXE` and displays `SYSBOOT>`. There you use `SYSGEN SET` and `SHOW` to adjust system parameters (the default parameter file on Alpha is `SYS$SYSTEM:ALPHAVMSSYS.PAR`), then `CONTINUE` to complete the boot.

```
>>> boot dqa0 -fl 0,1
... SYSBOOT>
SYSBOOT> SET WRITESYSPARAMS 0
SYSBOOT> CONTINUE
```

Boot flags -- Tru64 UNIX

Tru64 (OSF/1) also boots from the SRM console, but it has no `SYSGEN` conversational prompt; its analog is single-user mode. Tru64 uses letter flags stored in `boot_osflags` (or passed with `-fl`, which overrides the stored default): `a` = normal multi-user boot, `s` = single-user shell, `i` = interactive (prompt for an alternate kernel, then continue). So `boot dqa0 -fl s` brings up the single-user shell, from which the kernel and startup can be controlled before going multi-user.

Install from CD (the EmulatR acceptance path)

The standard install flow uses the ATAPI CD to boot an installer that writes onto the blank ATA disk, after which the installed disk is bootable:

```
>>> boot dqa1 (boot the installer from the CD / ISO)
... installer writes the system onto dqa0
>>> boot dqa0 (boot the installed system)
```

Requirement: a complete PCI interface

Reaching `SYSBOOT>` (and the OS device configuration that follows) requires the EmulatR PCI interface to be complete -- config cycles, BAR assignment, and dynamic BAR-to-range rebind. The SRM console probe is enough to enumerate `dqa`, but VMB, `SYSBOOT`, and the early OS adapter initialization traverse PCI config space to reach the boot device. Until PCI is complete, a conversational boot cannot be expected to reach `SYSBOOT>`. See Current Development Status.

See Also.

[Create Guest Disk Storage](#) ^{→ 96}
[boot](#) ^{→ 90}
[Current Development Status](#) ^{→ 21}

4.8 4.8 Logging & Diagnostics

EmulatR's diagnostics are driven by runtime `EMULATR_*` trace gates and a built-in retire profiler. Most output goes to the process's standard error, so a run is captured by redirecting `stderr` to a log (`2> run.log`).

In this section

- *Diagnostics & Debugging* -- the trace gates, probe / break gates, and the compile-time diagnostics.
- *SRM Performance Logging* -- where profiling output is written.

See Also.

[Diagnostics & Debugging](#) ^{→ 113}
[Environment Variables](#) ^{→ 216}
[Executing](#) ^{→ 74}

Developer.

[Diagnostic-Logging Subsystems](#) ^{→ 116}

4.8.1 SRM Performance Logging

Performance-run output depends on which "performance run" you mean.

EmulatR's own profiler

The `RetireProfiler` is an always-on retire-PC histogram in the `PipelineDriver`. It writes `profile_YYYYMMDD-HHMMSS_<tag>.txt` to the same resolved trace directory as the other sinks `-- EMULATR_RETIRE_TRACE_DIR` (the build-tree default otherwise). It moves automatically when that directory is overridden.

External profiling tools

A Software Verify Performance Validator (PV) run is not written by EmulatR -- that is an external tool profiling `Emulatr.exe`. Its session and results go wherever PV saves them (its `saveSession` path, or PV's default session folder under your Documents). Nothing in the codebase controls that; it is a PV-side setting.

See Also.

[Diagnostics & Debugging](#) ¹¹³

[Example - Performance](#) ⁷⁵

4.8.2 Diagnostics & Debugging

Diagnostics are turned on with runtime `EMULATR_*` environment variables, set in the launching shell before the process starts. The complete table lives in the Reference section's Environment Variables page; this page groups them by what they do.

Three gotchas to know first

- *Presence, not value.* Most gates test only that the variable is set, so `VAR=0` still *ENABLES* them. The few that parse a value (the *autosnapshot / marker / path / numeric knobs*) are noted in the reference.
- *Read once.* The trace and probe gates cache their result on first call, so they are effectively read at process start; exporting them mid-run has no effect.
- *Paths are CWD-relative.* Launch from the Emulatr root so relative output and snapshot paths resolve to the canonical set.

Device / subsystem traces

Presence-gated, read once, written to stderr: `EMULATR_IIC_TRACE` (PCF8584 IIC bus transactions), `EMULATR_FLASH_TRACE` (FlashRom command / erase / write activity), `EMULATR_PIC_TRACE` (the 8259 PIC pair -- IRQ assert / EOI). Redirect to per-trace logs, for example `2> iic.log`.

Chipset diagnostics and retire trace

The chipset CSR diagnostic stream is on by default; `EMULATR_CHIPSET_DIAG_OFF` silences it (note the inverted sense -- setting it turns diagnostics OFF). `EMULATR_TRACE_WINDOW` forces the retire-compact trace window, and `EMULATR_RETIRE_TRACE_DIR` sets the output directory for the retire / breakpoint / profiler trace files (see SRM Performance Logging).

Probe and break gates

For pinpoint debugging: `EMULATR_GCT_WATCH` (store-watch on the GCT / FRU config-tree region), `EMULATR_BREAK_ON_PA10` (conditional break on accesses to PA 0x10..0x17), `EMULATR_CHECKPOINTS` (named PC checkpoints), and `EMULATR_GATE` (a PC / condition gate controlling trace windowing). The ES40 bring-up probes -- `EMULATR_TIG_RESET`, `EMULATR_UDELAYWARP`, `EMULATR_TIG_TRACE`, and `EMULATR_RSCC_DIAG` -- are gated behind the compile-time `EMULATR_BRINGUP_PROBES` and are experimental.

Compile-time diagnostics (NOT runtime env vars)

Some `EMULATR_*` names are CMake options baked in at build time -- setting them in the shell does nothing. The important one is `EMULATR_MEMDIAG`: a per-retired-instruction `fprintf+fflush` that imposes roughly a 900x slowdown and must stay OFF for normal use. `EMULATR_DIAGNOSTIC_LOGGING` (centralized per-subsystem logging, non-Release builds) and the `EMULATR_LOG_*` `spdlog` level macros are also build-time.

For traced-boot command recipes, see Executing.

See Also.

[Environment Variables](#) ²¹⁶

[SRM Performance Logging](#) ¹¹³

[Executing](#) ⁷⁴

4.8.3 Bring-Up Probes (EMULATR_BRINGUP_PROBES)

Over hardware bring-up, many one-off probe prints were left in the run path -- per-step CPU samplers, memory watchpoints, PAL-transition traces, and end-of-run profiler dumps -- emitted as raw `fprintf(stderr)` on the hot loop. They are gathered behind a single COMPILE gate, `EMULATR_BRINGUP_PROBES`, so that when it is off they cost nothing (the code is not compiled at all, not merely skipped at run time). The gate defaults OFF: a normal build emits only initialization messaging, snapshot notices, and genuine error messages.

What it looks like when ON

If the probes are compiled in, the console / `stderr` carries lines such as:

```
PCSAMPLE cyc=67108864 pc=0x60093c pal=0 ra=0x6014bc
[HW_REI XITION #254] pc=0x...600351 REGISTER Rb=23 rawTarget=0x...6009a4 PAL->native e
```

Seeing these in a build you expected to be quiet -- including a `release` build -- means the gate is ON (see the cache note below).

Enabling and disabling

The gate is a CMake option, toggled at configure time from the repository root, then picked up on the next build:

Action	Command (from the repo root)
Enable (debug / relwithd ebinfo)	<code>cmake -DEMULATR_BRINGUP_PROBES=ON . && tools/build_emulatr.sh</code>
Disable	<code>cmake -DEMULATR_BRINGUP_PROBES=OFF . && tools/build_emulatr.sh</code>
Check the current cache value	<code>grep EMULATR_BRINGUP_PROBES CMakeCache.txt</code>

The cache gotcha

`EMULATR_BRINGUP_PROBES` is a `CACHED` CMake option. Its OFF default only applies the FIRST time a build tree is configured; once you have set it (for example - `DEMULATR_BRINGUP_PROBES=ON` during debugging), that value is remembered in `CMakeCache.txt` and persists across every subsequent build until you explicitly reconfigure with the opposite value. The build helper `tools/build_emulatr.sh` does NOT reset it. So if a release build is noisy, the fix is to turn the cached option off and rebuild:

```
cmake -DEMULATR_BRINGUP_PROBES=OFF .
tools/build_emulatr.sh release
```

As a rule, keep the gate OFF for any release or distribution build; enable it only in a `debug` or `relwithdebinfo` tree while actively chasing a bring-up issue, and turn it back off when done.

See Also.

[Diagnostics & Debugging](#) ¹¹³
[CMake Macro Guards](#) ¹³⁰
[Environment Variables \(EMULATR_*\)](#) ²¹⁶
[Building from Source](#) ⁴⁵

4.8.4 Logging Subsystems

EmulatR routes its diagnostic output through a small set of named subsystems. Each is the `<subsys>` argument accepted by the logging command-line options: `--log-disable`, `--log-only`, `--log-verbose`, and `--log-file`. Names are case-insensitive (Cbox, cbox, and CBOX all match). The facility is compiled in only when the binary is built with `EMULATR_DIAGNOSTIC_LOGGING=ON`; otherwise these options have no effect.

Subsystem	Default Throttle	What It Logs
Cbox	Loud-then-summary	EV6 Cbox (cache / system-bus interface) events -- the C_DATA / C_SHFT register chains, error registers, and MCHK-related Cbox polling.
Unalign	Loud-then-summary	Unaligned load/store emulation -- the fix-up path for misaligned memory accesses.
IntervalTimer	Loud-then-summary	Cchip interval-timer / clock-tick diagnostics -- tick scheduling and timer-interrupt delivery.
Snapshot	Unthrottled	Machine-state save / restore -- one line per .axpsnap save or load.
PalRelocation	Unthrottled	PALcode base relocation during cold boot -- the palBase move (one-shot per boot).
ChipsetCsr	Loud-then-summary	Tsunami / Titan chipset CSR accesses -- Cchip / Dchip / Pchip / TIG control & status register reads and writes (heavy during hardware bring-up).
StepD	Unthrottled	The SRM firmware-load "Step D" relocation step (one-shot per cold boot).
Misc	Loud-then-summary	Default catch-all for diagnostic streams without a dedicated subsystem.

Throttle Policies

Each subsystem has a default throttle that bounds how much it emits to stderr. The default-on policies keep a long run readable while still surfacing the first events.

`Unthrottled` – every event is emitted.

`Loud-then-summary` – the first 16 events emit in full, then one INFO heartbeat per 256K events.

`Loud-then-mute` – the first 32 events emit, then the subsystem goes silent on stderr.

`Off` – never emits to stderr (a configured file sink may still receive writes).

`--log-verbose <subsys>` forces a subsystem to `Unthrottled`; `--log-disable <subsys>` turns it `Off`; `--log-only <csv>` turns everything `Off` except the listed subsystems.

Examples

```
./tools/run_ds10.sh -- --log-only ChipsetCsr,Cbox
./tools/run_ds10.sh -- --log-disable IntervalTimer
./tools/run_ds10.sh -- --log-verbose Snapshot
./tools/run_ds10.sh -- --log-file ChipsetCsr=traces\csr.log
```

See Also: [Command Line Options \(EmulatR\)](#) ²⁷⁷ [Environment Variables \(EMULATR_*\)](#) ²¹⁶

5 5. Architecture & Internals

How EmulatR is built and how a cold boot actually runs. These pages are the operator lens; it presents enough detail to reason about the machine without reading the source. The register-level detail lives in the Appendices' Architecture Deep-Dives.

In this section

- *Overview -- the fidelity standard, the goals, the execution model, and the device fabric.*
- *Memory -- the GuestMemory physical store and the chipset arbiter.*
- *Addressing -- EV6 translation and the SPAM translation buffers.*
- *Firmware Decode & Decompression -- how the SRM image unpacks at cold boot.*
- *Execution Flight Plan -- the cold-boot sequence end to end.*
- *Clock Frequency MHz*
- *Chipset*

See Also.

[Overview](#) → 118

[Memory](#) → 132

[Execution Flight Plan](#) → 122

[Clock Frequency MHz](#) → 198

[Chipset](#) → 139

5.1 Overview

EmulatR is a full-system, high-fidelity emulator of the DEC Alpha AXP platform -- specifically the 21264 (EV6) processor and the 21272 (Tsunami / Typhoon) core logic as integrated in AlphaServer DS10 / ES40 / ES45-class systems. It is implemented in modern C++ (C++20) with a deliberately framework-free architectural core (Qt is confined to the periphery -- console UI and host glue), built with CMake. Its purpose is not to approximate an Alpha; it is to be one, faithfully enough that unmodified DEC SRM console firmware, PALcode, and the operating systems that ran on the metal run on EmulatR without knowing the difference.

This page describes the architecture of the current implementation (V4) -- the execution model, the memory and translation system, the privileged boundary, and the device fabric -- and the standard every subsystem is measured against. The chapters that follow expand each area; the deep-dive appendices carry the register-level detail.

The fidelity standard and its authoritative sources

EmulatR is governed by one standard: it must behave like real Alpha hardware in every architecturally visible way, even when that complicates the implementation. Correctness is defined by primary sources -- the **Alpha Architecture Reference Manual** and the original **DEC SRM console and PALcode sources**. These define what correct behavior is.

The test is black-box and adversarial, conducted by the most demanding guests available: the SRM console and PALcode exercise the privileged architecture directly, and a booting operating system exercises the memory model, the trap machinery, and the device fabric under real workloads. A second emulator (AXPBox) is consulted only as a convenience cross-check for differential triage -- it is explicitly **not** an authoritative source, because another implementation can be wrong, whereas the architecture manual and the firmware sources cannot. If a guest can distinguish EmulatR from an AlphaServer through architecturally defined behavior, the emulator is wrong.

Primary goals and objectives

- **Architectural correctness first.** EmulatR models Alpha's architectural semantics, not a particular chip's timing. Correctness always outranks speed.
- **Deterministic, debuggable execution.** Execution is reproducible and inspectable. The run loop is staged and cycle-counted, and the hot path is deliberately shallow so a stack trace at any point stays short and legible.
- **Explicit, per-instruction semantics.** Every instruction resolves to one faithful grain leaf; behavior is never inferred from an operand-shape family. Architectural results and exceptions become visible only at the Writeback stage.
- **Separation of concerns.** Decode, integer, floating-point, memory, control, and privilege are isolated into well-defined functional domains -- the "Boxes" -- with one-way dependency flow. No domain reaches into another's internal state.
- **SMP-aware by construction.** Per-CPU state, CPU-local translation buffers, and load-locked / store-conditional reservation tracking are built in, so a multiprocessor build is an extension rather than a rewrite (see the scope note below).

The execution model

Execution is driven by `PipelineDriver` (pipelineLib) through six in-order stages: `IF` (instruction fetch), `DE` (decode / dispatch), `GR` (grain-ready: read the register file into the execution context), `EX` (execute), `MEM` (memory drain), and `WB` (writeback). Each instruction is carried through the six stages on a `PipelineSlot`; architectural state -- the register file and the PC -- is committed only at `WB`, the single well-defined commit point that makes exceptions precise and replay deterministic.

Dispatch is flat. At `DE` a decoded instruction is resolved through the grain system to a single grain leaf -- a pure function with the signature `BoxResult(InstructionGrain const&, ExecCtx const&)` -- that implements its semantics. Execution is partitioned into the functional Boxes, each a namespace of grain leaves rather than a class hierarchy: **IBox** (control flow -- branches, jumps), **EBox** (integer ALU), **FBox** (floating point, IEEE and VAX,

over the fpBox backend), **MBox** (loads, stores, load-locked / store-conditional), **CBox** (branch prediction and target resolution), and **PalBox** (the privileged boundary). Dependencies flow one way: every Box depends on coreLib; most also on iprLib; MBox additionally on memoryLib.

A deliberate consequence is that the hot path is shallow -- from the driver, through the stage, into a grain leaf, the call-stack depth is typically two and rarely exceeds four. That keeps traces legible (serving debuggability) and minimizes the per-instruction scaffolding that profiling has shown -- rather than the opcode bodies -- to dominate runtime (serving performance without compromising correctness). The grain leaves are the leaves of that call tree by construction.

Dispatch tables are generated, not hand-maintained. `GrainMasterV4.tsv` is the single source of truth for instruction semantics; the `genGrains.py` codegen reads it and emits the dense primary table (indexed by the 6-bit opcode) plus the sparse secondary tables for multi-function opcodes, including the personality-aware PALcode dispatch (OSF/1 and VMS). There is no runtime trace cache or JIT; the generated tables are the decode artifact.

Memory and address translation

Physical backing. `GuestMemory` (memoryLib) is the single physical-memory interface -- a passive byte-store that pages the guest's physical address space in on demand, in 64 KiB pages, committed only as the guest first touches them, so a multi-gigabyte machine costs only its working set. The pages are backed by host `VirtualAlloc` on Windows and by `mmap` on Linux / ARM hosts (with a portable fallback). The earlier SafeMemory layer has been removed. `GuestMemory` itself does no MMIO routing or fault generation: the `TsunamiChipset` system-bus arbiter decodes addresses, routes device traffic, and raises range faults before raw RAM is ever touched. A `LockMonitor` tracks per-CPU load-locked / store-conditional reservations at cache-line granularity.

Guest translation. Address translation is performed by `Ev6Translator` (mmuLib) as the EV6 architecture specifies: PAL-mode bypass, the canonical virtual-address window check, and superpage (kseg) mapping, with non-mapped accesses returning an ITB or DTB miss. The translation buffers are modeled by the **SPAM** cache -- Set-Partitioned Associative Memory -- as `SPAMShardManager` instances resident in `CpuState` (`itbMgr` and `dtbMgr`, each 16 shards by 8 ways = 128 entries). Invalidation is epoch-based and lazy (`TlbEpoch`): a TBIA bumps a global counter and a TBIAP a per-process one, so a flush is O(1) and stale entries simply fail the liveness test at lookup. The translation buffers are CPU-local by design.

Faults, exceptions, and the privileged boundary

There is no central fault dispatcher. A leaf, or the memory drainer that detects a fault, sets a `faultCode` on the instruction's `BoxResult` (for example a DTB miss or an unaligned access). The hardware exception entry points are compile-time constants in `Ev6EntryVectors.h` (the vectors at `0x100` through `0x780`, spaced `0x80` apart). At `WB` the driver inspects the fault:

a halt sets the CPU's halted flag; otherwise, once `PAL_BASE` is established, control diverts to the computed entry vector. Privileged operations -- `CALL_PAL`, `HW_MTPR` / `HW_MFPR`, `HW_REI` -- are handled in `PalBox`, which applies IPR side-effects (`PAL_BASE` normalization, TLB invalidation, ASN context) inline. Internal processor registers are flat members of `CpuState`.

The machine and its device fabric

`Machine` (`systemLib`) is the top-level orchestrator: it owns the `CpuState`, the `GuestMemory`, and the `TsunamiChipset`, and provides firmware load, reset, and the step / run entry points. The chipset aggregates the three 21272 components -- the **Cchip** (system control, memory arbitration, interval timers, DRAM array configuration), the **Dchip** (data path), and the **Pchip** (PCI host bridge) -- together with the on-board devices. The PCI fabric is a standard bus / slot / function hierarchy (there is no separate "stratum" abstraction).

The southbridge is the **Cypress CY82C693** PCI-to-ISA bridge (`Cypress_CY82C693ISABridge`), which carries the ISA / SuperIO plane. The modeled devices include the CY82C693 IDE / ATAPI channel, the SuperIO controller (FDC / COM / LPT), the TOY / RTC, a 16550 UART for the serial console, the PCF8584 IIC controller, the flash / SRAM, and the dual 8259A interrupt controllers (`Pic8259Pair`). SCSI endpoints (`VirtualScsiDevice` and the ISO-CD variant) exist in the device library.

Lineage: four implementations, one set of objectives

The current emulator is the fourth implementation; the objectives above were fixed from the start, and each version reached for them by a different mechanism.

- *V1 dispatched each instruction through a virtual call to a per-instruction grain. It was correct and booted SRM, but the per-call overhead capped throughput.*
- *V2 chased performance by dispatching on instruction families (operand-shape signatures). It was fast, but families under-determined behavior -- instructions sharing a signature still needed per-instruction corner-case handling the model could not express.*
- *V3 tried to hybridize V1 and V2; the coupling was too tight and introduced silent-failure modes -- the worst outcome for an emulator whose value is fidelity.*
- *V4 keeps a flat dispatch path but resolves it to faithful per-instruction grain leaves via generated tables, with performance pursued as the sum of many marginal gains rather than one architectural bet. The dispatch mechanism evolved across all four; the objectives did not.*

Scope note: this page describes the V4 architecture and the standard it is held to. Some elements are infrastructure ahead of full activation -- multiprocessor execution and cross-CPU IPI, full PAL trap delivery, and SCSI / PCI device enumeration. For what is implemented and working in the current build versus what is still in progress, see [Current Development Status](#).

Source References

- `pipelineLib/PipelineDriver.h` -- *six in-order stages, WB single commit point.*

- `coreLib/InstructionGrain.h` -- *grain leaf signature over BoxResult / ExecCtx.*
- `systemLib/Machine.cpp` -- *top-level orchestrator owning CpuState/GuestMemory/chipset.*
- `chipsetLib/TsunamiChipset.cpp` -- *Cchip/Dchip/Pchip aggregate plus device fabric.*
- `grainFactoryLib/codegen/genGrains.py` -- *generated dense/sparse dispatch tables.*
- `coreLib/CpuState.h` -- *per-CPU state, IPRs, and the TLB managers.*
- `mmuLib/Ev6Translator.h` -- *virtual-to-physical address translation.*

See Also.

[Memory](#) ^{↗ 132}

[Addressing](#) ^{↗ 133}

[Execution Flight Plan](#) ^{↗ 122}

[Current Development Status](#) ^{↗ 21}

5.2 5.1 Execution, Decode & Internals

Developer-reference notes on individual subsystems: how floating point is approached, how the grain dispatch machinery is generated from tables, and how build-time behavior is gated by compile guards. These complement the operator-lens Architecture & Internals chapter and the register-level deep-dives in 8.3.

In this section

- *Floating-Point Considerations* -- the four formats, the FP-enable gate, FPCR, and the backend strategy.
- *grainFactoryLib* -- the table-driven grain codegen.
- *DispatchTables* -- the primary and per-kind dispatch tables that bind an opcode to a leaf.
- *CMake Macro Guards* -- the build-time options and diagnostic compile guards.

See Also.

[Architecture Deep-Dives](#) ^{↗ 131}

[Architecture Overview](#) ^{↗ 118}

5.2.1 Execution Flight Plan

A cold boot is a fixed sequence of stages from reset to the interactive prompt. Each instruction along the way is carried through the six in-order pipeline stages (IF, DE, GR, EX, MEM, WB) and commits at Writeback (see Overview); the stages below are the firmware-level milestones.

The stages

- **Reset / SRM hand-off.** The machine resets with the SRM parameters in registers R16-R21 and enters the firmware startup.
- **PAL and CPU setup.** The startup establishes `PAL_BASE`, sets the `I_CTL` SDE bits, flushes the ITB / DTB and instruction cache, and primes the shadow registers.
- **Decompression.** The firmware inflates its own compressed image (about 4M cycles; see Firmware Decode & Decompression).
- **Relocation.** A memory-copy moves the decompressed firmware to its run location and jumps to the SRM console entry.
- **SRM power-up.** Banner and memory sizing; the PCI / ISA probe; GCT / FRU initialization; the option-firmware scan of the CD and floppy; the Loadable Firmware Update utility (`UPD>`); system and disk testing; and finally the interactive `>>>` prompt. (That visible sequence is detailed in Reaching the SRM Prompt.)

Where the flight currently ends

The cold boot reaches the `>>>` prompt and is fully interactive. The remaining runtime work downstream of decompression centers on a clock-interrupt return path (the `hw_ret` R2 sequence) and, for booting a guest, the front-panel halt gate and PCI device enumeration. Those are tracked in Current Development Status.

Source References

- `pipelineLib/PipelineDriver.h` -- *six-stage IF/DE/GR/EX/MEM/WB loop, WB commit and fault divert.*
- `systemLib/Machine.cpp` -- *reset / firmware-load / step / run orchestration.*
- `systemLib/FirmwareLoader.cpp` -- *SRM/firmware image load and entry hand-off.*
- `coreLib/Ev6EntryVectors.h` -- *exception vectors reached once `PAL_BASE` is set.*
- `pipelineLib/MemDrainer.h` -- *decompression-window read intercepts along cold boot.*

See Also.

[Overview](#) ¹¹⁸

[Firmware Decode & Decompression](#) ¹⁵⁷

[Reaching the SRM Prompt](#) ⁶⁰

[Execution / Boot Internals \(appendix\)](#) ¹²³

5.2.2 Execution / Boot Internals

This appendix is the implementation-level companion to the Execution Flight Plan. It walks one instruction through V4's six pipeline stages, then describes how a fault is delivered to PALcode and how the firmware boot path is fed during the decompression loops. The code is in `pipelineLib/PipelineDriver.h`; the entry-vector table is in `coreLib/Ev6EntryVectors.h`.

The six-stage step

EmulatR runs a functional, single-slot pipeline: each instruction is carried by a `PipelineSlot` through all six stages in order before the next is fetched. The pipeline driver is the only component that mutates `cpu.pc` -- the leaves and the memory drainer never touch it.

Stage	Work
IF	Mint a slot from <code>cpu.pc</code> ; translate the VA to a PA (via <code>translateInstruction</code>) and read the 4-byte instruction word. The fetch consults an optional <code>IFetchOverride</code> first.
DE	Dispatch lookup against <code>grainFactory::g_primaryTable</code> (plus the sparse secondary tables) to bind the decoded grain leaf.
GR	Build the <code>ExecCtx</code> : read the register file (integer or FP per the grain) into the execution context.
EX	Invoke <code>grain.execFn(grain, ctx)</code> and store the returned <code>BoxResult</code> onto the slot (any memory effect is packed here, not performed).
MEM	<code>MemDrainer::drain(slot, cpu, memory)</code> -- translate any data effect, perform the load/store, sign-extend loads, commit the register file, and resolve LL/SC.
WB	Advance <code>cpu.pc</code> from the grain's next PC, a divert target, or a fault entry vector; intercept HALT and faults; increment the cycle count.

The driver exposes `step` (one slot through all six stages) and `run(maxCycles)`. The hot loop ships optimized; an `EMULATR_DEBUG_STEP` CMake option restores an `-O0` build for single-stepping.

Fault delivery to PALcode

Faults travel as a `faultCode` on the slot's `BoxResult` and are resolved at WB by `retire`. HALT is a graceful stop, not a trap: it sets `cpu.halted` and returns. Any other non-zero fault is delivered to PALcode by the sequence in the EV6 HRM (6.8.1): save the faulting PC in `EXC_ADDR` (bit 0 carries the pre-trap PALmode so `HW_REI` restores it on return), map the fault to an entry-vector offset with `coreLib::ev6::entryForFault`, divert PC to `palBase`

+ `offset`, and enter PAL mode. The mapping covers the architectural classes -- for example `UNALIGN` at `0x280`, `OPCDEC` at `0x400`, `DFAULT` at `0x380`, `DTB` single-miss at `0x300`, `ITB` miss at `0x580`, `MCHK` at `0x500` -- on the `0x80`-spaced vector grid from `0x100` to `0x780`. The absolute target is `computeHwExceptionEntry(palBase, offset)`, which ORs the offset into the 32K-aligned `palBase`.

Delivery falls back to halt-with-diagnostic only when PALcode is not loaded (`palBase == 0`, where diverting would land in low-memory garbage and lose the failure) or when the fault has no architectural entry (`kEntry_None`). Routine high-volume TB misses (`kFaultDtbMiss` / `kFaultItbMiss`) are excluded from the fault log; everything else is recorded to `logs/faults.log` for offline review.

Feeding the boot path

During the firmware's self-decompression loops the IF stage consults an optional `IFetchOverride` before the normal translate-and-read, so fetches can be served from an immutable SRM payload while the guest rewrites memory underneath itself. The cold-boot inflate is the dominant early cost (see Decompressor Internals). The `hw_ret R2` at PC `0xd954` -- once suspected of single-bit corruption -- was confirmed genuine firmware by the host decompressor oracle, not a defect. For the live boot status and current frontier, see Current Development Status.

Source References

- `pipelineLib/PipelineDriver.h` -- *step / run / retire; the six-stage slot walk; the only mutator of `cpu.pc`.*
- `pipelineLib/MemDrainer.h` -- *MEM-stage data-effect drain (translate, load/store, sign-extend, LL/SC).*
- `pipelineLib/IFetchOverride.h` -- *the IF-stage fetch hook; `systemLib/Machine.cpp` `onBeforeFetch` implements it (the decompression / Step D handoff).*
- `coreLib/Ev6EntryVectors.h` -- *`entryForFault` / `computeHwExceptionEntry` and the `kEntry_*` vector grid.*

See Also.

[Execution Flight Plan](#) ¹²²

[Memory Map & IPRs](#) ¹³⁴

[Current Development Status](#) ²¹

[Clock Frequency and the MHz Reading](#) ¹⁹⁸

5.2.3 grainFactoryLib

A grain is the unit of instruction semantics in V4: each decoded instruction resolves to a single grain leaf that implements its behavior. `grainFactoryLib` is where those leaves and the

tables that reach them are generated, from tab-separated source-of-truth tables rather than hand-maintained switch statements.

Table-driven codegen

The codegen reads two tables and emits generated C++ headers and sources.

`SemanticFlags.tsv` defines the semantic flag vocabulary; `GrainMasterV4.tsv` is the per-instruction master table. The generator `codegen/genGrains.py` validates the join between them and emits, under `generated/`: `SemanticFlagsEnum.h` (the flag enum, masks, and predicates), `DispatchKinds.h` (the dispatch-kind enum), `GrainsForward.h` (per-box leaf declarations), `DispatchTables.cpp` (the populated tables), and `GrainStubs.cpp` (stub leaves for not-yet-implemented rows). Hand-written leaves are carried in `codegen/handwritten.tsv` so a real implementation can supersede a generated stub.

The master table row

Each row of `GrainMasterV4.tsv` describes one instruction with these columns: `mnemonic`, `primaryOpcode`, `subDecode`, `dispatchKind`, `format`, `box` (the functional domain that owns the leaf -- IBox, EBox, FBox, MBox, CBox, or PalBox), `flags` (a bitwise-OR of `S_*` semantic flags), `notes`, and `leafBase`. The flags drive correctness-relevant behavior: for example a PAL row carries `S_PalFormat` | `S_PalEntry` | `S_ChangesPC` and one or more personality flags.

PALcode personalities

A single `CALL_PAL` opcode can mean different things under different operating systems, so rows carry personality flags -- `S_PalTru64`, `S_PalVms`, and `S_PalLinux`. For instance opcode `0x83` is `CHMK` (change mode to kernel) under VMS but `callsys` under Tru64 and Linux; the row marks all three. The codegen iterates the personalities to emit the right dispatch, which keeps the OS-specific PAL behavior in the table rather than in code.

Source References

- `grainFactoryLib/codegen/genGrains.py` -- *generator that joins the two TSVs and emits generated/*.*
- `grainFactoryLib/GrainMasterV4.tsv` -- *per-instruction master table (mnemonic/opcode/box/flags/leafBase).*
- `grainFactoryLib/SemanticFlags.tsv` -- *S_* semantic-flag vocabulary incl. PAL personalities.*
- `grainFactoryLib/generated/DispatchTables.cpp` -- *generated primary/secondary dispatch tables (do not hand-edit).*
- `grainFactoryLib/generated/GrainStubs.cpp` -- *stub leaves for unimplemented rows; handwritten.tsv supersedes.*

See Also.[DispatchTables](#) → 127[Execution Flight Plan](#) → 122[Floating-Point Considerations](#) → 128**5.2.4 DispatchTables**

The dispatch tables are how the Decode stage turns a raw instruction word into a grain leaf. They are generated into `DispatchTables.cpp` from the master table, so the decode path is data, not a hand-written switch.

Primary and secondary tables

The top-level lookup is `g_primaryTable`, indexed by the 6-bit primary opcode. Many Alpha opcodes are not fully decoded by that field alone -- arithmetic, floating point, the `HW_MxPR` instructions, and the PAL group all carry a sub-function in lower bits. For those, the primary entry points at a secondary (sparse) table keyed by the row's `subDecode` value, so the full decode is a primary index followed, where needed, by a sparse secondary lookup. The Decode stage performs exactly that: read the opcode, index the primary table, and follow the sub-decode where the kind requires it, yielding the grain leaf that the Execute stage will invoke.

Dispatch kinds

Each instruction is classified by a `DispatchKind` that names its decode family and selects which secondary table (if any) applies. The canonical set covers the integer families `IntArith`, `IntLogical`, `IntMul`, and `IntShift`; the floating-point families `FltIeee`, `FltVax`, `FltLogical`, `ItFp`, and `FpTiExt`; the privileged / hardware kinds `HwLd`, `HwSt`, `HwMfpr`, `HwMtpr`, `HwRei`, and `Pal`; and the control / miscellaneous kinds `Direct`, `JumpClass`, and `Misc`, plus a `Reserved` sentinel. A name helper (`dispatchKindName()`) is generated alongside the enum for trace and diagnostics.

Because the tables are generated, adding or correcting an instruction is a table edit plus a regeneration, not a hand change to decode logic -- the same source-of-truth discipline as the rest of `grainFactoryLib`. A row with no implemented leaf resolves to a stub (which logs and does not compute) or, if there is no row at all, the opcode takes a decode fault (OPCDEC).

Source References

- `grainFactoryLib/generated/DispatchTables.cpp` -- *the generated `g_primaryTable` + sparse secondary tables (GENERATED; do not hand-edit).*
- `grainFactoryLib/DispatchAccess.h` -- *`DispatchKind` + `dispatchKindName()`.*
- `grainFactoryLib/generated/GrainStubs.cpp` -- *the stub leaves for unimplemented rows.*

- The *grainFactoryLib* codegen (`genGrains.py`) -- edit the master table and regenerate, never the emitted *.cpp*.

See Also.[grainFactoryLib](#) ^{↗125}[Execution / Boot Internals](#) ^{↗123}[Execution Flight Plan](#) ^{↗122}

5.2.5 Floating-Point Considerations

Alpha floating point is wider than a single IEEE implementation, and modeling it faithfully has several moving parts: four data formats, a separate enable gate, a control register that selects rounding and trap behavior, and a choice of compute backend. This page records the considerations a developer needs when reading or extending the fBox; it is a design overview rather than a per-opcode status table, because the instruction coverage is actively being filled in.

Four formats

Alpha supports two IEEE formats -- S (single) and T (double) -- and the VAX formats F, G, and D. The distinction matters for guests: OpenVMS Alpha uses VAX G_float as its default double and F_float as its default single, while Tru64 and IEEE workloads use S and T. Data movement is format-complete: the loads and stores for all four memory formats (LDF/LDG/LDS/LDT and STF/STG/STS/STT) move values correctly, so a guest can hold VAX G_float values in registers. The work that distinguishes the formats is the arithmetic and the conversions between them, which is where build-out effort concentrates.

The FP-enable gate

Floating point is gated by the FEN (FP-enable) bit. The PAL path for it is in place: MTPR_FEN and MFPR_FEN move the bit, CLRFEN clears it, and an attempt to execute FP while disabled delivers through the FEN entry vector at offset 0x200. So the enable / disabled-trap machinery works independently of how much arithmetic is implemented behind it.

FPCR, rounding, and traps

The floating-point control register (FPCR) selects the dynamic rounding mode and the IEEE trap-enable bits; instruction qualifiers select trap mode and rounding per-instruction. Faithful FP must honor these rather than always rounding to nearest and never signaling. The implementation approach uses the host's `<cfenv>` facilities -- `fesetround`, `feclearexcept`, `fetestexcept` -- around a native host operation to realize the FPCR rounding mode and to detect and deliver IEEE exceptions. Move-to / move-from FPCR provide the storage; the qualifier handling is what connects it to each operation.

The backend abstraction

FP arithmetic is reached through a backend interface, `IFpBackend` (in `fpBoxLib`), so the grain leaves call an abstract operation set -- the IEEE S/T and VAX F/G add / subtract / multiply / divide / square-root, the compares, and the format conversions -- without binding to one implementation. Operands and results are passed as raw register images (`uint64_t`) plus an `FpExecCtx` that carries the rounding mode and qualifiers, and an `FpResult` that carries the value and the exception flags. Wiring this interface into every FP grain is the in-progress integration step.

Berkeley SoftFloat -- the deterministic oracle

The reference implementation of `IFpBackend` is `SoftFloatBackend`, built on the Berkeley SoftFloat Release 3 library (vendored in the tree as `berkeley-softfloat-3-master` and compiled into a `softfloat` static library, 8086-SSE specialization). It is the deterministic, bit-identical-across-hosts FP truth source: the rounding mode is passed to SoftFloat as data and the exception flags are harvested as data, so no host control word (MXCSR / FPSR) is ever touched and the result is identical on every host and every build. `SoftFloatBackend` implements the full surface, IEEE and VAX alike, including the VAX F/G arithmetic, compares, and conversions that OpenVMS depends on.

An optional native fast-path backend (`HostBackend`) provides the seam for using host FP hardware where it is faster; it inherits `SoftFloatBackend` so any operation it has not specialized still delegates to the deterministic path, and its results are validated against SoftFloat (the `fp_softfloat_vs_host_bench` test exists for exactly this). The build-time `EMULATR_FP_SOFTFLOAT` option selects SoftFloat as the active backend; the longer-term aim is to expose the SoftFloat-versus-native choice as a runtime setting. Where a leaf still uses the host directly, the FPCR rounding and IEEE-trap behavior is realized with `<cfenv>` as described above; the SoftFloat path needs no such host-state juggling because it carries rounding and flags as data.

Source References

- `fBoxLib/grains/Float.cpp`, `FloatVariants.cpp` -- *the FP grain leaves.*
- `fpBoxLib/` -- *IFpBackend, SoftFloatBackend, HostBackend; FpExecCtx / FpResult; fp_host_guard.h.*
- `berkeley-softfloat-3-master/` -- *the vendored Berkeley SoftFloat 3 (the deterministic oracle, built as the `softfloat` static lib).*

See Also.

[grainFactoryLib](#) ¹²⁵

[CMake Macro Guards](#) ¹³⁰

Attributions & Credits

5.2.6 CMake Macro Guards

A number of EmulatR behaviors are selected at build time through CMake options that define compile-guard macros, rather than at run time. This keeps diagnostic and debug scaffolding out of the shipping hot path: a guard that is off compiles to nothing. The options are set in the CMake configuration (for example through Qt Creator) and define the matching `EMULATR_*` macro for the build.

Build / behavior options

Guard	Effect
<code>EMULATR_DEBUG_STEP</code>	Restores an -O0 build of the per-instruction pipeline step for single-step debugging. Off by default so the hot loop ships optimized.
<code>EMULATR_FP_SOFTFLOAT</code>	Selects the SoftFloat compute backend for floating point instead of the native host path, for bit-exact host-independent results.
<code>EMULATR_USE_OS_PAGES</code>	Selects the OS-page (VirtualAlloc / mmap) backing mode for guest memory.

Diagnostic / trace guards

Guard	Effect
<code>EMULATR_TRACE_HOOKS</code>	Compiles in the per-step trace hooks used to capture an execution trace.
<code>EMULATR_MEMDIAG</code>	Compiles in the memory-diagnostic instrumentation (the cycle-windowed fetch / load / store dumps).
<code>EMULATR_CHIPSET_DIAG</code>	Enables chipset-level diagnostic output.
<code>EMULATR_DIAGNOSTIC_LOGGING</code>	Enables the broader diagnostic logging path.
<code>EMULATR_PA_DUMP</code>	Compiles in physical-address dump instrumentation.

Guard	Effect
EMULATR_BOOTSTRAP_ITB_BYPASS	Debug only: bypasses the ITB for the known reset / PAL entry page during bring-up. Must be removed before relying on permission bits or ASN isolation.
EMULATR_TRACE_CONSOLE / EMULATR_ISADIAG	Console-trace and ISA-diagnostic channels.

The default build leaves the diagnostic guards off; they are turned on only when chasing a specific problem, because the instrumentation they add (especially the memory and trace hooks) is heavy enough to change runtime substantially. Runtime behavior that does not need a recompile is controlled instead by the `EMULATR_*` environment variables.

See Also.

[Building from Source](#) ⁴⁵

[Qt Creator](#) ²⁴⁶

[Environment Variables \(EMULATR_*\)](#) ²¹⁶

5.3 5.2 Memory, Addressing & Deep Dives

These appendices are the register-and-address-level companions to chapter 5 (Architecture & Internals). Where chapter 5 explains a subsystem for an operator, the deep-dives name the V4 classes, the numeric IPR indices, the entry-vector offsets, and the source files -- the level a developer needs to read or extend the code.

In this section

- *Memory Map & IPRs -- the physical address regions and the internal processor register space.*
- *Addressing & TLB -- the Ev6 translator, the SPAM shard manager, and TLB epoch invalidation.*
- *Decompressor Internals -- the DEC inflate sources and the host_decompressor oracle.*
- *Execution / Boot Internals -- the six-stage pipeline and fault entry vectors.*

See Also.

[Architecture Overview](#) ¹¹⁸

[Appendices](#) ²³²

5.3.1 Memory

`GuestMemory` (`memoryLib`) is the single physical-memory interface -- a passive byte-store with no opinion about devices or faults. It pages the guest's physical address space in on demand, in 64 KiB pages, committing a page only when the guest first touches it. So a machine configured for gigabytes costs only its working set on the host. Pages are backed by `VirtualAlloc` on Windows and `mmap` on Linux / ARM hosts (with a portable fallback). The earlier `SafeMemory` layer has been removed.

The arbiter owns routing, not memory

`GuestMemory` does no MMIO routing and raises no faults. The `TsunamiChipset` system-bus arbiter decodes every physical address first: it routes device traffic to the right model, services the chipset CSRs, and raises range faults for non-existent memory before raw RAM is ever touched. That separation is what keeps device / MMIO traffic from ever aliasing real RAM.

Array configuration

The amount of RAM the guest sees is set by `[System] memorySize` in `EmulatRV4.ini` and configured into the Cchip's Array Address Registers (AAR). The Tsunami Cchip supports up to four arrays of up to 1 GB each (4 GB maximum); accesses above the configured limit are Non-eXistent Memory (NXM). The configured size is what the firmware reports under `show memory` and in the configuration table. The shipped default is 4 GiB (`[System] memorySize = 4294967296`), laid as four 1 GB arrays (AAR0..3) -- matching the `show config memory` table.

Reservations

A `LockMonitor` tracks load-locked / store-conditional reservations per CPU at cache-line (64-byte) granularity, for up to four CPUs -- the substrate for the EV6 `LDx_L` / `STx_C` primitives.

Source References

- `memoryLib/GuestMemory.cpp` -- *sparse-paging byte store for guest RAM.*
- `memoryLib/GuestMemory.h` -- *LockMonitor per-CPU cache-line LL/SC reservations.*
- `chipsetLib/TsunamiChipset.cpp` -- *system-bus arbiter: address decode/route/NXM before RAM.*
- `chipsetLib/TsunamiCchip.h` -- *Array Address Registers (AAR) DRAM config / max-RAM limit.*
- `memoryLib/ISystemBus.h` -- *the bus seam interface.*

See Also.

[Overview](#) 

[Addressing](#) ^{→ 133}

[SRM>>> show memory](#) ^{→ 90}

[Memory Map & IPRs](#) ^{→ 134}

5.3.2 Addressing

Virtual-to-physical translation is performed by `Ev6Translator` (`mmuLib`) exactly as the EV6 architecture specifies. In PAL mode the address bypasses translation ($VA = PA$). Otherwise the translator applies the canonical virtual-address window check and the superpage (`kseg`) mapping; an address that is neither a superpage nor a resident mapping returns an ITB or DTB miss for the PAL fill handler.

The translation buffers (SPAM)

The ITB and DTB are modeled by the **SPAM** cache -- Set-Partitioned Associative Memory -- as `SPAMShardManager` instances that live in `CpuState` (`itbMgr` and `dtbMgr`). Each is 16 shards by 8 ways = 128 entries: the manager hashes a lookup tag to pick a shard, and the shard does a short way-scan. The buffers are CPU-local by design -- each `CpuState` owns its own pair, and cross-CPU invalidation (in a future SMP build) would go through IPI machinery.

Invalidation is lazy (epoch-based)

Rather than walk and clear entries, invalidation bumps a counter (`TlbEpoch`): a full flush (TBIA) advances a global counter and a per-process flush (TBIAP) advances a process counter, so a flush is $O(1)$ and a stale entry simply fails its liveness test at the next lookup. Targeted single-entry flushes (TBIS / TBISI / TBISD) clear one slot. The page-table-entry format (`Ev6PteFormat`) carries the protection, granularity-hint, and fault bits the guest expects, and translation faults are delivered precisely at the Writeback stage.

Source References

- `mmuLib/Ev6Translator.h` -- EV6 VA->PA: PAL bypass, VA-window, superpage/kseg, ITB/DTB miss.
- `pteLib/SPAMShardManager.cpp` -- SPAM 16-shard x 8-way TLB manager (`itbMgr/dtbMgr`).
- `pteLib/TlbEpoch.h` -- lazy epoch invalidation (TBIA global / TBIAP per-process).
- `pteLib/Ev6PteFormat.h` -- PTE protection / granularity-hint / fault-bit format.
- `coreLib/CpuState.h` -- `itbMgr/dtbMgr` resident per-CPU.

See Also.

[Memory](#) ^{→ 132}

[Overview](#) ^{→ 118}

[Addressing & TLB \(appendix\)](#) ^{→ 136}

5.3.3 Memory Map & IPRs

This appendix is the register-level companion to the Memory and Addressing pages. It names the two internal-processor-register (IPR) namespaces EmulatR implements, the numeric indices a developer reads in a trace, and the physical-address regions the chipset arbiter routes. The authoritative reference is the Alpha 21264 Hardware Reference Manual (EC-RE2CA-TE, Table 5-4) together with V4's `coreLib/HW_IPR.h`.

Two IPR namespaces

EmulatR follows the architecture's split between registers the operating system reaches through `CALL_PAL` and registers only PALcode can touch with `HW_MFPR` / `HW_MTPR`.

1. PALcode-visible IPRs (0x0000 .. 0x00FF). These are `CALL_PAL` function codes, not hardware registers. The OS executes a `CALL_PAL` (for example `MFPR_PTBR` = 0x0015, `MTPR_TBIA` = 0x001B, `MFPR_WHAMI` = 0x003F); the PAL handler at the matching entry vector performs the actual access. Numbering follows the OSF/1 PALcode specification rather than the silicon. In V4 these are the `PAL_MFPR` / `PAL_MTPR` enums.

2. EV6 hardware IPRs (0x0100 .. 0x02FF). These are the real register indices encoded in the `scbd` field of an `HW_MFPR` / `HW_MTPR` instruction; only PALcode running in PAL mode may execute them. Numbering follows the 21264 HRM. The 0x0100 offset is an emulator convention so the hardware range cannot collide numerically with the PALcode-visible range: the decoder adds 0x0100 to the raw `scbd` value, except the `PAL_TEMP` block (raw 0x40..0x5F) which adds 0x01C0 so `PT0` lands at 0x0200.

Hardware IPR map (the indices you see in a trace)

The hardware IPRs are grouped by the box that owns them. Selected indices from `HW_IPR.h`:

Index	Name	Box / role
0x0106	<code>HW_EXC_ADDR</code>	IBox -- exception address; read = saved PC, write = <code>HW_REI</code> target. Bit[0] carries PAL mode.
0x0110	<code>HW_PAL_BASE</code>	IBox -- PAL base address (32K-aligned; low 15 bits zero).
0x0111	<code>HW_I_CTL</code>	IBox -- IBox control register (SDE and related bits).
0x0113	<code>HW_IC_FLUSH</code>	IBox -- I-cache flush all (action).

Index	Name	Box / role
0x0120 / 0x0121	HW_DTB_TAG0 / HW_DTB_PTE0	MBox -- DTB tag staging and PTE fill, bank 0 (writing the PTE triggers the fill).
0x0127	HW_MM_STAT	MBox -- memory-management status (read-only; fault detail).
0x0140	HW_PCTX	Process context -- ASN + FPE + PPCE packed.
0x01C0 / 0x01C1	HW_CC / HW_CC_CTL	Cycle counter and its control (offset + enable).
0x01C2 / 0x01C4	HW_VA / HW_VA_CTL	Faulting virtual address (read-only) and VA control (write-only, packed -- selects the 43/48-bit VA window).
0x01A0 .. 0x01A5	HW_DTB_*1	MBox -- DTB bank 1 (the EV6 dual-bank DTB).
0x0200 .. 0x0217	HW_PAL_TEMP_0 .. _23	PAL temporaries PT0..PT23 (PAL-mode scratch; OSF/1 assigns KSP, PTBR, PCBB, etc.).
0x0218 .. 0x021F	HW_PAL_TEMP_24 .. _31	PT24..PT31 -- EV5-vintage scratch, provisioned because bring-up SRM .exe code addresses PT31.

PAL temporaries

The EV6 provides 24 PAL temporary registers (PT0..PT23), reachable only via HW_MFPR / HW_MTPR in PAL mode. They are the backbone of PALcode: every PAL entry saves the guest GPRs into PAL_TEMP_s and every PAL exit restores them. The OSF/1 convention pins specific temporaries to architectural state exposed through CALL_PAL (PT0 = KSP, PT2 = PTBR, PT4 = PCBB, PT5 = PRBR, PT6 = SCBB, and so on). The OS never reads a PAL_TEMP directly -- it issues CALL_PAL MFPR / MTPR and the handler relays to the right temporary. V4 widens the range to PT31 because the SRM firmware used during bring-up carries EV5-vintage code that addresses PT24..PT31 even on EV6 (observed at PC 0x12f88 issuing `hw_mtpr Rx, 0x5F`); those extra eight are pure CpuState-backed scratch.

IPRs that map to architectural state live as flat fields on `CpuState` (`ptbr`, `i_ctl`, `pal_base`, `ps`, `excAddr`, `fpcr`). Selectors that are not yet `CpuState`-backed return `kFaultUnimplemented`; adding one later is a `CpuState` field plus a switch case in the `HW_MFPR` / `HW_MTPR` leaf.

Physical address regions

Guest physical addresses are routed by the Tsunami chipset arbiter. Reads and writes resolve to one of three destinations: main memory (the `GuestMemory` sparse backing, bounded by the `Cchip` AAR registers -- anything outside is a non-existent-memory / NXM fault), the chipset CSR space (`Cchip` / `Dchip` / `Pchip` control registers and the TIG region), or PCI / device MMIO windows. The halt-gate diagnostics read and write CSRs in this space; for example the console arming read targets `pmem:80130000FF8` in the TIG region. The exhaustive per-register CSR table is in the Tsunami / 21272 Hardware Reference Manual; the operator-facing view of how to read these locations is on the Evaluate Address page.

Source References

- `coreLib/HW_IPR.h` -- *the hardware IPR indices and the 0x0100 / PAL_TEMP offset convention.*
- `coreLib/CpuState.h` -- *the flat IPR fields (`ptbr`, `i_ctl`, `pal_base`, `ps`, `excAddr`, `fpcr`, `PAL` temporaries).*
- `palBoxLib/grains/PalEntries.cpp` -- *the CALL_PAL and HW_MFPR / HW_MTPR dispatch leaves.*
- `chipsetLib/TsunamiChipset.h` -- *arbiter routing of guest PAs to memory / CSR / device MMIO; AAR / NXM bound.*

See Also.

[Memory](#) ¹³²

[Addressing & TLB](#) ¹³⁶

[Evaluate Address](#) ⁹¹

5.3.4 Addressing & TLB

This appendix is the implementation-level companion to the Addressing page. It walks the V4 translation path from a virtual address to a physical address, then describes the translation-buffer front end and the epoch scheme that makes a TLB flush a single counter bump. The authoritative reference is the Alpha Architecture Reference Manual (sections 4.1.6 / 4.1.7) and the 21264 Hardware Reference Manual; the V4 code lives in `mmuLib/Ev6Translator.h` and `pteLib/`.

The translator

`Ev6Translator` is not an object -- its entry points are static functions that take a `CpuState const&` as the first argument, so per-CPU state travels through `CpuState` rather than a

translator instance. The MEM stage calls one of three entry points depending on the memory effect a leaf packed onto its `BoxResult` at EX:

- *translateData* -- a data load or store, no alignment check.
- *translateDataAligned* -- a data load or store with a size-class alignment check (1/2/4/8 bytes).
- *translateInstruction* -- an instruction fetch (longword-aligned, consults the I-side superpage enables).

Each returns a `TranslationResult`. On Success the caller's `pa_out` holds the physical address; any other value is a translation fault that the MEM drainer converts to a fault code (`mmuLib::toFaultCode`) and uses to short-circuit the register-file commit.

Translation path (data stream)

For a normal load or store, `translateData` proceeds in this order:

1. **Explicitly-physical accesses bypass upstream.** `HW_LD` / `HW_ST` / `LDQP` / `STQP` are tagged physical and the MEM drainer sets `pa = va` before ever calling the translator. Importantly, EV6 PAL mode does NOT blanket-disable D-stream translation -- a normal load or store in PAL mode still translates. (An earlier blanket PAL bypass mis-mapped VMS system-space kernel-stack addresses and was removed.)
2. **Canonical VA check.** `VA_CTL` bit 1 selects 48-bit VA mode when set, otherwise 43-bit mode. In 43-bit mode `VA<63:43>` must sign-extend `VA<42>`; in 48-bit mode `VA<63:48>` must sign-extend `VA<47>`. A non-canonical address faults before any TLB lookup.
3. **Kseg / superpage detection.** The three SPE bits (on `M_CTL` for data, `I_CTL` for instructions) gate three kernel-only superpage modes. `SPE[2]` maps `VA<43:13>` directly to PA when `VA<47:46> = 0b10`; `SPE[1]` and `SPE[0]` cover the other two windows. A kseg-shaped address from non-kernel mode is an access violation; a non-kseg address falls through.
4. **DTB lookup.** On a live hit, the PTE permission bits are checked and PA is composed as `(PFN << 13) | (VA & 0x1FFF)`. On a miss the result is `DtbMiss`, which the drainer maps to `kFaultDtbMiss`; PALcode's DTB-miss handler refills the buffer (`HW_MTPR DTB_TAG0` / `DTB_PTE0`) and the access retries.

The instruction path mirrors this with a 4-byte alignment check (PC bit 0 is the PALmode flag and is masked before the check), the I-side SPE bits, and an ITB lookup that misses to `kFaultItbMiss`. The EV6 base page is 8 KiB (shift 13); a PA is 44 bits wide.

The translation buffers (SPAM)

Each `CpuState` owns its own pair of `SPAMShardManager` front ends -- one ITB, one DTB. A manager holds an array of `SPAMBucket` shards, the epoch state, and a small hash that picks a shard from the tag. In `CpuState` the two managers are instantiated as

`SPAMShardManager<16, 8>` -- 16 shards by 8 ways = 128 entries each, matching the architectural EV6 ITB and DTB size (the shard count and ways are template parameters, so the geometry can be retuned without touching the lookup path). Its public surface mirrors the architectural operations: `lookup` (one per access), `insert` (the miss handler / `HW_MTPR` `ITB_PTE` / `DTB_PTE`), the three invalidate operations, and `clear` for cold boot. Because translation buffers are CPU-local by construction, the manager carries no synchronization primitives; any future SMP cross-CPU invalidation would arrive through IPI machinery processed on the target CPU's own thread.

Epoch invalidation

A naive TLB would walk every entry on a flush. V4 instead keeps two counters in the manager and stamps both into every entry at insert time. A flush of all entries is then a single counter bump; entries become stale the instant their stamped value fails to match the live counter on the next lookup. The mapping from PALcode flush operations is:

Operation	Action	Lazy?
TBIA	bump globalEpoch (invalidates everything)	yes
TBIAP	bump processEpoch (ASM=1 global entries survive)	yes
TBIS / TBISI / TBISD	locate the matching entry, clear it in place	no (targeted)

The liveness test applied at lookup is: an entry is live when its `globalEpoch` matches the manager's `globalEpoch` AND either the entry is ASM-global or its `processEpoch` matches. The ASM arm encodes the architecture's rule that an ASM=1 (global) mapping survives TBIAP. The counters are plain 64-bit integers, not atomics -- at one bump per 100 ns a 64-bit counter takes roughly 58,000 years to wrap, so wrap is not handled; a manager reset zeroes both counters and clears every shard if it ever matters.

Source References

- `mmuLib/Ev6Translator.h` -- *translateData / translateDataAligned / translateInstruction; TranslationResult; toFaultCode.*
- `coreLib/CpuState.h` -- *the per-CPU SPAMShardManager<16,8> ITB / DTB pair; VA_CTL / M_CTL / I_CTL SPE state.*
- `pipelineLib/MemDrainer.h` -- *the MEM stage that calls the translator and maps a fault to the register-commit short-circuit.*
- `pteLib/` -- *PTE decode / permission bits.*

See Also.[Addressing](#) ¹³³[Memory Map & IPRs](#) ¹³⁴[Execution / Boot Internals](#) ¹²³**5.4 5.3 Chipset & Devices**

In EmulatR the chipset is the adapter between the CPU(s) and the devices whose accesses it routes. Every CPU load, store, and fetch enters the chipset through its system-bus seam; the chipset decodes the physical address and steers it to DRAM, to a chipset control/status register (CSR), to a PCI device, or to the non-existent-memory path that raises a machine check.

Two distinct pieces of silicon are involved on the Alpha 21264 (EV6/EV68) machines EmulatR targets, and it is important not to confuse them.

Core-logic chipset (north side)

The core-logic chipset is the memory controller plus PCI/AGP host bridge -- the `Cchip` (command/address, interrupts, arbitration), `Dchip` (data slice), and `Pchip` (PCI host bridge). Two generations appear across the supported platforms:

Chipset	Part	Platforms	EmulatR
Tsunami / Typhoon	21272	DS10, DS20, ES40	Modeled (TsunamiChipset)
Titan	21274	DS15, DS25, ES45	Scaffold (TitanChipset)

Taxonomy note (corrected 2026-06-16): the part numbers are 21272 = Tsunami and 21274 = Titan. "Typhoon" is the higher-bandwidth (dual-Dchip) variant of the 21272, NOT a separate part. Earlier code mislabeled the 21274 as "Typhoon"; DS15/DS25/ES45 are Titan machines.

South bridge (legacy I/O side)

Hanging off PCI hose 0 is the south bridge, which provides the legacy ISA plane: the 8259 interrupt controllers, 8254 timers, MC146818 real-time clock, 16550 UARTs, keyboard controller, and an IDE controller. The south bridge is platform-specific and independent of the core-logic generation:

- Cypress CY82C693 -- DS10 / DS20 (PC264 lineage).
- ALi M1543C -- ES40 / ES45.

EmulatR selects the south bridge by machine model in `TsunamiChipset::wireDevices()`; the ISA devices behind it sit at fixed ports and are reused unchanged regardless of which bridge is wired.

Source References

- `chipsetLib/TsunamiChipset.cpp` -- *wireDevices() selects south bridge by model; arbiter seam.*
- `chipsetLib/TsunamiCchip.h` -- *north-side Cchip (with TsunamiDchip.h / TsunamiPchip.h).*
- `chipsetLib/Cypress_CY82C693ISABridge.h` -- *Cypress south bridge (DS10/DS20).*
- `chipsetLib/AliM1543C.h` -- *ALi M1543C south bridge (ES40/ES45).*
- `chipsetLib/TitanChipset.h` -- *21274 (Titan) scaffold counterpart.*

See Also.

[South Bridge](#) ¹⁴⁵

[Cypress](#) ¹⁴⁸

[Tsunami / Typhoon \(21272\)](#) ¹⁴⁰

[82077 Controller](#) ¹⁵⁰

[DECchip 21274 \(Titan\)](#) ¹⁴⁴

[Ali M1543C South-Bridge Interface](#) ¹⁴⁷

5.4.1 Tsunami \ Typhoon

Tsunami is the DEC/Compaq 21272 core-logic chipset for 21264 (EV6) systems -- the memory controller and PCI host bridge for the DS10, DS20, and ES40. Typhoon is the higher-bandwidth variant of the same 21272 (a doubled Dchip population for twice the memory bandwidth), used on the ES40-class boxes. They share one programming model; EmulatR treats them as variants of a single chipset.

Typhoon is NOT the 21274. The 21274 is the separate Titan chipset (DS15/DS25/ES45). Some earlier material mislabeled the 21274 as "Typhoon" -- corrected 2026-06-16.

Components

- `Cchip` -- command/address, the memory array address registers (AAR0-3), per-CPU device interrupt mask/request (DIM/DIR), the MISC register (NXM capture, inter-processor interrupt request, arbitration), and the interval timer.
- `Dchip` -- the data slice; carries the chip revision (DREV).
- `Pchip` -- the PCI host bridge (sparse/dense memory, I/O, Type-0/1 config, DMA windows). Pchip0 is always present; Pchip1 is populated on the larger boxes.

- **TIG** -- the byte-wide Timer / Interrupt / GPIO bus that carries the front-panel halt / SMI status register (SMIR) and the per-CPU halt registers.

Physical address map (the form firmware uses)

Block	Physical address
Pchip0 CSRs	0x801_8000_0000
Cchip CSRs	0x801_A000_0000
Dchip CSRs	0x801_B000_0000
Pchip1 CSRs	0x803_8000_0000
TIG (smir at +0x40)	0x801_0000_0000

Memory above the Cchip AAR limit is non-existent memory (NXM) and raises a machine check. On the 21272 -- Tsunami and Typhoon alike -- the DS / ES40-class firmware encodes the array size in a 3-bit ASIZ (0x7 = 1 GB x 4 arrays), capping DRAM at 4 GB; the effective limit is the min of the Cchip PA width and the firmware ASIZ width. The 32 GB reach is a Titan (21274) property (its wider 4-bit ASIZ), NOT Typhoon -- see the Titan topic.

EmulatR model

Modeled by `TsunamiChipset (chipsetLib/TsunamiChipset.{h,cpp})` with `TsunamiCchip`, `TsunamiDchip`, `TsunamiPchip`, and `TsunamiTig`. The variant (Tsunami vs Typhoon) is selected by model and latched at construction; the CSR layout, MMIO routing, and reset values live in `Tsunami21272_CsrSpec.h` / `Tsunami21272_RegisterMap.h`. The DS10 boots to the SRM >>> prompt on this model.

The Titan (21274) reuses these exact Cchip/Dchip/TIG register offsets and the same top-level PA map; the Titan-specific silicon is concentrated in the dual-port Pchip plus AGP. See the Titan topic.

Source References

- `chipsetLib/TsunamiChipset.{h,cpp}` -- *the top-level chipset; variant latch, MMIO routing, reset()*.
- `chipsetLib/TsunamiCchip.h`, `TsunamiDchip.h`, `TsunamiPchip.h`, `TsunamiTig.h` -- *the four sub-blocks.*
- `chipsetLib/Tsunami21272_RegisterMap.h`, `Tsunami21272_CsrSpec.h` -- *the CSR offsets / reset values (SSOT for the register map);* `TsunamiVariant.h` -- *Tsunami vs Typhoon.*

See Also.

[Chipset](#) ¹³⁹

[DECchip 21274 \(Titan\)](#) ¹⁴⁴

[South Bridge](#)  145

5.4.2 Tsunami Error Chain (PERROR / No Devsel)

EmulatR models the 21272 Pchip PCI error-reporting chain end to end (21272 HRM Ch 8.8 and 10.2.5.6-10.2.5.8). When a PCI transaction fails in a way the Pchip latches on real silicon, the emulated Pchip latches it the same way, raises the same error interrupt, and the guest firmware or operating system reads the same registers a real DS20 would present. If the SRM console prints:

```
Error detected by Pchip 0
  PERROR = 0070010011480100
    No Devsel
```

that is the SRM's own IRQ0 error handler decoding EmulatR's PERROR register. This topic explains the chain, how to decode the value, and what a report means for troubleshooting.

The chain, end to end

1. A CPU access reaches the Pchip's PCI dense-memory window and NO device claims the address (on real PCI: no DEVSEL, a master abort). The read returns all-ones; a write is discarded.
2. If the corresponding PERRMASK bit is set, the Pchip latches the error bit in PERROR together with the failing PCI address and command. The FIRST latched error freezes the register; later errors can only set LOST.
3. The Pchip asserts b_error to the Cchip, which raises DRIR bit 62 (the Pchip0 error interrupt, HRM Table 6-9). That is an IRQ0-class interrupt.
4. The guest's error handler reads PERROR, reports it, and writes 1s to clear (W1C). When all error bits clear, the register unfreezes and b_error deasserts.

Decoding a PERROR value

Worked example: PERROR = 0070010011480100

Field	Bits	Example decode
Error bits	<11:0>	0x100 = bit 8 = NDS (No Devsel)
CMD (PCI command of the failing cycle)	<55:52>	0x7 = memory write (0x6 = memory read, 0xB = config write)
ADDR (failing PCI address <31:2>)	<47:18>	(value >> 18 & 0x3FFFFFFF) << 2 = PCI 0x0100_1148
LOST	<0>	A second error arrived while the register was frozen (its details were lost)

Error bit positions (HRM Table 10-42): LOST<0>, SERR<1>, PERR<2>, DCRT0<3>, SGE<4>, APE<5>, TA<6>, RDPE<7>, NDS<8>, UECC<10>, CRE<11>.

What EmulatR latches today

Source	When it latches
NDS (No Devsel)	Unclaimed dense PCI-memory read or write (master abort), and master-aborted CONFIG WRITES. Exempt by the HRM itself: config READS and special cycles (HRM 8.8.2.1), and sub-64K legacy I/O ports (the south bridge claims ISA ranges subtractively).
SGE (scatter-gather error)	A DMA scatter-gather walk fetched an invalid page-table entry (V bit clear) inside an SG-enabled window (HRM 8.8.2.5).
All other sources	TA, RDPE, PERR, APE, SERR, DCRT0 are not yet wired (they require PCI master/target timing models). Documented in the source TODO table.

PERRMASK gating

Out of reset, `PERRMASK = 0`: NOTHING latches (HRM 10.2.5.7 -- the default is all errors disabled). Firmware enables reporting by setting mask bits; the DS20 SRM does this during console init, which is why boot-time BAR sizing probes never storm the handler but a post-init unclaimed access reports immediately. LOST is gated by mask bit 0 like every other bit. PERRSET (W1S) honors the same mask and the same freeze semantics.

When you see No Devsel: it is a real finding

As of 2026-08-02, every device EmulatR presents in PCI config space also DECODES the memory/IO windows its BARs claim -- behavioral models answer with real behavior, and config-only stub devices answer with an inert claimed window (reads float all-ones, writes absorbed, logged as `STUB-BAR` rows on stderr). A No Devsel report therefore names a GENUINELY unclaimed address: decode the ADDR field and ask what assigned that address and to whom.

Case study (the report shown at the top of this topic): the DS20's DE500 tulip was declared as a config-only stub, so the SRM assigned its memory BAR but nothing claimed the window. The ew driver's SROM/microwire polling at PCI `0x0100_1148` then master-aborted on every access -- a write/write/read rhythm of No Devsel reports. The correction was to bind the behavioral 21143 model (which claims its BARs) and to make stub devices claim their windows in general. The error chain was working exactly as documented; it made a real modeling gap audible.

See Also.[Chipset](#) ^{↳ 139}[Tsunami / Typhoon](#) ^{↳ 140}[Bring-Up Probes](#) ^{↳ 114}**5.4.3 DECchip 21274 (Titan)**

Titan is the DECchip 21274 core-logic chipset -- the memory controller and PCI/AGP host bridge for the 21264A/21264B (EV67/EV68) machines: the DS15, DS25, and ES45. It is the successor to the 21272 (Tsunami) and a separate part.

Naming: 21272 = Tsunami, 21274 = Titan. "Typhoon" is a 21272 variant, not the 21274. EmulatR's enum was corrected on 2026-06-16 so that DS15/DS25/ES45 map to the Titan variant.

Shared with the 21272

The most important architectural fact for EmulatR: Titan shares the top-level physical-address map and the Cchip/Dchip/TIG register offsets with the 21272. Cchip is at 0x801_A000_0000, Pchip0 at 0x801_8000_0000, Pchip1 at 0x803_8000_0000, hose stride h<<33. The Cchip MISC register is at +0x80 and the per-CPU device-interrupt requests DIR0-3 at +0x280 / +0x2C0 / +0x680 / +0x6C0 (confirmed against the ES45 Service Guide Appendix D). EmulatR therefore reuses TsunamiCchip, TsunamiDchip, and TsunamiTig unchanged.

New Titan silicon

- **Dual-port PA-chip.** Each of the two PA-chips has a G-port (PCI) at port offset +0x000 and an A-port (AGP) at +0x1000, each with four DMA windows (WSBA/WSM/TBA) and a port-control register (GPCTL/APCTL). This replaces the 21272's single-port Pchip.
- **Richer error set.** ERROR / GPERROR / APERROR / AGPERROR (plus enable/set companions) replace the 21272 single PERROR.
- **AGP.** Hose 2 (pachip0.A) and hose 3 (pachip1.A) are AGP; APCTL carries the AGP rate/SBA/enable/present bits.
- **Four hoses.** 0 = pachip0.G, 1 = pachip1.G, 2 = pachip0.A, 3 = pachip1.A; pachip1 presence is advertised in Cchip CSC bit 14.

EmulatR model (scaffold)

Implemented adjacent to the Tsunami model:

- `Titan21274_CsrSpec.h` -- authoritative constants (bases, Cchip/Dchip/Port offsets, PCTL/WSBA/error masks, hose map, machine-check vectors). Verified by `static_assert`.
- `TitanPchip.h` -- the new dual G/A-port PA-chip with reset window programming. Compiles and runs.
- `TitanChipset.h` -- top-level scaffold composing the reused Cchip/Dchip/TIG plus the Titan Pchip and the 4-hose interrupt mapping.

Status: scaffold, not yet a boot target. Faithful: all register offsets/identities, the interrupt/IPI/halt path (via the reused Cchip/TIG). Stubbed: per-hose PCI MEM/IO/config routing (TODO titan-pci-route, identical to the 21272 -- delegate to the Tsunami routing) and the ISystemBus + device layer (TODO titan-isystembus, mirror TsunamiChipset). Authoritative reference: Linux core_titan.c/.h and the ES45 Service Guide EK-ES450-SV Appendix D. Full detail in journal 20260616_titan_21274_interface.md.

CPU note

The Titan boxes run EV67/EV68 (21264A/B). The instruction set is essentially the same as EV6 for emulation purposes, so the CPU core is not the gap -- the chipset is. The ES45/DS25/DS15 south bridge is the ALi M1543C (shared with the ES40).

Source References

- `chipsetLib/Titan21274_CsrSpec.h` -- *authoritative bases/offsets/masks (static_assert-verified).*
- `chipsetLib/TitanPchip.h` -- *dual G/A-port PA-chip with reset-window programming.*
- `chipsetLib/TitanChipset.h` -- *top-level scaffold reusing TsunamiCchip/Dchip/Tig.*
- `chipsetLib/TsunamiCchip.h` -- *reused unchanged (shared offsets with 21272).*

See Also.

[Tsunami / Typhoon \(21272\)](#) ¹⁴⁰

[Chipset](#) ¹³⁹

[ALi M1543C South-Bridge Interface](#) ¹⁴⁷

5.4.4 South Bridge

The south bridge is the PCI-to-ISA bridge that carries the legacy I/O plane of an Alpha system: the 8259A interrupt-controller pair, the 8254 timers, the MC146818 real-time clock / CMOS, the 16550 UARTs (COM1/COM2), the 8042 keyboard controller, an IDE controller, and the Super I/O (floppy, parallel). It sits on PCI hose 0 and is one of the first devices the SRM console enumerates.

The south bridge is chosen by platform and is independent of the core-logic chipset generation. EmulatR models two:

South bridge	Vendor:Device	Platforms	Model class
Cypress CY82C693	0x1080:0xC693	DS10, DS20 (PC264)	Cy82C693IsaBridge
ALi M1543C	0x10B9:0x1533	ES40, ES45	AliM1543C

The device-attachment contract

Both bridges implement the same two interfaces from `chipsetLib/IDeviceHandlers.h`, so the chipset wires them identically:

- `IPciDeviceHandler -- pciConfigRead/Write(reg, value, width)` over a 256-byte PCI configuration space. This is how the SRM finds the bridge during slot enumeration.
- `IIoPortHandler -- ioRead/Write(port, value, width)` for the legacy ISA I/O window.

Model-gated selection

EmulatR picks the bridge in `TsunamiChipset::wireDevices()` from a `SouthBridge` enum (`m_southBridge`; the former `isAliPlatform(model)` string test was retired) -- ES40 / ES45 / DS25 wire the ALi, everything else wires the Cypress (the default). The SAME lever also selects the IDE controller (func 1) -- ALi M5229 or Cypress -- so bridge and IDE stay consistent by construction. The model string reaches the chipset because `Machine` constructs it from the ini `[System] model`. The ISA devices behind the bridge live at fixed ports (8259 at 0x20/0xA0, RTC at 0x70, COM1 at 0x3F8, ...) and are registered separately, so they are reused unchanged regardless of which bridge is selected.

Only the bridge identity and config space differ between the two. The SRM probes the bridge's vendor/device ID and then programs bridge-specific control and interrupt-routing registers; getting those config-space responses right is what keeps enumeration from stalling.

Source References

- `chipsetLib/TsunamiChipset.h` -- *wireDevices(); the m_southBridge lever selecting bridge + IDE (func 0 / func 1).*
- `chipsetLib/AliM1543C.h` -- *ALi M1543C ISA bridge (0x10B9/0x1533); M5229 IDE id 0x5229.*
- `chipsetLib/Cypress_CY82C693ISABridge.h` -- *the Cypress CY82C693 ISA bridge (0x1080/0xC693).*
- `chipsetLib/IDeviceHandlers.h` -- *IPciDeviceHandler / IIoPortHandler, the shared attachment contract.*

See Also.

[Chipset](#) ¹³⁹

[Cypress](#) ¹⁴⁸

[CY82C693](#) ¹⁴⁹

[Ali M1543C South-Bridge Interface](#) ¹⁴⁷

5.4.4.1 ALi M1543C South-Bridge Interface

The Acer Labs (ALi) M1543C is the PCI-to-ISA south bridge of the AlphaServer ES40 and ES45 -- the counterpart to the Cypress CY82C693 used by the DS10/DS20. EmulatR models it so that ES40 firmware enumeration finds the bridge instead of master-aborting. ES40 is Tsunami (21272) silicon, so the south bridge is the only platform piece that differs from the DS10.

PCI identity and functions

Function	Role	Vendor:Device
Func 0	PCI-to-ISA bridge (class 0x060100)	0x10B9:0x1533
M5229	IDE controller	0x10B9:0x5229
M5237	USB host	0x10B9:0x5237
M7101	Power management / SMBus	0x10B9:0x7101

Function 0 advertises header type 0x80 (multifunction). The bridge/interrupt-routing config registers the ES40 SRM programs live at config index 0x40-0x4B: ISA-bus / decode / IDE control (0x40-0x47) and PCI INTx (PIRQ) steering to ISA IRQs (0x48-0x4B).

EmulatR model

`chipsetLib/AliM1543C.h` mirrors the Cypress contract exactly: it implements `IPciDeviceHandler + IIoPortHandler` over a 256-byte config space, with read-only-protected vendor/device IDs and store-through config writes. A small `AliPciFunctionStub` gives the SRM's multifunction probe valid headers for the IDE/USB/PMU companions instead of a master abort. The header was ported from `Processor Support/ALi M1543_Datasheet` (Section 4) and verified standalone with `g++` (identity reads back 0x1533_10B9; PIRQ store-through and RO protection both correct).

Selection and wiring

`TsunamiChipset::wireDevices()` selects the bridge from the `m_southBridge` enum (the former `isAliPlatform(model)` string test was retired) -- ES40/ES45/DS25 wire the ALi (`m_ali`), everything else wires the Cypress (default). The SAME lever also selects the func1 IDE (ALi M5229 vs Cypress), so bridge and IDE stay consistent. DS10 still resolves to Tsunami / `cpuCount` 1, byte-identical to before, so the working path is untouched.

Faithful vs stubbed

Faithful: func0 identity, class, multifunction header, companion identities, config store-through with RO IDs, the model-gated selection, and -- since Phase 2B -- the func1 M5229 IDE via `AliM5229Ide` (selected by the same south-bridge lever; it answers the taskfile / IDENTIFY probe on `dqa0` / `dqb0`). Stubbed (TODO `ali-irq-route`): the PIRQ steering (`idx` 0x48-0x4B) and IDE INTA level/edge (0x44) currently store through with no field effect; the

USB (M5237) and PMU/SMBus (M7101) companion internals are not modeled beyond their identities.

Run the ES40 with [System] model=ES40, cpuCount=1 and [ROM] firmwareImage=firmware/es40_v7_3.exe. Build and smoke-test the DS10 first to confirm no regression. The most likely first divergence is ALi config/IRQ-routing init (only the identity is faithful so far), ahead of the halt-button check. Full detail: journal 20260617_es40_ali_m1543c_interface.md.

Source References

- `chipsetLib/AliM1543C.h` -- *the ISA-bridge model (config space, RO IDs, PIRQ store-through, companion stubs).*
- `deviceLib/Tsunami/AliM5229Ide.h` -- *the func1 M5229 IDE (0x5229), wired for ES40 / ES45 / DS25.*
- `chipsetLib/TsunamiChipset.h` -- *wireDevices(); the m_southBridge lever that gates bridge + IDE.*

See Also.

[South Bridge](#) ¹⁴⁵

[CY82C693](#) ¹⁴⁹

[Tsunami / Typhoon \(21272\)](#) ¹⁴⁰

[Chipset](#) ¹³⁹

5.4.4.2 Cypress

Cypress Semiconductor (PCI vendor ID 0x1080) is the maker of the CY82C693 "HyperCore" combination PCI-to-ISA bridge used as the south bridge on the DS10 and DS20 (the PC264 platform lineage). In EmulatR, "Cypress" and "CY82C693" refer to this part and its device model.

The CY82C693 is the south bridge for the platforms EmulatR drives end to end today, so it is the most exercised of the two bridge models. It provides the PCI-to-ISA bridge (function 0), an IDE controller (function 1), and a USB host (function 2), and fronts the legacy ISA devices.

The ES40 and ES45 do NOT use the Cypress -- their south bridge is the ALi M1543C. Do not assume Cypress for an ES-class machine.

The full part detail -- PCI identity, the multi-function layout, and the EmulatR device classes (`Cy82C693IsaBridge`, `Cy82C693Ide`) -- is on the dedicated part page.

Source References

- `chipsetLib/Cypress_CY82C693ISABridge.h` -- *fn0 ISA bridge (Cy82C693IsaBridge), vendor 0x1080.*
- `deviceLib/Tsunami/Cy82C693Ide.h` -- *fn1 IDE controller (Cy82C693Ide).*
- `chipsetLib/TsunamiChipset.cpp` -- *bridge wired at hose0/bus0/slot5 in wireDevices().*

See Also.

[CY82C693](#) ¹⁴⁹
[South Bridge](#) ¹⁴⁵
[Chipset](#) ¹³⁹

5.4.4.2.1 CY82C693

The Cypress CY82C693 is the combination PCI-to-ISA bridge / IDE controller / Super I/O used as the south bridge on the DS10 and DS20 (PC264 lineage). It is a multi-function PCI device on hose 0, bus 0, slot 5.

PCI identity and functions

Function	Role	Vendor:Device	Class
5.0	PCI-to-ISA bridge	0x1080:0xC693	0x060100
5.1	IDE controller (dqa0/dqa1)	0x1080:0xC693	0x010100
5.2	USB host (not modeled)	--	--

Function 0 sets header type `0x80` (multi-function), so the SRM probes functions 1 and 2 as well. Any function EmulatR does not model returns `0xFFFFFFFF` (master abort), which the SRM treats as "not present" -- safe and expected.

EmulatR model

- `chipsetLib/Cypress_CY82C693ISABridge.h` -- function 0 (Cy82C693IsaBridge): the PCI config-space header that prevents the SRM enumeration loop from stalling, plus the legacy ISA I/O fallback. Implements `IPciDeviceHandler + IIoPortHandler`.
- `deviceLib/Tsunami/Cy82C693Ide.h` -- function 1 (Cy82C693Ide): the IDE controller config space plus the fixed legacy ATA task-file ports (primary command `0x1F0` / control `0x3F6`, secondary `0x170` / `0x376`), carrying the ATA fixed disk (dqa0) and the ATAPI CD (dqa1).

The bridge is wired at hose 0 / bus 0 / slot 5 in `TsunamiChipset::wireDevices()`, which also registers the ISA devices (8259 pair, RTC, COM1/COM2, KBC, Super I/O) at their fixed ports. The ALi M1543C is its ES-class counterpart.

Source References

- `chipsetLib/Cypress_CY82C693ISABridge.h` -- *fn0 PCI header 0x1080:0xC693, hdrtype 0x80.*
- `deviceLib/Tsunami/Cy82C693Ide.h` -- *fn1 IDE config + legacy ATA ports (dqa0/dqa1).*
- `deviceLib/Tsunami/AtaTaskfileEngine.h` -- *taskfile engine answering the ATA/ATAPI units.*
- `chipsetLib/TsunamiChipset.cpp` -- *slot-5 wiring + ISA device registration.*

See Also.

[Cypress](#) ¹⁴⁸

[South Bridge](#) ¹⁴⁵

[Ali M1543C South-Bridge Interface](#) ¹⁴⁷

5.4.5 82077 Floppy Controller

EmulatR models a minimal Intel/DEC 82077AA-class floppy disk controller (FDC) at the legacy ISA `base 0x3F0`. Its purpose is firmware enumeration, not data transfer: it lets the SRM floppy probe for `dva0` reset, initialize, recalibrate, and detect an empty drive so the powerup option-firmware scan concludes "no media" and proceeds, rather than stalling to a retry timeout. It is deliberately NOT a working floppy -- there is no media, no DMA engine, and no real sector transfer (see Limitations).

The `0x3F0` register window is reached through the FDC37C669 Super-I/O, which forwards non-config accesses (and the `0x536` poll register) to the embedded FDC. Command completion is signalled on ISA IRQ6, fed into the 8259 PIC by the chipset.

Register Map

Base `0x3F0`. Offset `+6 (0x3F6)` is owned by the IDE controller, not the FDC. The `0x536` poll register is outside the `0x3F0` window and is reached via the Super-I/O.

Port	Register	Notes
<code>0x3F0 (+0)</code>	SRA (read-only)	Status Register A. Model returns 0x00.
<code>0x3F1 (+1)</code>	SRB (read-only)	Status Register B. Model returns 0x00.
<code>0x3F2 (+2)</code>	DOR (read/write)	Digital Output Register: drive select (bits 1:0), /RESET (bit 2), DMA/IRQ enable (bit 3), motor enable (bits 7:4). A 1->0 on bit 2 resets the FSM; a 0->1 raises the reset-completion IRQ6.

Port	Register	Notes
0x3F3 (+3)	TDR (read/write)	Tape Drive Register. Read returns 0x00.
0x3F4 (+4)	MSR (ro) / DSR (wo)	Main Status Register on read; Data-rate Select Register on write. See Main Status Register below.
0x3F5 (+5)	FIFO (read/write)	Command / parameter / result FIFO. Writes feed the command phase; reads drain the result phase.
0x3F6 (+6)	(IDE)	Owned by the IDE controller, not the FDC.
0x3F7 (+7)	DIR (ro) / CCR (wo)	Digital Input Register on read (returns 0x00 = no disk change); Configuration Control Register on write (data rate).
0x536	IRQ6 poll (TURBO)	Polled-mode floppy interrupt register (outside the 0x3F0 window, via Super-I/O). Read returns 0x80 while an FDC IRQ6 is pending, else 0x00. The 0x0A poll-select write is accepted and ignored.

Main Status Register (0x3F4 read)

Bit	Name	Meaning
7 (0x80)	RQM	Request for Master -- host may read/write a FIFO byte. Always asserted at idle so the firmware's "wait for RQM" poll never spins.
6 (0x40)	DIO	Data direction: 0 = host writes command/params, 1 = host reads results.
5 (0x20)	NDMA	Non-DMA execution mode. 0 during enumeration (no data phase exercised).
4 (0x10)	CB	Controller busy -- set during a command's result phase.

Bit	Name	Meaning
3:0	drive busy	Per-drive seek/busy. 0 in this model.

Phase transitions: command phase RQM=1/DIO=0; result phase RQM=1/DIO=1/CB=1 until the result FIFO is drained, then back to RQM=1/DIO=0/CB=0 (idle).

Command Set

The opcode byte's MT/MF/SK bits (0xE0) are stripped; dispatch is on bits 4:0. "Path" is the completion mechanism (see below). Result values are the empty-drive / not-ready model.

Op	Command	Param / Result	Path	Modeled Result
0x03	SPECIFY	2 / 0	none	accepted; no in
0x04	SENSE DRIVE STA	1 / 1	none	ST3 = 0x00 (Rea
0x05	WRITE DATA	8 / 7	B	ST0 = 0x40 (abn
0x06	READ DATA	8 / 7	B	ST0 = 0x40 (abn
0x07	RECALIBRATE	1 / 0	A	IRQ6 (seek-end)
0x08	SENSE INTERRUPT	0 / 2	A-ack	ST0 = 0x70 (abn
0x0A	READ ID	1 / 7	B	ST0 = 0x40 (abn
0x0C	READ DELETED DA	8 / 7	B	ST0 = 0x40; IRQ
0x0D	FORMAT TRACK	5 / 7	B	ST0 = 0x40; IRQ
0x0E	DUMPREG	0 / 10	none	10 zero bytes
0x0F	SEEK	2 / 0	A	IRQ6 (seek-end)
0x10	VERSION	0 / 1	none	0x90 (82077AA)
0x11	SCAN EQUAL	8 / 7	B	ST0 = 0x40; IRQ
0x12	PERPENDICULAR M	1 / 0	none	accepted; no in
0x13	CONFIGURE	3 / 0	none	accepted; no in
other	(invalid opcode)	0 / 1	none	ST0 = 0x80 (IC

Completion and Interrupt Model

The 82077 has two command-completion mechanisms; conflating them desyncs the controller, so the model branches on them:

Path A -- no result phase, cleared by SENSE INTERRUPT STATUS: RECALIBRATE (0x07), SEEK (0x0F). The command asserts IRQ6 and sets the interrupt-pending flag; the driver then issues SENSE INTERRUPT (0x08) to read ST0/PCN and clear it.

Path B -- 7-byte result phase, cleared by READING the result phase: READ DATA, READ ID, WRITE DATA, and the other data commands. The command asserts IRQ6 and pushes 7 result bytes (ST0,ST1,ST2,C,H,R,N); the interrupt clears when the host reads the last result byte from 0x3F5, NOT via SENSE INTERRUPT.

ISA IRQ6 is edge-triggered, so the model maintains three interrupt edges precisely (these were the fixes that let the option-firmware scan complete instead of timing out):

1. **Reset-completion IRQ** -- deasserting /RESET (DOR bit 2, 0->1) raises IRQ6, the completion interrupt enable_controller waits on.
2. **Command-busy deassert** -- accepting a command opcode clears the pending interrupt (controller goes busy), so the command's completion re-asserts it as a fresh 0->1 edge.

Without this a second IRQ-raising command (e.g. RECALIBRATE after the reset IRQ) would set an already-high line, produce no edge, and the driver's wait would hang to its timeout.

3. Path-B result-drain clear -- reading the final result byte clears the interrupt for result-phase commands.

The pending interrupt is visible two ways: as the 0x80 bit at the 0x536 polled register (polled-mode driver), and as the ISA IRQ6 line into the 8259 PIC (interrupt-mode driver), driven by the chipset's device-IRQ evaluation. SENSE INTERRUPT (0x08) and a controller reset clear it.

Configuration and Wiring

Base port 0x3F0; the FDC owns 0x3F0-0x3F5 and 0x3F7 (0x3F6 is IDE). These ports, plus the 0x536 poll register, are registered to the FDC37C669 Super-I/O, which forwards non-config-mode accesses to the embedded 82077. The chipset feeds the FDC's pending-interrupt line into the 8259 as ISA IRQ6 alongside the other ISA sources. A controller reset is driven by the DOR bit-2 /RESET line. The model carries no media and no

Source References

- `deviceLib/Tsunami/Floppy82077.h` -- *82077AA FDC at 0x3F0: MSR/DOR, IRQ6 edge model, command set.*
- `deviceLib/Tsunami/Smc37c669SuperIo.h` -- *Super-I/O forwards the 0x3F0 window to the embedded FDC.*
- `chipsetLib/TsunamiChipset.cpp` -- *feeds FDC pending-IRQ line into 8259 as ISA IRQ6.*
- `tests/deviceLib/test_floppy82077.cpp` -- *locks enumeration / IRQ-edge behavior.*

5.4.6 DE500 Ethernet (21143 Tulip)

EmulatR models a DEC DE500-class Fast Ethernet adapter built on the DECchip 21143 ("Tulip") PCI Ethernet MAC. This topic documents **Phase 1**: the CSR register file, software reset, serial-ROM (SROM) station-address readout, and the CSR5 status register. Phase 1 is bring-up scaffolding for firmware and OS enumeration -- it presents a correct PCI identity and a readable station (MAC) address so the SRM device probe recognizes the adapter, but it is deliberately NOT a working NIC: there is no transmit/receive datapath, no descriptor-ring DMA, and no PHY/SIA link yet (see Phase Roadmap and Limitations).

The adapter is declared in the platform catalog as `de500_tulip` (`systemLib/PlatformConfig.cpp`) at PCI hose 0, bus 0, slot 7, function 0, interrupt pin INTA. Prior to Phase 1 the catalog entry was a config-space-only `PciModel::Generic` shell; Phase 1 adds the behavioral CSR model behind the same PCI identity.

Note: identity, class code, and BAR sizes in the catalog are marked `_PROVISIONAL` and should be reconciled against a hardware dump; the Phase 1 columns below reflect the intended scaffolding and are to be confirmed against the landed implementation.

PCI Identity and BARs

Field	Value	Notes
Vendor ID	0x1011	Digital Equipment Corporation.
Device ID	0x0019	DECchip 21143.
Class code	0x020000	Ethernet network controller. _PROVISIONAL.
Location	bus 0 / bus 0 / slot 7 / func 0	Fixed in the platform catalog.
Interrupt pin	INTA (1)	Routed through the PCI / chipset interrupt path. Phase 1 raises no interrupts.
BAR0	I/O, 0x80 bytes	CSR access window (I/O space). _PROVISIONAL size.
BAR1	Memory, 0x80 bytes	CSR access window (memory space). _PROVISIONAL size.
Option ROM	none	No expansion ROM is presented.

The 21143 exposes the same sixteen CSRs through both BAR0 (I/O) and BAR1 (memory). Each CSR is 8 bytes apart, so the register block spans 0x80 (16 x 8).

CSR Register Map

Register offset = CSR number x 8. The **Phase 1** column states the intended bring-up behavior; entries marked _PROVISIONAL are to be reconciled against the landed model.

Offset	CSR	Name	Phase 1
0x00	CSR0	Bus Mode / Software Reset	Stored; SWR (bit 0) triggers software reset.
0x08	CSR1	Transmit Poll Demand	Accepted, no-op (no TX engine).
0x10	CSR2	Receive Poll Demand	Accepted, no-op (no RX engine).
0x18	CSR3	Receive Descriptor List Base	Stored; unused (no ring walk).
0x20	CSR4	Transmit Descriptor List Base	Stored; unused (no ring walk).
0x28	CSR5	Status	Returns idle/reset value. See Status Register.
0x30	CSR6	Operating Mode	Stored; TX/RX enable bits latched

Offset	CSR	Name	Phase 1
			but inert.
0x38	CSR7	Interrupt Enable	Stored; nothing raises a summary bit yet.
0x40	CSR8	Missed Frames / Overflow Counter	Reads 0.
0x48	CSR9	Serial ROM & MII	SROM read protocol -- station-address readout. Phase 1 core; see SROM below.
0x50	CSR10	Boot ROM / Diagnostic	Reserved; reads 0. _PROVISIONAL.
0x58	CSR11	General-Purpose Timer	Stored; no countdown.
0x60	CSR12	SIA Status (SIASAT)	Reset value. _PROVISIONAL.
0x68	CSR13	SIA Connectivity (SIACONN)	Stored (SIA reset control).
0x70	CSR14	SIA Transmit and Receive (SIATXRX)	Stored.
0x78	CSR15	SIA General (SIAGEN)	Stored.

CSR12-CSR15 are the 21143 SIA (Serial Interface Adapter) block that drives the on-chip 10/100 media interface. Phase 1 stores them and returns reset values; no link state is modeled.

Software Reset (CSR0 SWR)

Writing 1 to CSR0 bit 0 (SWR) issues a software reset. The bit is self-clearing: the model clears it and restores every CSR to its power-on default, re-primes the SROM read state, and returns CSR5 to its idle value. A PCI reset or device power-on performs the same restoration. The specific reset default values are _PROVISIONAL and should be confirmed against the implementation.

Serial ROM (SROM) and Station Address

The 21143 reads its configuration and 6-byte IEEE 802.3 station (MAC) address from an external 93C46-class serial EEPROM, accessed by bit-banging CSR9. Phase 1 models the SROM as a 128 x 16-bit (256-byte) image in the DEC SROM format; firmware clocks a read command and word address out through CSR9 and shifts the returned data back in one bit per clock.

SROM image layout (DEC format, offsets _PROVISIONAL -- confirm against implementation): an 18-byte header, an SROM format-version byte, the 6-byte station

address near `offset 0x14`, one or more controller info-leaf blocks describing the media, and a CRC in the final two bytes.

CSR9 read-protocol bit roles (exact positions `_PROVISIONAL`): serial-ROM chip select, serial clock, data-in, data-out, plus the SROM/MII select and read/write strobes. The MII management side of CSR9 is not exercised in Phase 1.

The Phase 1 default station address is TBD (confirm; a DEC OUI `08-00-2B` prefix is typical for DE500-class parts). Whether the address is fixed or configurable is an implementation choice to be recorded here once landed.

Status Register (CSR5)

Phase 1 holds the interrupt and process-state bits at their reset (idle) values; because no descriptor process runs, the RS/TS state fields report stopped. Write-1-to-clear behavior on the interrupt bits is `_PROVISIONAL`.

Bit	Name	Meaning
0	TI	Transmit Interrupt.
1	TPS	Transmit Process Stopped.
6	RI	Receive Interrupt.
7	RU	Receive Buffer Unavailable.
8	RPS	Receive Process Stopped.
13	FBE	Fatal Bus Error.
15	AIS	Abnormal Interrupt Summary.
16	NIS	Normal Interrupt Summary.
19:17	RS	Receive Process State. Reports stopped in Phase 1.
22:20	TS	Transmit Process State. Reports stopped in Phase 1.
25:23	EB	Error Bits (bus-error type). 0 in Phase 1.

Abbreviated to the bits firmware inspects during enumeration. The full CSR5 set (TU, TJT, UNF, RWT, ETI, GTE, ERI) is present in hardware but inert in Phase 1.

Phase Roadmap and Limitations

Phase 1 (this topic) -- CSR register file, software reset, SROM station-address readout, and CSR5 status. This is enough for firmware and OS to enumerate the adapter and read its MAC; it is identity and bring-up only.

Not yet modeled -- transmit/receive descriptor-ring DMA, actual packet movement, CSR6 operating-mode effects, SIA/PHY link negotiation, MII management, interrupt generation, and any host network backend. These are deferred to later phases and will be documented here as they land.

Source References

- `systemLib/PlatformConfig.cpp` -- `de500_tulip` *PCI catalog entry: identity* (`0x1011/0x0019`), *class, slot 7 / func 0, BAR0/BAR1, INTA*.
- `deviceLib/Tsunami/Dec21143Tulip.h` -- *Phase 1 CSR file, software reset, SROM readout, CSR5 status. Header-only device model per the deviceLib/Tsunami/ convention (alongside Floppy82077.h, Cy82C693Ide.h, IicPcf8584.h).*
- `EMULATR_TULIP_TRACE` -- *environment variable that traces this device (one-shot DMA-window dump plus per-translate trace). See Environment Variables.*
- *No dedicated unit test yet; a Phase 1 register / reset / SROM test will follow the tests/deviceLib/test_<device>.cpp convention.*

See Also.

[Chipset & Devices](#) ¹³⁹

[Feature Matrix](#) ¹⁸

[Known Gaps](#) ²³

5.5 5.4 Firmware Decode & Decompression

The shipped SRM image is compressed. At every cold boot the firmware decompresses itself on the emulated CPU, using the original DEC SRM decompressor -- a Huffman / inflate engine (Mark Adler's inflate, version c10p1) wrapped by a `DEC_decompress()` routine. EmulatR runs that code faithfully rather than substituting its own.

What the startup does

The guest assembly startup (`ev6_huf_decom.m64`) is more than a call to `decompress()`: it first sets up CPU and PAL state -- establishes `PAL_BASE`, sets the `I_CTL` SDE bits, flushes the ITB / DTB and the instruction cache, primes the shadow registers, and saves / restores the SROM parameters passed in registers R16-R21 -- and only then inflates the image. Any future "skip the inflate" optimization must preserve that exact end-state.

Cost

The inflate inner loop is about 4 million cycles per cold boot (the spin observed at PC `0x60111c`). That one-time cost is what the entry-snapshot boot acceleration removes on later runs: a snapshot captured at the post-decompression handoff (`entryPa`) is restored instead of re-running the inflate. Reaching `>>>` from there still involves the post-handoff initialization, so the snapshot skips the decompression, not the whole boot.

The host oracle

To verify the decompressed bytes are correct without trusting EmulatR's own CPU (or a second emulator), the project builds the **same** DEC source as a native host program in `tools/host_decompressor`. Its output matches EmulatR's decompressed image byte for

byte. This is a stronger reference than AXPBox's `decompressed.rom`, which runs the guest decompressor on AXPBox's own emulated CPU. The oracle settled the "single-bit PAL corruption" question: the `hw_ret R2` instruction at PC `0xd954` is genuine firmware, not a decompression artifact.

The faithful on-CPU path is the default and is the Oracle. A runtime skip is available via the entry-snapshot lever (`EMULATR_FAST_DECOMPRESS=snapshot`), which restores a saved post-decompression state rather than re-inflating. A host-native path that runs the inflate off-CPU while preserving the PAL side-effects remains a possible future addition.

Source References

- `systemLib/SrmLoader.h`, `systemLib/SrmLoader.cpp` -- *compressed-image load, WimC / stub-signature decode, descriptor (entryPa = targetPalBase + finalPC).*
- `systemLib/Machine.cpp` -- *loadSrmFirmware; onBeforeFetch (the post-decompression entryPa handoff trigger).*
- `tools/host_decompressor/ (oracle.c, inflate.c)` -- *offline byte-for-byte decompression oracle / regression guard.*
- `firmwareLib/ (inflate.c, decompressFirmware.c)` -- *in-tree clean-room inflate lift (host side).*
- `Processor Support\Palcode\palcode\apisrm\apisrm\ref\ (ev6_huf_decom.m64, decom.c, inflate.c)` -- *read-only DEC guest sources, reference only.*

See Also.

[Boot Acceleration - Entry Snapshot](#) ^{↗107}

[Execution Flight Plan](#) ^{↗122}

[Current Development Status](#) ^{↗21}

5.5.1 Host Decompressor

`tools/host_decompressor` is a native, host-compiled build of the original DEC SRM Huffman / inflate decompressor. It serves as a trusted reference -- an oracle -- for the compressed Alpha firmware images, producing the decompressed firmware byte-for-byte without running any emulation. This page is the tool view (build, run, outputs); the algorithm and its place in the boot path are covered under Decompressor Internals.

Why it exists

The guest firmware self-decompresses on the emulated CPU at every cold boot (about 4 million cycles -- the spin at PC `0x60111c`). To confirm that EmulatR's CPU produces the correct decompressed image without trusting either EmulatR's own CPU or AXPBox, the tool builds the same algorithm DEC used, from the same sources the firmware was compiled from, as a native host program. The reference image it produces matches EmulatR's emulated-CPU output byte for byte, which makes it both a regression guard and

a clean way to mint a decompressed ROM. It is strictly stronger as an oracle than AXPBox's `decompressed.rom`, which is generated by running the guest decompressor on AXPBox's own emulated CPU and so still depends on a correct CPU and a contrived entry sequence.

Sources

Under `src/`, pinned copies of the read-only DEC originals plus the harness:

- *`inflate.c` -- Mark Adler `inflate`, version `c10p1` (the engine).*
- *`decom.c` -- the DEC wrapper (main + decompress); reference only, not compiled.*
- *`decomp.h` -- types and the ROM / non-ROM switches.*
- *`ev6_huf_decom.m64` -- the guest assembly startup that calls `decompress()`; reference only (it also performs CPU / PAL state setup, which any in-emulator host path must preserve).*
- *`oracle.c` -- the harness: it locates the WimC header, sets the input / output pointers and compressed size, calls `inflate()`, and dumps the image.*

Build and run

Build with the bundled script (`build.sh` on Linux / `gcc`, `build.bat` on Windows), then run the oracle against a compressed firmware image:

```
sh build.sh
./oracle <path-to-compressed-firmware.exe>
```

Output is written to `ref_decompressed.bin` with a printed summary; a known-good run of `es45_v7_3.exe` is cached under `out/`. The `build_firmware_variants.sh` helper drives the oracle across multiple firmware images (the DS10 image, for example, yields `decompressed_ds10_v7_3.bin`, which is the input the Ghidra workflow analyzes).

Compressed-image layout

The oracle finds the firmware payload by a linear scan for the WimC magic (`0x436D6957`). For `es45_v7_3.exe` the header sits at file offset `0x2400`; its longwords give the compressed size (`0x1a6164`), the decompress target (`0x8000`), and the start of the compressed data (WimC + 20 = file `0x2414`). The decompressed output is `0x30cc00` bytes (3,197,952).

The regression guard

In the decompressed image the V4-runtime quad at PA `0xd950` is `0x7be2a000_b4de0030` -- the `hw_ret R2` at `0xd954` uses R2 (byte `0xe2`), never R6 (`0xe6`). This is the signature that settled the 2026-05-30 single-bit-PAL-corruption question: that instruction is genuine firmware, not a decompression artifact. If a future change makes the guest CPU produce R6 in that neighborhood, it is a CPU regression, not a fix.

The runtime boot skip is shipped separately as the entry-snapshot lever

(`EMULATR_FAST_DECOMPRESS=snapshot`, see Boot Acceleration - Entry Snapshot); this host

tool's role is the OFFLINE oracle described above. A clean-room lift of its `inflate.c` plus a WimC wrapper also lives in-tree as `firmwareLib`, but the shipped acceleration is the snapshot model, so `firmwareLib` is compiled-but-unused on the boot hot path.

Source References

- `tools/host_decompressor/ (src/oracle.c, src/inflate.c, src/decomp.h; build.sh / build.bat; build_firmware_variants.sh)` -- the native oracle harness and its outputs.
- `firmwareLib/ (inflate.c, decompressFirmware.c, decomp.h)` -- the in-tree clean-room lift of the same engine.
- `Processor Support\Palcode\palcode\apisrm\apisrm\ref\ (decom.c, ev6_huf_decom.m64)` -- read-only DEC guest sources, reference only.

See Also.

[Boot Acceleration - Entry Snapshot](#) ¹⁰⁷

[Decompressor Internals](#) ¹⁶⁰

[Ghidra](#) ²⁴³

[Firmware Decode & Decompression](#) ¹⁵⁷

5.5.2 Decompressor Internals

Diagnostics are turned on with runtime `EMULATR_*` environment variables, set in the launching shell before the process starts. The complete table lives in the Reference section's Environment Variables page; this page groups them by what they do.

Three gotchas to know first

- *Presence, not value.* Most gates test only that the variable is set, so `VAR=0` still **ENABLES** them. The few that parse a value (the autosnapshot / marker / path / numeric knobs) are noted in the reference.
- *Read once.* The trace and probe gates cache their result on first call, so they are effectively read at process start; exporting them mid-run has no effect.
- *Paths are CWD-relative.* Launch from the Emulatr root so relative output and snapshot paths resolve to the canonical set.

Device / subsystem traces

Presence-gated, read once, written to stderr: `EMULATR_IIC_TRACE` (PCF8584 IIC bus transactions), `EMULATR_FLASH_TRACE` (FlashRom command / erase / write activity), `EMULATR_PIC_TRACE` (the 8259 PIC pair -- IRQ assert / EOI). Redirect to per-trace logs, for example `2> iic.log`.

Chipset diagnostics and retire trace

The chipset CSR diagnostic stream is on by default; `EMULATR_CHIPSET_DIAG_OFF` silences it (note the inverted sense -- setting it turns diagnostics OFF). `EMULATR_TRACE_WINDOW` forces the retire-compact trace window, and `EMULATR_RETIRE_TRACE_DIR` sets the output directory for the retire / breakpoint / profiler trace files (see SRM Performance Logging).

Probe and break gates

For pinpoint debugging: `EMULATR_GCT_WATCH` (store-watch on the GCT / FRU config-tree region), `EMULATR_BREAK_ON_PA10` (conditional break on accesses to PA 0x10..0x17), `EMULATR_CHECKPOINTS` (named PC checkpoints), and `EMULATR_GATE` (a PC / condition gate controlling trace windowing).

Compile-time diagnostics (NOT runtime env vars)

Some `EMULATR_*` names are CMake options baked in at build time -- setting them in the shell does nothing. The important one is `EMULATR_MEMDIAG`: a per-retired-instruction `fprintf+fflush` that imposes roughly a 900x slowdown and must stay OFF for normal use. `EMULATR_DIAGNOSTIC_LOGGING` (centralized per-subsystem logging, non-Release builds) and the `EMULATR_LOG_*` `spdlog` level macros are also build-time.

For traced-boot command recipes, see Executing.

See Also.

[Environment Variables](#) ^{→ 216}

[SRM Performance Logging](#) ^{→ 113}

[Executing](#) ^{→ 74}

5.5.3 Post Flash

Source Function	Source Location
<code>platform_init1()</code>	kernel.c:1287 <- prints the flash message (pc264 chain) ... spinlocks, PCB, memory zone init ...
<code>filesystem_init()</code>	kernel.c:1703
<code>scb_init</code> / <code>ovly_init</code>	kernel.c:1707/1710 ... (further down) <code>initialize_hardware()</code> / <code>reset_hardware()</code> <code>pc264_init.c:472/513</code>
... <code>ddb_startup()</code>	<- first thing that prints "calling module %s" (never reached)

Source Function	Source Location
platform_init2()	kernel.c:1456 <- PC264 secondary platform/hardware init
dyn\$_init() (heap)	kernel.c:1690

Source References

- `systemLib/SrmLoader.cpp` -- *loads the SRM console image these functions live in.*
 - `tools/symbolication/apply_symbols_ds10.py` -- *function-name -> address symbol application.*
 - `systemLib/FirmwareLoader.cpp` -- *firmware / flash image load.*
- The mapped kernel.c / pc264_init.c functions are DEC apismr firmware SSOT, external to the EmulatR C++ tree.

5.6 5.5 PALcode, HWRPB & SMP

PAL\$CPU0_START_BASE	%x100
PAL\$CPU1_START_BASE	%x108
PAL\$PRIMARY	%x200
PAL\$CALLBACK	%x210
PAL\$HALT_SWITCH_IN	%x220

PAL\$HWRPB_BASE %x2000

PAL\$IMPURE_BASE	%x4000
PAL\$IMPURE_COMMON_SIZE	%x200
PAL\$IMPURE_SPECIFIC_SIZE	%x600
PAL__IMPURE_BASE	PAL\$IMPURE_BASE
PAL__IMPURE_COMMON_SIZE	PAL\$IMPURE_COMMON_SIZE
PAL__IMPURE_SPECIFIC_SIZE	PAL\$IMPURE_SPECIFIC_SIZE

%x6000

%x400

PAL\$TEMPS_BASE	%x7000
PAL\$TEMPS_SPECIFIC_SIZE	%x200
PAL__TEMPS_BASE	PAL\$TEMPS_BASE
PAL__TEMPS_SPECIFIC_SIZE	PAL\$TEMPS_SPECIFIC_SIZE

PAL\$PAL_BASE	%x8000	
PAL\$OSFPAL_BASE	%x18000	
PAL\$CONSOLE_BASE	%x24000	
PAL\$PAL_SIZE		PAL\$OSFPAL_BASE - PAL\$PAL_BASE
PAL\$OSFPAL_SIZE	PAL\$CONSOLE_BASE - PAL\$OSFPAL_BASE	
PAL__PAL_BASE	PAL\$PAL_BASE	
PAL__OSFPAL_BASE	PAL\$OSFPAL_BASE	
PAL__CONSOLE_BASE	PAL\$CONSOLE_BASE	
/*		
/* DEFINE ADDRESSES OF LOCATIONS IN THE IMPURE		
/* COMMON AREA. ASSUMES BASE AT 0X4000 AND LOCATIONS		
/* STARTING AT 0X100.		
PAL\$EVMSVERSION	PAL\$IMPURE_BASE + %x100	
PAL\$EVMSBUILD_BITMASK	PAL\$IMPURE_BASE + %x108	
PAL\$OSFVERSION	PAL\$IMPURE_BASE + %x110	
PAL\$ENTER_CONSOLE_PTR	PAL\$IMPURE_BASE + %x140	
PAL\$ENTER_OSF	PAL\$IMPURE_BASE + %x180	
PAL__ENTER_OSF	PAL\$ENTER_OSF	
IRQ_DEV__M	%x2	/* IRQ1 */
IMPURE\$PCB_OFFSET	%x10	
IMPURE\$PROF_OFFSET	%x40	
IMPURE\$QUILT_OFFSET	%x40	
IMPURE\$UART_OFFSET	%x70	
IMPURE\$CPU_TYPE_OFFSET	%x80	
MP\$RESTART	1	
MP\$HALT	2	
MP\$CONTROL_P	3	

MP\$BREAK

4

```
/*=====
/* Define cserve funtions
/*=====
```

8

9

16

17

18

19

64

65

66

67

68

69

70

101

CSERVE__START
CSERVE__CALLBACK

CSERVE\$START
CSERVE\$CALLBACK

Source References

- `coreLib/Ev6Pc264PalDefs.h` -- *PAL\$ base addresses, IMPURE offsets, CSERVE\$ function codes.*
- `palBoxLib/grains/PalEntries.cpp` -- *CALL_PAL / CSERVE dispatch consuming those bases/codes.*

5.6.1 HWRPB Key/Value Offset Map

This page documents the `kKeyValue` offset map for the HWRPB (Hardware Restart Parameter Block) and its chained structures, as encoded in `deviceLib/Hwrpb.h`. It is a navigation aid; the authoritative layout is the AARM Section III "Common Architecture, Console Interface" (Table 26-1 header, Table 26-4 per-CPU slot), cross-referenced against the read-only `apism/srmapi` SSOT and the live DS20 SRM dump. EmulatR is the only tree edited; `apism/srmapi` is never modified.

Source References

- `deviceLib/Hwrpb.h` -- *kKeyValue offset keys, HwrpbHeader/PerCpuSlot structs, static_asserts.*
- `deviceLib/HwrpbBuilder.cpp` -- *builds the HWRPB (intrclock_freq / cycle_count_freq, SYSVAR).*
- `deviceLib/HwpcbContext.h` -- *per-CPU HWPCB slot context.*

See Also. journals/HWRPB_PerCpuSlot_FieldMap_AARM_20260629.md (the full AARM-vs-Hwrbp.h reconciliation), journals/HWRPB_Region_Fidelity_and_Resume_20260624.md (region methodology).

The kKeyValue pattern

Each HWRPB field has a convenience constexpr "key = value" constant (value is the field's byte offset from its region base), in the same spirit as kAlphaPageSize. A field can then be reached two equivalent ways: by typed struct member, or by offset key over a raw byte window through a View union. The two paths are pinned together by static_asserts so they can never silently drift.

```
// offset keys (constexpr uint64_t, like kAlphaPageSize = 8192)
constexpr uint64_t kSlotStateFlags = 0x080;
constexpr uint64_t kSlotCycleCounterFreq = 0x270;
constexpr uint64_t kSlotSize = 0x280;

// View union: overlay a raw byte window with the typed struct
union PerCpuSlotView { uint8_t raw[kSlotSize]; PerCpuSlot fields; };

// typed-member path and offset-key path are equivalent:
uint64_t a = view.fields.state.raw;
uint64_t b = peek<uint64_t>(view.raw, kSlotStateFlags); // a == b
poke<uint64_t>(hdrBase, kHwrbpSystemVariation, 0x1805); // DS20 SYSVAR
```

peek/poke are memcpy-based, so they are alignment- and strict-aliasing-safe wherever the base points in guest memory. Host (x86_64) and guest (Alpha) are both little-endian and the structs are #pragma pack(1), so the overlay is byte-faithful.

HWRPB header offset keys (AARM Table 26-1)

Key	Offset	AARM field	HwrbpHeader member
kHwrbpPhysAddr	0	self physical address	hwrbp_pa
kHwrbpId	8	"HWRPB" identifier	identifier
kHwrbpRevision	16	HWRPB revision	revision
kHwrbpSize	24	total HWRPB size	hwrbp_size
kHwrbpPrimaryCpuld	32	primary WHAMI	primary_cpu_id
kHwrbpPageSize	40	page size (8192)	page_size
kHwrbpPaSize	48	PA size in bits	pa_size_bits
kHwrbpExtVaSize	52	extended VA size	extended_va_size
kHwrbpMaxValidAsn	56	max valid ASN	max_valid_asn
kHwrbpSysSerialNum	64	system serial (octaword)	system_serial_number
kHwrbpSystemType	80	SYSYTYPE	system_type

Key	Offset	AARM field	HwrpbHeader member
kHwrpbSystemVariation	88	SYSVAR (badge member)	system_variation
kHwrpbSystemRevision	96	system revision (ASCII)	system_revision
kHwrpbIntrClockFreq	104	interval-clock freq	intrclock_freq
kHwrpbCycleCountFreq	112	cycle-counter freq (fallback)	cycle_count_freq
kHwrpbVptbVa	120	virtual page-table base VA	vptb_va
kHwrpbReservedArch	128	reserved for architecture	reserved_arch
kHwrpbTbbOffset	136	-> TB hint block	tbb_offset
kHwrpbCpuSlotCount	144	number of per-CPU slots	cpu_slot_count
kHwrpbCpuSlotSize	152	size of each slot (= > 0x280)	cpu_slot_size
kHwrpbCpuSlotOffset	160	-> per-CPU slot array	cpu_slot_offset
kHwrpbCtbCount	168	number of CTBs	ctb_count
kHwrpbCtbSize	176	size of each CTB	ctb_size
kHwrpbCtbOffset	184	-> CTB table	ctb_offset
kHwrpbCrbOffset	192	-> Console Routine Block	crb_offset
kHwrpbMddtOffset	200	-> Memory Data Descriptor Table	mddt_offset
kHwrpbCdbOffset	208	-> Configuration Data Block	cdb_offset
kHwrpbFruOffset	216	-> FRU table	fru_offset
kHwrpbTermSaveVa	224	save-state routine VA	terminal_save_va
kHwrpbTermSavePv	232	save-state routine PV	terminal_save_pv
kHwrpbTermRestoreVa	240	restore-state routine VA	terminal_restore_va
kHwrpbTermRestorePv	248	restore-state routine PV	terminal_restore_pv
kHwrpbCpuRestartVa	256	CPU restart routine VA	cpu_restart_va
kHwrpbCpuRestartPv	264	CPU restart routine PV	cpu_restart_pv
kHwrpbReservedSw	272	reserved for software	reserved_software
kHwrpbReservedHw	280	reserved for hardware	reserved_hardware
kHwrpbChecksum	288	header checksum	checksum
kHwrpbRxtxBlock	296	RX/TX serial extension	rxtx_block
kHwrpbDsrdBOffset	312	-> DSRDB section	dsrdb_offset

Key	Offset	AARM field	HwrpbHeader member
kHwrpbHeaderSize	320	fixed-header size	sizeof(HwrpbHeader)

Per-CPU slot offset keys (AARM Table 26-4)

The slot stride is 0x280 (640 bytes): the AARM fields end at Cycle Counter Frequency (+624, last byte +631), padded to an octaword. This matches the live DS20 SRM. The five fields at +592..+624 were previously hidden in an opaque filler, and an in-slot DSRDB had inflated the slot to 0x400 (corrected 2026-06-29).

Key	Offset	AARM field	PerCpuSlot member
kSlotHwpcb	0x000	HWPCB (128 B)	hwpcb
kSlotStateFlags	0x080	STATE FLAGS (BIP/RC/PA/PP/...)	state
kSlotPalMemLen	0x088	PALcode memory length	pal_mem_len
kSlotPalScrLen	0x090	PALcode scratch length	pal_scr_len
kSlotPalMemPa	0x098	PA of PALcode memory	pal_mem_pa
kSlotPalScrPa	0x0A0	PA of PALcode scratch	pal_scr_pa
kSlotPalRevision	0x0A8	PALCODE REVISION (qword)	pal_rev
kSlotPalVariation	0x0AC	(low-half split; see note)	pal_var
kSlotProcType	0x0B0	PROCESSOR TYPE	cpu_type
kSlotProcVariation	0x0B8	PROC VARIATION (PE=bit2/IEEE/VAX)	cpu_var
kSlotProcRevision	0x0C0	PROCESSOR REVISION (ASCII)	cpu_rev
kSlotProcSerial	0x0C8	PROC SERIAL (octaword)	serial_no
kSlotLogoutPa	0x0D8	PA of logout area	logout_mem_pa
kSlotLogoutLen	0x0E0	logout area length	logout_length
kSlotHaltPcbb	0x0E8	HALT PCBB	halt_pcbb
kSlotHaltPc	0x0F0	HALT PC	halt_pc
kSlotHaltPs	0x0F8	HALT PS	halt_ps
kSlotHaltArgList	0x100	HALT ARG LIST (R25)	halt_arglist
kSlotHaltRetAddr	0x108	HALT RETURN ADDR (R26)	halt_ret_addr
kSlotHaltProcValue	0x110	HALT PROC VALUE (R27)	halt_proc_value

Key	Offset	AARM field	PerCpuSlot member
kSlotReasonForHalt	0x118	REASON FOR HALT	halt_code
kSlotReservedSw	0x120	reserved for software	reserved_sw
kSlotRxtxBuffer	0x128	RXTX BUFFER AREA (168 B)	icba
kSlotPalcodeAvailable	0x1D0	PALCODE AVAILABLE ([0]=fw rev)	palcode_revs
kSlotSwCompat	0x250	+592 SW- compatibility	sw_compat
kSlotConsoleLogPa	0x258	+600 console data- log PA	console_log_pa
kSlotConsoleLogLen	0x260	+608 console data- log length	console_log_len
kSlotCacheDescriptor	0x268	+616 cache descriptor	cache_descriptor
kSlotCycleCounterFreq	0x270	+624 cycle-counter freq	cycle_count_freq
kSlotSize	0x280	slot stride (640 bytes)	sizeof(PerCpuSlot)

Note (PALCODE REVISION): the AARM defines +168 as one bit-packed quadword (bits 63:48 max processors sharing this PALcode, 47:32 compatibility, 23:16 variation, 15:8 major, 7:0 minor). EmulatR keeps a pal_rev/pal_var u32 split at 0x0A8/0x0AC: the offset is correct, but the high 32 bits (max-procs + compat) are not separately modeled. Widen when the PAL-rev publishing path is reworked.

Note (REASON FOR HALT): the AARM halt codes (0 boot, 1 operator crash, 2 kstack-not-valid, 3 invalid SCBB, 4 invalid PTBR, 5 CALL_PAL HALT in kernel, 6 double-error, 7 machine-check in PALcode) do not match EmulatR's HaltCode enum numbering. Reconcile before relying on the halt reason cross-process.

DSRDB section offset keys

The DSRDB (Dynamic System Recognition Data Block) is a separate top-level HWRPB section reached via kHwrpbDsrdboffset (+312), found live on DS20 at PA 0x2ac0. It is NOT part of the per-CPU slot (it was incorrectly embedded there before 2026-06-29). Validate this layout field-by-field against the live 0x2ac0 dump before relying on it.

Key	Offset	Field
kDsrdbsmm	0x00	system marketing-model code
kDsrdblurtOffset	0x08	offset to LURT table
kDsrdbynameOffset	0x10	offset to system name
kDsrdblurtCount	0x18	number of LURT entries
kDsrdblurtTable	0x20	LURT table (20 qwords)

Key	Offset	Field
kDsrdNameCount	0xC0	length of system name
kDsrdName	0xC8	system name (7 qwords)
kDsrdSize	0x100	DSRDB size (256 bytes)

Correctness: keys pinned to struct offsets

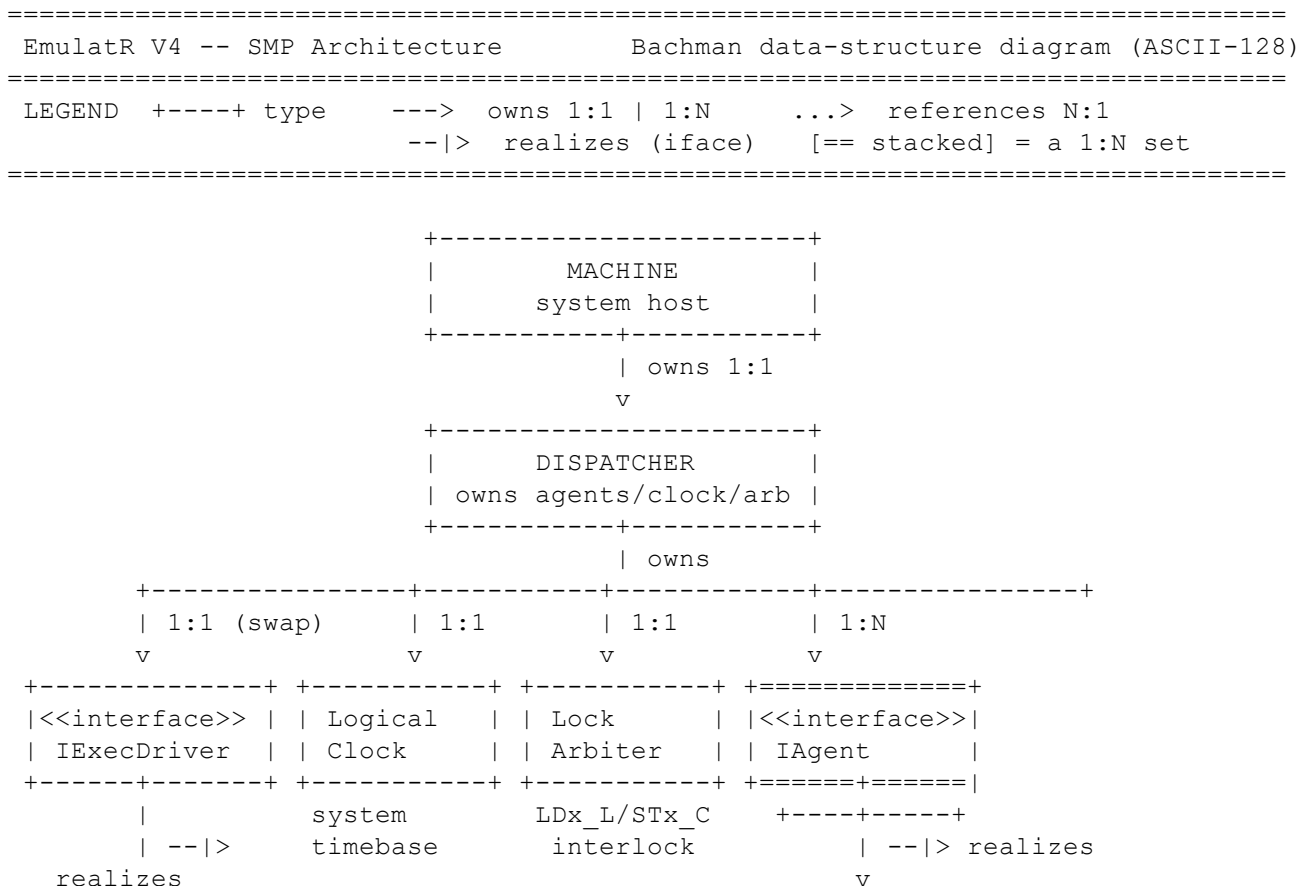
A block of static_asserts at the end of Hwrpb.h ties each key to the matching offsetof() in its struct, so the typed-member path and the offset-key path can never silently diverge as the layout evolves:

```
static_assert(kSlotCycleCounterFreq == offsetof(PerCpuSlot,
cycle_count_freq), "key/cycle_count_freq");
static_assert(kHwrpbSystemVariation == offsetof(HwrpbHeader,
system_variation), "key/sysvar");
```

Verified with g++ -std=c++17: the header compiles clean and every static_assert passes (PerCpuSlot = 0x280, HwrpbHeader = 320, DsrdName = 0x100, cycle_count_freq at 0x270).

See Also. [What is EmulatR](#)⁶

5.6.2 SMP (MultiProcessor) Design



```

+-----+-----+
v          v
+-----+ +-----+
|Sequent.| | Threaded |
|Driver  | | Driver   |
+-----+ +-----+
determ.   barrier
default   opt-in

+-----+
|      AlphaCpuAgent      |
| step(); id()->cpuSlot |
+-----+
| owns 1:1
v
+-----+
|      CpuState          |
+-----+
| contains
v
+-----+
| CpuState members (owned, per-agent) |
|  intReg[0..31]   fpReg[0..31]      |
|  pc  palBase  scbb  ASN             |
|  cycleCount (PCC, per-CPU) + ccOffset |
|  cpuSlot          <-- STEP 4 set-site |
|  itbMgr, dtbMgr : SPAMShardManager<16,8> |
|  reservedCacheLine, hasReservation (LL/SC) |
|  IER / ISUM  shadow SP {K,E,S,U}  I_CTL |
+-----+

```

SHARED SERVICES (Machine owns 1:1 each; AlphaCpuAgent references N:1)

```

+-----+ +-----+ +-----+
| GuestMemory | | TsunamiChipset | | TraceSink |
+-----+ +-----+ +-----+
one instance   per-cpuId arrays:   single ordered
(shared)       dim[]/iic[]/         stream
               pendingIrq2[]/3[]
AlphaCpuAgent ...> { GuestMemory, TsunamiChipset, TraceSink } (N:1, not owned)

```

OWNERSHIP SETS (Bachman)		CARD.	notes
Machine	-> Dispatcher	1:1	
Machine	-> GuestMemory	1:1	shared service
Machine	-> TsunamiChipset	1:1	shared service
Machine	-> TraceSink	1:1	shared service
Dispatcher	-> IExecDriver	1:1	swappable policy
Dispatcher	-> LogicalClock	1:1	system timebase (cycle-unit)
Dispatcher	-> LockArbiter	1:1	LDx_L / STx_C interlock
Dispatcher	-> IAgent	1:N	* the SMP set
AlphaCpuAgent	-> CpuState	1:1	
CpuState	-> (members)	1:1	contains

REALIZES (interface):

```

SequentialDriver, ThreadedDriver --|> IExecDriver
AlphaCpuAgent                    --|> IAgent

```

REFERENCES (N:1, NOT ownership):

```

AlphaCpuAgent ...> GuestMemory, TsunamiChipset, TraceSink

```

NOTES

* Determinism boundary: per-CPU state is AGENT-owned; clock, arbiter, memory,


```

    chipset, trace are DISPATCHER-owned shared services (agent-referenced).
* Execution policy is a swap behind IExecDriver -- SequentialDriver is the
  deterministic default; ThreadedDriver (barrier-synchronised) is opt-in.
* TsunamiChipset is ~80% SMP-ready: per-cpuId arrays + MISC CUID injection
  + read/write(offset,cpuId) overloads.
* reservation fields migrate out of CpuState to a ReservationManager (Phase 3).
* cpuSlot is default 0 today (single agent); STEP 4 sets it from agent id().
=====

```

Source References

- `schedLib/AlphaCpuAgent.{h,cpp}` -- *the per-CPU agent (step over Machine::stepCycle); realizes IAgent.*
- `schedLib/SmpDrivers.h` -- *SequentialDriver (deterministic default) and ThreadedDriver; SmpHarness.h -- the Dispatcher.*
- `systemLib/Machine.cpp` -- *run()'s Dispatcher / SequentialDriver setup; stepCycle = cpuKernel (PipelineDriver::step) + systemTick.*
- `coreLib/CpuState.h` -- *the per-agent CpuState (registers, IPRs, cpuSlot, reservation fields).*

5.6.3 ES40 TIG Module Reset (Experimental)

On the ES40 in silicon mode, exiting the firmware update utility (or an `update srm` followed by the console restart) can stall at `Initializing...` with no second boot banner. The root cause is an unmodeled TIG-bus module reset: the firmware asks the platform to reset itself and then spins, expecting the machine to reboot.

Root cause (stable)

The console restart path issues a co-gated pair of writes to the TIG control bus and then a branch-to-self, expecting a hardware reset to break the loop: an ARM write to control offset `+0x280` (PA `0x80130000280`) followed by a COMMIT write to offset `+0x600` (PA `0x80130000600`), then `BR .` at the dead-loop site. If the emulator treats those TIG registers as passive stores, the reset never happens and the CPU spins forever -- the observed hang. This is ES40 (Typhoon) specific; the DS10 / DS20 (Tsunami) firmware never issues the triad and soft-re-inits in software.

The model (experimental)

The reset is modeled behind an opt-in knob and is not yet a settled feature. `TsunamiTig` detects the co-gated ARM-then-COMMIT protocol (never a bare commit) and raises a reset request; `TsunamiChipset` exposes it; and `Machine::systemTick` applies a module reset at a clean tick boundary per HRM 12.1.1 -- re-initialize the chipset CSRs to their Chapter-10 reset values, re-seed the firmware image into guest memory (the SROM's job), and re-enter the loaded firmware entry. The reset is armed only when both `EMULATR_BRINGUP_PROBES` is compiled in and `EMULATR_TIG_RESET` is set; it is inert by default.

The commit bit pattern is provisional (marked in code) pending confirmation against the HRM / RMC-TIG register map. Treat this page as a description of work in progress, not final behavior.

Source References

- `chipsetLib/TsunamiTig.h` -- *co-gated reset-triad detect (`kResetArm +0x280`, `kResetCommit +0x600`), `resetRequested`.*
- `chipsetLib/TsunamiChipset.h` -- *`tigResetRequested()` / `clearTigResetRequest()`; `chipset reset()`.*
- `systemLib/Machine.cpp` -- *`systemTick reset block`; `reseedFirmwareForReset` (re-lay the firmware image, leave flash).*
- `journals/hmdoc_20260713_es40_tig_reset_and_cchip_register_audit.md` -- *the investigation and the HRM 12.1.1 basis.*

See Also.

[Firmware Hangs / Slow Boot](#) ²²⁸
[Execution / Boot Internals](#) ¹²³
[Firmware Update Utility \(LFU\)](#) ¹⁰⁹

5.7 5.6 Platform Execution Modes & Clock

The Alpha console firmware runs in one of two modes, and the choice changes its entire initialization strategy. Virtual mode (ISP) is DEC's pre-silicon software simulator path: the firmware assumes it runs on the Instruction-Set-Processor engineering model rather than on chips, and deliberately skips the real-hardware probe, timing, and device-driver-attach steps. REAL_HW is the actual-silicon path: the firmware drives the full probe, reset, timing, and attach sequences. EmulatR answers "virtual (ISP)" by default, which is why the console reaches the operator prompt cleanly and quickly, and equally why real device discovery does not happen in that mode.

The mode flag: `platform()`

The firmware carries its own detector: a single 32-bit read of physical address `0x0000_BFFC`. On the ISP simulator, DEC's startup deposits the sentinel `0xCAFEDEED` there; on real hardware that location holds something else, so the detector returns REAL_HW.

```
int platform( ) {
    /* ISP flag at 0xBFFC. ISP startup deposits 0xCAFEDEED there. */
    if ( (*(int *)0xBFFC == 0xCAFEDEED) )
        return ISP_MODEL; /* pre-silicon software simulator */
    else
        return REAL_HW; /* actual silicon */
}
```

Reference: `apismr_pc264.c:173`; `ISP_MODEL / REAL_HW` defined in `pc264_io.h` and `emulate.h (ISP_MODEL = 1)`.

How EmulatR selects the mode

EmulatR does not deposit anything into memory; it presents the sentinel as a read intercept, so the value survives the SRM self-decompressor (which overwrites image offset `0x3FFC == PA 0xBFFC`, so a one-time store would not last). The environment knob `EMULATR_PLATFORM` selects the mode at runtime; no rebuild is needed.

```
// EMULATR_PLATFORM unset|"isp" => ISP (default; reaches >>>)
// "silicon" => REAL_HW (no intercept)
if (s_isp && pa == 0x0000000000000BFFCull)
    br.data = 0xCAFEDEEDFull; // platform() => ISP_MODEL
```

Reference: `pipelineLib/MemDrainer.h:458-486`. There is intentionally one flag; the firmware's own `platform()` is the single source of truth.

Execution-path differences

The same firmware takes two very different paths depending on the mode:

Stage	Virtual (ISP_MODEL)	REAL_HW
<code>platform()</code> result	Returns <code>ISP_MODEL</code> ; <code>0xBFFC</code> reads the <code>0xCAFEDEED</code> sentinel	Returns <code>REAL_HW</code> ; <code>0xBFFC</code> holds non-sentinel data
Device probe / attach	Short-circuited -- e.g. <code>dq_initialize</code> returns <code>msg_failure</code> before reset / IDENTIFY	Full probe -> reset -> init -> IDENTIFY runs
Micro-delays (<code>kern\$ micro_delay</code>)	Return immediately (no wait)	Busy-wait on <code>RSCC</code> for <code>cpm*us</code> scaled cycles
IDE taskfile	No <code>0x1F0/0x170</code> accesses; IDENTIFY never issued	Taskfile driven; <code>dqa</code> enumerates
show config vs show dev	Controller shown in <code>show config</code> (PCI topology walk); no <code>dqa</code> in <code>show dev</code>	<code>dqa/dqb</code> enumerate in <code>show dev</code>
Net result	Reaches <code>P00>>></code> quickly; no real device datapath	Real discovery, but stalls on unmodeled timing loops (needs warp / modeling)

Why ISP is fast and silicon grinds

The single mechanism behind the speed difference is the micro-delay. In virtual mode `krn$_micro_delay` returns immediately; in silicon mode it busy-waits on the scaled cycle counter (RSCC) for the full requested interval. Probe and reset sequences call it constantly, so silicon mode spends millions of emulated instructions spinning where ISP mode spends none.

```
void krn$_micro_delay (INT us) {
    if (platform() == ISP_MODEL) return; // ISP: no delay at all
    cpm = cycles_per_microsecond;
    rsccl(&start); // CALL_PAL RSCC
    limit = (cpm * us) + start;
    do { rsccl(&stop); } while (stop < limit); // silicon: busy-wait on RSCC
}
```

EmulatR models `RSCC = cycleCount/5594`. The faithful acceleration for silicon mode advances `cpu.cycleCount` coherently during a recognized pure delay spin so RSCC crosses limit and the loop exits (the same approach as `EMULATR_IDLEWARP`). It must never blindly jump the counter or write timebase state out-of-band; the quarantined `EMULATR_RSCCWARP` did (rewriting `0x3c970`) and corrupted the boot.

Choosing a mode

Virtual (ISP) is the frozen faithful-oracle-to-prompt mode: it reaches `P00>>>` with device presence via the topology walk, but no device datapath. `REAL_HW` is required for actual device discovery and, eventually, OS boot -- but it runs to completion only once the real-hardware timing loops are modeled or warped past. Any device or behavior the firmware gates behind `platform() == REAL_HW` is invisible in ISP mode, no matter how faithfully the device itself is modeled -- so mode is the first thing to check when reading a "device not discovered" symptom.

References

Firmware: `pc264.c:173 platform()`; `pc264_io.h / emulate.h ISP_MODEL`;
`krn$_micro_delay (RSCC busy-wait)`; `dq_driver.c:435 ISP gate`;
`dq_driver.c:676 dq_init_ide (prog-IF + Cypress special case)`.
EmulatR: `pipelineLib/MemDrainer.h:458-486 (0xBFFC read-intercept +`
`EMULATR_PLATFORM knob)`; `RSCC = cycleCount/5594`;
`EMULATR_IDLEWARP (coherent cycleCount advance)`.
Runner: `tools/run_es40_showdev.sh (PLATFORM / WARP knobs, stamped)`.

Source References

- `pipelineLib/MemDrainer.h -- 0xBFFC read-intercept => 0xCAFEDEED,`
`EMULATR_PLATFORM knob, RSCC / IDLEWARP.`

- `deviceLib/Tsunami/AtaTaskfileEngine.h` -- *IDE taskfile driven only on the REAL_HW path.*
- `deviceLib/BadgeMhzGauge.h` -- *records the ISP default.*

The firmware decision points (`pc264.c platform()`, `dq_driver.c`) are DEC apism SSOT, external to the tree.

5.7.1 Platform Execution Mode: ISP vs Silicon

1. Why this matters (the significance)

EmulatR presents the Alpha console firmware with a single, firmware-visible flag that tells it whether it is running on **DEC's pre-silicon ISP software simulator** or on **real hardware**. That one flag changes the firmware's entire initialization strategy. By default EmulatR answers "ISP simulator," and in that mode the SRM console **deliberately skips the real-hardware probe, timing, and device-driver-attach steps** that V4 does not yet model -- which is exactly why the console reaches `P00>>>` cleanly, and equally why platforms like the ES40 do **not** enumerate their IDE (`dqa/dqb`) drives in `show dev`.

This is not a device bug and not a chipset bug. It is a **whole-machine execution-mode choice**. Any device or behavior the firmware gates behind `platform() == REAL_HW` is invisible in ISP mode, no matter how faithfully the device itself is modeled. Understanding this lever is a prerequisite for reading any "device does not appear / is not discovered" symptom on an EmulatR platform.

EmulatR presents `0BFFC -> 0xCAFEDEED` -> firmware `platform() - ISP_MODEL -> firmware skips the real-hardware timing/probe/IDE steps -> reaches >>>`.

2. The firmware mechanism (single source of truth: `platform()`)

The console firmware carries its own detector. apism `pc264.c` line 173:

```
int platform( ) {
    int platform;
    /* ISP flag at location 0xBFFC. In your ISP startup command
     * procedure, deposit 0xBFFC with CAFEDEED. */
    if ( (*(int *)0xBFFC == 0xCAFEDEED) )
        platform = ISP_MODEL;
    else
        platform = REAL_HW;
    return platform;
}
```

- `ISP_MODEL` and `REAL_HW` are defined in `pc264_io.h` (`enum {ISP_MODEL = 1, REAL_HW}`) and `emulate.h` (`#define ISP_MODEL 1`).

- ISP == DEC's Instruction-Set-Processor / engineering *virtual* model (the pre-silicon software simulator DEC used to bring up the SRM before chips existed). REAL_HW == actual silicon.
- The detector is a single 32-bit read of physical address 0x0000_BFFC. On the ISP simulator, DEC's startup deposits the sentinel 0xCAFEDEED there; on real hardware that location holds something else, so `platform()` returns REAL_HW.

3. How EmulatR selects the mode (the lever)

EmulatR does NOT deposit anything. It presents the sentinel as a **read intercept** so the value survives the SRM self-decompressor (which overwrites image offset 0x3FFC == PA 0xBFFC, so a one-time store would not last). See `pipelineLib/MemDrainer.h` lines 458-486:

```
// env -> 0xBFFC answer -> firmware platform().
// EMULATR_PLATFORM: unset or "isp" => ISP (default; reaches >>>);
// "silicon" => REAL_HW (no intercept).
static bool const s_isp = <EMULATR_PLATFORM != "silicon">;
if (s_isp && pa == 0x0000000000000BFFCull)
    br.data = 0xCAFEDEEDFull; // platform() => ISP_MODEL
```

Environment knob:

```
EMULATR_PLATFORM = (unset) | isp -> ISP_MODEL (DEFAULT)
EMULATR_PLATFORM = silicon -> REAL_HW
```

There is intentionally ONE flag; the firmware's own `platform()` is the single source of truth. (`BadgeMhzGauge.h` and others note the ISP default explicitly.)

4. What ISP mode skips (the consequence)

In ISP mode the firmware short-circuits every step that would touch real hardware it assumes the simulator does not model. The `platform() == ISP_MODEL` guard appears throughout the driver set, e.g.:

```
dq_driver.c:435 IDE -- "IDE controller is not emulated", return failure
dac960_driver.c:406 Mylex RAID
esc_nvram_driver.c ESC NVRAM (several sites)
ew_driver.c:5436 Ethernet
n810_driver.c:563 n810
kernel.c:1852 kernel init branch
```

For the IDE specifically, `dq_initialize()` returns `msg_failure` BEFORE calling `dq_init_port -> dq_reset -> dq_init_ide -> dq_identify`. So in ISP mode the SRM **never issues IDENTIFY over the taskfile** -- which is precisely the observed signature on the ES40: the console lists the controller and channel names in `show config` (that comes from

the PCI topology walk), but `show dev` shows no `dqa`, and an IDE I/O trace shows zero `0x1F0/0x170` taskfile accesses.

5. The IDE path in silicon mode (what SHOULD happen)

In `REAL_HW` mode `dq_init_ide()` (`dq_driver.c:676`) runs and chooses the taskfile base from the controller's programming interface:

```
progif = incfgeb(pb, 0x09);
// primary channel:
if (progif & 0x01) csr = incfgl(pb, 0x10) & ~3; // NATIVE: BAR0
else csr = 0x1F0; // COMPAT: fixed port
// + a Cypress-only special case:
if (id == 0xC6931080 && class == 0x0101) { csr = 0x1F0; program BARs; }
```

Consequences for EmulatR's controllers:

- **DS10/DS20 (Cypress CY82C693, id 0xC6931080)** hit the special case that forces `csr = 0x1F0` -- this is *why they enumerate IDE*.
- **ES40/ES45/DS25 (ALi M5229, id 0x522910B9)** do NOT match the special case, so they use the prog-IF path. EmulatR's `AliM5229Ide` presents prog-IF **0x00** (compatibility mode), so `dq_init_ide` selects the fixed `csr = 0x1F0` -- which the shared `AtaTaskfileEngine` already answers. No native/BAR/DMA path is required for console enumeration.

So the ALi IDE config (identity + compat prog-IF) is already correct for discovery; it simply never runs while EmulatR is in ISP mode.

NOTE (open): DS10/DS20 currently enumerate IDE under EmulatR's ISP default, yet the `dq_initialize` ISP gate is upstream of the Cypress special case. Either the shipped ds20 (pc264) console's IDE path lacks that gate, or its device-dispatch differs from the reference source. To be confirmed against the ds20 binary.

6. The tension (why EmulatR defaults to ISP)

ISP mode is a deliberate, honest simplification: it lets the console reach the operator prompt on a machine model that does not yet implement the real-hardware timing loops, register handshakes, and probe sequences the silicon path drives. The MemDrainer note is explicit that the `REAL_HW` path is "currently unmodeled -> stalls." So today:

Mode	Behavior
ISP mode	Reaches <code>P00>>></code> , but NO real device discovery (IDE/SCSI/etc.).
Silicon mode	Real device discovery, but stalls on unmodeled real-HW timing/looping counters (needs warp, and/or modeling).

This maps directly onto the V5 trajectory. Phase A (the faithful Oracle to the SRM prompt) is served by ISP mode with device PRESENCE/PROBE via the topology walk. Actual device DATAPATH discovery -- and eventual OS boot -- requires the silicon path, which is a Phase-B/optimization concern: model or warp the real-hardware timing so REAL_HW mode runs to completion.

7. How to run each mode

```
# ISP (default): reaches P00>>>, IDE not discovered
./tools/run_es40_showdev.sh

# Silicon + warp experiment: real discovery; bypass the looping counters
PLATFORM=silicon WARP=1 ./tools/run_es40_showdev.sh
# -> EMULATR_PLATFORM=silicon (drop the 0xBFFC intercept -> REAL_HW)
# -> WARP=1 sets EMULATR_IDLEWARP (skip idle/tick delay loops)
```

The run log's build stamp records `platform=` and `warp=` so every run is self-identifying. No rebuild is needed to switch modes: the lever is a runtime read-intercept keyed off the environment.

8. Open questions / next steps

1. Does `PLATFORM=silicon WARP=1` reach the ES40 IDE probe, or stall earlier on another unmodeled real-HW step? The console tail + trace locate the stall.
2. Enumerate every `platform() == ISP_MODEL` site the ES40 SRM hits on the silicon path; each is an unmodeled-hardware step to model or warp past.
3. Confirm the DS20 (pc264) IDE-under-ISP behavior vs the reference `dq_driver.c` ISP gate (Section 5 NOTE).
4. Decide the long-term policy: is silicon mode the Phase-B execution target for OS boot, with ISP retained as the frozen-Oracle-to-prompt mode?

Source References

- `pipelineLib/MemDrainer.h` -- *0xBFFC intercept + EMULATR_PLATFORM selection.*
- `deviceLib/Tsunami/Alim5229Ide.h` -- *ALi M5229 identity, compat prog-IF (forces csr=0x1F0 on ES40).*
- `deviceLib/Tsunami/AtaTaskfileEngine.h` -- *fixed-port taskfile answering the ES40 IDE probe.*
- `deviceLib/BadgeMhzGauge.h` -- *records the ISP default.*

The firmware `dq_init_ide` probe is DEC apismr SSOT, external to the tree.

See Also


```
Firmware: pc264.c:173 platform(); pc264_io.h:48 / emulate.h:33 ISP_MODEL;
dq_driver.c:435 (ISP gate) / :676 dq_init_ide (prog-IF + Cypress
special case); dac960/esc_nvram/ew/n810/kernel ISP_MODEL sites.
EmulatR: pipelineLib/MemDrainer.h:458-486 (0xBFFC read-intercept +
EMULATR_PLATFORM knob); deviceLib/BadgeMhzGauge.h (ISP-default note);
deviceLib/Tsunami/Alim5229Ide.h (ALi identity, compat prog-IF);
deviceLib/Tsunami/AtaTaskfileEngine.h (fixed-port taskfile).
Runner: tools/run_es40_showdev.sh (PLATFORM / WARP knobs, stamped).
```

5.7.2 Reset into Secondary Boot (CSERVE START Handoff)

This topic documents the execution mode EmulatR enters when the SRM console hands control to a secondary bootstrap -- the guest operating-system loader. It covers the reset re-entry after firmware decompression, the PALcode power-up path (`sys__reset / sys__reset_init`), and the CSERVE START handoff that restores the OS context and transfers to the loaded image. The walk-through is DS20-specific (`ds20_v7_3`) but the mechanism generalizes to DS10 and ES40. All addresses and instruction encodings are taken from live EmulatR DIAG-PC, FaultEventLog, and PA-watch traces; values are runtime-observed unless noted.

The durable architectural lesson, established once boot0 actually executed, is stated up front: the OS-boot handoff is a SEQUENCE of CALL_PAL CSERVE functions, and each must RUN THE GUEST PALcode -- not be intercepted and replicated in C+. The rest of this topic derives that lesson from the live evidence.

The handoff at a glance

On EV6, transferring from the SRM console to the operating system is a PAL restart, not a plain jump. The console boot command (`ref/boot.c`) first builds the 3-level page table (VA `0x20000000` -> `base_pfn`), populates the per-CPU HWRPB slot, and writes the restart context into the impure / CNS save area. It then invokes CSERVE START, and PALcode runs `cfw_start` -> `sys__exit_console` -> `pal__restore_state` -> `HW_REI` to the OS entry at `0x20000000`. `restore_state` reconstructs the OS context by dereferencing `p_temp` (register `r21`) then `PT__IMPURE(p_temp)` then the CNS save frame -- from which it restores PTBR, PC, PS, SP, the GPRs, and FPCR. A valid `p_temp` is the precondition for a correct restart, and -- critically -- the restart must carry the OS execution MODE (PS / IPL / VPTB), not just the PC.

The faithful handoff sequence (`sys__exit_console`)

This is the reference mechanism the guest PAL runs, from the DEC `apisrm` source (read-only authority). Option A drives EmulatR into exactly this code:

```
cfw_start (ev6_vms_pc264_pal.mar:3956): br r31, sys__exit_console
sys__exit_console (:4245):
    bsr p7, pal__restore_state ; restore PTBR/PC/PS/SP/GPRs/FPCR from the CNS
save area
    hw_mtptr r31, EV6__ITB_IA ; flush ITB (clears any stale 0x20000000)
    hw_mtptr r31, EV6__DTB_IA ; flush DTB
    ... set up + tear down the 1:1 mapping, clear locks ...
```

```
hw_mtptr r31, EV6__IC_FLUSH ; flush icache
hw_ret_stall (p23) ; RESTART at the restored PC (0x20000000)
pal_restore_state (ev6_vms_pal.mar:6228): hw_ldq/p r1,
PT__IMPURE(p_temp) ; then CNS__PTBR/PC/PS/SP/GPRs/FPCR
```

The terminating `hw_ret_stall(p23)` is what makes the PAL linkage register `p23` load-bearing: it is the RESTART, and it uses R23. Any handoff that reaches this instruction with a stale R23 restarts to the wrong place (a reset).

The defect: CSERVE START stubbed as a no-op

The firmware-to-OS handoff at VA `0x20000000` is CSERVE function `0x42` (START). EmulatR's dispatch (`palBoxLib/grains/PalEntries.cpp`, case `0x42`, ~line 628) originally stubbed it: "no secondaries to start; return with nothing done." That is correct for an SMP-secondary start, but WRONG for the PRIMARY boot transfer, which routes through the same `START` -> `sys__exit_console` path. The no-op strands the whole handoff -- no PTBR install, no ITB flush, no restart -- and HALTs at `0x20000000`. It is also why AXPBox / es40 boot VMS and a stubbed V5 did not.

Live reproduction (DS20, Mac; DIAG probes compiled in): with `EMULATR_PTBR_DIAG=1` and `EMULATR_ITBPROBE_VA=0x20000000`, driving `b dqa0` shows CSERVE entry `func=0x42` `START pc=0x1ae398 cyc=874809139`, then halted CPU 0 / halt code = 0 / PC = `20000000`. The diagnostics confirm it is the RESTART, not translation: PTBR writes `(SWPCTX) = 0` and `ITBPROBE 0x20000000 hits = 0`. VMB ran (`cyc ~184M -> 874M` setup), then START fired immediately before the halt.

The reset re-entry path

To reach a correct START, `sys__reset` must complete. After the decompressor inflates the console/PAL image, its final `hw_ret` is routed to the PAL power-up vector, NOT the console entry. The power-up vector branches to `sys__reset`, which flushes the translation buffers and programs the Ibox/Mbox control registers before reaching the scaffold instruction.

Address / instruction	Effect
<code>0x60079c hw_ret</code>	Decompressor exit -> POWERUP vector <code>0x8000</code>
<code>0x8000 br -> 0x8004 -> 0x13540</code>	Enters <code>sys__reset</code>
<code>0x135a0 HW_MTPR ITB_IA; 0x135a4 HW_MTPR DTB_IA</code>	Flush the I-stream and D-stream translation buffers
<code>0x135cc write i_ctl / m_ctl = 0x387</code>	Program Ibox / Mbox control
<code>0x13654</code>	The unassigned-IPR (0x2d) scaffold (next section)

The unassigned-IPR (0x2d) scaffold

At `0x13654`, `sys__reset` executes `HW_MTPR r?, <scbd 0x2d>` (encoded `0x77e72d40`) -- a write to an UNASSIGNED IPR index. Real EV6 silicon IGNORES writes to unassigned IPRs: architecturally this is a no-op (per the AARM), with no fault. EmulatR instead raises `kFaultUnimplemented` and traps to the OPCDEC vector `0x8400` with `excAddr=0x13655`,

which ABORTS `sys__reset` before `sys__reset_init` can run. FaultEventLog records this as `FAULT[0] pc=0x13655 encoded=0x77e72d40 op=0x1d fault=3`.

The fault is a deliberate SCAFFOLD, gated behind `EMULATR_2D_NOOP` in

`palBoxLib/grains/PalEntries.cpp`. It was kept because aborting `sys__reset` lets the SRM reach >>> by SKIPPING the real-hardware init -- which is exactly the step that would otherwise build `p_temp`. Setting `EMULATR_2D_NOOP=1` makes the write a faithful no-op and restores the real path. The downstream differs per platform: DS10 routes to a device poll at `0x13d38`, ES40 to an MMIO DtbMiss at `0x1b7dd4`, DS20 to the settling delay at `0x13e40` described later -- so keep the no-op DS20-scoped or fix those first.

sys__reset_init: building p_temp and the impure frame

With `0x2d` no-op'd, `sys__reset` continues into `sys__reset_init`, which establishes the PAL temp region and links the impure / CNS save frame. From the DEC PAL source

(`ev6_vms_pc264_pal.mar`, `sys__reset :4315 -> sys__reset_init :4475 -> :4610`):

```
GET_32CONS p_temp, PAL__TEMPS_BASE, r31 ; p_temp = PAL__TEMPS_BASE
addq p_temp, r1, p_temp ; + whami * PAL__TEMPS_SPECIFIC_SIZE (0 for
primary)
```

```
GET_32CONS r2, PAL__IMPURE_BASE, r31 ; impure base
```

```
hw_stq/p r0, PT__WHAMI(p_temp) ; store whami @ +0x98
```

```
hw_stq/p r2, PT__IMPURE(p_temp) ; link impure base @ +0x88
```

EmulatR observes `p_temp` built at runtime by `0x138b8 LDAH r21,0 ; 0x138bc LDA`

`r21,0x7000(r21)`, giving `PAL__TEMPS_BASE = 0x7000` for DS20 `ds20_v7_3`. The PC264 def files list `0xF000 / 0x5000` -- the WRONG variant for DS20; trust the runtime `0x7000`. It sits below `palBase 0x8000` in free low DRAM (HWRPB at `0x2000`, impure near `0x5000`, PAL temps at `0x7000` -- distinct, all below the Tsunami MMIO gate `0x800000000000`), so there is no image overlap. On the default (2D-faulting) path `r21` instead stays the decompressor scratch `0xf01` (`LDA R21,0xf01(R31) @0x600594`), so `PT__IMPURE(0xf01)` reads garbage at the OS restart.

Selected CNS / PT frame offsets (from `ev6_pal_impure.mar / ev6_pal_temps.mar`):

Field	Offset
PT__IMPURE (link to impure base)	0x88
PT__WHAMI	0x98
CNS__PTBR	0x238
CNS__KSP	0x250
CNS__VPTB	0x268
CNS__P_MISC	0x308
CNS__VA_CTL	0x328
CNS__EXC_ADDR (restart PC)	0x330
CNS__I_CTL	0x360
CNS__M_CTL	0x390

Breaking the wall: from RESET(0) to boot0 executing

Two fixes take DS20 from a dead RESET(0) at OS entry to boot0 actually executing. They are driven together by the wrapper `tools/run_ds20_bplus.sh` (exports

`EMULATR_2D_NOOP=1 + EMULATR_DELAYWARP=1 + the CSERVE mode, then execs run_ds20_showdev.sh.`

- **FIX 1** -- `EMULATR_2D_NOOP=1:sys__reset_init` runs and builds `p_temp=0x7000`. **CONFIRMED.**
- **FIX 2** -- `EMULATR_DELAYWARP=1` (a general SUBQ-countdown warp in `PipelineDriver.h`): collapses the settling delays at `0x13e40 / 0x13e80 / 0x13ec0` (~990M cyc) and reaches `P00>>>` at ~88 MHz effective. **CONFIRMED.**

With both applied, `b_dqa0` gives a valid boot block, reads 1226 blocks, and prints the full VMB banner (`base=5bc000 image_bytes=99400, init HWRPB @ 0x2000, page table @ 0x3ff04000, "jumping to bootstrap code"`). The halt PC advances from `0x20000000` (RESET) to `0x20000010` (boot0's first data access) -- boot0 is now EXECUTING. CAVEAT: do NOT run `u_srm` in the LFU; `update+exit` triggers a ~407e9-cyc memory re-init spin, whereas a plain LFU exit re-inits in ~30e9 cyc.

boot0 disassembly and the mode gap

boot0 at VA `0x20000000` (PA `0x5bc000`, leaf pfn `0x2de`) is REAL VMB / APB code. Its first five instructions are position-independent setup that computes the HWRPB address and issues the first data load:

Address encoding instruction	Note
<code>0x20000000 d3800000 bsr r28,0x20000004</code>	PIC: r28 = own PC
<code>0x20000004 201f0001 lda r0,1</code>	
<code>0x20000008 203f001c lda r1,28</code>	
<code>0x2000000c 48010720 sll r0,r1,r0</code>	<code>r0 = 1<<28 = 0x10000000</code>
<code>0x20000010 a4800050 ldq r4,0x50(r0)</code>	LOAD VA <code>0x10000050</code> -- boot0's FIRST data access; the deterministic exit PC

The exit at `0x20000010` is boot0's first DATA reference. VA `0x10000000` IS mapped: `13[0] pte=0x100001101 leafPfn=0x1 leafPa=0x2000` -- the HWRPB, valid kernel R/W. So boot0 reads `HWRPB+0x50`, which is legitimate; the console's page table is COMPLETE and CORRECT. The cycle-ordered sequence is: seed intact -> ITBPROBE MISS then HIT `va=0x20000000 pfn=0x2de` (code fetch resolved) -> boot0 runs 4 instructions -> the `ldq` DTB-misses VA `0x10000000`. The conclusion is decisive: this is NOT a missing map -- it is a MODE gap. A handoff that transfers the PC but NOT the OS execution mode / IPL / PS leaves boot0's data-miss to be serviced in an inconsistent context.

Why C++ replication (Option B) is a dead end

Option B (`EMULATR_CSERVE_START_MODE=cpp / EMULATR_CSERVE_START_RESTART`) has EmulatR's own C++ replicate `exit_console`: it reads the CNS via `PT__IMPURE` and installs `PTBR = 0x1ff82` (the page table at PA `0x3ff04000`) and `VPTB = 0x200000000` (taken from `hwrpb.vptb_va + 120`, merged into `VA_CTL / I_CTL`), flushes ITB+DTB, and diverts to the

halt PC. This got boot0 to FETCH (the JRN-VMB-001/002/003 wall broke), but live tracing then proved it cannot HOLD the machine in OS mode.

With `EMULATR_PA_WATCH=0x7000`, Option B seeds `PT__VPTB(p_temp=0x7000) = 0x200000000`, and ~480 cyc later a PA-WATCH STORE `v=0x0 pc=0x1333c ra=0x62f6c` ZEROES it. The guest `DTBM_DOUBLE_3` VPTB self-test then reads 0 and crashes with halt code `0xA`. The clobber is not a bug in the seed -- it is the guest firmware asserting itself: `pc=0x1333c` decodes exactly to `ev6_vms_pc264_pal.mar:4162-4173 = sys__enter_console's "clear VPTB, switch VA_CTL to 48-bit 1-1 console mode."` Its caller `0x62f6c` is compiled SRM CONSOLE C code (`LDQ r27,0x30(r2); BSR r26 -> 0x1ae328`). In other words: after Option B diverts the PC to `0x20000000`, control does NOT stay in OS boot0 -- it lands back in the SRM console, which runs `enter_console` and wipes `PT__VPTB`. This is hard proof that a C++ PC-divert plus PAL-temp seeding cannot hold the CPU in OS virtual mode. The mode is not a value you can set from outside; it is state the guest PAL owns.

The architectural model: CSERVE dispatch runs the guest PAL

The OS-boot handoff is NOT one intercept -- it is a SEQUENCE of `CALL_PAL` CSERVE functions the SRM console issues, each of which on real hardware runs a guest PAL handler. EmulatR's earlier mistake was intercepting CSERVE functions in C++ and either stubbing them or replicating their effects; that fights the PAL state machine and loses side effects. The correct, faithful model -- proven by AXPBox, which boots VMS -- is: route each CSERVE function to its guest PAL handler and let the real firmware execute. There is zero effect replication.

A note on references. AXPBox is a HANDOFF reference because it runs the real DEC PALcode. QEMU is NOT a handoff reference: it executes its own bespoke PALcode (it boots Linux with a different console/PAL), so it is useful only for EV6 instruction semantics, never for the console-to-OS handoff. EmulatR will NOT implement or imitate QEMU's PAL. SimH has no Alpha / EV6 target at all.

The guest CSERVE dispatcher and the AXPBox reference

The guest dispatcher is `sys__cserve (ev6_vms_pc264_pal.mar:3852)`: it reads `r16` (the function code) and branches to a per-function handler. Two handlers seen so far:

`cfw_start (:3956 = br sys__exit_console, the 0x42 START)`, and `cfw_mp_work_request (:4017, saves r18 -> CNS__WORK_REQUEST, the 0x65 MP_WORK_REQUEST)`.

`CNS__WORK_REQUEST` (checked at :886) is the console-to-CPU work mailbox; stubbing 0x65 means the item is never saved, so the coordination never completes and the console re-inits.

AXPBox (`axpbox-1.1.2/src/AlphaCPU_vmspal.cpp, vmspal_call_cserve`) is literally this: its FIRST line saves the `CALL_PAL` return PC into the PAL linkage register (`p23 = state.pc`), then a `switch(r16)` does `set_pc(<guest PAL handler addr>)` and lets the firmware execute the full faithful path (`restore_state -> mode/vptb/IPL/TBIA -> hw_ret_stall(p23)`). Its `r16 -> handler map` (ES40 layout; DS20 addresses differ, locate by scan):

CSERVE r16	Guest PAL handler (ES40)
0x10 / 0x11 (hw_ldl / hw_stl)	inline
0x12 (LDLP)	0x12e21
0x13 (STLP)	0x12f95
0x14	0x13115
0x15	0x131c1
0x40 (HALT)	0x13249
0x41 (WHAMI)	inline (hw_ldq p21+0x98)
0x42 (START)	0x13781 (cfw_start)
0x43	0x13261
0x44 (MTPR_EXC_ADDR)	set_pc(r17)
0x45	0x13289
0x65 (MP_WORK_REQUEST)	0x132bd (cfw_mp_work_request)
0x66	0x133e9
0x3e	0x1344d

The routing invariant and Option A (mirror AXPBox)

Option A (EMULATR_CSERVE_START_MODE=guest, now the wrapper default) mirrors AXPBox exactly for the routed function. The invariant, to be applied for EACH CSERVE function routed to the guest PAL:

- Set p23 (R23) = the CALL_PAL return PC = g.pc + 4. Set BOTH banks (intReg[23] and intShadow[7]) so the value survives the PAL-shadow swap around the divert. This is the PAL linkage register; the guest handler's terminating hw_ret_stall / hw_rei uses it. (Earlier Option A set excAddr / divertTarget but NOT p23, so exit_console's terminating hw_ret_stall(p23) landed on a stale R23 and reset.)
- Divert the PC to the guest handler entry (PALmode, PC<0>=1). For START, the entry is the guest sys__exit_console, located by scanning the PAL image for the ITB_IA / DTB_IA signature pair (encodings 0x77FF0300 / 0x77FFA300, mask 0xFC00FF00) -- which is exactly cfw_start's br target.
- Do NOT replicate the handler's effects in C++. The guest PAL sets mode / VPTB / IPL / the CNS frame / the work mailbox itself.

Result of applying this to START: the 0xA crash is gone, exit_console runs clean and returns to the console -- and the console then loops on CSERVE 0x65 MP_WORK_REQUEST, which EmulatR currently STUBS, so it re-inits. That is the point, not a regression: the next stubbed pure-PAL function becomes the next wall. The CLEANER GENERALIZATION is to scan for the sys__cserve ENTRY once and route ALL currently-stubbed CSERVE functions there with r16 intact -- the guest dispatcher then dispatches on r16 to the correct handler, so one divert covers every pure-PAL function uniformly. The console-I/O functions (PUTS / GETS / GET_ENV) legitimately need host interaction and MAY stay C++, but the pure-PAL coordination functions (START, MP_WORK_REQUEST, and the rest AXPBox set_pc's) MUST run the guest PAL.

The two homes of PT__VPTB

VPTB (the self-mapped page-table base) is stored in TWO places that must stay coherent: the PAL temp copy, and the impure frame at `CNS__VPTB = impure_base + 0x268`, where `impure_base = mem[p_temp + 0x88] = mem[0x7088].restore_state` reads `CNS__VPTB` from the impure frame. The value `0x200000000` is NOT garbage -- it is the real VMS VPTB, sourced from `hwrpb.vptb_va + 120`. The Option B failure is the concrete lesson: when EmulatR seeds that derived VPTB into the PAL temp while the guest still owns the mode, the guest's own `enter_console` zeroes it. The faithful rule (P-1) is to let the guest's own restart machinery be the SOLE writer of the derived pal-temps, so the two homes are coherent by construction and the value's provenance is traceable to a single guest instruction.

The 0x13e40 settling delay (downstream of the 0x2d no-op)

No-op'ing 0x2d exposes a calibrated settling-delay loop at `0x13e40`, a pure SUBQ countdown armed after programming Pchip1's DMA-window CSRs (WSBA0-3).

```
0x13e14-20 : HW_ST -> PA 0x803_8000_0000/40/80/c0 ; Pchip1 WSBA0-3 DMA
windows
0x13e2c : LDAH R12,0xe5 ; R12 ~= 0xE4E1C0 (~15,000,000)
0x13e30 : LDA R12,-0x1e40(R12)
0x13e40 : SUBQ R12,#1,R12 ; pure countdown spin, no device read inside
0x13e44 : BEQ R12, exit
0x13e48 : BR -12 (back)
```

The older warps (`EMULATR_IDLEWARP / UDELAYWARP / TICKWARP / RSCCWARP`) target other loops (RSCC `0x7c304` etc.) and do NOT recognize `0x13e40`, so `2D_NOOP` alone stalls here (a further settle at `0xb740` follows). The fix (FIX 2, `EMULATR_DELAYWARP=1`) is a GENERAL SUBQ Rn-countdown-to-zero warp: recognize the SUBQ Rn / BEQ Rn / BR-back idiom, fast-forward Rn to 0, and advance the cycle counter by the skipped count. This covers `0x13e40`, `0xb740`, and siblings, and reproduces the delay's only architectural effect (elapsed time) without needing silicon's exact iteration count.

Platform lever: ISP vs silicon

The 0x2d fault and ISP mode are the same strategy -- skip the real-hardware init EmulatR does not yet model. In ISP (the default), EmulatR read-intercepts `PA 0xBFFC -> 0xCAFEDEED` so the firmware's `platform()` returns `ISP_MODEL` and routes down the pre-silicon simulator paths that skip the unmodeled timing / probe / IDE steps. See [Platform Execution Mode: ISP vs Silicon](#)¹⁸¹ for the full lever, and [WARP Variants and Cycle-Accounting](#)²⁰⁰ for the delay-warp family.

Current status and next step

FIX 1 (2D no-op) and FIX 2 (delay warp) are CONFIRMED; the wall is broken and boot0 executes. Option B (C++ replicate) is a proven dead end (the seed is clobbered by console re-entry -- the mode gap). Option A (route START to the guest PAL, mirroring AXPBox's `p23` save) is in flight and removes the `0xA` crash. The well-scoped next step is to route `CSERVE 0x65` (`MP_WORK_REQUEST`) to the guest PAL under the same invariant -- or, preferred, to

locate the `sys__cserve` entry and route ALL currently-stubbed CSERVE functions there, so the guest dispatcher handles 0x65 and every future one. Then iterate `b_dqa0` and surface the next guest-PAL requirement. Diagnostics to remove once Option A lands: the CSERVE-DISASM dump, PTWALK BOOT0DATA/BOOT0VPTE, the CSERVE-START-BPLUS seeding, and the PA_WATCH default in the wrapper.

Key addresses and values (DS20 ds20_v7_3)

Name	Value
Decompressor entry	0x9000000
POWERUP vector (br <code>sys__reset</code>)	0x8000
<code>sys__reset</code>	0x13540
0x2d fault instruction	0x13654 enc=0x77e72d40
OPCDEC trap vector	0x8400
<code>sys__reset_init p_temp build</code>	0x138b8 / 0x138bc -> r21 = 0x7000
0x13e40 settling delay	0x13e40..0x13e48 R12 ~= 0xE4E1C0
CSERVE START entry (live)	func 0x42 @ pc=0x1ae398
<code>cfw_start / sys__exit_console / restore_state</code>	:3956 / :4245 / ev6_vms_pal.mar:6228
enter_console VPTB clear (clobber)	0x1333c (pal :4162-4173); caller 0x62f6c
boot0 entry / first data access	0x20000000 ; ldq @ 0x20000010
boot0 HWRPB read target	VA 0x10000000 -> leafPa 0x2000 (HWRPB)
OS entry / restart PC	0x20000000 -> pfn 0x2de (PA 0x5bc000)
PTBR / page table ; VPTB	0x1ff82 (PT @ 0x3ff04000) ; 0x20000000 (hwrpb.vptb_va+120)
Option B halt code (seed clobbered)	0xA
Platform flag PA	0xBFFC -> 0xCAFEDEED (ISP_MODEL)
Tsunami MMIO gate ; Pchip1 base	0x800000000000 ; 0x803_8000_0000

Diagnostic knobs

Environment variable	Effect
EMULATR_2D_NOOP=1	FIX 1: flip the 0x2d MTPR to a no-op so <code>sys__reset_init</code> runs and builds <code>p_temp=0x7000</code> .
EMULATR_DELAYWARP=1	FIX 2: general SUBQ-countdown warp; collapses the 0x13e40/0x13e80/0x13ec0 settling delays to reach >>>.
EMULATR_CSERVE_START_MODE=guest cpp off	Option A (guest PAL, faithful) / Option B (C++ replicate, dead end) / no-op. Case 0x42 in <code>PalEntries.cpp</code> .
EMULATR_PA_WATCH=<pa>	Watch stores to a physical address (used on 0x7000 to catch the VPTB seed clobber).

Environment variable	Effect
EMULATR_PTBR_DIAG=1 + EMULATR_ITBPROBE_VA=<va>	Count PTBR (SWPCTX) installs and ITBPROBE hits for a VA -- proved START never ran (both 0).
EMULATR_PLATFORM=silicon	Real-hardware path (no 0xBFFC intercept); default is ISP.
EMULATR_DIAG_WREG=<n> [+ WMIN] [+ PCLO/PCHI]	Register last-writer trace, optionally gated by cycle and PC region (isolated the r21 / VPTB writers).
EMULATR_DIAG_PCLO/PCHI + CYCLO/CYCHI + CAP	PC-window instruction trace (DIAG-PC) -- the source of the ASM in this topic.
EMULATR_PTEMP_PROBE=1	Dump r21 (both banks) on a double-miss plus a one-shot PAL-temp-region dump.

See Also: [Platform Execution Mode: ISP vs Silicon](#)¹⁸¹, [SRM Boot Call-Stack](#)²⁴¹, [PALcode, HWRPB & SMP](#)¹⁶², [WARP Variants and Cycle-Accounting](#)²⁰⁰.

Source References

- `palBoxLib/grains/PalEntries.cpp` -- *execCserve; case 0x42 = CSERVE START (~line 628; Option A kStartGuest, Option B EMULATR_CSERVE_START_RESTART); EMULATR_2D_NOOP MTPR path (~2489) / MFPR path (~1958).*
- `pipelineLib/PipelineDriver.h` -- *delay-loop warp sites (~286-405, EMULATR_DELAYWARP); DIAG-PC / DIAG-WREG PC-region gating.*
- `pipelineLib/MemDrainer.h` -- *EMULATR_PLATFORM ISP intercept of PA 0xBFFC (581-608); PA_WATCH.*
- `tools/run_ds20_bplus.sh` -- *the FIX1 + FIX2 + CSERVE-mode wrapper over run_ds20_showdev.sh.*
- *Reference (real DEC PAL): axpbox-1.1.2/src/AlphaCPU_vmspal.cpp (vmspal_call_cserve). QEMU is NOT a handoff reference (bespoke PALcode); SimH has no Alpha target.*
- *Journals: 20260722_JRN-VMB-016_0x20000000_wall_end_to_end_rootcause.md (end-to-end root cause + live PC results + the run-the-guest-PAL model) and 20260722_JRN-VMB-004_cserve_start_boot_handoff_root_cause.md (START = the handoff; reference mechanism; Option A/B). Companion memory notes: emulatr5-cserve-start-boot-handoff.md, emulatr5-axpbox-handoff-mechanism.md, emulatr5-qemu-reference.md.*
- *Processor Support (read-only authority): ev6_vms_pc264_pal.mar (sys_reset :4315, sys_reset_init :4475/:4610, sys_cserve :3852, cfw_start :3956, sys_exit_console :4245, cfw_mp_work_request :4017, enter_console :4162-4173), ev6_vms_pal.mar (pal_restore_state :6228), ev6_alpha_defs.mar, ref/boot.c.*

5.7.3 Virtualization Methods

EmulatR can present the firmware with one of two execution-path identities. The choice is made by a single firmware routine -- the SRM console's `platform()` (`apisrm pc264.c`) -- which reads the 32-bit longword at physical address `0xBFFC` and branches: the constant `0xCAFEDEED` selects `ISP_MODEL` (the pre-silicon simulator path), any other value selects `REAL_HW` (the faithful hardware path). EmulatR controls which branch the firmware takes through the `EMULATR_PLATFORM` environment variable; there is no second knob, because the firmware's own `platform()` is the single source of truth.

Method	Description
ISP (virtual) path;	Default
Silicon	Hardware Initialization <code>EMULATR_PLATFORM=silicon</code>

How the selection works

`platform()` reads `*(int *)0xBFFC`. EmulatR answers that read with a `READ-INTERCEPT`, never a deposit: `0xBFFC` sits at firmware image offset `0x3FFC` and is overwritten by the SRM self-decompressor during cold boot, so a one-time store would not survive to the moment `platform()` runs. The lever therefore lives in the load path (`pipelineLib/MemDrainer.h`): when the ISP method is active and the guest reads physical `0xBFFC`, EmulatR substitutes `0xCAFEDEED` (sign-extended to match the firmware's constant). On the silicon method the read is not intercepted, so `0xBFFC` returns its real memory contents and `platform()` resolves to `REAL_HW`.

ISP (virtual) path -- default

Active when `EMULATR_PLATFORM` is unset or set to any value other than the exact string "silicon" (the legacy alias "isp" also selects it). `platform()` returns `ISP_MODEL`, and the firmware takes its pre-silicon simulator branches -- skipping the real-hardware timing, probe, and IDE steps that EmulatR does not yet model. This is the path that reaches the SRM console prompt `P00>>>`. It is the out-of-box behavior and the path used for all current console-and-boot work.

Silicon path -- hardware initialization

Active when `EMULATR_PLATFORM` is set to exactly "silicon".

`EMULATR_PLATFORM=silicon` `platform()` returns `REAL_HW`, and the firmware runs the faithful hardware-initialization sequence (the same code path real Alpha silicon would execute).

This exercises the real timing/probe/device-init steps, so it takes substantially longer to initialize. IMPORTANT: `REAL_HW` is currently largely unmodeled in EmulatR, so this path may not reach `P00>>>` -- it

can grind in the hardware-probe steps and run until the cycle limit (`MaxCyclesExceeded`).

Stop a run cleanly by creating the file `EMULATR_STOP` in the run directory (this flushes flash `NVRAM` on exit).

The value of running it today is comparative:

it shows whether the firmware's deterministic init sequence (for example, the order of IIC bus init versus `HWRPB` construction, and the resulting system-type badge) behaves differently from the ISP path.

Invocation examples

From the build/run directory (`out/build/release`). Console is on `TCP 10023`.

```
# ISP (virtual) path -- default; reaches the SRM prompt
./run_fw.sh ds20 cold

# Silicon path -- full hardware init (slow; may not reach the prompt)
export EMULATR_PLATFORM=silicon
export EMULATR_FLASH_ROM=ds20_flash.rom; rm -f ds20_flash.rom # factory-
fresh cold
./run_fw.sh ds20 cold 2>&1 | tee fw_ds20_silicon.out
touch EMULATR_STOP # stop a long silicon run cleanly
```

Notes

The `EMULATR_PLATFORM` lever (2026-06-19) supersedes the temporary `EMULATR_CPU1_ALIVE` flag. Do not add a second host-side sim/silicon flag -- the firmware's `platform()` is the single source of truth, and EmulatR's only role is to answer the `0xBFFC` read consistently with the chosen method. Selection is evaluated once per run (a function-local static), so changing `EMULATR_PLATFORM` requires a fresh run.

Source References

Lever: `pipelineLib/MemDrainer.h` (the `0xBFFC` read-intercept). Firmware decision: `apism pc264.c platform()` (reads `0xBFFC`; `0xCAFEDEED => ISP_MODEL`, else `REAL_HW`). The ISP flag convention is documented per `apism pc264.c`.

See Also. CMake Macro Guards; `HWRPB` Key/Value Offset Map. [What is EmulatR](#) ⁶

5.7.4 Platform Execution

Every supported model is launched the same way, through one script. The model is not really a flag you choose -- it follows the firmware image the run loads.

The launcher

```
tools/emulatr_launch.sh ds20 bgate
```

The full form is `tools/emulatr_launch.sh <ds10|ds15|ds20|ds25|es40|es45> [purpose] [-- extra Emulatr args]`. The platform argument is the single input, and it fixes all three identity channels together -- firmware, memory ceiling and ini model -- so a mis-badged or over-allocated launch cannot be typed by hand. For one run the script:

- resolves `firmware/<stem>.exe` and passes it as `--firmware`;
- requests 4 GiB of guest memory, clamped down to that model's silicon ceiling;
- sets the `[System] model` line in `config/Emulatr.ini` and restores the original on exit;
- always raises the PuTTY console -- it unsets `EMULATR_NO_PUTTY` and sets `EMULATR_CONSOLE_MIRROR` -- because diagnostic runs are driven by an operator at the console;
- writes a stem-keyed log to `logs/<stem>_<purpose>_<timestamp>.log`;
- passes everything after -- to `Emulatr.exe` verbatim, and forwards diagnostic environment variables (`EMULATR_VALUE_GATE` and the like) from the calling shell;
- on exit, echoes the platform-latched line and flags any `PLATFORM MISMATCH`.

The run directory is `$RUN_DIR`, defaulting to `out/build/relwithdebinfo`. Set it in the environment to run a different configuration; the script changes into it before launching, so relative paths resolve the same way whichever build you point at.

Platform-named wrappers

Six wrappers exec straight into the launcher with the platform already supplied. They hold no argument logic of their own, so there is only ever one place where the per-platform arguments are decided.

```
tools/run_ds20.sh bgate -- --no-autoload --autosnapshot off
```

The models

Model	Command	Status
DS10	<code>tools/run_ds10.sh</code>	Boots to >>>
DS15	<code>tools/run_ds15.sh</code>	Wrapper present; firmware not yet staged
DS20	<code>tools/run_ds20.sh</code>	Boots to >>>
DS25	<code>tools/run_ds25.sh</code>	Manifest present; not yet driven to the prompt
ES40	<code>tools/run_es40.sh</code>	Boots to >>>; init does not yet complete
ES45	<code>tools/run_es45.sh</code>	Firmware decompresses; platform not yet driven

DS10, DS20 and ES40 additionally form a standing three-platform gate: all three must still reach the prompt before any shared core, chipset or device change is committed.

DS15 is the newest entry and the least finished. Its wrapper exists, but no `ds15_v7_3.exe` is staged in the run directory, so the launcher stops with a FATAL before the emulator starts; the v7.3 image is in the firmware archive and staging it is what the platform waits on. DS15 is also the one model EmulatR itself does not recognize -- see [Platform Identity & Memory Geometry](#)⁷¹.

Cold boots

The launcher has no `cold` keyword. A genuinely cold boot is requested by passing the emulator flags through: `tools/run_ds20.sh cold -- --no-autoload`, where the first word is only the log tag. Stale `auto_*.axpsnap` files are not purged for you, so a run that must not restore from a snapshot should pass `--no-autoload` explicitly rather than rely on the snapshot directory being empty.

What the launcher does not decide

It still rewrites the `ini model` line, but that is belt and braces rather than the authority: the model is derived from the firmware stem whenever `--firmware` is given, so `ds10_v7_3.exe` means DS10 regardless of what the ini says. The rewrite covers the case where the binary in the run directory predates that rule.

Memory is clamped twice, and for most models the two agree. The launcher clamps its request to the model's ceiling before launching, and EmulatR clamps again at argument resolution. On a DS10 the 4 GiB default becomes 2 GiB at the first clamp and passes the second untouched. DS15 is the exception: EmulatR does not know the model, so only the launcher's clamp applies, and a bare `Emulatr.exe` invocation for a DS15 is unprotected. Both clamps announce themselves on `stderr`; neither is a failure.

run_fw.sh, and why it is superseded

`run_fw.sh` is the earlier launcher, still staged in the run directory and still working for the five models it knows. It is no longer the recommended entry point. It requests a flat 4 GiB with no per-model ceiling of its own, leaving over-allocation to be caught by EmulatR rather than prevented; it does not know DS15; it writes `fw_<name>_<timestamp>.out` to the run-directory root instead of the stem-keyed name under `logs/`; and it copies the firmware image in from the source tree on every run. Prefer `emulatr_launch.sh` or a platform wrapper for new work.

Which build to run

The run directory is `out/build/<config>`, where `<config>` is one of `release`, `relwithdebinfo` or `debug`. The launcher lives in the repository's `tools/` directory rather

than being staged beside the executable, and changes into `$RUN_DIR` itself, so it can be invoked from anywhere in the tree.

Use `relwithdebinfo` or `debug` for any diagnostic capture. The `EMULATR_DIAG_*` retire-time facility is compiled out of `release` entirely, so a release build will silently produce no diagnostic output rather than warn you.

See Also.

[Platform Identity & Memory Geometry](#) ⁷¹

[Platform Memory Reference](#) ²³⁸

[Running from a Run Directory](#) ⁵⁶

[run_fw.sh](#) ²⁵²

[Execution Modes](#) ¹⁷⁸

5.7.5 Clock Frequency and the MHz Reading

The SRM banner and `show config` report a CPU speed -- on the DS10 profile, 268 MHz. That number is not a hardcoded banner string and it is not read from a stored constant; the firmware derives it by measuring the CPU cycle counter against the chipset interval-timer interrupts. This page explains how that figure is produced and how to change it.

The profile clock

EmulatR has one build-time constant for the modeled CPU frequency,

`EMULATR_PROFILE_ALPHA_CLOCK_HZ`, defined in `chipsetLib/Tsunami21272_CsrSpec.h`.

The current DS10 value is:

```
EMULATR_PROFILE_ALPHA_CLOCK_HZ = 268435456 (2^28 Hz = 268.435456 MHz)
```

Everything downstream -- the interval-timer cadence and the frequency the firmware reports -- follows from this one number.

The interval timer cadence

The Cchip interval timer is designed to fire at the architectural nominal of 1024 Hz. The cycle target per tick is the profile clock divided by 1024, rounded to the nearest power of two so the fire test is a single mask compare:

```
kCchipIntervalTimerCycles = kProfileAlphaClockHz / 1024
kCchipTimerBit = roundLog2Nearest(kCchipIntervalTimerCycles)
fire when: (cycleCount & kCchipTimerMask) == 0 && cycleCount != 0
```

At 2^{28} Hz this is exact: $2^{28} / 1024 = 2^{18}$, already a power of two, so the rounding introduces zero error and the timer fires every 2^{18} cycles -- a clean 1024 Hz.

How the firmware turns that into MHz

The SRM measures its own clock: it reads the CPU cycle counter (PCC / RPCC) and counts how many cycles elapse across a known number of 1024 Hz interval-timer interrupts, then multiplies cycles-per-tick by the tick rate.

```
measured clock = cycles-per-tick x tick-rate
                = 2^18 cycles x 1024 Hz
                = 2^28 Hz = 268.435456 MHz -> "268 MHz"
```

So the reported MHz is the profile clock, recovered by measurement -- the calibration path comes back to exactly the value that drove the timer in the first place.

Why not the HWRPB constant?

The Hardware Restart Parameter Block does carry a cycle-counter frequency field (`cycle_count_freq`) and an interval-clock field (`intrclock_freq`, interrupts per second times 4096), built by `HwrpbBuilder`. But the `cycle_count_freq` spec value is 500 MHz, while the banner shows 268 -- so on the DS10 the firmware is clearly using its own measurement rather than the stored constant. (Firmware that instead trusted the HWRPB field would report 500.) The interval-clock field is set consistently with the 1024 Hz cadence.

Changing the reported speed

Override the profile clock at configure time, for example for an ES40 profile:

```
cmake -DEMULATR_PROFILE_ALPHA_CLOCK_HZ=600000000 (ES40, ~600 MHz)
```

One caveat follows directly from the calibration mechanism: a clock that is not a power-of-two multiple of 1024 makes `kCchipIntervalTimerCycles` round to the nearest power of two, so the tick rate drifts slightly from 1024 Hz and the firmware's measured MHz drifts with it (for ES40 600 MHz the tick lands at bit 19, about 1144 Hz, within ~12 percent of nominal -- harmless to a rate-agnostic OS but visible if a guest reads the calibrated speed). Powers of two like 2^{28} calibrate exactly; other values trade a small reported-speed skew for the chosen cadence.

Source References

- `chipsetLib/Tsunami21272_CsrSpec.h` -- `EMULATR_PROFILE_ALPHA_CLOCK_HZ`, `kCchipIntervalTimerCycles/Bit`.
- `chipsetLib/CchipIntervalTimer.h` -- *1024 Hz interval-timer cadence / mask fire test*.
- `deviceLib/HwrpbBuilder.cpp` -- `cycle_count_freq` / `intrclock_freq` HWRPB fields.

See Also.

[HWRPB - DS20](#) ¹⁷⁰

[Execution / Boot Internals](#) ¹²³

[Memory Map & IPRs](#) ¹³⁴

[show config](#) ⁸⁹

5.7.6 WARP Variants and Cycle-Accounting

WARP collapses idle and delay spins by fast-forwarding the simulation clock to a known target instead of executing every cycle. This topic catalogs the WARP variants, the counters each one touches, the single safety rule that separates the clean variant from the quarantined ones, and a cycle-accounting scheme that turns the warp deltas into an honest "real vs effective" performance number. It closes by relating that number to the SRM CPU-speed field so the badge and the perf gauge stay cleanly separated.

STATUS — the WARP levers were REMOVED from EmulatR on 2026-08-01. Every variant catalogued below (`EMULATR_IDLEWARP`, `EMULATR_RSCCWARP`, `EMULATR_TICKWARP`, `EMULATR_DELAYWARP`, `EMULATR_UDELAYWARP_LEGACY_UNSAFE`) is gone from the execution path, and those names are now stripped from a child environment if they are still exported in a shell. They fabricated guest-observable state: all of them advanced the guest clock, and one wrote a guest register directly to force a countdown loop to exit. EmulatR is the primary Oracle, and performance is not a reason to break fidelity; a deliberate performance pass will come later, measured against the Oracle.

Two things survive and are NOT covered by that removal. `EMULATR_UDELAYWARP` remains as a single documented exception — it advances only to the next timer edge, so no interval-timer boundary is ever skipped. And `EMULATR_SPINSKIP` is the proven successor: it refuses to collapse a loop unless it can prove the body is free of memory, CSR and IPR access with a closed-form trip count, and it replays every timer edge crossed. That is a provably equivalent transformation rather than a fabrication.

The **cycle-accounting** material further down remains current and is now more useful, not less: on a clean run the accounting reports `warp_cyc=0` with `K=1` and `MHz_real == MHz_eff`. That is the signature of a run in which every cycle was executed and none fabricated, and it is the quickest way to confirm a build is honest. The variant table below is retained as a historical record of what was removed and why.

The three counters (do not conflate)

1. Simulation clock -- `cpu.cycleCount`. The fundamental time base; advances per retired instruction AND by every WARP jump. The interval timer fires against it (`cycleCount & kCchipTimerMask`). This is the counter WARP is allowed to move.

2. Architectural cycle counter (RPCC / HW_CC) -- the 2026-05-11 split. It is `cpu.cycleCount + ccBias`, where `ccBias` is the offset the guest last wrote via `HW_MTPR HW_CC`. RPCC and `HW_MFPR HW_CC` both return that expression (`IntArith.cpp`). It is derived from counter 1, not independent.

3. Guest tick counter -- DRAM word at `0x3c970`, the SRM uptime-in-ticks, incremented by the interval-timer ISR. This is guest state and must be advanced only by the real ISR, never written out-of-band.

WARP variants (REMOVED 2026-08-01 — historical record)

Variant (gate)	Where	What it does	Status
IDLEWARP (<code>EMULATR_IDLEWARP</code>)	<code>krn\$_idle</code> loop, pc <code>0x7bad0..0x7bb10</code>	Advances <code>cycleCount</code> to the NEXT tick edge: <code>(c0 kCchipTimerMask) + 1</code> . The FIRE path then runs the real ISR (bumps <code>0x3c970</code>). One tick per hit.	CLEAN, active
Tick-delay warp (<code>EMULATR_RSCCWARP</code>)	pc <code>0x7c314</code>	Data-driven: reads the firmware's own counter (R5 addr, R6 target, R0 cur); writes counter to target and advances <code>cycleCount</code> by <code>delta * kTickInterval</code> .	QUARANTINED
RSCC-deadline warp (<code>EMULATR_RSCCWARP</code>)	pc <code>0x7c304</code>	Data-driven: R19 elapsed, R07 duration; advances <code>cycleCount</code> by (duration - elapsed) AND writes <code>0x3c970</code> out-of-band.	QUARANTINED (<code>0x3c970</code> rewrite = <code>0x7f4xx</code> corruption)

The safety rule

WARP may advance **only the simulation counter** (`cpu.cycleCount` / the system clock). The guest tick counter `0x3c970` must be delivered by the real interval-timer ISR through the normal FIRE path, never written directly. IDLEWARP obeys this -- it moves the clock and lets `fireIntervalTimer` deliver the tick. The RSCC-deadline warp violates it (out-of-band `0x3c970` write) and is quarantined. The delay-loop warps are otherwise sound (they are data-driven -- they read the firmware's OWN loop bounds, so they adapt to any clock), so re-homing them onto the IDLEWARP model (advance `cycleCount`, let the ISR post the ticks) is the path to un-quarantining them.

Cycle-accounting: the one new counter

`cpu.cycleCount` conflates executed and warped cycles. Add a single accumulator that captures only the warp deltas; everything else derives from the invariant `cycleCount = execCycles + warpCycles`.

```
uint64_t s_warpCycles; // NEW: sum of every warp advance
// at each warp: cpu.cycleCount += delta; s_warpCycles += delta;
uint64_t execCycles = cpu.cycleCount - s_warpCycles; // real executed cycles
```

The math ($T = \text{kProfileAlphaClockHz} / 1024$ cycles per tick; W = host wall-clock seconds)

Quantity	Meaning	Formula
cycleCount	system/guest clock (warped)	execCycles + warpCycles
warpCycles	NEW: accumulated warp advance	sum of warp deltas
execCycles	real executed cycles	cycleCount - warpCycles
ticks_total	all ticks (== 0x3c970)	cycleCount / T
ticks_warp	ticks fast-forwarded	warpCycles / T
ticks_exec	ticks actually executed	ticks_total - ticks_warp
MHz_real	honest EmulatR throughput	execCycles / W / 1e6
MHz_eff	warp-inflated (guest wall view)	cycleCount / W / 1e6
K (warp factor)	over-performance ratio	cycleCount / execCycles = ticks_total / ticks_exec = MHz_eff / MHz_real
relation	effective is K times real	MHz_eff = K * MHz_real

Why the guest cannot see the warp (and where the gauge must live)

Inside the guest, RPCC (`cycleCount`) and the tick counter (`0x3c970`) advance **coherently** through every warp. Any guest-side measurement -- cycles divided by ticks, times 1024 -- therefore recovers exactly `kProfileAlphaClockHz`, warp-invariant. The over-performance K exists only relative to the HOST clock, so it is observable only by correlating guest cycles against host wall-time W . That is precisely what `warpCycles` plus a host timestamp provide. The gauge is an EmulatR-side readout (PROFILE line), NOT a guest-visible register.

Relation to the CPU-speed badge

The SRM banner "%3d MHz" comes from `krn$_timer_get_cycle_time() / 1e6`, which on the ISP path reads the HWRPB cycle-counter-frequency field at PA `0x2070` (the same self-built HWRPB whose SYSVAR is at `0x2058`). That field is **functional** -- the firmware's delay/timeout loops key off it, and the WARP thresholds are tuned against the current value -- so it must NOT be repurposed as a raw performance gauge. Keep it frozen (or set it to a true model clock only as part of the warp refactor, since a realistic clock enlarges the delay loops and only the re-homed delay-loop warp collapses them for free). The build-relative performance number (`MHz_real`, `MHz_eff`, K) lives EmulatR-side, computed from `warpCycles` + wall-time, independent of the guest.

Implementation order

1. Add `s_warpCycles` and increment it at every warp site (IDLEWARP now; the delay-loop warps when re-homed). Emit `MHz_real / MHz_eff / K` on the PROFILE line. Zero guest impact; immediate build-relative perf gauge. 2. Re-home the delay-loop warps onto the clean sim-counter model (advance `cycleCount`, let the ISR post 0x3c970) to un-quarantine them. 3. Only then, if a realistic banner is wanted, set the model clock at PA 0x2070 -- now free, because the re-homed warp collapses the enlarged delay loops.

Source References

- `pipelineLib/MemDrainer.h` -- *IDLEWARP/RSCCWARP warp sites; advances `cpu.cycleCount`.*
- `eBoxLib/grains/IntArith.cpp` -- *RPCC / HW_MFPR HW_CC return `cpu.cycleCount` + `ccBias`.*
- `chipsetLib/CchipIntervalTimer.h` -- *kCchipTimerMask FIRE path posting the real tick.*
- `deviceLib/BadgeMhzGauge.h` -- *SRM MHz badge gauge (kept separate from the perf gauge).*

See Also. Model Badging: SRM Platform Identity from an IIC Discriminator Device; Diagnostics: GuestMemory Store-Watch and Retire-Trace Arm; [HWRPB - DS20⁵⁻¹⁷⁰](#).

5.7.7 Model Badging (IIC Discriminator)

The SRM firmware brands each machine with a platform name in its power-up banner ("AlphaServer DS20", "AlphaPC 264DP", and so on). That name is not configured anywhere in EmulatR -- the firmware derives it at boot from the `member` id in the HWRPB `SYSVAR` word, and it computes that member by probing the IIC bus for a **model-specific discriminator device**. For EmulatR to badge a model correctly, its platform manifest must populate the exact IIC device that model's firmware probes. This topic states the general pattern, the DS20 case (root-caused 2026-06-30), and how to diagnose badging for any model.

This topic supersedes "DS20 Badge: Polled IIC Completion". That earlier thesis -- a missing polled IIC completion times out the node-0x40 verify -- was disproven: the completion is delivered correctly, node 0x40 reads cleanly, and the badge never depended on node 0x40 at all.

Symptom

The banner shows the base / wrong model. The HWRPB scan reports `SYSSTYPE=0x22` (DEC_TSUNAMI) and `SYSVAR=0x405`. `Member = (SYSVAR >> 10) & 0x3F = 1`, which the SRM banners as "AlphaPC 264DP". The correct DS20 is member 6 (`SYSVAR=0x1805`), bannered "AlphaServer DS20".

How the SRM computes the badge

`build_hwrpb` calls `get_sysvar()` (`pc264.c:636`). `get_sysvar` starts with the base `sysvar = HWRPB$_SYSVAR = 0x5`, then attempts `fopen()` on a model-specific IIC device; on success it ORs `member << 10` into `SYSVAR<15:10>`, else it takes the member-1 default. The value is stored to the HWRPB SYSVAR word at physical address `0x2058`.

Retire-trace of the running V7.3-2 binary confirms the exact chain: `build_hwrpb` stores base `0x5` to PA `0x2058` (`pc 0x5c3e0`), calls `get_sysvar` (`pc 0x7f5c0`), which byte-walks the discriminator device name and on failure branches at `BEQ pc=0x7f608 (R16 == 0)` to load the member-1 default `0x405` as a literal (`pc 0x7f660`); `build_hwrpb` then stores `0x405` to PA `0x2058` (`pc 0x5c3f8`).

Member map (SYSTYPE 0x22, DEC_TSUNAMI / PC264)

Member <15:10>	SYSVAR	Banner	Discriminator device (IIC node)
1	0x405	AlphaPC 264DP (default / fallback)	none -- reached when the others fail to open
6	0x1805	AlphaServer DS20	<code>iic_rcm_temp @ 0x9e</code>
8	0x2005	AlphaServer DS20E (Goldrack)	<code>iic_8574_ocp @ 0x4E</code>

Root cause (DS20)

The DS20 discriminator is `iic_rcm_temp` -- the temperature sensor on the KCRCM server-management card, which per the `get_sysvar` source comment is "only present on the AlphaServer DS20". It is a static node-table entry at IIC node `0x9e` (`iic_driver.c:264`, `IIC_LED_TYPE`). EmulatR's `IicPcf8584` populates its bus from the platform manifest; any node absent from the manifest NAKs. The DS20 manifest did not list node `0x9e`, so the bus scan NAK'd it (`IIC-TXN addr=0x9e dir=R -> NAK`), `iic_init` never registered `iic_rcm_temp`, `fopen("iic_rcm_temp")` returned `NULL`, and `get_sysvar` fell to the member-1 default.

Why node 0x40 (`iic_ocp0`) was a red herring

An earlier analysis, and an earlier manifest fix, targeted `iic_ocp0 @ 0x40`, believing it the discriminator. It is not. `iic_ocp0` is the OCP LED panel, present on **both** AlphaPC 264DP and AlphaServer DS20, so it cannot distinguish them. The reference apism `get_sysvar` code reads `fopen("iic_ocp0")`, but its own *comment* and the shipped V7.3-2 binary use the temperature sensor. The retire-trace of the running binary is authoritative: in the entire `get_sysvar` window it byte-walks `iic_rcm_temp` and never references `iic_ocp0`. Decisive proof: node `0x40` was already populated and read cleanly in every trace -- had it been the gate, the badge would already have been member 6.

Resolution

Populate the model's discriminator device in its platform manifest. For DS20, add one entry

to `ds20_v7_3_platform.json` under `iic_devices`:

```
{ "name": "iic_rcm_temp", "address": "0x9e", "class": "status", "byte": "0x19" }
```

With node `0x9e` ACKing, `iic_init`'s verify read succeeds, `iic_rcm_temp` registers, `fopen` succeeds, and `get_sysvar` returns member 6 -> `SYSVAR=0x1805` -> "AlphaServer DS20".

Registration gates on the read *succeeding* (`rec_count`), not on the byte value, so byte `0x19` (~25 C) is cosmetic. The platform manifest is loaded at runtime -- **no rebuild**; edit the source manifest and sync the run-dir copy. Do **not** add `iic_8574_ocp` (`0x4E`) -- that yields member 8 = DS20E / Goldrack.

How to diagnose model badging

Arm the retire window on the base-SYSVAR store so the `get_sysvar` path is captured, and watch the SYSVAR word directly:

```
unset EMULATR_TRACE_ARM_ON_IIC # do not consume the window at the ~185M IIC read
export EMULATR_TRACE_ARM_PA=0x2058 # arm on the HWRPB SYSVAR store...
export EMULATR_TRACE_ARM_VAL=0x5 # ...value 0x5 (base), just before get_sysvar
export EMULATR_TRACE_ARM_INSTRS=2000000 # window over get_sysvar
export EMULATR_GMEM_WATCH=0x2058 # log every store to the SYSVAR word
export EMULATR_IIC_TRACE=1 # log IIC START / device reads + NAKs
```

Read it as follows. The `GMEM-WATCH` (`0x2058`) final store value encodes the member (`0x405=1`, `0x1805=6`, `0x2005=8`). In the retire `.trc` between the base (`0x5`) and final store, read which device name `get_sysvar` byte-walks (the `strlen` loop) -- that is the discriminator the running firmware actually probes. Cross-check its IIC node against the bus scan: an `IIC-TXN ... -> NAK` at that node means EmulatR does not populate the device.

General rule

Every model's SRM brands itself by probing for a signature IIC device that is unique to that model. To badge a new model correctly: trace the running binary's `get_sysvar` (the **trace**, not just the reference source, which can disagree with the shipped image), identify the device name and its IIC node, and populate a matching device at that node in the model's platform manifest. If the banner shows the base / wrong model, the discriminator node is NAKing -- add it.

Source References

- `deviceLib/Tsunami/IicPcf8584.h` -- *PCF8584 IIC bus; manifest nodes, NAK for absent discriminator.*
- `deviceLib/HwrpbBuilder.cpp` -- *builds SYSVAR word the badge derives from.*
- `deviceLib/Hwrpb.h` -- *SYSVAR / SYSTYPE HWRPB fields.*

See Also. DS20 Badge: Polled IIC Completion [SUPERSEDED]; Diagnostics: GuestMemory Store-Watch and Retire-Trace Arm; [HWRPB - DS20](#)^{▷170}; [Virtualization Methods](#)^{▷194}.

6 6. Reference

Look-up material: the full environment-variable table, a glossary of Alpha and EmulatR terms, the build and runtime dependencies, and the project's attributions.

In this section

- *Environment Variables* -- every `EMULATR_*` runtime gate and the compile-time flags.
- *Glossary* -- terms and acronyms.
- *Dependencies* -- the frameworks, libraries, and tools EmulatR relies on.
- *Attributions & Credits* -- sources, heritage, and licensing acknowledgements.

See Also.

[Environment Variables](#) → 216

[Glossary](#) → 225

[Dependencies](#) → 48

[Attributions & Credits](#) → 160

6.1 Windows Registry Settings

Every key EmulatR persists is listed here. You should not normally need this topic -- every value below has a control in the launcher UI -- but it is the reference for auditing a machine, scripting a deployment, or recovering from a damaged configuration.

The product hive

All persistent launcher state lives in ONE per-user hive:

```
HKEY_CURRENT_USER\Software\eNVy Systems\EmulatR
```

Four design rules explain its shape:

- *The product is EmulatR; the launcher and Platform Editor are SUBSYSTEMS of it. Each namespaces its keys with a prefix -- `launch_` today, with `platedit_` reserved. One `regedit` path shows the whole product, and subsystems can never collide.*
- *HKCU scope, never HKLM. No administrator rights are needed, and a kit install or uninstall -- which touches Program Files and the HKLM uninstall entries -- never removes it. Named systems survive every upgrade.*
- *Not keyed by version. Deliberate: an upgrade must never orphan the system catalog.*
- *The registry is the CATALOG, not the substance. It stores names, pointers and small per-system knobs. The substance of a system -- `EmulatR.ini`, the platform manifest copy, disks,*

*firmware, logs -- lives in that system's profile directory on disk. If the hive is ever lost, **Add Existing...** re-registers any profile directory. Nothing load-bearing exists only in the registry.*

Values are written through Qt QSettings in native format: strings land as `REG_SZ`, integers as `REG_DWORD`, booleans as `REG_SZ` holding "true" or "false", and list values as `REG_MULTI_SZ`. The only binary blob is the saved window geometry.

launch_systems -- one subkey per named system

Each system is a subkey `launch_systems\<guid>\`. The GUID is assigned at creation and is immutable: renames and run-directory moves never change it, so per-system settings can never be orphaned. Deleting a system in the launcher removes the entire subtree in one operation, because all per-system state keys off that GUID by design.

value	type	meaning
name	REG_SZ	display name; user-editable, unique ignoring case
platform	REG_SZ	DS10, DS20 or ES40. Bound at creation, immutable
runDir	REG_SZ	absolute path of the profile directory -- the one-to-one pointer from catalog to substance
binaryConfig	REG_SZ	which emulator this system launches: empty or discovered (the tester default), release, relwithdebinfo, debug, or custom
binaryCustomPath	REG_SZ	the explicit executable path when binaryConfig is custom
snapshotMode	REG_SZ	snapshot cadence: empty or safety (emulator default), diagnostic (dense 1B-cycle saves), off
platformMode	REG_SZ	execution mode: empty or isp (default, boots to the SRM), silicon (real-hardware path, experimental)
env\<VARIABLE>	REG_MULTI_SZ	per-system selection for one <code>EMULATR_*</code> variable: first element 1 or 0 for enabled, second the value. A variable

value	type	meaning
		with no entry is ABSENT from the child environment, never set to empty

The console port is deliberately NOT here. It lives in the profile's `Emulatr.ini` under `[SRMConsole] port`, because the emulator itself reads it from there. The registry never duplicates a fact the ini owns.

launch_ui -- launcher UI state

value	type	meaning
<code>lastSelectedSystem</code>	REG_SZ	GUID of the system selected when the launcher last ran; restored at startup
<code>showDeveloperVariables</code>	REG_SZ bool	whether the environment panel shows developer-tier variables
<code>windowGeometry</code>	REG_BINARY	saved window size and position

launch_paths -- operator override paths

All three are optional. When one is absent, discovery runs instead, in this order: explicit override, then the installed location `C:\Program Files\eNVy Systems, Inc\ASA-EmulatR`, then the newest development-tree build.

value	type	meaning
<code>emulatrExe</code>	REG_SZ	override path to <code>Emulatr.exe</code>
<code>platEdExe</code>	REG_SZ	override path to the Platform Editor
<code>terminalExe</code>	REG_SZ	override path to the console terminal client; default discovery finds PuTTY

launch_run -- run behavior

`shutdownTimeoutMs` (REG_DWORD) sets how long Stop waits, after writing the `EMULATR_STOP` sentinel, before OFFERING escalation. The default is 10000 and the floor is 1000. The launcher never force-kills on its own.

Legacy hive migration

Launcher builds before 2026-08-14 used a private hive at

`HKEY_CURRENT_USER\Software\eNVy Systems\EmulatrLaunch`. On startup the current launcher performs a ONE-TIME, READ-ONLY migration: if the product hive has no

`launch_systems` group and the legacy hive has keys, every legacy key is copied into the product hive with the `launch_` prefix -- `systems\...` becomes `launch_systems\...`, `ui\...` becomes `launch_ui\...`, and so on.

The legacy hive is left untouched and can be deleted by hand once you have confirmed the migration. The gate -- that the product hive must not already have `launch_systems` -- guarantees the migration can never overwrite real state.

What survives what

event	registry catalog	profile directories
Launcher restart	kept	kept
Kit uninstall, reinstall or upgrade	kept -- HKCU is not touched	kept, including customized manifests
Registry hive deleted	lost -- recover with Add Existing... per profile	kept
Profile directory deleted	entry remains, flagged Broken by preflight	lost -- the profile IS the system
Windows user profile migration	moves with HKCU	move the directories, then Relocate or Add Existing

Editing by hand

Prefer the launcher UI -- every key above has a control. Hand-editing the hive is safe while the launcher is NOT running; it reads the hive at startup and on selection changes, and concurrent instances are last-writer-wins. Two rules if you do edit by hand:

1. `runDir` must remain an absolute path to a directory holding an `Emulatr.ini`. Otherwise preflight flags the system Broken -- it is never silently dropped.
2. Do not invent keys outside your subsystem's prefix. Unknown keys are ignored today, but the prefix IS the collision contract between subsystems.

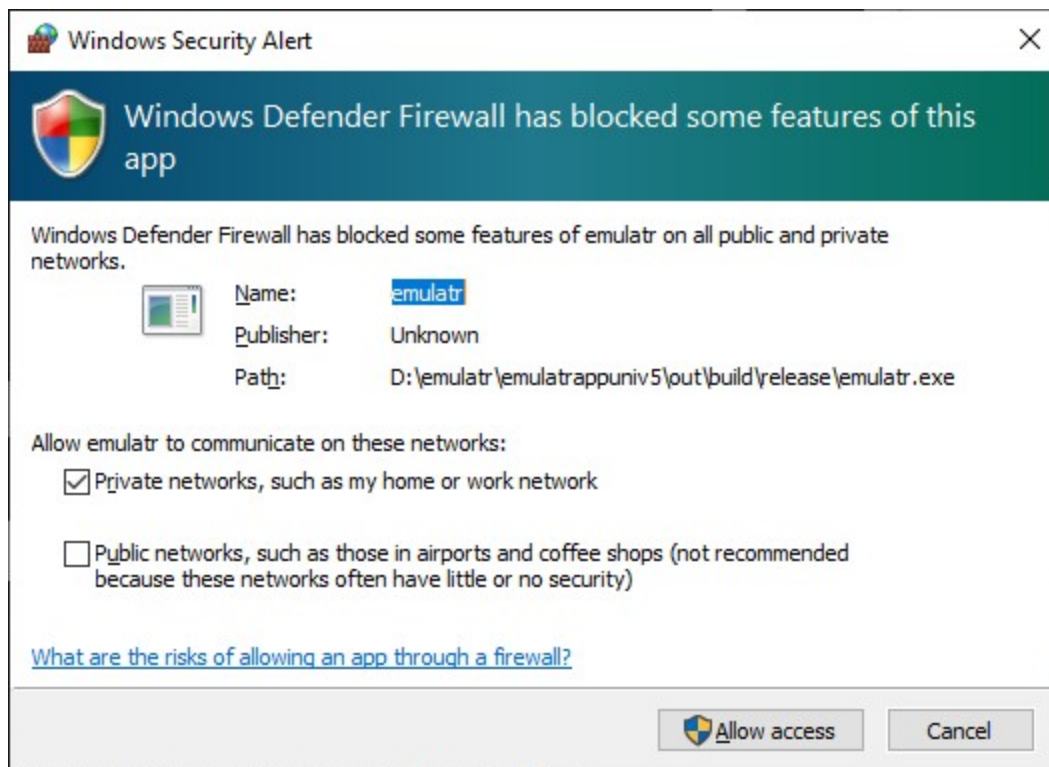
See Also.

[Configuring a System with the Launcher](#) ⁶³
[Environment Variables \(EMULATR_*\)](#) ²¹⁶
[System Disk Configuration](#) ⁹⁹

6.2 Windows Defender Firewall

Select button: Allow Access.

Disable Antivirus for the directory tree which contains the EmulatR storage `.vDisk`, `vTable`, and `.img` files.



6.3 Execution Matrix (Models x Modes)

A quick reference for the testing teams: each supported model crossed with its two execution modes -- Virtual (ISP) and REAL_HW (Silicon) -- plus the orthogonal firmware.axpsnap snapshot accelerator, and what each combination is expected to reach TODAY. Cells describe current, verified behavior; open diagnostics are cross-referenced to the Known Gaps IDs (G-*nn*). With every lever unset, all models run the faithful decompressor on the ISP default and constitute the deterministic Oracle.

Model / chipset	Virtual (ISP) mode	REAL_HW (Silicon) mode	firmware.axpsnap (snapshot boot)	Current milestone / notes
DS10 (Tsunami 21272)	Reaches SRM P00>>> reliably (~200 iters). <code>kern\$_micro_delay</code> returns immediately; no real device discovery. Fastest to console.	Full RSCC timing grind. Use <code>EMULATR_IDLEWAIT</code> to reach the prompt in practical time.	Supported. <code>firmware/ds10_*.axpsnap</code> skips self-decompression; restore-except-flash honors live set / update srm.	Reference-stable path. Faithful Oracle when all levers are unset.
DS20 (Tsunami 21272)	Reaches SRM prompt. Enumerates both	Timing grind; <code>EMULATR_IDLEWAIT</code> available.	Supported. Per-firmware keyed <code><stem>.axpsnap</code> .	DS20-under-ISP enumeration

Model / chipset	Virtual (ISP) mode	REAL_HW (Silicon) mode	firmware.axpsnap (snapshot boot)	Current milestone / notes
	IDE units (dqa0 / dqa1) even under the ISP default (anomaly G-07).			anomaly (G-07) still unreconciled.
ES40 (Typhoon 21272)	Reaches P00>>> but <code>init</code> does not complete (G-01 / G-02). PCHIP1 CSR / IIC-base blockers (G-04 / G-10).	Does NOT yet initialize -- OUTSTANDING DIAGNOSTIC (G-24). Under warp the RSCC spin exits (G-08) but <code>init</code> still does not complete.	Supported. Typhoon-variant keyed; the validator blocks a Typhoon snapshot from restoring into a Tsunami (DS10 / DS20) machine.	Release-checkpoint target. Active diagnostic surface in BOTH modes -- see Known Gaps.
ES45 / Titan (21274)	Scaffold only -- not booting. Titan CSR / Pchip scaffold present.	Not applicable yet (scaffold).	Not applicable until boot.	V5 trajectory (Titan chipset = DS15 / DS25 / ES45).

Mode and lever legend

- `Virtual / ISP (Engineering)` -- *the DEFAULT* (`EMULATR_PLATFORM` unset). *The 0xBFFC platform read is intercepted to return 0xCAFEDEED, `krn$micro_delay` returns immediately, and no real device discovery runs. Fastest path to the console.*
- `REAL_HW / Silicon` -- *opt in with* `EMULATR_PLATFORM=silicon`. *Real RSCC-based micro-delay timing and the device-discovery path are exercised; pair with `EMULATR_IDLEWARP` to make the timing grind practical.*
- `firmware.axpsnap` -- *opt in with* `EMULATR_FAST_DECOMPRESS=snapshot`. *Orthogonal to the two modes: it captures a complete-machine snapshot at entryPa and restores it, skipping the guest self-decompression. Per-firmware keyed (firmware/<stem>.axpsnap), restore-except-flash, validated by checksum + version + chipset variant.*

Other testing considerations

- *Determinism / Oracle: all levers unset => faithful decompressor + ISP default = the deterministic Oracle. Warp, snapshot, and silicon are all opt-in and must be named when a run trades determinism.*
- *Warp knobs: `EMULATR_IDLEWARP` is safe. `EMULATR_RSCCWARP` is QUARANTINED (jumped ticks corrupt the boot) -- do not enable it.*
- *Output placement: run and console logs go to `./logs`, execution / retire traces to `./traces`, timestamped `purpose_YYYYMMDD_HHMMSS.ext`.*

- *Cross-model isolation: a snapshot never restores across chipset families -- a Typhoon (ES40) snapshot cannot load into a Tsunami (DS10 / DS20) machine; a mismatch falls back to a faithful decompress.*

See Also.

[Execution Modes](#) ^{→ 178}

[Boot Acceleration - Entry Snapshot](#) ^{→ 107}

[Known Gaps](#) ^{→ 23}

[Environment Variables \(EMULATR *\)](#) ^{→ 216}

6.4 Per-CPU Halt Codes

When an Alpha processor stops and drops into console I/O mode, PALcode records WHY in the processor's per-CPU slot of the HWRPB. A developer reading an EmulatR trace or a console banner sees this value as the halt code. This page is the lookup table behind those numbers: the architected reason-for-halt set from the Alpha Architecture Reference Manual (AARM), the EV6 (21264) PALcode symbols EmulatR's PAL layer actually emits, and the companion halt-request field that system software sets to direct the console's next action. The authoritative references are the AARM (Table 26–4, per-CPU slot fields) and the apismr EV6 PALcode sources under `Processor Support/Palcode/`.

Where the code lives

Each processor owns a per-CPU slot in the HWRPB. When a halt condition is encountered, PALcode captures the halting context and the reason into these slot fields (offsets relative to the slot base, AARM Table 26–4):

- `+256 HALT ARGUMENT LIST` – value of R25 at the halt.
- `+264 HALT RETURN ADDRESS` – value of R26 at the halt.
- `+272 HALT PROCEDURE VALUE` – value of R27 at the halt.
- `+280 REASON FOR HALT` – the code from the first table below. Set to 0 at console initialization.

The restart PC (HALT PC) and PS the console uses to resume live earlier in the slot; the boot handoff parks HALT PC at `0x20000000` (see Reset into Secondary Boot).

Architected reason-for-halt codes (AARM Table 26–4)

These are the values the architecture defines for the REASON FOR HALT quadword. Codes `0x8..0xFFF` are reserved; values above that are implementation-specific (the EV6 realization is in the next table).

Code	Reason	Meaning
0x0	Bootstrap / start / powerfail restart	Default. Set at console init and at every system bootstrap, processor start, or powerfail restart. Not an error.
0x1	Operator-requested crash	Console operator requested a system crash.
0x2	Kernel stack not valid	PALcode could not push state because the kernel stack was not valid (or misaligned) – a kernel-stack-not-valid halt.
0x3	Invalid SCBB	The System Control Block Base was invalid when PALcode tried to dispatch through the SCB.
0x4	Invalid PTBR	The Page Table Base Register was invalid (or a page-table error occurred) during translation.
0x5	CALL_PAL HALT in kernel mode	The processor executed a CALL_PAL HALT instruction in kernel mode – the ordinary software-directed halt.
0x6	Double error abort	A second error was encountered while handling a machine check – a double-error abort.
0x7	Machine check in PALcode	A machine check occurred while executing in the PALcode environment.
0x8 .. 0xFFF	Reserved	Reserved by the architecture.
Other	Implementation-specific	Reserved for implementation-specific halt / console-entry

Code	Reason	Meaning
		reasons – see the EV6 realization below.

EV6 (21264) PALcode realization

The apism EV6 PALcode (`ev6_alpha_defs.mar`, macro `HALT_DEFS`) names the architected codes as `HALT__*` symbols and adds implementation-specific ones for the D-cache tag-parity halt and the several console-entry dispatch reasons. These are the codes EmulatR's PAL layer works with; `0x8` and everything at `0x20` and above fall in the architected "implementation-specific" range.

Code	Symbol	Meaning
0x00	<code>HALT__RESET</code>	Bootstrap / reset.
0x01	<code>HALT__HW_HALT</code>	Console-requested halt. EmulatR's CSERVICE HALT delivers this.
0x02	<code>HALT__KSP_INVALID</code>	Kernel stack not valid.
0x03	<code>HALT__SCBB_INVALID</code>	Invalid SCBB.
0x04	<code>HALT__PTBR_INVALID</code>	Invalid PTBR or other page-table error.
0x05	<code>HALT__SW_HALT</code>	CALL_PAL HALT executed.
0x06	<code>HALT__DBL_MCHK</code>	Double error abort.
0x07	<code>HALT__MCHK_FROM_PAL</code>	Machine check while in PAL mode.
0x08	<code>HALT__DC_TAG_PERR_FROM_PAL</code>	D-cache tag parity error while in PAL mode (EV6 implementation-specific).
0x20	<code>HALT__START</code>	Console-entry dispatch: processor start.
0x21	<code>HALT__CALLBACK</code>	Console-entry dispatch: console callback.
0x22	<code>HALT__MPSTART</code>	Console-entry dispatch: multiprocessor (secondary) start.
0x23	<code>HALT__LFU_START</code>	Console-entry dispatch: firmware update (LFU) start.
0x24	<code>HALT__FAULT_RESET</code>	Console-entry dispatch: fault reset.
0x40	<code>HALT__JUMP0</code>	Jump through 0 – a wild control transfer to VA 0 was trapped.

Halt-request codes (the requested action)

Distinct from the reason-for-halt is the HALT REQUESTED field carried in the per-CPU slot STATE. System software (or firmware) writes it to tell the console what to do when the processor next halts; the console clears it after acting. The apismr HWRPB definition (`hwrpb_def.sdl`, `HALT_REQUEST$` prefix) enumerates:

Code	Symbol	Requested action on halt
0x0	HALT_REQUEST\$DEFAULT	Default – take the console's normal halt action.
0x1	HALT_REQUEST\$SV_RST_TERM	Save state and restart / terminate.
0x2	HALT_REQUEST\$COLD_BOOT	Perform a cold bootstrap.
0x3	HALT_REQUEST\$WARM_BOOT	Perform a warm bootstrap.
0x4	HALT_REQUEST\$REMAIN_HALTED	Stay in console mode; do not restart.
0x5	HALT_REQUEST\$POWER_OFF_SYSTEM	Power off the system.
0x6	HALT_REQUEST\$RESERVED	Reserved.
0x7	HALT_REQUEST\$MIGRATE	Migrate (transfer this CPU's role); used in the SMP / secondary path.

How EmulatR surfaces a halt

In V5 a halt condition is delivered as the `kFaultHalt` fault. The clearest path is the console-service `HALT: C$ERVE$HALT` (function 0x40) in `palBoxLib/grains/PalEntries.cpp` mirrors the SRM `cfw_halt` – it corresponds to `HALT__HW_HALT` and returns `kFaultHalt`, and the run loop stops with `StopReason::HaltedClean`. The per-CPU slot's HALT PC parks at 0x20000000 for the OS boot handoff, so a stop at 0x20000000 in a trace is the transfer point, not a fault.

Source References

- Processor Support/alpha_arch_ref.txt – *AARM Table 26–4 (per-CPU slot fields); REASON FOR HALT at offset +280, architected codes 0..7.*
- Processor Support/Palcode/.../ref/ev6_alpha_defs.mar – *the HALT_DEFS macro; HALT__* symbols including the EV6 implementation-specific 0x08 and 0x20..0x40 console-entry codes.*
- Processor Support/Palcode/.../ref/hwrpb_def.sdl – *the per-CPU slot STATE.HALT_REQUESTED field and the HALT_REQUEST\$ enumeration.*
- palBoxLib/grains/PalEntries.cpp – *C\$ERVE\$HALT (0x40) delivering kFaultHalt / HALT__HW_HALT.*

See Also.

[Halt Button is IN, BOOT NOT POSSIBLE](#) ²²⁹

[Memory Map & IPRs](#) ¹³⁴

[PALcode, HWRPB & SMP](#) ¹⁶²

[Reset into Secondary Boot \(CSERVE START Handoff\)](#) ¹⁸⁵

6.5 Environment Variables (EMULATR_*)

The runtime environment variables EmulatR reads at launch, plus the compile-time flags that look like env vars but are CMake options. All values are read with `std::getenv -- set` them in the launching shell before starting the process (inline bash assignment is the normal form).

Three gotchas

- **Presence, not value.** Most gates test only that the variable is set, so `VAR=0` still ENABLES them. Exceptions that parse a value are noted below.
- **Read once.** The trace / diagnostic gates cache their result on first call; exporting them mid-run has no effect.
- **Paths are CWD-relative.** The stop sentinel and the snapshots directory resolve against the launch directory; launch from the EmulatR root.

Snapshot / autoload

Variable	Default	Effect
EMULATR_CONSOLE_SNAPSHOT	off	Arms the console-marker snapshot watch; a matching input line writes a .axpsnap. EMULATR_CONSOLE_SNAPSHOT=1 arms a console marker watch. The marker (default, overridable via EMULATR_SNAPSHOT_MARKER) is the literal line: >>>set oem_string snapshot
EMULATR_SNAPSHOT_MARKER	set oem_string snapshot	The exact line that triggers a console snapshot (internal whitespace NOT normalized).
EMULATR_AUTOSNAP	on	=off disables periodic auto saves. Mirrors --autosnapshot off.
EMULATR_NO_AUTOLOAD	unset	Skip autoload-newest; force a genuine cold boot. Mirrors --no-autoload.
EMULATR_SNAPSHOT_PRUNE	unset	Enable pruning of old snapshots (off by default).

Process / platform

Variable	Default	Effect
EMULATR_STOP_FILE	EMULATR_STOP	Path of the graceful-stop sentinel; touch it to flush flash NVRAM and exit cleanly.
EMULATR_NO_PUTTY	unset	Do not auto-launch PuTTY; the listen socket still comes up.
EMULATR_CONSOLE_PORT	10023	TCP console port (PuTTY auto-launch follows it). Give each instance its own port.
EMULATR_FLASH_ROM	<firmware>.rom	Backing file for the flash / NVRAM image. Isolate it per instance to avoid corruption. When unset the backing is DERIVED from the firmware image name (<code>es40_v7_3.exe -> es40_v7_3.rom</code> in the same directory); the fixed <code>ds10_flash.rom</code> is only the legacy final fallback. Resolution order: <code>EMULATR_FLASH_ROM > ini [ROM]flashImage > <firmware>.rom > ds10_flash.rom</code>. example bash: <code>EMULATR_CONSOLE_SNAPSHOT=1</code> Adding <code>EMULATR_FLASH_ROM=ds20_flash.rom</code> configures DS20 with the persisted NVRAM ROM image (serial + memory_test none). No <code>--max-cycles --</code> (cold default - cycles count are unlimited). Build clean (PROBES off) first if you can -- the marker feature works on any build, but it kills the <code>DIVERT-REI</code> spam and gives a production-grade snapshot.
EMULATR_TOY_MODE	unset	TOY / RTC time-source mode (SPEC-TOY-001). <code>unset</code> (default) = dead-battery lifecycle: Reg D VRT reads 0 and the clock fields read invalid until the guest sets the time via the MC146818 SET protocol, after which the time is valid, advances on the emulated cycle clock, and persists. <code>fixed</code> = always valid, epoch 2026-01-01 + cycles -- a pure function of guest cycles; MANDATORY for do-no-harm / differential / Oracle gate runs. <code>host</code> = seeded from the host wall clock at startup, then advances on cycles; best usability but BREAKS bit-reproducibility -- never valid for a gate run.

Variable	Default	Effect
EMULATR_TOY_NVRAM	nvrn/toy_cm os.bin	Backing file for the 256-byte RTC CMOS array plus the time origin, relative to the run directory. Loaded at startup (absent/short/corrupt/mode-mismatch => UNSET semantics plus one loud stderr line), flushed on every CMOS store, so a hard kill loses nothing. A store written by one EMULATR_TOY_MODE is never consumed by another mode -- point a gate run at a scratch path or rely on the mode-mismatch refusal. SEPARATE from the SRM environment-variable flash store (EMULATR_FLASH_ROM).
EMULATR_PLATFORM_CONFIG	derived DS10	Override the platform manifest.

Logging / diagnostics and device traces

Variable	Effect
EMULATR_CHIPSET_DIAG_OFF	Silences the chipset CSR diagnostics (inverted sense: setting it turns them OFF).
EMULATR_TRACE_WINDOW	Forces the retire-compact trace window.
EMULATR_RETIRED_TRACE_DIR	Output directory for retire / breakpoint / profiler trace files.
EMULATR_IIC_TRACE	Trace PCF8584 IIC bus transactions.
EMULATR_FLASH_TRACE	Trace FlashRom command / erase / write activity.
EMULATR_PIC_TRACE	Trace the 8259 PIC pair (IRQ assert / EOI).

The variables below were added from a full v5 source audit (2026-07-21). Entries marked (value) parse their value rather than testing presence only; several diagnostic and probe gates are compiled only in EMULATR_BRINGUP_PROBES builds and have no effect in a stock build.

Additional device & bus traces

Variable	Effect
EMULATR_FDC_TRACE	Trace 82077 floppy controller commands and IRQ6 edges.
EMULATR_IDE_TRACE	Trace the ALi M5229 IDE controller taskfile activity.
EMULATR_IIC_CTRL_TRACE	Trace PCF8584 IIC controller-register (S0/S1) accesses.
EMULATR_PCI_CFG_TRACE	Trace PCI configuration-space reads and writes at the Pchip.
EMULATR_UARTWATCH	Watch and trace 16550 UART register accesses.

Variable	Effect
EMULATR_CONSOLE_MIRROR	Mirror the guest console stream to the host stdout.
EMULATR_VGA_DUMP	Path to dump the VGA text framebuffer to (value = file path).
EMULATR_CHIPSET_DIAG_ON	Explicitly enable the chipset CSR diagnostics (counterpart to EMULATR_CHIPSET_DIAG_OFF).
EMULATR_TULIP_TRACE	Trace the DEC 21143 (DE500 Tulip) NIC: one-shot DMA-window dump plus per-translate trace.

Memory & PA watches

Variable	Effect
EMULATR_GMEM_WATCH	Watch guest-memory accesses at a single physical address (value = hex PA).
EMULATR_GMEM_WATCH_LO	Low bound of a guest-memory watch range (value = hex PA).
EMULATR_GMEM_WATCH_HI	High bound of a guest-memory watch range (value = hex PA).
EMULATR_PA_WATCH	Physical-address base for the MemDrainer store/load watch (value = hex PA).
EMULATR_PA_WATCH_LEN	Length in bytes of the EMULATR_PA_WATCH window (value).
EMULATR_VECWATCH_VAL	Watch guest memory for a specific value (value; EMULATR_DIAGNOSTIC_LOGGING builds).
EMULATR_START_WATCH	Watch the SMP secondary-CPU START handshake accesses.
EMULATR_SYSVAR_WATCH	Watch the IIC sysvar / OCP model-badge probe accesses.
EMULATR_IIC_WATCH	Watch IIC accesses at the MemDrainer seam.
EMULATR_VAL_WATCH	Watch for a specific value as it executes during b dqa0 (value; capped).
EMULATR_VAL_WATCH_CYCLO	Cycle floor for EMULATR_VAL_WATCH; skips early cold-boot noise (value).

Gated trace arming / windows

Variable	Effect
EMULATR_TRACE_ARM_PA	Arm the retire trace when a store hits this physical address (value = hex PA).

Variable	Effect
EMULATR_TRACE_ARM_VAL	Value that, written to EMULATR_TRACE_ARM_PA, arms the trace (value).
EMULATR_TRACE_DISARM_PA	Disarm the retire trace when a store hits this physical address (value = hex PA).
EMULATR_TRACE_DISARM_VAL	Value that, written to EMULATR_TRACE_DISARM_PA, disarms the trace (value).
EMULATR_TRACE_ARM_INSTRS	Number of instructions to record once the trace is armed (value; default 8000000).
EMULATR_TRACE_ARM_ON_DTBM	Arm the trace on the next DTB miss.
EMULATR_TRACE_ARM_ON_IIC	Arm the trace on the next IIC access.
EMULATR_TRACE_ON_BOOTSTRAP	Arm the trace at the bootstrap (0x20000000) handoff.
EMULATR_PC_GATE	Gate the decode-listing trace to a specific PC (value = hex PC).
EMULATR_VALUE_GATE	Gate the decode-listing trace on a specific value (value).
EMULATR_VALUE_GATE_FLOOR	Lower bound applied to EMULATR_VALUE_GATE (value).

MMU / translation probes

Variable	Effect
EMULATR_HOOKA_VA	Target VA for the ACVPROBE translation hooks; dumps issuing PC and GPRs on match (value = hex VA).
EMULATR_HOOKA_CYC_FLOOR	Suppress the ACVPROBE hook until this cycle is reached (value).
EMULATR_ITBPROBE_VA	Target VA for the ITB probe (value = hex VA).

Pipeline & PAL diagnostic gates

Variable	Effect
EMULATR_DIAG_PCLO	Low bound of the retire-diagnostic PC window (value = hex PC).
EMULATR_DIAG_PCHI	High bound of the retire-diagnostic PC window (value = hex PC).
EMULATR_DIAG_CYCLO	Low bound of the retire-diagnostic cycle window (value).
EMULATR_DIAG_CYCHI	High bound of the retire-diagnostic cycle window (value).

Variable	Effect
EMULATR_DIAG_WREG	Register the retire diagnostic watches (value).
EMULATR_DIAG_WMIN	Minimum value threshold for the retire diagnostic (value).
EMULATR_DIAG_CAP	Cap on retire-diagnostic output lines (default 200).
EMULATR_DTBMISS_DIAG	Enable the DTB-miss diagnostic.
EMULATR_VACTL_DIAG	Log every VA_CTL and I_CTL write (the VACTL-DIAG probe).
EMULATR_VPTB_DIAG	Diagnostic for MTPR_VPTB / VPTB writes.
EMULATR_PTBR_DIAG	Diagnostic for the PT_PTBR page-table base.
EMULATR_IIC_DIAG	PAL-level IIC diagnostic.
EMULATR_BOOTTRACE	Enable the pipeline boot trace.
EMULATR_2D_NOOP	Treat the CALL_PAL 0x2d leaf as a no-op instead of the labeled fault scaffold (bring-up gate).
EMULATR_CSERVE_START_MODE	Selects how the B+ handoff is taken at CSERVE START (0x42). Default <code>guest</code> (Option A) mirrors AXPBox <code>vm spal_call_cserve</code> exactly -- it sets p23 to the CALL_PAL return PC and diverts to the guest <code>cfw_start/exit_console</code> , letting the real PAL do the mode/VPTB/IPL transition. Set <code>cpp</code> for the old C++ B+ replica.
EMULATR_CSERVE_AUDIT	When set (=1), the CSERVE default case captures the missing contract for every func EmulatR still stubs, emitting <code>CSERVE-CONTRACT-MISSING: func=0x...</code> R16/R17/R18/R0 <code>callerPc=...</code> (one record per distinct func) so each unhandled func can be cross-referenced to its guest <code>cfw_*</code> handler. Set empty to disable.
EMULATR_CSERVE_ROUTE	When set (=1), the CSERVE default case DIVERTs to the guest <code>sys__cserve</code> dispatcher (mirror-AXPBox p23 + divert, R16 intact) instead of no-op'ing. Closes 0x65 MP_WORK_REQUEST (the MP\$RESTART boot post): guest <code>cfw_mp_work_request</code> saves R18 to CNS_WORK_REQUEST so the console proceeds past the re-init loop. Faithful for every func (unknown codes hit the guest's

Variable	Effect
	own hw_ret (p23) no-op). Emits CSERVICE-ROUTE lines. Set empty to disable.
EMULATR_DTBM_PROBE	Enable the DTB-miss probe (silent and zero-cost when unset).
EMULATR_HALT_DIAG	Pin where the console halts on the b dqa0 handoff: the swppal / console_exit transfer PC and halt code.
EMULATR_IFETCH_DIAG	I-fetch diagnostic at the missing OS-side base: dumps i_ctl / va_ctl VPTB, ptbr, and palBase at that point (capped).

RPCC read logging

Variable	Effect
EMULATR_RPCC_LOG_AFTER	Begin logging RPCC (cycle-counter) reads after this count (value).
EMULATR_RPCC_LOG_MAX	Maximum number of RPCC log entries (value).
EMULATR_RPCC_LOG_FILE	Output file path for the RPCC read log.

Spin-skip (idle acceleration)

Variable	Effect
EMULATR_SPINSKIP	Enable spin-loop skipping (idle acceleration).
EMULATR_SPINSKIP_THRESH	Iteration threshold before a detected spin is skipped (value).
EMULATR_SPINSKIP_VERBOSE	Verbose spin-skip logging.

Platform & scan

Variable	Effect
EMULATR_PLATFORM	Platform selector consulted at the MemDrainer seam (e.g. ds10 / ds20 / es40).
EMULATR_SCAN_PATTERN	String pattern for the HWRPB system scan (value; default HEAX1PEER).
EMULATR_HWRPB_SCAN_FILE	Output file for the HWRPB scan results.
EMULATR_FP_SOFTFLOAT	Force the Berkeley SoftFloat FP backend at runtime.
EMULATR_IIC_IRQ_BIT	Override the Cchip DRIR bit used for the IIC interrupt (value; default -1 = auto).

Probe / experimental

EMULATR_GCT_WATCH, EMULATR_BREAK_ON_PA10, EMULATR_CHECKPOINTS, and EMULATR_GATE are pinpoint debug gates. The ES40 bring-up probes EMULATR_TIG_RESET (model the TIG module reset) and EMULATR_UDELAYWARP (collapse the 0x6a514 micro-delay loop, advancing only to the next timer edge so no timer boundary is skipped), together with the trace gates EMULATR_TIG_TRACE / EMULATR_RSCC_DIAG, are gated behind the compile-time EMULATR_BRINGUP_PROBES and remain experimental / provisional.

Removed 2026-08-01 — no longer recognised. EMULATR_IDLEWARP, EMULATR_RSCCWARP, EMULATR_TICKWARP, EMULATR_DELAYWARP and EMULATR_UDELAYWARP_LEGACY_UNSAFE have been taken out of the execution path. They advanced the guest clock, and one of them wrote a guest register directly to force a delay loop to exit — fabricating state the emulated machine would never have reached. Setting any of them now has no effect, and the launcher strips them from the child environment so a stale export in an operator shell cannot silently change a run. Delay and idle spins execute at their real cost. See the Release Notes for v1.6 and the WARP Variants topic.

Compile-time flags (NOT runtime env vars)

These are CMake options baked in at build time -- setting them in the shell does nothing:

EMULATR_MEMDIAG (per-instruction logging, ~900x slowdown, keep OFF),
EMULATR_DIAGNOSTIC_LOGGING (non-Release per-subsystem logging), and the
EMULATR_LOG_* spdlog level macros.

CLI flags that mirror env vars

--autosnapshot off = EMULATR_AUTOSNAP=off; --no-autoload =
EMULATR_NO_AUTOLOAD=1. --max-cycles, --snapshot-on-pc, and --snapshot-name-tag
have no env equivalent.

See Also.

[Boot Acceleration - Entry Snapshot](#) ^{↳107}

[Configuring](#) ^{↳69}

[Diagnostics & Debugging](#) ^{↳113}

[Executing](#) ^{↳74}

[Warp Variants and Cycle Accounting](#) ^{↳200}

6.6 Development Conventions

The standing conventions for working in the EmulatR tree. The file-placement rules are also encoded as Cowork skills so an assistant follows them automatically; the source rules are enforced by review and the MSVC build.

Scripts and output files

- *Runnable shell scripts live in `EmulatRAppUniV4/Emulatr/tools` -- never the repo root or a build directory. (Skill: `emulatr-bash-scripts`.)*
- *Run and console logs go in the run directory's `./logs`; execution / retire / CPU traces go in `./traces` (plural) -- never loose in the run-dir root. (Skill: `emulatr-log-trace-placement`.)*
- *Name every log / trace artifact `<purpose>_<YYYYMMDD>_<HHMMSS>.<ext>` so runs sort chronologically and never collide.*

Source files

- *ASCII(128) only in all file content -- no smart quotes, em dashes, Unicode arrows, or box-drawing glyphs in source, comments, docs, or generated artifacts. The MSVC pipeline expects unformatted ASCII.*
- *A copyright / attribution header on every generated source and header (and Markdown specs as an HTML comment), per ADR-0001; the template is `docs/notes/templates/header_cpp.txt`. No "small file" exemption.*
- *Include guards, never `#pragma once`. Pattern `#ifndef <DIR>_<FILE>_H` (e.g. `PIPELINELIB_PIPELINEDRIVER_H`). Qt MOC plus `#pragma once` under MSVC /permissive- causes LNK2001.*
- *Hex radix for all switch / case dispatch labels; convert decimal to hex by `VALUE (16 -> 0x10`, never digit substitution).*

C++ and tests

- *doctest: use `CHECK` only, never `REQUIRE` -- exceptions are disabled in V4.*
- *Never name an enum's printable helper `toString` (doctest ADL clash) -- use `<typeName>Name(T)` plus `operator<<`.*
- *Mark guessed IPR / SCBD values `_PROVISIONAL` and HRM-verify them before any `HW_MFPR` / `MTPR` decode matches them, or silent PAL corruption follows.*

Source References

- `docs/notes/ADR-0001-source-file-headers.md`, `docs/notes/templates/header_cpp.txt` -- *the header rule and template.*
- *Cowork skills `emulatr-bash-scripts` and `emulatr-log-trace-placement` -- the tools/ and logs/ + traces/ placement rules.*
- `config/EmulatorSettings.h` -- *trace file defaults now under `traces/` (plural).*

See Also.

[CMake Macro Guards](#)  130

[Diagnostics & Debugging](#)  113

6.7 Glossary

Terms and acronyms used throughout this guide.

Alpha AXP -- Digital's 64-bit RISC architecture. EmulatR models the 21264 (EV6) generation.
21264 / EV6 (EV67) -- the Alpha CPU implementation EmulatR emulates; EV67 is the later 21264A.

21272 / Tsunami / Typhoon -- the core-logic chipset: Cchip (control), Dchip (data), Pchip (PCI).

Cchip / Dchip / Pchip -- the three 21272 components: system control and memory arbitration, the data path, and the PCI host bridge.

CY82C693 -- the Cypress PCI-to-ISA southbridge (also the IDE / ATAPI controller and SuperIO).

SRM -- the Alpha console firmware (the >>> prompt) that EmulatR runs unmodified.

PALcode -- Privileged Architecture Library code: the low-level routines (OpenVMS and Tru64 personalities) that implement privileged operations.

SROM -- the serial ROM whose parameters seed the CPU at reset (passed in registers R16-R21).

LFU / UPD> -- the Loadable Firmware Update utility and its prompt, used to update firmware in flash.

NVRAM -- the non-volatile store (backed by the flash image) holding SRM environment variables.

IPR -- Internal Processor Register, accessed in PAL mode via HW_MTPR / HW_MFPR.

ITB / DTB -- the instruction and data Translation Buffers (the EV6 TLBs).

TLB -- Translation Lookaside Buffer: the cache of virtual-to-physical mappings.

SPAM -- Set-Partitioned Associative Memory: EmulatR's sharded, set-associative TLB model.

kseg -- the EV6 superpage region where a virtual address maps directly to physical (no page table).

ASN -- Address Space Number, the tag distinguishing per-process translations.

Box -- a functional execution domain in EmulatR: IBox (control flow), EBox (integer), FBox (floating point), MBox (memory), CBox (cache / predict), PalBox (privileged).

grain -- a single per-opcode leaf function that implements one instruction's semantics.

GCT / FRU -- the firmware's Configuration Tree and Field-Replaceable-Unit data, built during power-up.

HWRPB -- the Hardware Restart Parameter Block, the firmware-to-OS handoff structure.

VMB / SYSBOOT -- the OpenVMS primary and secondary bootstraps; a conversational boot stops at `SYSBOOT>`.

dqa / dka / dva / ewa -- SRM device names: IDE / ATAPI, SCSI, floppy, and Ethernet respectively.

ATAPI -- the packet command set for optical / removable devices on an IDE channel.

RZ-series -- DEC SCSI disk models (RZ26, RZ28, RZ29, ...).

.axpsnap -- an EmulatR machine-state snapshot file.

NXM -- Non-eXistent Memory: a physical access above the configured RAM limit.

RPCC -- the Alpha cycle-counter instruction (Read Processor Cycle Counter).

Key	Value
SCBL	SCSI Bus Control Lines

See Also.

[Overview](#)  118

[What is EmulatR](#)  6

7 7. Troubleshooting

What to check when EmulatR will not start, hangs, boots slowly, is missing a device, or refuses a boot. Several of these are expected behaviors of the current build rather than failures -- the pages say which.

In this section

EmulatR will not start -- launch-time failures.

Firmware Hangs, Slow Boot, and Missing Devices -- long pauses and absent devices.

Console Connection Issues -- can't attach a terminal.

Halt Button is IN, BOOT NOT POSSIBLE -- the halt-gate symptom.

Example (Console) execution -- a console debugging session.

evaluate Address -- the examine / deposit console commands.

See Also.

[EmulatR will not start](#) ²²⁷

[Halt Button is IN, BOOT NOT POSSIBLE](#) ²²⁹

[Current Development Status](#) ²¹

7.1 EmulatR will not start

Console port already in use. Check with `netstat -ano | grep 10023` and free it, or set `EMULATR_CONSOLE_PORT`.

Firmware not found. The ini default is the bare name and resolves against the CWD -- pass -- `firmware ./firmware/ds10_v7_3.exe` or launch from the EmulatR root.

Missing platform manifest. A missing `<model>_platform.json` is a hard error with no fallback.

*Two instances from one directory share `ds10_flash.rom` and `snapshots/`; concurrent NVRAM writes can corrupt the flash. Isolate with `EMULATR_FLASH_ROM=<copy>` (see *Executing Two Instances*).*

See Also.

[Firmware Requirements](#) ⁴⁴

[Executing Two Instances](#) ⁷⁵

[Configuring](#) ⁶⁹

7.2 Firmware Hangs / Slow Boot / Missing Devices

Firmware hangs (usually timed loops, not crashes)

Some long pauses are time-based firmware loops rather than emulator hangs:

- *-The option-firmware scan -- after "Checking dva0.0.0.0.0 for the option firmware files. . ." the firmware enters a `kern$_sleep`-based wait (the `0x7bef0` software-tick loop). It is a timed sleep, not a device probe; a profiled gap of roughly 36 minutes has been observed here (open item).*
- *`dqa0` "no media present" is followed by a stall (suspected timer loop).*
- *GCT/FRU initialization can be significantly delayed.*
- *ES40 silicon mode: a stall at "Initializing...." after an LFU exit / `update srm` is the (experimental) TIG module reset, not a crash -- see the ES40 TIG Module Reset topic.*

Slow boot (shorten it)

Boot time is dominated by power-up diagnostics and several bounded timeout loops. To speed iteration, turn the heavy diagnostics off at the prompt:

```
>>> set memory_test none
>>> set full_powerup_diags off
```

Bounded device-wait loops (approximate wall-clock): `dq_wait_on_busy` ~5000 x 100us; `dq_wait_for_drq` ~5000 x 100-200us; `dq_long_wait_on_busy` ~310000 x 100us (about 31 s). The `EMULATR_IDLEWARP` gate fast-forwards idle waits without altering committed time. On a repeated run, `EMULATR_FAST_DECOMPRESS=snapshot` skips the self-decompression entirely by reusing an entry snapshot (see Boot Acceleration - Entry Snapshot).

Missing devices

If a device is absent from `show device`, check the IDE / ATAPI enumeration path. The `dqa` CD enumeration gap is a known open item: the firmware does not always observe the device during the option-firmware scan. Tracing `0x1F0..0x1F7` plus the PCI config of bus 0 / dev 5 / func 1 distinguishes a port-never-initialized case from a probe-protocol mismatch.

See Also.

[ES40 TIG Module Reset](#) ¹⁷⁷

[Boot Acceleration - Entry Snapshot](#) ¹⁰⁷

[set](#) ⁹⁰

[ATAPI Devices](#) ⁹²

[Current Development Status](#) ²¹

7.3 Console Connection Issues

For refused, timed-out, or silent console connections, the full client coverage is in the Terminal Clients page. In brief, check three things:

The listener is up -- the log shows "Listening on TCP port ...".

The port matches your client (the default is 10023; an instance may have been started on another with EMULATR_CONSOLE_PORT).

The client is in RAW mode -- the socket is raw, with no Telnet option negotiation.

See Also.

[Terminal Clients](#) ⁸¹

[TCP Console Overview](#) ⁸⁰

7.4 Halt Button is IN, BOOT NOT POSSIBLE

Symptom

At power-up the console prints "Halt Button is IN, AUTO_ACTION ignored", and at the >>> prompt any boot attempt (for example `b dqal`) is refused with "Halt Button is IN, BOOT NOT POSSIBLE".

What it means

The firmware reads the front-panel RUN/STOP (halt) switch and finds it asserted (IN). A halted machine refuses to boot and ignores `auto_action` -- on real hardware you would release the halt switch to allow a boot.

In EmulatR - RESOLVED

This was a real defect in EmulatR and it is FIXED. A normal machine now powers up with the halt switch released, honours `auto_action`, and accepts a boot at the >>> prompt. If you are reading this because you saw the message, see "If you still see this" below.

Root cause

The firmware reads the halt-switch state from a system-interface status register (`smir`, in the TIG register block). EmulatR did not model that register block at all, so reads of it fell through to the unmapped-space default of all-ones - every bit set, including the bit that means "halt switch asserted". The firmware was reading the machine correctly; the machine was lying to it. The fix was to model the TIG faithfully so the register returns real state.

Two plausible explanations were investigated and ruled out with evidence before the real one was found: a PALcode halt-switch flag, and an interrupt line being misread as a halt request. Neither was involved.

This is the textbook form of the most common class of emulator defect: an unmodeled region returns a value that looks plausible (all-ones or all-zeros), the firmware branches on it, and the symptom appears far away from the actual defect. It is why EmulatR's standing rule is that every register the console reads must be decoded and faithful.

What it used to look like

Kept for reference, since the message itself is genuine DEC console output you may meet on real hardware:

```
AlphaServer DS10 268 MHz Console V7.3-2, Feb 27 2007 13:25:53
Halt Button is IN, AUTO_ACTION ignored
>>> b dqal
Halt Button is IN, BOOT NOT POSSIBLE
>>>
```

If you still see this

On a current build the message means the halt state is genuinely asserted, not that the register is floating. Check that the platform manifest for your machine actually loaded - an invalid or misnamed manifest silently falls back to the built-in default device tree, which can leave your machine without the device that supplies this register (see Current Platform Support). Report it if the machine is otherwise correctly configured.

See Also.

[Current Development Status](#) ²¹

[Guest Boot](#) ¹¹⁰

[boot](#) ⁹⁰

7.5 Example (Console) execution

A console debugging session on a current build (1024 MB, DS10 268 MHz). Note the cosmetic "ISA table corrupt" message at power-up (the firmware reinitializes the ISA table to defaults), the "system serial number not set" advisory, and the use of `e` (examine) and `d` (deposit) to inspect physical memory.

```
1024 Meg of system memory
probing hose 0, PCI
probing PCI-to-ISA bridge, bus 1
ERROR: ISA table corrupt!
Initializing table to defaults
type >>>init to use these changes
bus 0, slot 5 -- dqa -- Cypress 82C693 IDE
*** system serial number not set. use set sys_serial_num command.
initializing GCT/FRU at 3ff32000
Checking dqal.1.0.105.0 for the option firmware files. . .
...
AlphaServer DS10 268 MHz Console V7.3-2, Feb 27 2007 13:25:53
Halt Button is IN, AUTO_ACTION ignored
```

```
>>> e pmem:80130000040
pmem: 80130000040 000000000000000000
>>> e pmem:801300003C0
pmem: 801300003C0 000000000000000000
>>> d pmem:80130000040 1
>>> e pmem:80130000040
pmem: 80130000040 000000000000000000
```

See evaluate Address for the command syntax, and Halt Button is IN, BOOT NOT POSSIBLE for the halt-gate context.

See Also.

[evaluate Address](#) → 91

[Halt Button is IN, BOOT NOT POSSIBLE](#) → 229

[Example - Performance](#) → 75

8 8. Appendices - Developer Reference

The deep developer lens: per-platform reference material and register-level engineering notes that sit beneath the operator-facing chapters.

In this section

8.1 Platform Appendices -- per-model notes (DS10 in depth; DS20 / ES40 / ES45).

8.2 Subsystem Internals -- floating point, the grain codegen, CMake guards.

8.3 Architecture Deep-Dives -- the register-level companions to Architecture & Internals.

8.4 Tools -- developer tooling (Ghidra).

See Also.

[Architecture Deep-Dives](#) ^{↳ 131}

[Architecture Overview](#) ^{↳ 118}

[Ghidra](#) ^{↳ 245}

[Building from Source](#) ^{↳ 45}

8.1 8.1 Platform Appendices

Per-model reference material. The DS10 is the only platform EmulatR currently models and drives end to end, so it carries a full appendix with detailed sub-topics; the larger Tsunami-family machines (DS20, ES40, ES45) are scaffolded but not yet validated and are documented here as planned work.

In this section

Appendix - DS10 -- the modeled platform: CPU, chipset, device manifest, and detail sub-topics.

Env Vars Operation Ref / Kill the EmulatR / Known Limitations -- DS10 operational reference.

SRM Boot / SRM Commands Prompt / SuperIO Controller -- DS10 firmware and SuperIO analysis.

Task Matrix / TimeOut Loops -- engineering status and the bounded firmware wait loops.

Appendix DS20 / ES40 / ES45 -- planned platforms (not yet validated).

Platform Memory Reference -- vendor memory specifications, memory architecture and reference sources for every modeled platform.

See Also.

[Appendix - DS10](#) ^{↳ 233}

[Current Platform Support](#) ^{↳ 17}

[Platform Memory Reference](#) ^{↳ 238}

8.1.1 Appendix - DS10

Reference material specific to the DS10 platform -- EmulatR's reference model, driven end to end to the SRM prompt (DS20 also boots to the prompt; ES40 is in active bring-up). This page summarizes the hardware EmulatR presents and the device manifest that defines it; the sub-topics carry the environment-variable reference, the SRM and SuperIO analysis, and the engineering status.

Hardware summary

The AlphaServer DS10 is a single-processor Tsunami-family workstation. EmulatR presents a single 21264 (EV6) CPU, the 21272 (Tsunami) core-logic chipset, and a Cypress CY82C693 combination ISA bridge / IDE controller / SuperIO. The default configuration reports 1024 MB of memory and a 268 MHz cycle clock. The platform is defined declaratively by `ds10_platform.json`; a missing manifest is a hard startup error with no fallback.

PCI device manifest

All devices sit on hose 0, bus 0. From `ds10_platform.json`:

Slot.Func	Device	Vendor:Device	Class	Role
5.0	Cypress ISA bridge	0x1080:0xc693	0x060100	ISA bridge + embedded SuperIO (COM/LPT/FDD).
5.1	Cypress IDE	0x1080:0xc693	0x010100	IDE controller; fixed legacy ports (primary 0x1F0 / 0x3F6, secondary 0x170 / 0x376).
7.0	DE500 tulip	0x1011:0x0019	0x020000	21143 Ethernet; currently a generic stub (full model is planned).

The IDE controller carries two storage units on its primary channel: `dqa0` (primary master) is the bootable ATA fixed disk (raw flat image, 512-byte sectors, byte offset = LBA x 512), and `dqa1` (primary slave) is the ATAPI CD. Media for each is resolved from the manifest against the `[Storage] diskDir` in the ini; an empty media string means no disk / no disc.

IIC / FRU devices

The DS10 manifest also models the IIC (I2C) bus the firmware probes during GCT/FRU initialization: two system-status devices (0x70, 0x72), an SMB FRU EEPROM for the DS10

board (0xA2) and one for the EV6 CPU (0xA4), and a 256-byte RCM NVRAM (0xC0). The PCI vendor/device IDs and BAR sizes are marked provisional in the manifest pending a real hardware dump.

Source References

- `systemLib/PlatformConfig.{h,cpp}` -- *loads ds10_platform.json (hard error on a missing manifest).*
- `chipsetLib/Cypress_CY82C693ISABridge.h`, `deviceLib/Tsunami/Cy82C693Ide.h` -- *the Cypress ISA bridge + IDE.*
- `licPcf8584 + ToyRtc`, `chipsetLib/FlashRom.h` -- *the IIC / FRU EEPROM / RCM NVRAM devices.*
- `firmware/ds10_platform.json` -- *the declarative device manifest.*

See Also.

[SuperIO Controller](#) ^{→ 235}

[Task Matrix / TimeOut Loops](#) ^{→ 236}

[Current Platform Support](#) ^{→ 17}

[Env Vars, Shutdown & Known Limitations](#) ^{→ 234} [SRM Boot, Commands & SuperIO](#) ^{→ 235} [Task Matrix & TimeOut Loops](#) ^{→ 236}

8.1.1.1 Env Vars, Shutdown & Known Limitations

The DS10 operational quick reference: where the environment-variable controls live, how to stop a running instance cleanly, and the limitations a DS10 operator should expect today. The full catalogues are on the dedicated reference pages; this page is the platform-scoped summary and the pointers.

Environment variables

EmulatR reads a set of `EMULATR_*` environment variables that override ini defaults -- the console TCP port (`EMULATR_CONSOLE_PORT`), the flash ROM path (`EMULATR_FLASH_ROM`), the idle fast-forward gate (`EMULATR_IDLEWARP`), trace directory, and others. The complete list with semantics is the Environment Variables reference page; the SRM >>> variable catalogue (the firmware's own environment, set with `set`) is covered under Operating EmulatR.

Stopping an instance cleanly

Stop EmulatR by closing the console / GUI session so NVRAM and the flash ROM flush normally; avoid killing the process mid-write. The DS10 keeps its non-volatile state in `ds10_flash.rom` and the RCM NVRAM, so an interrupted write during a flash update can corrupt persisted firmware settings. When running two instances from one directory, isolate the flash with `EMULATR_FLASH_ROM` so concurrent NVRAM writes cannot collide.

Known limitations (DS10)

The SRM firmware boots all the way to the interactive >>> prompt; what is not yet complete is booting a guest operating system. No bootable disk is enumerated yet and the on-board PCI devices are not walked, and b dqa1 currently reports "BOOT NOT POSSIBLE" (see Current Development Status).

The on-board DE500 Ethernet is a generic stub: the firmware tolerates the missing NIC, but it is not enumerated as a real tulip yet.

The dqa1 CD is not always observed during the option-firmware scan (a known enumeration gap).

PCI vendor/device IDs and BAR sizes in the manifest are provisional pending a hardware dump.

See Also.

[Environment Variables \(EMULATR_*\)](#) ²¹⁶

[Executing Two Instances](#) ⁷⁵

[Current Development Status](#) ²¹

8.1.1.2 SRM Boot, Commands & SuperIO

The DS10 firmware reference. SRM boot and the >>> command set are covered operationally in the Quick Start and Operating EmulatR chapters; this appendix records the DS10-specific SuperIO analysis -- the part of the platform that is unique reference material rather than general operation.

SRM boot and commands (pointers)

Reaching the >>> prompt and the boot sequence are described under Quick Start; the console command catalogue (`show config`, `show device`, `boot`, `set`, and the rest) is under Operating EmulatR. The notes below assume that context and focus on the SuperIO programming the firmware performs during initialization.

SuperIO controller -- word-fold (confirmed)

The Cypress CY82C693 SuperIO is configured through 16-bit writes that fold an index byte and a data byte into one word: the low byte is the index, the high byte is the data. This byte order is locked and confirmed -- the 0x22 / 0x23 power-control writes carry their Table 61 default data bytes, and the configuration splits cleanly into `port 0x3F0 (index)` / `port 0x3F1 (data)`:

```
SIO-TRACE W port=0x3F0 val=0x60 [cfg] index 0x60
SIO-TRACE W port=0x3F1 val=0x03 [cfg] data 0x03 (FDD base high)
SIO-TRACE W port=0x3F0 val=0x70 [cfg] index 0x70
SIO-TRACE W port=0x3F1 val=0x06 [cfg] data 0x06 (IRQ6)
```

Logical device map

The firmware programs every logical device in turn (selecting each via index 0x07 = LDN, then base / IRQ / DMA / activate). The resulting assignment matches `show config` exactly:

LDN	Device	Base	IRQ / DMA
0	Floppy (FDD)	0x03F0	IRQ6 / DMA2 (Activate set)
3	Parallel (LPT1)	0x03BC	IRQ7
4	Serial (COM1)	0x03F8	IRQ4
5	Serial (COM2)	0x02F8	IRQ3

The configuration is grounded end to end in the HRM trace: 93 SIO-TRACE config lines, every one splitting cleanly into index / data, with the FDD logical device landing at 0x03F0 / IRQ6 / DMA2. The SuperIO is properly modeled and the byte-fold is confirmed beyond doubt.

Source References

- `deviceLib/Tsunami/Smc37c669SuperIo.h` -- *word-fold index/data config + LDN device map (FDD/COM/LPT).*
- `chipsetLib/Cypress_CY82C693ISABridge.h` -- *CY82C693 SuperIO/ISA plane fronting the ports.*
- `deviceLib/SRMConsole.cpp` -- *>>> command handling (show config etc.).*

See Also.

[Appendix - DS10](#) ²³³

[show config](#) ⁸⁹

[Task Matrix / TimeOut Loops](#) ²³⁶

8.1.1.3 Task Matrix & TimeOut Loops

Two pieces of DS10 engineering reference: the concrete bounds of the firmware's device-wait loops (useful when judging whether a long pause is a hang or a timed wait), and a snapshot of the open engineering work on the platform.

TimeOut loops

The firmware's IDE device-wait loops have concrete, bounded iteration counts. They are timed waits, not hangs; the longest is about half a minute of wall-clock per occurrence.

Loop function	Approx. wall clock	Notes
<code>dq_wait_on_busy</code>	5000 x 100 us	Wait for the IDE BSY bit to clear.
<code>dq_wait_for_drq</code>	5000 x 100--200 us	Wait for the data-request (DRQ)

Loop function	Approx. wall clock	Notes
		handshake.
dq_long_wait_on_busy	310000 x 100 us (about 31 s)	The long device-busy wait.

The `EMULATR_IDLEWARP` gate fast-forwards these idle waits without altering committed time. A separate, much longer pause -- the `kern$_sleep`-based option-firmware scan after "Checking dva0.0.0.0 for the option firmware files. . ." -- is a timed sleep rather than a device-wait loop and is a known performance item.

Task matrix (engineering status)

A snapshot of open DS10 engineering work, grouped by theme. This is a moving target; treat it as orientation, not a contract.

Networking. Promote the DE500 from a generic stub to a real DC21143 (tulip) model: enumerate ewa0 with a MAC read over the CSR9 SROM bit-bang, then add link / media select (SIA/MII) and the TX/RX rings for network boot.

Storage. Wire the media-backed read path (ATA READ 0x20 + ATAPI READ(10) to the file backend) so booting dqa0 works, and close the absent-unit IDE taskfile-read behavior.

PCI enumeration. Implement a real PCI bus walk so on-board devices are discovered and given valid BARs, replacing the firmware's all-ones fallback (the source of the repeated unhandled-outer-write probes).

Performance. Fast-forward the option-firmware scan sleep; keep the chipset off the hot path with a DRAM fast-path for fetch and load/store; event-driven console RX instead of a per-step poll; compile-guarded trace hooks.

Operational. Make system memory size resolvable from the QSettings ini; neutralize the "Halt Button is IN, BOOT NOT POSSIBLE" gate in software; centralize trace / diagnostic output under the build directory.

See Also.

[Firmware Hangs, Slow Boot, and Missing Devices](#) ^{↗ 228}

[Halt Button is IN, BOOT NOT POSSIBLE](#) ^{↗ 229}

[Current Development Status](#) ^{↗ 21}

8.1.2 Appendix DS20 / ES40 / ES45

The larger multi-CPU machines. DS20 is modeled and boots to the SRM >>> prompt; ES40 is in active bring-up (Typhoon variant + ALi M1543C south bridge + silicon-mode path); ES45 (a Titan box) has its firmware decompressing but is not yet driven to the prompt. This appendix records each platform's current status and manifest.

DS20

Modeled and validated to the SRM >>> prompt. DS20 is a Tsunami box that wires the Cypress CY82C693 south bridge (like the DS10) and boots the SRM console. Run with `[System] model=DS20`.

ES40

In active bring-up. ES40 is a Typhoon (21272) box: EmulatR models the Typhoon chipset variant, the ALi M1543C south bridge with the M5229 IDE, and a silicon-mode boot path. Recent work landed the TIG module-reset model and the entry-snapshot boot acceleration. Run with `[System] model=ES40` and firmware `es40_v7_3.exe`; see the ALi M1543C and ES40 TIG Module Reset topics.

ES45

Planned. ES45 (EV67 / 21264A) is a Titan (21274) box with the ALi M1543C south bridge; its compressed firmware (`es45_v7_3.exe`) decompresses correctly via the host oracle, but the Titan chipset and device modeling around it is not yet driven to the prompt. Manifest `es45_platform.json` is scaffolded.

Note that the compressed firmware oracle work (`host_decompressor`) was developed against `es45_v7_3.exe`, so the ES45 firmware image decompresses correctly even though the full platform is not yet driven; what remains is the device and chipset modeling around it.

See Also.

[Ali M1543C South-Bridge Interface](#) ¹⁴⁷

[ES40 TIG Module Reset](#) ¹⁷⁷

[Boot Acceleration - Entry Snapshot](#) ¹⁰⁷

[Appendix - DS10](#) ²³³

[Current Platform Support](#) ¹⁷

[Decompressor Internals](#) ¹⁶⁰

8.1.3 Platform Memory Reference

This appendix records the vendor memory specifications for the AlphaServer models EmulatR models or plans to model: how much memory each chassis physically holds, how its memory subsystem is organized, and where each figure came from. The maximum-memory column is the basis for EmulatR's per-model silicon clamp; what the clamp does, and when it applies, is described in Platform Identity & Memory Geometry rather than here.

Model	Maximum memory	DIMM basis	Chipset
DS10	2 GB	Webbbrick, 4 DIMM slots	Tsunami (21272)

Model	Maximum memory	DIMM basis	Chipset
DS15 / DS15a	4 GB	1 GB DIMM modules, ECC	Titan (21274)
DS20 / DS20e	4 GB	four 1 GB banks	Tsunami (21272)
DS25	16 GB	16 DIMM slots	Titan (21274)
ES40	32 GB (Model 2); 16 GB (Model 1)	32 DIMMs at 8 per memory motherboard; Model 1: 16 DIMMs	Typhoon (21272 high-bandwidth)
ES45	32 GB	32 DIMM slots, MS620 kits of 4	Titan (21274)

The vendor figures are quoted in GB as the source documents write them. EmulatR implements each ceiling as the binary equivalent -- a 2 GB DS10 is clamped at 2147483648 bytes -- which is the reading these memory specifications intend.

DS15 / DS15a

Maximum memory is 4 GB. The DS15 is built around the single-CPU Alpha 21264C (EV68C) platform architecture, and uses standard error-correcting code (ECC) memory configurations, reaching its maximum via 1 GB DIMM modules.

DS20 / DS20e

Maximum memory is 4 GB. DS20 is a Tsunami (21272) box; EmulatR models the ceiling as four 1 GB banks.

DS25

Maximum memory is 16 GB across 16 DIMM slots, on the Titan (21274) chipset. This figure is the one EmulatR already applies, and it is consistent with the chassis slot count, but it is the one entry in this appendix with no named source document behind it -- the reference set that supplied the other platforms grouped DS25 under a DS20 heading without giving it a separate maximum. Treat the 16 GB as sound but uncited until a DS25 QuickSpec is attached.

ES40

The ES40 ships in two memory builds. Model 1, the entry-level configuration, holds up to 16 GB across 16 DIMM slots on its memory motherboards. Model 2, the expanded configuration, holds up to 32 GB across 32 DIMM slots, at 8 DIMMs per memory motherboard.

The memory subsystem is built on a switch-based interconnect driven by a chipset of one C-chip (command), eight D-chips (data paths) and two P-chips (PCI interface). It carries two 256-bit memory buses supporting multiple memory arrays, which yields high-bandwidth interleaving of up to 5.2 GB/sec.

EmulatR clamps ES40 at the Model 2 figure of 32 GB. That bounds the more capable build, so a Model 1 machine modeled under the same name is not protected by the clamp at its own 16 GB ceiling.

ES45

Maximum memory is 32 GB, using up to 32 DIMM slots across 8 memory options and arrays.

The ES45 design is an evolution of the mid-range enterprise architecture and supports up to four high-speed EV68CB processors. It incorporates dual memory buses operating in a multi-way interleaved configuration -- up to 4-way interleaving when the arrays are equally populated -- to lower latency and maximize throughput. Memory options are structured in groups of 4 DIMMs per kit; the MS620 kits range from 512 MB up to 4 GB in capacity.

DS10

Maximum memory is 2 GB across the Webbrick's 4 DIMM slots. The DS10 figure predates this reference set and is carried here so the summary table covers every modeled platform.

Reference sources

Platform	Source
DS15 / DS15a	Global IT Corporation, AlphaServer DS15 specifications; and Island Computers, HP AlphaServer DS15 and DS15a.
DS20 / DS20e	Vendor QuickSpecs.
DS25	No named source; see the note in the DS25 section above.
ES40	Manx Docs / Compaq, AlphaServer ES40 Systems Technical Summary.
ES45	Hewlett Packard Enterprise (HPE) / Compaq, AlphaServer ES45 Linux-Ready technical documentation.
DS10	Project-established value; not part of this reference set.

See Also.

[Platform Identity & Memory Geometry](#) ⁷¹

[Appendix - DS10](#) ²³³

[Appendix DS20 / ES40 / ES45](#) ²³⁷

[Current Platform Support](#) ¹⁷

[Memory](#) ¹³²

8.1.4 SRM Boot Call-Stack

This appendix names the ordered path from power-up to the >>> prompt, and where each stage lives. Its practical use is localization: when a boot goes quiet, the last line the console printed identifies the stage it died in, and the stage identifies the file to open.

Where the console sources actually are

One scope correction saves most of the hunting. `apism/ref` is the platform and chip reference LEAF -- device drivers (`dq_driver.c`), PAL macros (`ev6_huf_decom.m64`), the definition files (`ev6_defs.mar`, `osfalpha_defs.mar`) and the IPR examine / deposit driver (`ev6_ipr_driver.c`). The boot-to-prompt SPINE -- the kernel executive, the secondary-start logic, and the shell that actually prints >>> -- lives in the shared `apism` source directories, not under `ref/`. A call stack confined to `ref/` is therefore incomplete by construction. [Source Cross Reference](#)²⁶² covers the tree layout.

The ordered path

The first three stages run before the console has a working output path, so they are invisible unless a trace is armed. Everything from the banner down is observable on the console.

Stage	What happens	Evidence that it completed
PAL reset entry	Power-up enters PALcode at <code>sys__reset</code> . No console output exists yet.	None -- arm a trace
Self-decompression	The shipped SRM image is compressed and decompresses itself on the emulated CPU (<code>ev6_huf_decom.m64</code>). Roughly four million cycles on a cold boot.	None -- arm a trace
Reset init	<code>sys__reset_init</code> builds <code>p_temp</code> and the impure frame, establishing the per-CPU state the console will run on.	None -- arm a trace
Banner and memory sizing	The console prints its banner and sizes guest memory.	1024 Meg of system memory
Device probe	PCI and ISA enumeration, then GCT / FRU initialization at <code>3f32000</code> .	The probe listing
Option-firmware scan	The CD and floppy devices are scanned for option firmware.	Scan lines for <code>dqa</code> devices

Stage	What happens	Evidence that it completed
LFU	The Loadable Firmware Update utility runs. <code>exit</code> returns to the console; <code>update srm</code> refreshes firmware.	UPD>
Test and prompt	System and disk testing, then the interactive console shell.	>>> (or P00>>> on a multiprocessor build)

By default the console halts at the prompt rather than booting an operating system (`auto_action = HALT`). A full annotated transcript is in [Example - Performance](#)⁷⁵.

The CSERVE seam

CSERVE is the call gate between the console and PALcode, and it is where most boot-path work actually lands. The console loads a function code into `R16` and traps; EmulatR dispatches it in `palBoxLib/grains/PalEntries.cpp`. The architecture is run-the-guest-PAL: wherever possible a CSERVE call is routed into the guest's own handler rather than replicated in C++.

Code	Name	What it does
0x40	CSERVE\$HALT	Machine halt
0x41	CSERVE\$WHAMI	Returns the executing agent's real SMP slot (<code>cpuSlot</code>). With a single agent that is 0, which is byte-identical to the hardcoded 0 it replaced, and becomes per-CPU as soon as SMP lands.
0x42	CSERVE\$START	Releases a secondary CPU -- and carries the PRIMARY operating-system boot transfer, which routes through the same path. See below.
0x44	CSERVE\$MTPR_EXC_ADDR	The console hand-off continuation used by the decompression switch
0x46	CSERVE\$IIC_WRITE	Packs a PCF8584 I2C write -- the path that badges the machine
0x66	(masked PAL_BASE)	Returns PAL_BASE with the low bits masked

The `START` path is worth understanding before touching it. On a uniprocessor model there are no secondaries to start, which once made a no-op look harmless -- but the primary OS boot transfer runs through the same entry. It reaches `sys__exit_console`, which restores the boot context from the per-CPU HWRPB slot, flushes the ITB and DTB, and restarts fetch at the slot's halt PC. Stubbing it stranded the whole handoff. The current default routes faithfully into the guest PAL; `EMULATR_CSERVE_START_MODE` selects `guest` (the default), `cpp` (a C++ replica, for testing) or `off`. [Reset into Secondary Boot](#)¹⁸⁵ covers that handoff in full.

Unimplemented CSERVE codes are a known divergence

A function code with no case falls to the dispatch default, which returns with `R0` untouched. That is a DIVERGENCE from real execution, not a tolerated no-op: the guest `sys__cserve` would dispatch the same code to a handler with real side-effects.

The distinction was settled empirically. `0x65 MP_WORK_REQUEST` disproved the earlier assumption that these were silicon-tolerated -- the console loops on it and re-initializes, because the guest handler never saves `R18` into `CNS__WORK_REQUEST`. Codes still landing in the default arm include `0x0A`, `0x0B-0x0D`, `0x32-0x37`, `0x45 JUMP_TO_ARC` and `0x65 MP_WORK_REQUEST`.

Set `EMULATR_CSERVE_AUDIT=1` to capture the missing contract for each: the function code and its inputs, the caller PC, and whether the call is one-shot or a polling loop. That capture is what lets a code be closed against the corresponding guest handler in the apism source.

Localizing a stall

Read the last console line and map it to the table above. A boot that stops after a device-probe line and before any prompt is in enumeration; one that stops at `UPD>` is waiting in LFU, not hung. A boot that produces no output at all never reached the banner, which puts it in one of the three pre-console stages -- and there the only instrument is a trace, because the console cannot yet report on itself.

Two failure shapes look like hangs but are not. A long pause during self-decompression is the guest doing several million cycles of real work. And a memory geometry the machine cannot support does not fail -- it churns its memory configuration indefinitely, alive but never converging; see [Platform Identity & Memory Geometry](#)⁷¹.

Source References

Source	What it settles
<code>palBoxLib/grains/PalEntries.cpp</code>	The CSERVE dispatch and every function code above
<code>apism ev6_vms_pc264_pal.mar</code>	The guest PAL: <code>sys__cserve</code> , <code>sys__exit_console</code>
<code>apism ref/ev6_huf_decom.m64</code>	The self-decompression path

Source	What it settles
deviceLib/Hwrpb.h	The HWRPB and its per-CPU slots
schedLib/SmpHarness.h	The multi-agent substrate the secondary path will use
chipsetLib/TsunamiCchip.h	The Cchip, including the CPU-present reporting the console reads

See Also.

[Reset into Secondary Boot \(CERVE START Handoff\)](#) ^{→185}

[Reaching the SRM Prompt](#) ^{→60}

[Firmware Decode & Decompression](#) ^{→157}

[Execution / Boot Internals](#) ^{→123}

[Source Cross Reference](#) ^{→262}

8.2 8.2 Tools

The external tools used to build, profile, reverse-engineer, and document EmulatR. None of these are required to run the emulator; they are the developer-side toolchain behind the project.

In this section

- *Ghidra* -- reverse-engineering the decompressed SRM firmware.
- *Help & Manual* -- the authoring tool that produces this documentation.
- *Software Verify* -- the Performance Validator profiler used to find hot spots.
- *Qt Creator* -- the IDE and CMake build environment.
- *DOCTest* -- the unit-test framework (doctest 2.5.0, single vendored header); CHECK-only, exceptions disabled.
- *Tools Script Reference* -- consolidated inventory of the `tools/` shell and Python scripts and where they deploy.
- *Platform Editor* -- the Qt desktop application (`editors/platedit_qt`) for editing the `<firmware>_platform.json` device manifest.

See Also.

[Required Tools & Frameworks](#) ^{→44}

[Building from Source](#) ^{→45}

[scripts](#) ^{→247}

[Tools Script Reference](#) ^{→254}

[Run-Directory Contents](#) ^{→57}

[Platform Editor](#) ^{→259}

8.2.1 Ghidra

Ghidra is the NSA's open-source software reverse-engineering (SRE) suite. In EmulatR it is the tool used to read the SRM firmware: once the compressed image is decompressed to a flat binary, Ghidra disassembles and decompiles it to C so that firmware addresses, functions, and behaviors can be identified by name rather than by raw PC.

Workflow

The input is the decompressed firmware produced by the host decompressor (for the DS10, `decompressed_ds10_v7_3.bin`). Analysis itself is done interactively in the Ghidra GUI -- import the binary, let auto-analysis finish, then save. The repetitive part is automated: `tools/decompile_ds10.sh` drives `analyzeHeadless` against the already-analyzed program (with `-noanalysis`) and runs the `DecompileToDir.java` post-script to dump C source plus an index file. Because headless needs the project lock, the GUI project must be closed and saved first.

Configuration notes

The project layout is fixed in the script: the Ghidra project lives at `EmulatRAppUniV4/ghidra`, the program is `decompressed_ds10_v7_3.bin`, and decompiled output is written to `tools/host_decompressor/out/decompiled_src`. One pitfall is pinned in the script: Ghidra 11.3.x must run on JDK 21. An early headless run on JDK 26 failed with `osgi.ee=UNKNOWN` because Ghidra's script compiler could not resolve the Java execution environment and never compiled the post-script, so `JAVA_HOME` is set to a JDK 21 in the script.

This is how firmware addresses cited elsewhere in the documentation -- the decompression spin, the `decompress()` entry, the `hw_ret R2 at PC 0xd954` -- were located and confirmed against real decompiled C.

See Also.

[Decompressor Internals](#) ¹⁶⁰

[Required Tools & Frameworks](#) ⁴⁴

8.2.2 Help & Manual

Help & Manual (EC Software) is the documentation authoring tool that produces this help project. Every page you are reading is a Help & Manual topic; the project organizes those topics into the table of contents and compiles them to the published output.

How this documentation is structured

The project is one topic per subject. Each topic is an XML file with a title, a keyword index, and a body; the table of contents is a separate map that nests the topics into the eight-section structure. Pages cross-reference each other through topic links rather than copied text, so a fact lives on one page and is linked from the others.

Authoring conventions

A consistent house style keeps the project uniform: body text uses the Normal style, section headings use the Heading styles, console and code text uses the Code Example style, and a See Also block of topic links closes most pages. Two rules matter for searchability and portability -- the keyword index on each page names every noun on that page, and the text is ASCII-only (a double hyphen stands in for a dash). These are the same conventions the rest of this documentation follows.

See Also.

[Attributions & Credits](#) ▷ 160

[Required Tools & Frameworks](#) ▷ 44

8.2.3 Software Verify

Software Verify (Software Verify Limited) makes a family of C/C++ analysis tools for Windows; the one used in EmulatR development is Performance Validator, a sampling and instrumenting profiler. It is the tool used to find where the emulator spends its time so optimization effort lands on the code that matters.

What it provides

Performance Validator attributes run time to functions and lines and presents call graphs and call trees, so a hot path is visible rather than guessed. For an instruction-set emulator this is essential: a small number of routines dominate execution, and a few percent shaved off the inner loop is worth more than any amount of tuning elsewhere.

How it is used here

Profiling confirmed that the pipeline driver's per-instruction `step()` is the overwhelming majority of run time (on the order of 82 percent), which is why that loop ships optimized and the -O0 debug build is gated behind a separate CMake option. It also quantified the cold-boot decompression spin and the bounded firmware wait loops -- the measurements behind the performance items elsewhere in this documentation. The profiler is a developer tool only; it is not involved in running the emulator.

See Also.

[Execution / Boot Internals](#) ▷ 123

[CMake Macro Guards](#) ▷ 130

8.2.4 Qt Creator

Qt Creator is the IDE used to develop and build EmulatR. It drives the CMake build, manages the compiler kit, and provides the editor and debugger; the project's reference build kit is Qt 6.10.2 with the MSVC 2022 64-bit toolchain.

Build environment

EmulatR builds with CMake and a C++20 MSVC compiler. Qt Creator configures the CMake project against the selected kit and produces the usual Debug and RelWithDebInfo build trees. The emulator core is deliberately Qt-free -- Qt is used only by the console / UI subsystem -- so the dependency on Qt is confined to the front end even though the whole project is built through the Qt Creator environment.

Build options that matter

Several behaviors are CMake options surfaced as build configuration rather than runtime flags. The most notable is `EMULATR_DEBUG_STEP`, which restores an `-O0` build of the per-instruction step for single-step debugging; it is off by default so the hot loop ships optimized. Other compile-time guards (trace hooks, diagnostic channels) follow the same pattern and are described under CMake Macro Guards.

See Also.

[Building from Source](#) ⁴⁵

[CMake Macro Guards](#) ¹³⁰

[Required Tools & Frameworks](#) ⁴⁴

8.2.5 script

The tools topic-hive documents the developer and operator scripts that live in the EmulatR V4 source `tools/` directory. The shell scripts are deployed beside `Emulatr.exe` in `<run-dir>/tools/` by the CMake `POST_BUILD` step and anchor to the run-dir root; the Python utilities run directly from the source tree. All shell scripts target Git Bash on Windows.

Most scripts locate the run-dir by anchoring to their own directory and expect `Emulatr.exe`, the firmware, and `config/EmulatrV4.ini` beside it. None of them build except where the name says so (`build_test_run`); the rest are run-only or diagnosis-only.

Build, run, and boot

[build_test_run](#) ²⁴⁹ -- Rebuild (RelWithDebInfo), run the chipset halt-register doctest, then hand off to the OpenVMS boot.

[run_fw.sh](#) ²⁵² -- Launch a `v7_3` firmware image with 4 GiB RAM for a chosen model; cold or warm boot.

[launch_vms_boot](#) ²⁵⁰ -- Cold-boot DS10/EV6 and boot OpenVMS Alpha V8.2 from `dqa1`.

Diagnostics and tracing

[diag_halt_irq4](#) ²⁵⁰ -- IRQ4 / halt-gate runbook: arm a window retire trace and snapshot interrupt state from the console.

[trace_halt](#) ²⁵³ -- Console-armed retire trace plus the TIG bring-up canary.

Trace analysis and profiling

[analyze_retire_trace](#)^{↗249} -- Find the dominant PC loop (the wedge body) in a retire-compact trace.

[profile_resolve](#)^{↗252} -- Annotate RetireProfiler dumps with SRM/PAL symbols and region rollups.

Firmware and disassembly

[alpha_diasm](#)^{↗248} -- Minimal Alpha (EV6) disassembler for reading PAL and console code by file offset.

[decompile_ds10](#)^{↗249} -- Dump C source from an already-analyzed Ghidra project (headless, decompile-only).

Snapshots and disk images

[snapmap](#)^{↗253} -- Decode an `.axpsnap` and hex-map its header, chipset block, and HWRPB.

[mkdisk](#)^{↗251} -- Generate a raw, stamped, writable disk image for the `dqa` install target.

Acceptance gate

[phase1_dispatch_gate](#)^{↗251} -- Prove the `AlphaCpuAgent` dispatcher boot is byte-identical to the legacy boot.

See Also

[What is EmulatR](#)^{↗6}

8.2.5.1 alpha_disasm

`tools/alpha_disasm.py` is a minimal Alpha (EV6) disassembler for inspecting raw firmware and decompressed PAL images. It is deliberately NOT a full disassembler: it covers the integer, memory, branch, and PAL (`HW_MFPR` / `HW_MTPR` / `HW_LD` / `HW_ST` / `HW_RET`) formats needed to read PAL and console routines. It is handy against the host-decompressor `out/*.bin` images to read a routine by file offset.

Usage

Offsets are byte offsets into the file (little-endian). Each line prints the file offset, the raw instruction word, and a decoded mnemonic.

```
python3 tools/alpha_disasm.py <image-file> <start-offset-hex> [count]
python3 tools/alpha_disasm.py decompressed.bin 0x8ab0 40
```

See Also.

[Host Decompressor](#)^{↗158}

[Ghidra](#)^{↗245}

[scripts](#)^{↗247}

8.2.5.2 analyze_retire_trace

Finds the dominant PC loop in a `TRACE_RETIRE_COMPACT` retire trace. It counts hot PCs and locates the longest consecutive repeating instruction-group in the tail of the file -- the candidate wedge pattern that emerges when a run gets stuck cycling through one tight loop body.

Usage

```
python analyze_retire_trace.py <trace_file> [--tail N] [--top N] [--max-period N]
```

Arguments

--tail N: analyze only the last N RET records (default 200000; 0 = whole file). Bounds memory at roughly 50 MB on the default.

--top N: report the top N hottest PCs (default 15).

--max-period N: maximum loop body length to consider (default 64).

Notes

The RET line parser accepts both the current `rpcc=` field and the pre-2026-06-20 `cyc=` form, and tolerates the optional `cpu=` SMP slot tag. Output flags whether the detected loop dominates the analyzed window (run almost certainly wedged) or is merely significant.

8.2.5.3 build_test_run

Rebuilds the project (RelWithDebInfo), runs the chipset halt-register doctest, then hands off to `launch_vms_boot.sh`. Deployed to `<run-dir>/tools/` by the CMake `POST_BUILD` step.

Usage

```
./build_test_run.sh
```

Notes

Requires the MSVC tools (`cl` and `ninja`) on `PATH` -- run it from a VS Developer Git Bash, or build in the IDE and then run `tools/launch_vms_boot.sh` directly. The test step runs the TsunamiChipset ISystemBus arbiter subcase; the script aborts if the build does not produce `Emulatr_tests.exe` or if the test fails.

8.2.5.4 decompile_ds10

Dumps C source from an already-analyzed Ghidra project. This script only runs the decompile post-script against the saved program -- it does not import or analyze. Analysis is performed in the Ghidra GUI first.

Before running

Let the GUI auto-analysis finish, save the program (Ctrl+S) so the functions are on disk, and close the project in the GUI so the headless run can take the lock.

Configuration

GHIDRA points at the Ghidra install. `JAVA_HOME` must be pinned to JDK 21 (the official choice for Ghidra 11.3.x); running on JDK 26 fails with `osgi.ee=UNKNOWN` because the script compiler cannot resolve the Java execution environment. The `.gpr` project name is auto-detected from `PROJECT_DIR`.

Output

`OUT_DIR/decompressed_ds10_v7_3.bin.c` plus a `.index.txt`, with the full headless log at `D:/EmulatR/traces/decompile.log`. If the log reports the project is LOCKED, close the Ghidra GUI and retry.

8.2.5.5 diag_halt_irq4

2026-06-14 runbook that disambiguates the IRQ4 / halt gate: World A (a `clr_irq*` write clears a real interrupt source) versus World C (the code is modelling a source read). Diagnosis only -- no rebuild. Deployed to `<run-dir>/tools/` by CMake `POST_BUILD`; anchors to the run-dir root.

Environment

Sets `EMULATR_TRACE_WINDOW=1` (required -- builds the window-only retire sink; arming is a no-op without it) and `EMULATR_TIG_TRACE=1` (a canary that logs any access to an unmodeled TIG-window register).

Operator steps at the >>> prompt

1) Snapshot the interrupt state:

e pmem:801a0000300 -- Cchip DRIR

e pmem:801a0000280 -- Cchip DIR0

e pmem:80130000040 -- smir (expect 0)

2) Arm the retire trace with a READ to `pmem:80130000FF8` -- it must return `7FFFFFFFFFFFFFFFFF`; a deposit to that PA disarms.

3) Boot: b dqa1

4) Stop cleanly from a second Git Bash to flush the `_srm.trc`: `touch <run-dir>/EMULATR_STOP`

Triage

After the graceful stop the script greps the log for the TIG canary, HALTPROBE, and halt lines, then surfaces the TIG (`0x80130xxxx`) and Cchip (`0x801a0xxxx`) accesses and the last retire lines from the newest `_srm.trc`.

8.2.5.6 launch_vms_boot

Cold-boots EmulatR (DS10/EV6) and boots OpenVMS Alpha V8.2 from `dqa1`. Deployed to `<run-dir>/tools/` by CMake `POST_BUILD`; lives in `tools/` but operates on the run-dir root.

Usage

```
./launch_vms_boot.sh
```

Preflight

Verifies `Emulatr.exe` and the firmware are present and that the platform manifest (`ds10_platform.win`) references the OpenVMS ISO (`alpha082.iso`) -- otherwise `dqa1` is not wired to OpenVMS and the script aborts.

Notes

Launches with `--no-autoload` (cold boot) and tees the console to `boot_vms_<timestamp>.log`. The SRM console listens on TCP `127.0.0.1:10023`. At the `>>> prompt: e pmem:80130000040 (smir = 0)`, then `b dqa1`. The triage pass greps memory, halt, and frontier lines.

8.2.5.7 mkdisk

Raw disk-image generator for the emulated ATA fixed disk (the `dqa` install target). It creates a blank, writable, raw flat disk image with 512-byte sectors by default. One artifact serves two roles: the read-proof target (so a read of LBN 0 returns verifiable bytes) and the empty install target the OS installer writes onto. It does not synthesize a bootable OS or a filesystem.

Usage

```
python tools/mkdisk.py dqa0.img --type EMULATR-DISK-4G
python tools/mkdisk.py data.img --size 9G --sector 512
python tools/mkdisk.py dqa0.img --verify
```

Sizing and stamping

Size comes from `--type` (a `config/disk_types.json` catalog key mapping to `total_sectors`) or `--size` (K/M/G/T, 1024-based). The image is sparse where the filesystem supports it. With `--stamp` (the default) the first `--stamp-sectors` low LBNs plus the last sector get a 32-byte header (magic, LBN, FNV-1a checksum) so a read can be proven to return the correct LBN. Output is deterministic: identical arguments produce identical bytes. `--verify` checks the stamps instead of creating.

8.2.5.8 phase1 dispatch gate

`AlphaCpuAgent` Phase-1 acceptance gate. It proves that the dispatcher-driven boot (`EMULATR_DISPATCH=1`, one `AlphaCpuAgent` under `SequentialDriver` calling `Machine::stepCycle`) is byte-identical to the legacy `Machine::run boot`. Both paths delegate to the identical per-cycle kernel, so a passing gate confirms the loop, clock, and dispatch wiring is pure -- it does not re-verify the CPU kernel. This script does not build.

Usage

```
./phase1_dispatch_gate.sh [model] [max-cycles] -- model is ds10|ds20|ds25|
es40|es45 (default ds20), max-cycles is a hex or decimal cap (default 0x40000000).
```

Method

Purge `*_flash.rom` before each run so both boots build NVRAM identically; run two cold boots via `run_fw.sh` (legacy and `EMULATR_DISPATCH=1`); normalize volatile tokens (timestamps, snapshot and log filenames, the RPCC-probe line, the dispatch banner); then diff the normalized host logs. An empty diff is a PASS.

Notes

Locates `Emulatr.exe` by probing `EMULATR_RUNDIR`, the current directory, and the script directory -- run it from the build-run dir or set `EMULATR_RUNDIR` first. For belt-and-suspenders, the guest SRM console on TCP 10023 can be captured with `plink` and diffed the same way.

8.2.5.9 `profile_resolve`

Annotates RetireProfiler dumps with SRM and PAL symbols. It reads one or more RetireProfiler dump files, labels each bucket with the best-known SRM/PAL symbol or region, and prints two tables per file: the labeled top buckets and a rollup by region.

Usage

```
py profile_resolve.py <profile_*.txt> [more dumps...]
```

Notes

The symbol table is hand-curated from live-trace work. Ranges are in PA space of the DS10 SRM V7.3 decompressed image (base 0), plus the OS-PAL window at `0x8000` and the pre-takeover staging at `0x600000` and up. Matching is by overlap (a 1 KiB bucket that overlaps a region belongs to it), and `REGIONS` is ordered specific-first so the narrowest matching label wins.

8.2.5.10 `run_fw.sh`

Launches EmulatR V4 against one `v7_3` firmware image with 4 GiB of RAM. Resolves the model variant, sets the [System] model line in `config/EmulatrV4.ini` for the run (backing up and restoring the ini on exit), copies the firmware into the `build-dir` `firmware/` directory, and launches `Emulatr.exe` with console and stderr teed to `fw_<name>_<ts>.out`. Git Bash on Windows; anchors to the script's own directory as the build dir.

Usage

```
./run_fw.sh <ds10|ds20|ds25|es40|es45> [cold] [extra Emulatr args...]
```

Arguments

model (required): one of `ds10`, `ds20`, `ds25`, `es40`, `es45` -- maps to the [System] model written into the ini.

cold (optional 2nd arg): forces a genuine cold boot -- passes `--no-autoload` and purges *stale auto_*.axpsnap* (predig_* fast-forward anchors are preserved). Without it the run is

warm: autoloads the newest snapshot and keeps writing periodic autosnaps.

extra: any further arguments are passed through verbatim to Emulatr.exe (e.g. --max-cycles 0x10000000).

Notes

Memory is fixed at 4 GiB. The model is ini-driven; there is no --model flag. The SRM console listens on TCP 10023 -- connect with PuTTY/plink in raw mode to reach the >>> prompt.

bash - Examples

```
export EMULATR_FLASH_ROM=ds20_flash.rom
unset EMULATR_IIC_WATCH           # storm's gone; drop the probe noise
cd /d/Emulatr/EmulatrAppUniv4/Emulatr/out/build/release
./run_fw.sh ds20 cold --max-cycles 0x480000000 2>&1 | tee ds20_to_prompt_$(date +%H%M)
# clean-stop sentinel from another shell once you see >>> (flushes flash NVRAM):
# touch /d/Emulatr/EmulatrAppUniv4/Emulatr/out/build/release/EMULATR_STOP
```

8.2.5.11 snapmap

Region mapper for Emulatr. .axpsnap snapshot files, producing a 010-Editor-style hex map. It decodes the snapshot header, the chipset block, and the HWRPB (AARM Table 26-1), then emits a hex dump (top axis, address gutter, ASCII pane) of named critical guest-physical regions. Output is ASCII(128).

Usage

```
python3 snapmap.py <file.axpsnap>
```

What it reports

Header fields (magic, format/cpuState/chipset versions, timestamp, cycleCount, comment); the located memory size and guest-PA-0 file offset; the chipset block (variant, model, cpuCount, chipset memory size); and, when present, the HWRPB header and per-CPU slot states. If the HWRPB validation qword is absent the map notes the SRM has not yet built it at this capture point. It finishes with hex dumps of the PAL entry (palBase 0x600000) and the console/panic region at 0x602000.

8.2.5.12 trace_halt

Console-armed retire trace plus a TIG bring-up canary. Deployed to <run-dir>/tools/ by CMake POST_BUILD. It cold-boots to the >>> prompt with no trace noise; the trace is armed from the prompt, after which b dqa1 streams only the boot command to the _srm.trc as HW_LD a=<pa> v=<value> lines.

Operator steps at the >>> prompt

```
>>>e pmem:80130000040 -- smir, the Halt gate (expect 0 now). b dqa1 -- should cross into
the OpenVMS bootstrap. If still blocked, arm the trace with
>>>e pmem:80130000FF8 and then b dqa1.
```

Notes

Sets `EMULATR_TRACE_WINDOW=1` (window-only sink, so no multi-GB cold-boot stream) and `EMULATR_TIG_TRACE=1` (logs any access to an unmodeled TIG register, so a new one cannot hide behind a plausible 0). HALTPROBE auto-logs nonzero TIG reads. The triage pass greps HALTPROBE and the TIG canary, then the halt lines, and tails the newest `_srm.trc`.

8.2.6 Tools Script Reference

A consolidated inventory of every shell (`*.sh`) and Python (`*.py`) script in the EmulatR source `tools/` directory. This is the reader-facing companion to the on-record manifest `HMDocs/Maps/file_index.md`; both are regenerated from the same source tree.

How the scripts are deployed

On every build, the CMake `POST_BUILD` step (root `CMakeLists.txt`) stages the tool scripts into the run directory (`out/build/<config>/`) using a file(`GLOB ... CONFIGURE_DEPENDS`) over `tools/*.sh` and `tools/*.py`, so a newly added top-level script deploys automatically. Two rules apply:

- *`run_fw.sh` is special: it is staged to the run-dir `ROOT` (beside `Emulatr.exe`) because it anchors to its own directory and launches the binary directly. It is superseded by `emulatr_launch.sh`, which stays in `tools/` and changes into `$RUN_DIR` itself.*
- *Every other top-level script is staged to `<run-dir>/tools/` and locates `config/`, `firmware/`, and the manifests as siblings of the run-dir root. The `tools/host_decompressor/` and `tools/ghidra_scripts/` subtrees are NOT runtime tools and are deliberately excluded.*

All shell scripts target Git Bash / MSYS2 on Windows (the supported PC host). Each carries a uniform platform guard that detects the host via `uname -s` and prefers `Emulatr.exe`, falling back to a bare `Emulatr` for a macOS/Linux build; sibling scripts are invoked through `bash` so they do not depend on the executable bit (which an NTFS checkout drops). The two host-aware SRM launchers (`run_srm_trace_full.sh`, `run_es40_srm_trace_full.sh`) and `vsenv.sh` already branch for macOS. Python utilities require a Python 3 interpreter on the host.

Firmware launch & SRM boot

Script	Purpose
<code>emulatr_launch.sh</code>	THE platform-safe launcher, and the single source of truth for the per-platform arguments. <code>emulatr_launch.sh <plat> [purpose] [-- extra args]</code> takes the platform as its only real input and fixes firmware stem, silicon-max memory and the ini model together, so a mis-badged or over-

Script	Purpose
	allocated launch cannot be typed by hand. Always raises the PuTTY console; logs stem-keyed to <code>logs/<stem> <purpose> <ts>.log.</code>
<code>run_ds10.sh</code>	Platform wrapper: DS10, 2 GiB ceiling. Thin exec into <code>emulatr_launch.sh</code> .
<code>run_ds15.sh</code>	Platform wrapper: DS15, 4 GiB ceiling. Firmware not yet staged, so this currently stops with a FATAL before launching.
<code>run_ds20.sh</code>	Platform wrapper: DS20, 4 GiB ceiling.
<code>run_ds25.sh</code>	Platform wrapper: DS25, 16 GiB ceiling.
<code>run_es40.sh</code>	Platform wrapper: ES40, 32 GiB ceiling.
<code>run_es45.sh</code>	Platform wrapper: ES45, 32 GiB ceiling.
<code>run_fw.sh</code>	LEGACY, superseded by <code>emulatr_launch.sh</code> . Launch one v7_3 firmware image for a chosen model (ds10/ds20/ds25/es40/es45); cold or warm boot. Staged to the run-dir root. Requests a flat 4 GiB with no per-model ceiling, does not know DS15, and logs to the run-dir root rather than the stem-keyed name under <code>logs/</code> .
<code>run_srm_trace_full.sh</code>	Host-aware, model-parameterized SRM cold-boot with a full comprehensive trace. The general launcher the per-model wrappers delegate to.
<code>run_es40_srm_trace_full.sh</code>	ES40 (Clipper) SRM cold boot with the full comprehensive trace.
<code>run_ds10_trace.sh</code>	Thin wrapper: DS10 comprehensive SRM trace (delegates to <code>run_srm_trace_full.sh</code>).
<code>run_ds20_trace.sh</code>	Thin wrapper: DS20 comprehensive SRM trace.
<code>run_es40_trace.sh</code>	Thin wrapper: ES40 comprehensive SRM trace.
<code>run_ds20_showdev.sh</code>	Cold-boot DS20 to P00>>> with a PuTTY console attached so "show dev" confirms both dqa0 (disk) and dqa1 (CD) on the CY82C693 IDE.
<code>run_es40_showdev.sh</code>	Cold-boot ES40 to P00>>> with PuTTY and the IDE config-space trace, to diagnose ES40 device enumeration.

Script	Purpose
<code>run_ds20_ide_trace.sh</code>	Cold-boot DS20 with the IDE probe/command trace on (retire trace off); boots to P00>>> for show dev / boot dqa0/dqa1.
<code>run_ds20_putty.sh</code>	Cold-boot DS20 with an attached PuTTY console so the cosmetic MHz badge rewrite is visible live and in the log.
<code>run_ds20_bplus.sh</code>	DS20 cold boot with the B+ primary-bootstrap restart stack enabled; a thin wrapper that arms the CSERVE handoff knobs and delegates to <code>run_ds20_showdev.sh</code> .
<code>run_ds20_optfw_trace.sh</code>	Capture the DS20 option-firmware to LFU decision with the DIAG retire facility; a thin wrapper that arms the cycle window and delegates to <code>run_ds20_showdev.sh</code> .
<code>run_es40_hookb.sh</code>	Run the ES40 boot from the CLI build and pull the ACVPROBE HOOKB PTE-verdict lines (valid/kre/pfn).
<code>run_es40_rscclab_warp.sh</code>	ES40 RSCC A/B, Case A (warp ON): reproduce the LFU "Initializing...." hang with idle-tick warp enabled; delegates to <code>run_es40_showdev.sh</code> .
<code>run_es40_rscclab_nowarp.sh</code>	ES40 RSCC A/B, Case B (warp OFF): the faithful-timing control run; delegates to <code>run_es40_showdev.sh</code> .
<code>run_initloop_trace.sh</code>	Capture the ES40 "Initializing...." stuck-loop body (0x12c270) with the retire-trace facility; delegates to <code>run_es40_showdev.sh</code> .
<code>launch_vms_boot.sh</code>	Cold-boot DS10/EV6 and boot OpenVMS Alpha V8.2 from dqa1.

Build

Script	Purpose
<code>build_emulatr.sh</code>	Host-aware build of Emulatr in any config (relwithdebinf/debug/release), trace-ready.
<code>build_diag.sh</code>	One-command command-line build of Emulatr.exe with the EMULATR_BRINGUP_PROBES scaffolding on (Hook B, VPTB-DIAG), outside the VS IDE.
<code>build_es40_relwithdebinf.sh</code>	Turnkey RelWithDebinf build of Emulatr.exe for the ES40 target.

Script	Purpose
<code>build_kbd_vga.sh</code>	Build EmulatR V5 with the keyboard (8042) + VGA interfaces enabled.
<code>build_test_run.sh</code>	Rebuild (RelWithDebInfo), run the chipset halt-register doctest, then hand off to the OpenVMS boot.

Diagnostics, probes & verification

Script	Purpose
<code>diag_ds20_badge_ab.sh</code>	DS20 badge A/B diagnostic (polled IIC completion).
<code>diag_ds20_nowarp.sh</code>	DS20 boot-progress diagnostic with warp off.
<code>diag_halt_irq4.sh</code>	IRQ4 / halt-gate runbook: arm a window retire trace and snapshot interrupt state from the console.
<code>diag_msvc.sh</code>	Diagnose why vcvars64.bat is not populating cl / INCLUDE in Git Bash (run it, do not source).
<code>diag_sde_swap_trace.sh</code>	Capture the PAL shadow-bank (I_CTL[SDE]) swap ledger.
<code>probe_es40_preseed_p00.sh</code>	ES40 CONFIRM-3 preseed-to-P00 probe (zero-risk, no source edits).
<code>probe_vmb_itbmiss.sh</code>	JRN-VMB-001 Step 1: capture the ITB miss at the VMB entry.
<code>snap_vmb_capture.sh</code>	JRN-VMB-002 E-6 deterministic state capture.
<code>phase1_dispatch_gate.sh</code>	AlphaCpuAgent Phase-1 acceptance gate: prove the dispatcher boot is byte-identical to the legacy boot.
<code>trace_halt.sh</code>	Console-armed retire trace plus the TIG bring-up canary.
<code>verify_aar_fix.sh</code>	Verify the ASIZ decode-width (AAR) fix, the ES40 memtest ACV root cause.
<code>test_kbd_vga.sh</code>	Boot-frontier test run for the keyboard (8042) + VGA interfaces.

Trace analysis & firmware utilities (Python + Ghidra)

Script	Purpose
<code>alpha_disasm.py</code>	Minimal Alpha (EV6) disassembler for reading PAL and console code by file offset.

Script	Purpose
<code>analyze_retire_trace.py</code>	Find the dominant PC loop (the wedge body) in a retire-compact trace.
<code>diff_traces.py</code>	Normalize and compare two EmulatR trace logs.
<code>mkdisk.py</code>	Generate a raw, stamped, writable disk image for the dqa install target.
<code>profile_resolve.py</code>	Annotate RetireProfiler dumps with SRM/PAL symbols and region rollups.
<code>snapmap.py</code>	Decode an .axpsnap and hex-map its header, chipset block, and HWRPB.
<code>trace_reg_backtrace.py</code>	Data-flow backtrace of a register through an EmulatR trace.
<code>decompile_ds10.sh</code>	Dump C source from an already-analyzed Ghidra project (headless, decompile-only).

Environment setup (source, do not execute)

Script	Purpose
<code>env.sh</code>	Put the VS2022-bundled CMake + Ninja (and Qt / MSVC toolchain) on PATH for Git Bash command-line builds. Source it: <code>source tools/env.sh</code> .
<code>vsenv.sh</code>	Cross-platform sibling of env.sh: uname-gated MSVC import on Windows, no-op-with-warning on macOS/Linux.

Packaging / release

Script	Purpose
<code>make_redist.sh</code>	Package a built run directory (out/build/<config>) into a versioned zip under <code>redist/</code> , named after the H&M documentation version. Excludes logs, traces, the licensed <code>firmware/*.exe</code> images, and itself. Usage: <code>./tools/make_redist.sh [config]</code> .

Session / Git commit helpers

Script	Purpose
<code>commit_es40_p00.sh</code>	Stage, commit, and push the ES40 "boots to P00>>>" work (guarded).

Script	Purpose
<code>commit_20260713_tig_reset.sh</code>	Guarded end-of-session commit for the TIG-reset work (dated snapshot helper).

Note: the dated `commit_*.sh` helpers and the developer build/diagnostic scripts are session tooling. They deploy with the `run` directory but are not needed to operate the emulator; beta testers use the launch scripts above.

See Also

[script](#) ²⁴⁷

[Run-Directory Contents](#) ⁵⁷

[run_fw.sh](#) ²⁵²

8.2.7 Platform Editor

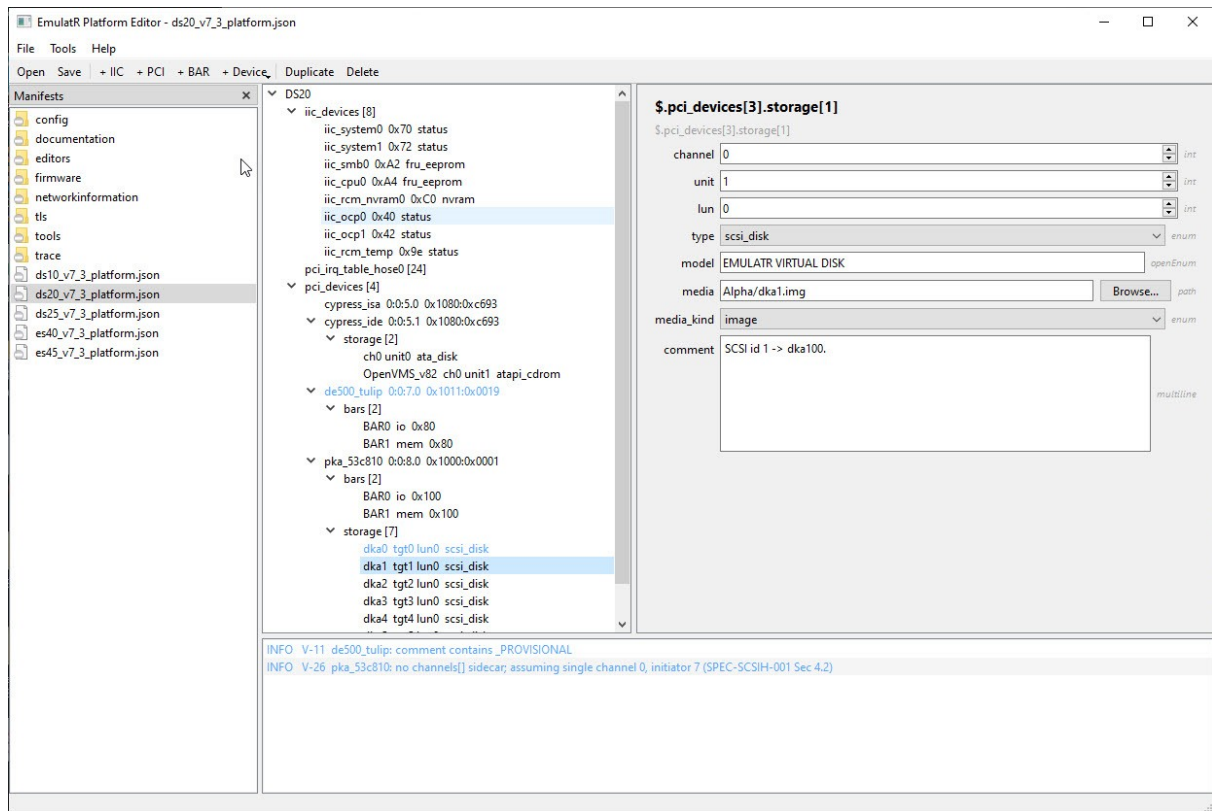
The Platform Editor is a desktop application for authoring and validating an EmulatR platform device manifest -- the `<firmware>_platform.json` file (e.g. `ds20_v7_3_platform.json`) that describes the guest device tree: the IIC devices, the PCI devices, and the storage media attached to each controller. Editing the manifest lets an operator re-populate the emulated board with a JSON edit instead of a rebuild.

Where it lives

The editor builds with every EmulatR build and is staged into the `run` directory at `<run-dir>/editors/`, beside the emulator executable:

- `editors/platedit_qt.exe` (*Windows*; `platedit_qt.app` *on macOS*, `platedit_qt` *on Linux*) -- the Qt Widgets GUI. On Windows the Qt runtime DLLs are deployed beside it, so it launches with a double-click.
- `editors/platedit_tui` -- a terminal-UI variant over the same core, for headless hosts.
- `editors/schema/` *and* `editors/catalog/` -- the data-driven field policy and the device catalog the editor reads at startup.
- `editors/webui/mockup.html` -- the original browser prototype, retained as a non-functional visual reference (its tree is hard-coded demo data; it does not load manifests).

Launch it directly, optionally with a manifest path argument (`platedit_qt ds20_v7_3_platform.json`), or use `File > Open Folder...` to point the Manifests panel at a directory (for example the `run` directory itself) and switch between the platform files it lists with a click. The last folder is remembered across launches.



Status: the Qt Widgets application is the shipping frontend (SPEC-PLATED-001 D-025). All editing is live: scalar edits write through the format-preserving core (an unchanged file round-trips byte-identically; a single field edit produces a one-token diff), and structural add/duplicate/delete regenerates only the touched container while every untouched section keeps its authored bytes, comments, and key order.

Layout

A menu bar and a device toolbar sit above the work area:

- **Manifests** (dock) -- the platform files of the opened folder; click to load.
- **Structure** (left) -- the manifest as a tree: the platform root, then `iic_devices[]` and `pci_devices[]`, with each PCI controller expanding to its `storage[]`, `bars[]`, and `channels[]` children. Rows with validation findings are tinted by severity.
- **Properties** (right) -- the selected node's fields as a typed form, in authored order.
- **Issues** (bottom) -- the live validation list, recomputed on every edit and CRUD action; clicking an issue selects the offending node in the tree.

Menu bar: File (Open, Open Folder, Reload, Save, Save As, Quit), Tools > Validate, Help > About. Toolbar: Open, Save, then + IIC, + PCI, + BAR, + Device (a bus-aware picker), Duplicate, and Delete.

Field tiers

Each field renders a widget chosen by the data-driven policy in

`editors/schema/platform_schema.json`, so edits stay well-typed and adding a manifest field never requires a rebuild:

Tier	Widget / meaning
int / size	Spin box, bounded by the policy's min/max.
hex	Validated 0x entry (addresses, vendor/device IDs, class codes).
enum	Closed dropdown (e.g. storage type, media_kind, BAR kind, channel width).
openEnum	Editable combo with suggestions (e.g. PCI model, manufacturer). With no suggestion list (e.g. a device_name) it renders as a plain text field.
bool	Check box.
path	Text input with a Browse button (media / image path).
multiline	Text area (comment fields).

For a PCI device with a known model, a field that disagrees with the device catalog default carries a `diverged` badge (catalog value shown). `manifest_version` is read-only.

Required-interface guards

Delete asks for confirmation, and is refused outright on the interfaces the emulator presence-checks, so a manifest cannot be edited into an unbootable state:

- *DS10 power/FRU set: IIC 0x70 (iic_system0), 0x72 (iic_system1), 0xA2 (iic_smb0 FRU).*
- *DS20 OCP set: IIC 0x40 / 0x42 (OCP badge / data), and the 0x9e get_sysvar discriminator.*

Live validation

The Issues pane recomputes on every change; each issue has a code, a severity (error / warn / info), and the offending node. Representative checks:

Code	Severity	Check
V-01	error	Duplicate PCI BDF (hose:bus:slot.func).
V-02	error	Duplicate IIC address.
V-06	error	IIC address is odd (the emulator rejects the manifest).
V-15	error	Invalid PCI vendor id (0, 0xffff, or non-hex).
V-25	error	SCSI target equals the HBA initiator_id.

Code	Severity	Check
V-26	error / info	Storage channel has no channels[] entry (error when a channels[] sidecar exists; informational single-channel assumption when the controller has none).
V-27	error	Narrow SCSI bus with a target > 7.
V-07	warn	Authored PCI identity diverges from the device catalog default for its model.

Adding storage devices

The + `Device` picker is bus-aware: it reads the selected controller's class code, offers Virtual Disk / Virtual CD-ROM (and Virtual Tape / host CD-ROM on SCSI), maps the choice to the correct (type x media_kind) pair, and assigns the next free address on the bus -- SCSI walks target ids skipping the HBA's initiator id; IDE walks master/slave on channels 0 and 1. Set the new row's `media` to the disk image and it is ready to boot.

Saving

Save writes through the format-preserving core: JSON layout, key order and comments survive, only edited tokens (or structurally changed containers) differ, and the previous file is kept as `.bak` on overwrite. Reload discards in-memory edits after confirmation.

See Also

[Run-Directory Contents](#) → 57

[Tools Script Reference](#) → 254

8.3 8.3 Source Cross Reference

Every claim EmulatR makes about the machine is meant to be traceable to something that is not EmulatR. This appendix says where those authorities live, and how the topics point at them.

The convention

Most engineering topics end in a `Source References` block naming the exact files behind the behavior they describe -- 49 topics carry one today. That block is not decoration. When a topic, a test, and the code disagree, it is where the argument gets settled, because it names the thing that is allowed to decide.

Which source decides

Tier	What it is	Weight
1	The DEC / Compaq firmware images and the apism console sources	Ground truth. It decides.
2	EmulatR's own C++	What is actually implemented -- never the authority on what is correct
3	AXPBox, SimH and other emulators	Secondary corroboration only; never primary over EmulatR

The ordering matters most when a test fails. A test encodes an expectation, and an expectation can be wrong; a tier-1 source cannot be argued with. The rule is to let the source say which of the code and the test is mistaken, rather than adjusting whichever is easier to change.

Where the sources live

The reference library is `Processor Support/`. It is large, and scanning it is the wrong move: start at `Processor Support/REFERENCE_INDEX.md`, which lists every document with a one-line description and ends in a "what should I read for X?" lookup, then open only what that points at.

The SRM console sources sit at `Processor Support/PalcodeBitsavers/apism/apism/`. One scope correction saves a great deal of hunting: `ref/` is the platform and chip reference LEAF, not the console's spine. It holds the device drivers (`dq_driver.c`, `n810_driver.c`), the PAL macros (`ev6_huf_decom.m64`), the definition files (`ev6_defs.mar`, `osfalpa_defs.mar`) and the IPR examine / deposit driver (`ev6_ipr_driver.c`). The boot-to-prompt spine -- the kernel executive and the secondary-start logic -- is elsewhere in the tree; `Hierarchy.txt` maps it.

The EmulatR side

A Source References block names EmulatR files by library, so it helps to know what each one owns:

Library	Owns
<code>coreLib</code> , <code>cpuLib</code> , <code>pipelineLib</code>	CPU state, instruction execution, and the retire pipeline
<code>iBoxLib</code> , <code>eBoxLib</code> , <code>mBoxLib</code> , <code>fBoxLib</code> , <code>fpBoxLib</code> , <code>cBoxLib</code>	The EV6 functional boxes, mirroring the hardware's own partitioning
<code>mmuLib</code> , <code>pteLib</code>	Address translation and page-table entries
<code>iprLib</code>	Internal processor registers

Library	Owns
palBoxLib, grainFactoryLib	PALcode dispatch and the generated grain tables
chipsetLib	Tsunami / Typhoon core logic -- Cchip, Dchip, Pchip
deviceLib	Device models and the SRM console surface
memoryLib	Guest physical memory
systemLib	Machine assembly, the platform manifest, and snapshots
schedLib	The CPU agent and the SMP harness
jitLib, traceLib	Translation caching and the diagnostic trace facility

See Also.
[SRM Boot Call-Stack](#) ²⁴¹
[Testing](#) ²⁶⁴
[Known Gaps](#) ²³

8.4 Testing (draft)

EmulatR is validated two ways: a doctest unit-test suite, and a boot-gate that a build must still pass at the console level. The determinism of the V4 Oracle (faithful decompressor plus the ISP default) is what makes both repeatable.

Unit tests (doctest)

The unit suite uses doctest 2.5.0 (a single vendored header, `tests/doctest.h`). Because V4 builds with C++ exceptions disabled, tests assert with `CHECK` only -- never `REQUIRE`. They build as the `Emulatr_tests` target and run as a standalone executable:

```
cmake --build out/build/relwithdebinfo -j --target Emulatr_tests
./out/build/relwithdebinfo/Emulatr_tests.exe
```

Coverage spans the device models (for example `test_floppy82077`, `test_cy82c693ide`, `test_scsi_iso`), the decode / dispatch tables, and the floating-point backend -- the `fp_softfloat_vs_host_bench` differential checks the native FP path against the Berkeley SoftFloat oracle bit-for-bit.

Boot gate

Beyond the unit suite, a build must still clear the console boot-gate: DS10 and DS20 reach the SRM `P00>>>` prompt, and ES40 reaches its prompt with no regression against the prior build. Running with all levers unset (faithful decompressor, ISP default) makes the run the deterministic Oracle, so a behavioral drift shows up as a diff against the golden traces rather than as a flaky pass.

Useful DOCTEST flags

```
Emulatr_tests.exe --list-test-cases - see what's registered
Emulatr_tests.exe --test-case="*NDS*" - run just the new PERROR<NDS> case
Emulatr_tests.exe --test-case="*pchip*,*dma*" - the Pchip DMA set
```

add -s to show passing assertions too

See Also.

[Building from Source](#) ⁴⁵

[Execution Matrix \(Models x Modes\)](#) ²¹⁰

[Known Gaps](#) ²³

[Supported Test Cases](#) ²⁶⁵

8.4.1 Supported Test Cases

```
$ Emulatr_tests.exe --list-test-cases
```

[doctest] listing all test case names

```
=====
cBox::execTrapb -- propagates S_TrapBarrier, no effect
cBox::execExcb -- propagates S_ExcBarrier, no effect
cBox::execMmb -- propagates S_Mb, no effect
cBox::execWmb -- propagates S_Wmb, no effect
cBox::execEcB -- propagates S_PrefetchOnly, no effect; opB ignored
FieldSpec lsb + width within 64-bit container
MISC field positions match HRM Table 10-12
IIC field positions match HRM Table 10-20
mask(): bit positions are correct for representative fields
extract / deposit round-trip
MISC W1C_MASK includes NXM, IPINTR, ITINTR -- and only those
MISC W1S_MASK includes ABT and ABW -- and only those
MISC WO_MASK includes DEVSUP, ACL, IPREQ -- and only those
MISC composite masks do not overlap
roundLog2Nearest matches known table
kcchipIntervalTimerCycles matches profileAlphaClockHz / 1024
Mask fire predicate fires exactly once per interval
MISC REV reset value depends on chipset variant
Dchip DREV reset value is byte-sliced 0x0101... on both variants
FakePciDevice returns programmed config and records the read
FakePciDevice masks a narrow read to its width
FakePciDevice records writes and reset
FakeIoPortDevice returns programmed port value and records the read
FakeIoPortDevice records the written byte and port
FakeCpu carries its id
FakeCpu reservation survives a write to a different cache line
FakeCpu reservation clears on a write to the same cache line
DeterministicMemory round-trips a quadword
DeterministicMemory ignores an out-of-range access
FlashRom factory state is erased 0xFF
FlashRom autoselect returns AMD Am29F016 id, then read-array
FlashRom erase handshake returns 0x80 twice then read-array
FlashRom program after erase round-trips a byte
FlashRom sector erase clears only its own 64 KB sector
FlashRom missing backing file leaves factory 0xFF
```

```

IDE wiring: CY82C693 IDE enumerates at bus0/dev5/func1
IDE wiring: legacy taskfile routes to the controller (CD on slave)
IDE wiring: IDENTIFY DEVICE through the chipset posts the ATAPI signature
Tsunami: registry contains exactly one Tsunami range after construction
Tsunami: findRange resolves Cchip CSC PA to the Tsunami range
Tsunami: findRange returns nullptr for a PA below the MMIO window (DRAM region)
Tsunami: findRange returns nullptr for a PA above the Tsunami MMIO window
Tsunami: tryRead at Cchip CSC PA dispatches and returns a value
Tsunami: write to Cchip CSC followed by read returns the written value (or its masked form)
Tsunami: tryRead at a DRAM PA returns false and does not write valueOut
Tsunami: tryWrite at a DRAM PA returns false
GuestMemory routes a Cchip CSC PA through the attached MMIO hook
GuestMemory: PA below MMIO window still hits DRAM (hook returns false)
GuestMemory is a dumb byte-store: out-of-range PAs read back zero, not a fault
unregisterIoPortRange retires the claim; dispatch reverts to float
unregisterPciMemRange retires the claim; dispatch reverts to all-ones
unregister with a non-matching range is a logged no-op
BAR re-program unregisters the old window and registers the new one
BAR re-program moves live decode (chipset integration)
dmaWriteBytes/dmaReadBytes round-trip across a 4 KiB chunk boundary
Pre-init: edges latch nothing deliverable; output stays low
pc264 init sequence completes; everything masked except cascade
Unstick invariant: masked edge delivers on the OCW1 unmask
acknowledge drops the output and holds the level until EOI
Edge mode: a held level does not re-trigger after ack + EOI
OCW3 poll returns 0x80|level and acknowledges
Cascade: slave request presents on master IR2 with slave vector
IMR gates output but never capture; remask hides a pending edge
Chain: UART THREE kick-start reaches DRIR<55> through the PIC
variantFromModel maps platform models to the right variant
ChipsetVariantInfo caps match the documented AAR limits
All three sub-chips agree on variant (ES45 / Typhoon, 4 CPUs)
All three sub-chips agree on variant (ES40 / Tsunami)
Model-string ctor derives variant and reports the model
Unknown model normalizes to Tsunami; consistency guard holds
DREV reads byte-sliced per-Dchip revision, variant-independent
routeMmioOffset maps each realm region and leaves gaps as None
MMIO read at Cchip CSC routes to the Cchip
MMIO read at Dchip DREV returns byte-sliced per-Dchip revision
MMIO read in the Pchip1 half returns off-bus all-ones
Static shim reaches the same Cchip register as the instance method
cpuId threads to Cchip MISC.CPUID; static shim defaults to CPU 0
MMIO write to a Cchip RW register round-trips deterministically
step() fires the Cchip interval timer at the mask interval
CycleInjector::runCycles drives step() across an interval boundary
Tsunami 1GB: AAR0 = 1GB at base 0, AAR1-3 disabled
Tsunami 4GB: 4 x 1GB arrays at bases 0, 1G, 2G, 3G
Typhoon 32GB (4-bit-ASIZ firmware): 4 x 8GB arrays, ASIZ = 0xA
Typhoon 2GB and 4GB single-array ASIZ (4-bit-ASIZ firmware)
AAR base + size never exceeds 2^35
AAR0 honours writes (RW per HRM), not read-only
AAR write/read round-trips through the chipset MMIO surface
M2: AAR total via firmware formula == configured memSize (Tsunami)
M2: 64MB -> AAR0 low word 0x3009; 1GiB -> 0x7009 (firmware-observed)
M3: Typhoon default 3-bit firmware sizes are pc264-consumable
M3: Typhoon ES40 4GB -> 4 x 1GB, ASIZ 0x7 (root-cause regression #6)
DRIR assert with matching DIM sets DIR for that CPU only
DRIR assert with mismatched DIM does not set DIR
Deassert clears DIR for all CPUs
Per-CPU masks are independent
PCI INTx -> DRIR bit follows the v4 convention
raisePciInterrupt lands in DRIR; lowerPciInterrupt clears it
MISC: w1S ABT[0] auto-promotes to ABW[0] (HRM 12.2 single-CPU arbitration win)
MISC: ACL (bit 24, WO) clears both ABT and ABW; ACL itself does not persist
MISC: re-arm cycle (ABT set -> ACL clear -> ABT set again) re-promotes ABW
MISC: w1S ABT[0] auto-promotes to ABW[0] (HRM 12.2 single-CPU arbitration win)
MISC: ACL (bit 24, WO) clears both ABT and ABW; ACL itself does not persist
MISC: re-arm cycle (ABT set -> ACL clear -> ABT set again) re-promotes ABW
step() fires the interval timer after one full interval
step() short of an interval does not fire

```

```

step() fires when a single large delta crosses a boundary (no skip)
step() fires again on the next interval after a clear
step() fires for all enabled CPUs (4-CPU ES45)
cycleInjector reaches an interval via overshooting chunks
Pchip WSBA0 holds its last write (live CSR)
Empty PCI config slot returns 0xFFFFFFFF
Registered FakePciDevice answers config reads at its slot
Cypress ISA bridge identifies at its slot (vendor 0x1080 / dev 0xc693)
Cypress ISA bridge reports the ISA-bridge class code 0x060100
Sparse I/O offset decodes back to the intended port and byte width
Sparse I/O read dispatches to a registered handler at the decoded port
Sparse I/O write dispatches the byte to the registered handler
Unregistered sparse I/O port reads 0xFF (ISA floating bus)
Chipset-owned COM1 LSR reads transmitter-ready via sparse I/O
Chipset-owned COM1 THR write is accepted (null backend tolerated)
Chipset-owned COM2 LSR reachable via sparse I/O
TsunamiTig -- fresh TIG holds no CPU-start request (UP-boot posture)
TsunamiTig -- CPU-START write latches a per-CPU start request
TsunamiTig -- each CPU-START register maps to its own id bit
TsunamiTig -- latched id maps to the PAL impure entry slot
byteops: EXTQH aligned Rb (offset 0) passes the value through
byteops: EXTWH/EXTLH aligned Rb apply only the width mask
byteops: EXTxH matches the AARM shift model for all offsets
byteops: pre-BWX signed byte-load idiom works at every X mod 8
byteops: two-LDQ_U unaligned quadword idiom (EXTQL|EXTQH)
byteops: EXTxH non-aligned offsets are unchanged by the fix
byteops: full-file audit -- byte-lane reference model, all offsets
byteops: ZAP/ZAPNOT lane selection (header-comment examples)
vptb: I_CTL 47:30 store + SEXT read round-trips the VMS base
vptb: VA_FORM compositions for the VMS base (HRM Figs 5-5/6/7)
byteops: INSXH/MSKXH aligned cases keep their AARM semantics
computeVaForm -- three HRM forms (5-5/5-6/5-7)
composeIsumSw -- SI[n] requires SIRR[n] AND IER.SIEN[n] (T1)
composeIsumSw -- AST[m] needs ASTRR+ASTER+ASTEN+(CM >= m) (T2)
composeIsum -- staged HW causes OR live SW; staged SW filtered (T3)
FileBlockMedia: 512-byte RW round-trip (write then read back)
FileBlockMedia: 2048-byte ISO is read-only (write -> ReadOnly)
FileBlockMedia: open() of a missing file -> NotOpen
FileBlockMedia: create_if_missing makes a blank writable image of the right size
FileBlockMedia: create_if_missing never creates for read-only media
FileBlockMedia: create_if_missing never overwrites an existing file
MockBlockMedia: status contract -- Ok / OutOfRange / NoMedia / ReadOnly
MockBlockMedia: fault injection returns the chosen status at the LBA
makeBlockMedia: image/iso/absent open a file; host + unknown fail closed
Cy82C693Ide: present ATAPI master -- ready, no BSY
Cy82C693Ide: IDENTIFY DEVICE aborts with the ATAPI signature
Cy82C693Ide: absent slave -- C1 BSY-clear, never 0xFF
Cy82C693Ide: alt-status mirrors status without side effects
Cy82C693Ide: unknown command aborts
Cy82C693Ide: empty controller -- both units absent answer C1
Cy82C693Ide: PACKET INQUIRY round-trip returns CDROM (type 0x05)
Cy82C693Ide: PACKET TEST UNIT READY (no media) -> not-ready sense
Cy82C693Ide: PIDENTIFY returns an ATAPI identify (word0 config)
Cy82C693Ide: ATA disk presents non-ATAPI ready signature
Cy82C693Ide: ATA IDENTIFY DEVICE reports LBA, capacity and model
Cy82C693Ide: READ SECTORS streams the seeded LBA content
Cy82C693Ide: READ SECTORS past end aborts, never hangs BSY
Cy82C693Ide: missing/empty image leaves the slot absent (C1)
Floppy82077: idle controller reports no pending interrupt at 0x536
Floppy82077: RECALIBRATE raises the IRQ6 poll bit at 0x536
Floppy82077: ide_poll sequence (write 0x0A, read 0x536) sees SEEK pending
Floppy82077: SENSE INTERRUPT clears 0x536 and returns ST0=0x70/PCN=0
Floppy82077: SENSE DRIVE STATUS does not raise the poll bit
HWPCB guest I/O: full image reads back what was written
HWPCB save set: writes KSP+AST+CPC(longword) and nothing else
FEN quad packing: FEN<0> | PME<62> | DAT<63> unpack + repack
CC arithmetic (F-1 packed): CPC round-trips across a swap pair
IicPcf8584: reset state -- PIN set, bus free, no error bits
IicPcf8584: ireg_test readback -- S0' / S2 / S3 store and return
IicPcf8584: probe sequence -- instant completion with NAK, never LAB
IicPcf8584: iic_chip_reset byte family leaves the model idle

```

IicPcf8584: NAK'd address -- status holds LRB, S0 floats
 IicPcf8584: rcm_nvram0 read sequence -- ACK, dummy, data, NACK done
 IicPcf8584: EEPROM write txn stores, read txn streams back
 IicPcf8584: controller reset preserves EEPROM content
 IicPcf8584: unknown nodes still NAK fast -- LAB never set
 IicPcf8584: non-byte access floats reads and drops writes
 IicPcf8584: determinism -- identical byte streams, identical state
 IicPcf8584: empty bus by default -- every node absent
 IicPcf8584: configured FRU device is present and streams content
 IicPcf8584: 0x70 status register reads its byte (build_power_hw)
 IicPcf8584: status register absorbs data writes (read-mostly)
 IicPcf8584: content image roundtrip preserves writes
 InlineWorkQueue runs work synchronously (fork-1)
 submit delivers the result inline (fork-1 completion)
 IoResult OWNS its data-in (C-VALUE: a copy, not a view into target storage)
 IoResult carries CheckCondition + sense by value (NOT READY / 3A no media)
 IoCommand owns its CDB + data-out (value copy is independent)
 53C810 config identity is the io_device_list bind row
 SCRIPTS INQUIRY transaction end-to-end
 Selection of an empty id raises STO (how the pk driver maps the bus)
 SCRIPTS READ(10) moves media blocks into guest RAM
 Unsupported LUN answers INQUIRY with qualifier 011b (no phantom units)
 BAR rebind callbacks fire on 53C810 BAR re-program
 MODE SELECT(6) with no parameter list is accepted
 MODE SELECT(6) accepts a block descriptor that keeps the block size
 MODE SELECT(6) rejects a block size the media does not have
 MODE SELECT(6) with a truncated parameter list reports a length error
 MODE SELECT(10) shares the contract with the 6-byte form
 VirtualIsoDevice: INQUIRY reports a removable CD-ROM (type 0x05)
 VirtualIsoDevice: TEST UNIT READY with no media -> 02/3A/00
 VirtualIsoDevice: READ10 no media is FAIL-FAST, not retry-eligible
 VirtualIsoDevice: REQUEST SENSE returns the medium-not-present sense
 VirtualIsoDevice: unknown opcode -> illegal request (0x20)
 VirtualIsoDevice: loadMedia + TEST UNIT READY good, READ CAPACITY
 VirtualIsoDevice: READ10 returns image content at the requested LBA
 VirtualIsoDevice: READ10 past end of medium -> 05/21/00 (LBA out of range)
 VirtualIsoDevice: READ TOC reports a single data track at LBA 0
 VirtualIsoDevice: ejectMedia returns to no-media fail-fast
 VirtualIsoDevice: READ(10) over MockBlockMedia + isPresent toggle (no-media end to end)
 VirtualIsoDevice: MediaStatus maps to sense -- OutOfRange 0x21, IoError 0x11
 SRMConsoleDevice: start + stop lifecycle (ephemeral port)
 SRMConsoleDevice: putchar('A') reaches connected client socket
 SRMConsoleDevice: client write -> getchar() returns byte
 SRMConsoleDevice: round-trip 'HELLO\r\n' device->client and back
 ToyRtc: Reg D VRT -- set in fixed mode, clear in unset (SPEC-TOY-001)
 ToyRtc: unset mode -- invalid fields, clock does not advance
 ToyRtc: w-2 write protocol -- SET/stage/clear latches the origin
 ToyRtc: B1 latch -- data port returns index-write-time snapshot
 ToyRtc: lazy time materialization from the cycle source (BCD)
 ToyRtc: binary + 24-hour modes honored via Reg B
 ToyRtc: krn\$_reset_toy sequence -- RegA=0x26, RMW RegB SQWE
 ToyRtc: Reg B SET holds off materialization for time staging
 ToyRtc: Reg C read-clears and is write-protected
 ToyRtc: non-byte access floats reads and drops writes
 ToyRtc: determinism -- same cycles, same bytes
 ToyRtc: timestampMMDDhhmm (CSERVE get_time) matches a 24h/BCD RTC read
 ToyRtc: persistence -- write, flush, re-init from file
 ToyRtc: corrupt / mode-mismatched store -> UNSET, never consumed
 ToyRtc: C-2 register dump at a boot-representative instant
 Reset state: no interrupt pending, FIFO empty, TX ready
 Kick-start: IER=0x03 with THR already empty asserts THRE source
 IER write without ETBEI edge does not assert; repeated 0x03 is edge-gated
 THR write re-arms the THRE source (per-byte fresh edge)
 RX level: feed -> pending; RBR drain -> level falls
 IIR priority: RX-avail reported above THRE; THRE latch survives
 Masking defers, never drops: IER gate off/on preserves RX level
 RX FIFO depth 16; overflow sets sticky LSR overrun, clears on read
 FCR RX-reset clears the FIFO and the RX level
 DLAB redirects +0x01 to DLM; divisor writes never touch the IER path
 MCR stores low 5 bits; OUT2 stored but does not gate intPending

```

reset() clears latch, FIFO, sticky errors
eBox::execAddl -- small positive operands
eBox::execAddl -- result sign-extends from 32 to 64 bits
eBox::execAddl -- 32-bit overflow wraps and sign-extends
eBox::execAddl -- only the low 32 bits of operands enter the add
eBox::execAddl -- semFlags propagate from grain to result
eBox::execAddl -- reachable through grain.execFn
eBox::execSubl -- basic positive subtraction
eBox::execSubl -- underflow sign-extends to 64-bit all-ones
eBox::execAddq -- 64-bit positive add
eBox::execAddq -- 64-bit overflow wraps to zero
eBox::execSubq -- basic 64-bit subtraction
eBox::execSubq -- 64-bit underflow wraps to UINT64_MAX
eBox::execCmpeq -- equal operands return 1
eBox::execCmpeq -- unequal operands return 0
eBox::execCmpeq -- full 64-bit comparison (high bits matter)
eBox::execAddlv -- non-overflow stores the ADDL result
eBox::execAddlv -- overflow still stores sign-extended wrap
eBox::execSublv -- non-overflow stores the SUBL result
eBox::execSublv -- overflow still stores sign-extended wrap
eBox::execAddqv -- non-overflow stores the ADDQ result
eBox::execAddqv -- overflow into bit 64 still stores wrap
eBox::execSubqv -- non-overflow stores the SUBQ result
eBox::execSubqv -- overflow still stores wrap
eBox::execMullv -- non-overflow stores the MULL result
eBox::execMullv -- overflow still stores sign-extended low 32
eBox::execMulqv -- non-overflow stores the MULQ result
eBox::execMulqv -- 128-bit overflow still stores low 64 bits
eBox::execMulqv -- negative operands, representable product
eBox::execRc -- reads intrFlag into Ra and clears it
eBox::execRc -- preserves prior zero state and remains zero
eBox::execRc -- Ra=R31 suppresses commit, side effect still fires
eBox::execRs -- reads intrFlag into Ra and sets it to 1
eBox::execRs -- prior set value returns 1, stays 1
eBox::execRc -> execRs -- atomic toggle round-trip
eBox::execRpcc -- packed CC: offset<63:32> | counter<31:0> (F-1)
eBox::execFtois -- AARM bit reorder with sign extension
softfloat vs host double: oracle agreement (RNE, bit-exact)
softfloat vs host double: cycles/op delta (for grins)
fBox::execCpys -- sign of Fa onto magnitude of Fb
fBox::execCpys -- positive sign carries through
fBox::execCpysn -- negated sign of Fa onto Fb
fBox::execCpysn -- negative Fa flips to positive on Fb
fBox::execCpyse -- sign+exp of Fa, fraction of Fb
fBox::execAdddt -- 1.5 + 2.5 = 4.0
fBox::execAdddt -- additive inverse cancels to zero
fBox::execSubdt -- 5.0 - 2.0 = 3.0
fBox::execMuldt -- 2.0 * 3.0 = 6.0
fBox::execMuldt -- zero times finite is zero
fBox::execDivdt -- 10.0 / 2.0 = 5.0
fBox::execDivdt -- 1.0 / 0.0 yields IEEE +infinity
fBox::execCmpteq -- equal operands -> 2.0
fBox::execCmpteq -- unequal operands -> 0.0
fBox::execCmpteq -- NaN compares unordered (false) -> 0.0
fBox::execCmptlt -- 1.0 < 2.0 -> 2.0
fBox::execCmptlt -- 2.0 < 1.0 is false -> 0.0
fBox::execCmptlt -- equal operands are not less than -> 0.0
fBox::execCmptle -- 1.0 <= 1.0 -> 2.0
fBox::execCmptle -- 2.0 <= 1.0 is false -> 0.0
fBox::execCmptle -- NaN -> 0.0 (unordered)
fBox::execCpys -- explicit sign bit comparison
fBox::execCvtqlv -- in-range quadword: repositioned, no IOV
fBox::execCvtqlv -- 2**31 overflows: truncated result + IOV sticky
fBox::execCvtqlv -- negative in-range longword does not overflow
fBox::execCvtqlsv -- overflow above 32 bits: IOV recorded
fBox::execCvtql -- plain variant records sticky IOV too (AARM 4.7.5)
SoftFloatBackend: exact T arithmetic, no flags
SoftFloatBackend: inexact div + sqrt set INE
SoftFloatBackend: directed rounding is honored (1 ULP apart)
SoftFloatBackend: overflow -> +inf, OV+INE
SoftFloatBackend: underflow to zero -> UNF+INE

```

```

SoftFloatBackend: compare (T) true/false + unordered
SoftFloatBackend: CVTTQ rounding + IOV on overflow
SoftFloatBackend: CVTQT integer->double exact
SoftFloatBackend: S-format is single-rounded (not double-rounded)
VAX addsub: same-binade effective subtraction orders |a| >= |b| (FV-1)
VAX CVTQG: exact-halfway quad rounds AWAY from zero, not to even (FV-2)
VAX underflow is recorded when the qualifier enables it (FV-3)
VAX CMPG: top-binade G operands compare as numbers, not IEEE NaN (FV-4)
VAX CMPG: reserved operand signals INV in every trap mode (FV-4/FV-8)
VAX dirty zero: INV in default mode, clean zero under /S (FV-8)
VAX CVTGF: F exponent window checked, OvF/Unf recorded (FV-6)
VAX CVTGF: VAX half-up rounding at F precision; /C chops (FV-6)
VAX CVTQF: rounds to F precision with VAX tie rule; /C chops (FV-6)
VAX CVTQG overflow stores the true result's low 64 bits (FV-7)
IEEE CVTTQ overflow stores the true result's low 64 bits (FV-7)
host FP self-test agrees with the SoftFloat oracle (no-abort path)
GrainSem -- bitwise operators
GrainSem -- compound assignment
GrainSem -- category masks contain expected members
GrainSem -- categories do not bleed
GrainSem -- bit positions are ABI
DispatchKind -- name helper round-trip
DispatchKind -- canonical count
iBox::execBr -- forward 1 instruction
iBox::execBr -- backward to itself (disp = -1)
iBox::execBr -- R31 destination suppresses link via kNoRegwrite
iBox::execBsr -- positive disp links Ra to pc+4
iBox::execBeq -- taken when Ra == 0
iBox::execBeq -- not taken when Ra != 0
iBox::execBne -- taken when Ra != 0
iBox::execBne -- not taken when Ra == 0
iBox::execBlr -- taken on signed-negative Ra
iBox::execBlr -- not taken on zero Ra
iBox::execBge -- taken on zero Ra (>= 0)
iBox::execBge -- not taken on signed-negative Ra
iBox::execBle -- taken on zero (Ra <= 0)
iBox::execBle -- taken on signed-negative
iBox::execBle -- not taken on positive Ra
iBox::execBgt -- taken on positive Ra
iBox::execBgt -- not taken on zero
iBox::execBgt -- not taken on signed-negative
iBox::execBlbc -- taken when low bit clear (even Ra)
iBox::execBlbc -- not taken when low bit set (odd Ra)
iBox::execBlbs -- taken when low bit set (odd Ra)
iBox::execBlbs -- not taken when low bit clear (even Ra)
iBox::execJmp -- target masks low 2 bits of opB
iBox::execJsr -- links Ra to pc+4, target from opB
iBox::execRet -- target from opB; Ra typically R31
iBox::execJsrCoroutine -- both link and indirect
mBox::execLda -- positive displacement
mBox::execLda -- negative displacement sign-extends
mBox::execLda -- R31 destination is just kNoRegwrite
mBox::execLdah -- displacement scaled by 65536
mBox::execLdah -- negative disp shifts and sign-extends correctly
mBox::execLdbu -- packs memSize=1 and EA
mBox::execLdwu -- packs memSize=2 and EA
mBox::execLdl -- packs memSize=4 (drainer sign-extends)
mBox::execLdq -- packs memSize=8
mBox::execLdqu -- masks low 3 bits of computed EA
mBox::execLdqu -- already-aligned EA passes through
mBox::execLdlL -- packs memSize=4, propagates S_Locked
mBox::execLdqL -- packs memSize=8, propagates S_Locked
mBox::execStb -- packs memSize=1, memData=opA, no register effect
mBox::execStw -- packs memSize=2
mBox::execStl -- packs memSize=4
mBox::execStq -- packs memSize=8
mBox::execStqu -- masks low 3 bits of computed EA
mBox::execStlc -- packs memEffect AND regwriteIdx=Ra
mBox::execStqc -- packs memEffect AND regwriteIdx=Ra
mBox::execFetch -- propagates semFlags only, no effect
GuestMemory -- default size is 64 MiB

```

```

GuestMemory -- zero-initialised on construction
GuestMemory -- byte round-trip
GuestMemory -- word round-trip; little-endian byte layout
GuestMemory -- longword round-trip
GuestMemory -- quadword round-trip
GuestMemory -- enforces the configured byte size (out-of-range faults)
GuestMemory -- misaligned access at in-range PA returns Ok (alignment delegated to
translator)
GuestMemory -- writes are independent across non-overlapping ranges
LockMonitor -- LL then SC by the same CPU succeeds once
LockMonitor -- reservation is cache-line granular (64B)
LockMonitor -- a load does not clear another CPU's reservation
LockMonitor -- exactly one STx_C wins a contended line
LockMonitor -- a plain store breaks every OTHER CPU's reservation
LockMonitor -- device/DMA store (exceptCpu=-1) breaks ALL reservations
palBox::execHalt -- raises kFaultHalt; no other effect
palBox::execHwMfpr -- reads HW_CC from CpuState::cycleCount
palBox::execHwMfpr -- reads HW_EXC_ADDR
palBox::execHwMfpr -- reads HW_CM as integer mode bits
palBox::execHwMfpr -- unknown selector raises kFaultUnimplemented
palBox::execHwMfpr -- silent-stub IPR returns 0 with no fault
palBox::execHwMtptr HW_CC -- packed format: writes OFFSET half only
palBox::execHwMtptr -- writes HW_VA_CTL into CpuState::va_ctl
palBox::execHwMtptr -- writes HW_CM clamps to bottom 2 bits
palBox::execHwMtptr -- unknown selector raises kFaultUnimplemented
palBox::execHwMtptr -- silent-stub IPR swallows write with no fault
palBox::execHwMtptr -- HW_PAL_BASE sources from Rb (regression: 2026-05-10)
palBox::execHwMtptr -- HW_PAL_BASE rejects opA noise (regression: 2026-05-10)
PCTX: EV6__ASTER write (0x42) updates only the ASTEN nibble; 0x5F read composes
PCTX: ASN field write (0x41) takes value<46:39>; ASTRR write (0x44) takes <12:9>
PCTX: full write (0x5F) sets all five fields incl. FPE->fen, PPCE->pme
PCTX: bare read form 0x60 (01xx xxxx) is a legal composed read, not a fault
palBox::execHwRei -- STACKED form diverts to EXC_ADDR and clears palMode
palBox::execHwRei -- REGISTER form diverts to Rb and tracks palMode bit
palBox::execBpt_tru64 -- stub raises kFaultUnimplemented
palBox::execBpt_vms -- stub raises kFaultUnimplemented
palBox::execChme_vms -- diverts to CALL_PAL entry palBase+0x3080
palBox::execChmk_tru64 -- stub raises kFaultUnimplemented
palBox::execCserve -- function 0x44 (MTPR_EXC_ADDR) diverts to R17, no R0 write
palBox::execCserve -- function 0x65 (MP_WORK_REQUEST) tolerated no-op; R0 untouched,
no divert
palBox::execCserve -- function code in low 8 bits only
palBox::execCserve -- unhandled function is tolerated (no-op, no fault)
palBox::execLdqp_vms -- packs phys-load memEffect targeting R0
palBox::execStqp_vms -- packs phys-store memEffect from R17
palBox::execLdqp_vms -- unaligned R16 raises kFaultUnaligned
palBox::execStqp_vms -- unaligned R16 raises kFaultUnaligned
palBox::execLdqp_vms -- 8-byte aligned address passes alignment check
palBox::execMfprVptb_vms -- returns cpu.vptb in R0; no fault, no divert
palBox::execMfprVptb_vms -- returns 0 when cpu.vptb is unset
palBox::execMtptrVptb_vms -- updates cpu.vptb, both IPRs, AND PT_VPTB
palBox::execMtptrVptb_vms -- an unusable p_temp FAULTS, never skips
palBox::execMtptrVptb_vms -> execMfprVptb_vms -- round-trips via cpu.vptb
palBox::execWtint -- returns 0 in R0; no fault, no divert
palBox::execMfprWhami -- returns 0 in R0 (single-CPU)
palBox::execCallPalDispatch -- privileged func 0x29 (MFPR_VPTB) computes 0x602A40
palBox::execCallPalDispatch -- unprivileged func 0x86 (IMB) hits 0x3000 base
palBox::execCallPalDispatch -- excAddr captures prior PALmode in bit 0
palBox::execCallPalDispatch -- preserves palBase high bits above 32K window
palBox HW_SIRR -- MTPR drops reserved bits; MFPR round-trips SIR<15:1> (T4)
palBox HW_ISUM -- derived read; MTPR stays a permissive no-op (T5)
pendingSoftInt + the run-loop delivery gate shape (T6)
SIRR liveness loop: request -> deliverable -> clear -> quiescent (T7)
SWPCTX T1+T5: A->B->A round-trips save-set fields and CPC
SWPCTX T2+T3+T4: TB semantics -- TBIAP, ASM survival, GH span
SWPCTX T6: misaligned R16 faults with NO state change; success path is a single clean
grain
MemDrainer -- LDA-shaped result commits Ra without memory
MemDrainer -- regwriteIdx == kNoRegwrite suppresses commit
MemDrainer -- LDQ commits 64-bit fill into Ra
MemDrainer -- LDL sign-extends 32-bit fill to 64 bits

```

```

MemDrainer -- LDBU zero-extends 8-bit fill
MemDrainer -- LDWU zero-extends 16-bit fill
MemDrainer -- STQ publishes 64-bit memData
MemDrainer -- STB publishes 1-byte memData
MemDrainer -- LDQ_L sets reservation at cache-line granularity
MemDrainer -- STQ_C succeeds when reservation matches; writes 1 to Ra
MemDrainer -- STQ_C fails when reservation cleared; writes 0; no publish
MemDrainer -- STQ_C fails when reservation tags a different cache line
MemDrainer -- pre-existing faultCode aborts before drain work
MemDrainer -- GuestMemory OOR populates kFaultBusError + mm_stat
MemDrainer -- unaligned LDQ raises kFaultUnaligned (trap enabled)
PipelineDriver -- bare HALT halts immediately
PipelineDriver -- LDA / ADDQ / HALT sequence sets R1 and R2
PipelineDriver -- STQ / LDQ round-trip via GuestMemory
PipelineDriver -- BR diverts past a no-op into HALT
PipelineDriver -- BEQ falls through when Ra is non-zero
PipelineDriver -- BEQ takes when Ra is zero
PipelineDriver -- maxCycles caps a non-halting program
PipelineDriver -- IF fetch beyond GuestMemory halts with excAddr set
PipelineDriver::step -- returns false once halt fires
PipelineDriver -- trap delivery swaps PAL shadow regs when SDE set
PipelineDriver -- FltVax dispatch: CVTQG/CVTGQ round-trip reaches the VAX-FP leaves
PipelineDriver -- ItFp dispatch: SQRTT func 0x0AB does not alias onto SQRTT_C
PipelineDriver -- FltIeee dispatch control: CVTQT reaches its leaf
PipelineDriver -- FltVax dispatch bounds: unpopulated function still OPCDECS
AlphaPte::makeValid composes the standard kernel-RX mapping
AlphaPte::canRead respects fault-on-read regardless of mode bits
AlphaPte::canWrite is per-mode
isEpochLive enforces the TBIA + TBIAP truth table
TlbEntry::matches checks VPN, ASN, and respects ASM=1
SPAMBucket<4> round-robin replacement + targeted invalidate
SPAMShardManager TBIA flushes every entry in O(1)
SPAMShardManager TBIAP preserves ASM=1 entries
SPAMShardManager invalidateSingle is targeted
SPAMShardManager clear resets epochs and drops slots
GH blocks: fill via page N, hit via page k, PA lands on page k
lock_arbiter_semantics
determinism_equivalence
driver_toggle
parked_agent_no_deadlock
determinism_across_quantum_sequential
Phase3 LockArbiter -- a load does not clear another CPU's reservation
Phase3 LockArbiter -- exactly one STX_C wins a contended granule
Phase3 LockArbiter -- a plain store breaks every OTHER CPU's reservation
FirmwareLoader -- load 8 bytes at PA 0; contents match
FirmwareLoader -- PAL bit on startPa sets palMode
FirmwareLoader -- low bit clear means palMode false
FirmwareLoader -- file not found
FirmwareLoader -- image larger than guest memory
FirmwareLoader -- non-zero loadPa places image at offset
Machine -- HALT at PC 0 returns HaltedClean
Machine -- LDA, LDA, ADDQ, HALT computes 0x42 + 0x10 = 0x52
Machine -- maxCycles caps a non-halting program
Machine -- reset clears regfile and halt flags
Machine -- step returns false when CPU halts
Machine::tryFetch -- without SRM load returns false
Machine::tryFetch -- serves payload before Step D, disarms after
Machine -- loads to R31/F31 never access memory or fault
PlatformConfig: built-in DS10 default validates clean
PlatformConfig: good manifest parses, no fallback
PlatformConfig: malformed JSON falls back to default
PlatformConfig: unsupported manifest_version falls back
PlatformConfig: duplicate IIC address is a hard error
PlatformConfig: odd IIC address is a hard error
PlatformConfig: fru_eeeprom missing required fields is a hard error
synthesizeFruImage: JEDEC fields + checksums
synthesizeFruImage: non-FRU entry is all zero
synthesizePciConfig: header fields + BAR masks
PlatformConfig: storage 'enabled' flag -- implicit true; disabled alt shares a slot
PlatformConfig: create_if_missing parses size + validates (disk only, 512-aligned)
Snapshot: round-trip preserves CpuState

```



```

Snapshot: round-trip preserves GuestMemory bytes
Snapshot: round-trip preserves Tsunami chipset CSR storage
Snapshot: round-trip preserves interrupt-chain devices (v2)
Snapshot: rejects file with bad magic
Snapshot: rejects memory-size mismatch
Snapshot: autoloadLatest returns no-file on empty dir
Snapshot: autoloadLatest picks newest by mtime
Snapshot: pruneOldSnapshots keeps newest N auto_ files only
Snapshot: round-trip preserves SRM staging fields
SrmLoader -- signature at 0x300 with palBase + finalPC
SrmLoader -- file too small returns error
SrmLoader -- signature absent returns error
SrmLoader -- PAL_BASE zero returns error
SrmLoader -- image larger than guest memory range-rejected
DeclistingSink -- emits one listing line and one machine line
DeclistingSink -- TRACE_REGFILE emits REG line in machine channel
DeclistingSink -- PAL window auto-on dumps lookback on entry
Disassembler -- LDA R1, 0x42(R31) renders correctly
Disassembler -- LDQ R7, -8(R30) renders signed disp as hex
Disassembler -- ADDQ R1, R2, R3 renders as triple
Disassembler -- ADDQ R1, #0x42, R3 renders literal form
Disassembler -- BR R31, +8 longwords
Disassembler -- BEQ R3, -16 longwords negative
Disassembler -- JMP renders Ra, (Rb)
Disassembler -- CALL_PAL HALT renders 0x000000
Disassembler -- MISC TRAPB renders 0x0000
Disassembler -- ADDT renders Fa, Fb, Fc
Disassembler -- HW_MFPR renders Ra, IPR(idx)
formatResult -- integer write
formatResult -- FP write
formatResult -- kNoRegWrite returns empty
RetireProfiler: bucket math -- 1KiB granularity, PAL bit masked
RetireProfiler: PCs above the 16MiB span fold into catch-all
RetireProfiler: control poke -- dump then mark; since-mark resets

```

```
[doctest] unskipped test cases passing the current filters: 561
```

Index

- . -

.axpsnap 18, 106

- [-

[System] model 71, 195

- _ -

_emulatr_runenv.sh 56

- > -

>>> 7, 253

- 0 -

0x10B9 147

0x1533 147

0x3c970 200

0xBFFC 178, 181, 194

- 1 -

10023 60, 69

1024 MB 90

- 2 -

21143 153

21264 6, 118, 233, 237

21264A 6, 237

21264C 238

21272 6, 17, 89, 118, 139, 140, 233, 238

21272 Tsunami 75

21272-CA 6, 17

21272-DA 6, 17

21272-EA 6, 17

21274 139, 144, 238

264DP 203

268 MHz 198

- 5 -

53C810 265

- 8 -

82077 150

82077AA 150

- A -

AAR 132

AAR0 265

AAR1 265

AAR2 265

AAR3 265

AARM Table 26-1 170

AARM Table 26-4 170

ABT 265

ABW 265

ACL 265

Add Existing 63, 206

Add/Remove Programs 42

AGP 144

ALi M1543C 23, 145, 147

ALi M5229 23, 181

AliM1543C 147

Alpha AXP 6, 118, 225

Alpha EV6 17

AlphaCpuAgent 175, 251

AlphaPte::makeValid 265

AlphaServer 6, 232, 238

AlphaServer DS10 17, 233

AlphaServer DS20 83, 203

AlphaServer ES40 83

Am29F016 265

analyze_retire_trace 249

analyzeHeadless 245

apisrm 241, 262

A-port 144

appendices 232

arbiter 17, 132

architectural CC 200

architectural correctness 15

architecture 118

argument resolution 71
 ASCII 223
 ASIZ 265
 ASM 136, 265
 ASN 133, 136, 265
 AST 265
 ASTEN 265
 ASTER 265
 ASTRR 265
 ATA 251
 ATA disk 96
 ATA fixed disk 105
 ATAPI 6, 18, 89, 92, 96, 104, 265
 ATAPI signature 92
 atapi_cdrom 104
 AtaTaskfileEngine 181
 attributions 206
 authoring tool 245
 authority tiers 262
 auto*.axpsnap 195
 auto_action 60, 90, 91, 110, 229
 autoload-newest 106
 --autosnapshot 50, 51, 74, 77, 106
 AXPBox 158, 185, 262
 axpsnap 107, 210, 253

- B -

b dka0 99
 b dqa1 21, 99, 229
 b_error 142
 Bachman diagram 175
 background process 51
 badge 203
 BAR 233, 265
 bare image 96
 bash 51
 Bcache 17
 BCD 265
 Berkeley SoftFloat 128
 beta 7, 42
 binaryConfig 206
 blank-4G 96
 blocker 23
 boot 82, 89, 90, 110, 250
 boot call stack 241
 boot flags 23
 boot gate 264

boot milestone 21
 BOOT NOT POSSIBLE 229
 boot problems 227
 boot transcript 75, 83
 boot_osflags 23, 90, 91, 110
 boot0 185
 bootdef_dev 90, 91, 110
 bootstrap handoff 185
 Box 15, 225
 BoxResult 118, 123
 bring-up probes 47, 114
 bugcheck 7
 build 42, 47, 246, 247, 249
 build of record 56
 build output 57
 build prerequisites 44
 build.sh 158
 build_emulatr_diag.bat 47
 build_hwrpb 203
 build_power_hw 23, 265
 build_test_run 249
 byte-identical 251

- C -

C++20 6, 44, 48
 CAFEBEEF 178, 181, 194
 calibration 198
 call graph 246
 CALL_PAL 134
 CALL_PAL HALT 61
 canonical VA 136
 canRead 265
 canWrite 265
 CBox 116, 118
 cBox::execEcb 265
 cBox::execExcb 265
 cBox::execMb 265
 cBox::execTrapb 265
 cBox::execWmb 265
 Cchip 6, 17, 89, 118, 132, 139, 140, 265
 C-chip 238
 CD001 94
 CDB 265
 cfenv 128
 changelog 7
 CHECK 264
 CheckCondition 265

- chipset 139, 249
- chipset block 253
- ChipsetCsr 116
- chipsetLib 262
- ChipsetVariantInfo 265
- clamp-and-continue 71
- clean shutdown 61, 234
- CLI 77
- CLRFEN 128
- CMake 42, 44, 45, 48, 56, 246
- CMake guards 122
- CMake option 130
- CMakeCache 114
- CNS__WORK_REQUEST 241
- codegen 122, 125
- cold boot 50, 74, 122, 157, 195, 250, 252
- COM1 6, 17, 91, 235, 265
- COM2 6, 17, 235, 265
- command line 51
- command line options 77
- command-line 77
- commercial license 14
- commit point 15
- compile gate 114
- compile guard 130
- composelsum 265
- composelsumSw 265
- computeVaForm 265
- concurrent instances 62, 75
- CONFIG 56
- config/Emulatr.ini 195
- CONFIGURE 45, 150
- configuring 62
- connection refused 81, 229
- console 60, 62, 79, 80, 82, 227
- console entry 212
- console log 51
- console port 80, 227
- console server 80
- console_restart 177
- conventions 223
- conversational boot 110
- copyright 14
- core logic 89, 139
- coreLib 262
- CPC 265
- CPU pipeline 18
- cpuCount 69
- CPUID 265
- cpuLib 262
- cpuSlot 241
- CPU-START 265
- CpuState 118
- CpuState.h 175
- crash dump 7
- create_if_missing 96
- CSC 265
- CSERVE 185, 241, 265
- CSERVE START 185
- CSERVE\$CALLBACK 162
- CSERVE\$CLEAR_HWE 162
- CSERVE\$HALT 162, 241
- CSERVE\$IIC_WRITE 162, 241
- CSERVE\$JUMP_TO_ARC 162
- CSERVE\$LDBP 162
- CSERVE\$LDLP 162
- CSERVE\$MP_WORK_REQUEST 162
- CSERVE\$MTPR_EXC_ADDR 162
- CSERVE\$SET_HWE 162
- CSERVE\$START 162, 241
- CSERVE\$STBP 162
- CSERVE\$STLP 162
- CSERVE\$WHAMI 162, 241
- CSERVE__CALLBACK 162
- CSERVE__START 162
- CSR0 153
- CSR5 153
- CSR9 153
- Ctrl/P 7
- current directory 51
- CVT 265
- CVTTQ 265
- CY82C693 18, 92, 93, 96, 148, 149, 265
- CY82C693 IDE 89
- Cy82C693Ide 149, 265
- Cy82C693IsaBridge 149
- cycle counter 198
- cycle counter frequency 170
- cycle_count_freq 198, 200
- cycleCount 200
- CycleInjector::runCycles 265
- Cypress 148, 149
- Cypress 82C693 17, 21, 75, 89
- Cypress CY82C693 6, 118, 145, 181, 233, 235
- Cypress ISA bridge 265

- D -

- d command 91
- d pmem 230
- DAT 265
- Dchip 6, 17, 118, 139, 140, 265
- D-chip 238
- ddb_startup() 161
- DE500 153, 233
- de500_tulip 153
- debug 195
- DEC 21143 153
- DEC Alpha 6
- dec_disk_media_types 96
- decode 127
- decom.c 158
- decompile 249
- decompile_ds10 249
- decompile_ds10.sh 245
- decompiler 245
- DecompileToDir.java 245
- decompress 157
- decompressed.bin 248
- decompressed_ds10_v7_3.bin 158
- decompression 118, 122
- deep-dive 131, 232
- dependencies 206
- deposit 91, 265
- determinism 15, 107
- DeterministicMemory 265
- developer 6
- developer lens 131
- developer reference 232
- development status 21
- development tools 244
- device discovery 178, 181
- device enumeration 181
- deviceLib 262
- DEVSUP 265
- diag_halt_irq4 250
- diagnostic build 130
- diagnostic logging 116
- diagnostics 62, 112, 247
- differential validation 15
- DIM 265
- DIMM 238
- DIR 265
- disassembler 248
- disk geometry 96
- disk image 251
- disk_types.json 96, 99
- diskDir 63, 99
- disown 51
- dispatch 251
- dispatch tables 122
- DispatchKind 127, 265
- DispatchTables.cpp 127
- Display 109
- distribution 56, 57
- dka 21
- dka0 99, 110
- DLAB 265
- DLM 265
- dmaReadBytes 265
- dmaWriteBytes 265
- doctest 223, 249, 264
- documentation 245
- DOR 150
- double machine check 212
- dq_driver 181
- dq_driver.c 241, 262
- dq_init_ide 178, 181
- dq_initialize 23, 178
- dq_long_wait_on_busy 236
- dq_wait_for_drq 236
- dq_wait_on_busy 228, 236
- dqa 93, 181, 251
- dqa0 18, 75, 89, 92, 96, 105, 110, 228, 233
- dqa1 99, 104, 110, 233, 250
- DQB 181
- DREV 265
- DRIR 265
- DRIR 62 142
- DS10 6, 17, 23, 69, 71, 140, 148, 149, 195, 210, 232, 238, 249, 250, 265
- ds10_flash.rom 75
- ds10_platform.json 17, 69, 233
- ds10_v7_3.exe 6, 44, 51
- DS15 71, 144, 195, 238
- DS15a 238
- DS20 23, 71, 140, 148, 181, 195, 210, 232, 237, 238, 265
- ds20_v7_3 185
- ds20_v7_3.exe 7
- DS20e 238

DS25 71, 144, 195, 238
 DSRDB 170, 203
 DSSI 95
 DTB 18, 133, 136
 DTB_IA 185
 DUMPREG 150
 DUMPSTYLE 7
 dva0 21, 89, 150
 dva0.0.0.0 105

- E -

e command 91
 e pmem 230
 EBox 118
 eBox::execAddI 265
 eBox::execAddIV 265
 eBox::execAddq 265
 eBox::execAddqV 265
 eBox::execCmpeq 265
 eBox::execFtois 265
 eBox::execMulIV 265
 eBox::execMulqV 265
 eBox::execRc 265
 eBox::execRpcc 265
 eBox::execRs 265
 eBox::execSubI 265
 eBox::execSubIV 265
 eBox::execSubq 265
 eBox::execSubqV 265
 ECC 238
 EEPROM 265
 effective MHz 200
 ejectMedia 265
 EmulatorSettings 69
 EmulatR 6, 112, 247
 Emulatr root 48
 EMULATR_VIRTUAL_CDROM 92
 EmulatR.exe 45, 50, 77
 Emulatr.ini 69
 EMULATR_ 91
 EMULATR_2D_NOOP 7, 185
 EMULATR_AUTOSNAP 106, 216
 EMULATR_BOOTSTRAP_ITB_BYPASS 130
 EMULATR_BREAK_ON_PA10 113, 160
 EMULATR_BRINGUP_PROBES 114
 EMULATR_CHECKPOINTS 113, 160
 EMULATR_CHIPSET_DIAG 130
 EMULATR_CHIPSET_DIAG_OFF 113, 160
 EMULATR_CONSOLE_MIRROR 74
 EMULATR_CONSOLE_PORT 69, 74, 75, 80, 216, 227, 234
 EMULATR_CONSOLE_SNAPSHOT 74, 106, 216
 EMULATR_CONSOLE_TRACE 44
 EMULATR_CSERVE_AUDIT 241
 EMULATR_CSERVE_START_MODE 241
 EMULATR_DEBUG_STEP 130, 246
 EMULATR_DIAG 195
 EMULATR_DIAGNOSTIC_LOGGING 116, 130
 EMULATR_FAST_DECOMPRESS 107, 210
 EMULATR_FDC_TRACE 150
 EMULATR_FLASH_ROM 75, 216, 227, 234
 EMULATR_FLASH_TRACE 74, 113, 160
 EMULATR_FP_SOFTFLOAT 128, 130
 EMULATR_GATE 113, 160
 EMULATR_GCT_WATCH 113, 160
 EMULATR_IDE_TRACE 74
 EMULATR_IDLEWARP 178, 181, 210
 EMULATR_IIC_TRACE 74, 113, 160
 EMULATR_ITBPROBE_VA 185
 emulatr_launch.sh 195
 EMULATR_MEMDIAG 45, 113, 130, 160, 216
 EMULATR_NO_AUTOLOAD 50, 74, 106, 216
 EMULATR_NO_PUTTY 48, 60, 80, 216
 EMULATR_PA_DUMP 130
 EMULATR_PIC_TRACE 113, 160
 EMULATR_PLATFORM 178, 181, 194, 210
 EMULATR_PLATFORM_CONFIG 17, 69, 216
 EMULATR_PROFILE_ALPHA_CLOCK_HZ 198
 EMULATR_PTBR_DIAG 185
 EMULATR_RETIRE_TRACE_DIR 113, 160
 EMULATR_SNAPSHOT_MARKER 106
 EMULATR_SNAPSHOT_PRUNE 106
 EMULATR_STOP 61, 63, 206
 EMULATR_STOP_FILE 216
 Emulatr_tests 264
 EMULATR_TOY_MODE 7, 216
 EMULATR_TOY_NVRAM 7, 216
 EMULATR_TRACE_DIR 51
 EMULATR_TRACE_HOOKS 130
 EMULATR_TRACE_WINDOW 113, 160, 253
 EMULATR_USE_OS_PAGES 130
 EmulatrLaunch 63, 99
 EmulatRLauncher 96
 engineering status 236
 entry vector 123

- entryPa 107
- environment variable 234
- environment variables 206, 216
- eNVy Systems 14
- EOI 265
- epoch invalidation 136
- error interrupt 142
- ES40 6, 17, 23, 71, 140, 147, 178, 181, 195, 210, 232, 237, 238, 265
- es40_platform.json 237
- ES45 6, 17, 71, 144, 147, 195, 210, 232, 237, 238, 265
- es45_platform.json 237
- es45_v7_3.exe 158
- ETBEI 265
- Ethernet 153
- EV6 6, 17, 18, 75, 89, 118, 233, 248
- EV6__ASTER 265
- ev6_defs.mar 241, 262
- ev6_huf_decom 157
- ev6_huf_decom.m64 241, 262
- ev6_ipr_driver.c 241, 262
- EV67 6, 17, 237
- EV68 144
- EV68C 238
- EV68CB 238
- Ev6EntryVectors 118
- Ev6Translator 118, 133, 136
- evaluate 253
- ewa0 110, 153
- examine 91
- EXC_ADDR 123, 134
- excAddr 265
- ExecCtx 118, 123
- execution matrix 210
- execution mode 63, 178
- execution model 118
- execution path 194
- Exit 109
- explicit serialization 15
- extract 265
-
- F -**
-
- F_float 128
- faithful 107
- faultCode 123
- fBox 23, 118, 128
- fBox::execAddt 265
- fBox::execCmpteq 265
- fBox::execCmptle 265
- fBox::execCmptlt 265
- fBox::execCpys 265
- fBox::execCpyse 265
- fBox::execCpysn 265
- fBox::execCvtql 265
- fBox::execCvtqlSv 265
- fBox::execCvtqlV 265
- fBox::execDivt 265
- fBox::execMult 265
- fBox::execSubt 265
- FCR 265
- FDC 6, 150
- FDC37C669 150
- FDD 235
- Feature Matrix 18
- FEN 128, 265
- fidelity standard 15
- fidelity-faithful 15
- FieldSpec 265
- FileBlockMedia 105, 265
- filesys_init() 161
- findRange 265
- Firewall 209
- firmware 6, 42, 50, 51, 63, 77
- firmware 44, 50, 51, 69, 71, 77, 227
- firmware integrity check 44
- firmware launch 252
- firmware path resolution 44
- firmware stem 69, 71, 195
- firmware update 62, 109
- firmwareImage 44, 69
- firmwareSha256 44, 69
- first run 48
- first startup 48
- Flash 18
- flash corruption 75
- flash isolation 75
- FlashRom 265
- floating point 122
- floppy 17, 150
- floppy controller 150
- Floppy, Virtual 105
- Floppy82077 265
- Fitleee 265
- FitVax 265

forked process 51
 FORMAT TRACK 150
 fpBox 118
 fpBoxLib 128
 FPCR 23, 128
 FPE 265
 FRU 23, 60, 241, 265
 FRU EEPROM 233
 full_powerup_diags 75, 90, 228

- G -

G_float 128
 g_primaryTable 123, 127
 GAP-PLAT-001 71
 gate 251
 GCT 60, 122, 241
 GCT/FRU 228
 genGrains.py 125
 get_sysvar 203
 getChar 265
 getenv 216
 GH 265
 Ghidra 44, 244, 245, 249
 Git Bash 47
 glossary 206, 225
 goals 15
 golden boot 83
 golden trace 264
 G-port 144
 graceful-stop 61
 grain 118, 123, 125, 225
 grain factory 122
 grain.execFn 265
 GrainMasterV4 118
 GrainMasterV4.tsv 125
 GrainSem 265
 GrainStubs.cpp 125
 ground truth 262
 GuestMemory 118, 132, 265

- H -

halt 18, 60, 83, 90, 250
 halt button 227, 229
 halt code 212
 halt gate 229

halt request 212
 halt switch 229
 HALT__ 212
 HaltedClean 265
 handwritten.tsv 125
 Hardware, Initialize 161
 Hardware, Reset 161
 HBA 94
 headless 245, 249
 help 89
 Help & Manual 244, 245
 help project 245
 hex dump 253
 Hierarchy.txt 262
 hive 206
 HKCU 206
 host_decompressor 157, 158
 HostBackend 128
 hot path 246
 hot PC 249
 Huffman 157
 HW_CC 265
 HW_CM 265
 HW_EXC_ADDR 265
 HW_IPR 134
 HW_ISUM 265
 HW_MFPR 118, 134, 265
 HW_MTPR 23, 118, 134, 265
 HW_PAL_BASE 265
 HW_REI 123
 HW_REI_XITION 114
 hw_ret 122, 185
 HW_SIRR 265
 HW_VA_CTL 265
 HWPCB 265
 HWRPB 170, 198, 212, 253
 Hwrpb.h 170, 241

- I -

I_CTL 134, 265
 IBox 118
 iBox::execBeq 265
 iBox::execBge 265
 iBox::execBgt 265
 iBox::execBlbc 265
 iBox::execBlbs 265
 iBox::execBle 265

iBox::execBlt 265
 iBox::execBne 265
 iBox::execBr 265
 iBox::execBsr 265
 iBox::execJmp 265
 iBox::execJsr 265
 iBox::execJsrCoroutine 265
 iBox::execRet 265
 IC_FLUSH 185
 IDE 6, 17, 18, 92, 96, 149, 181, 244, 246
 IDE controller 93
 ide_poll 265
 IDENTIFY 178
 IDENTIFY DEVICE 92, 265
 IDENTIFY PACKET DEVICE 92
 IDLEWARP 200
 IEEE S 128
 IEEE T 128
 IER 265
 IER.SIEN 265
 IFetchOverride 123
 IFpBackend 128
 IIC 18, 23, 233
 iic_8574_ocp 203
 iic_chip_reset 265
 iic_ocp0 203
 iic_rcm_temp 203
 licPcf8584 203, 265
 IloPortHandler 145
 IIR 265
 IMB 265
 impure frame 185
 IMPURE\$CPU_TYPE_OFFSET 162
 IMPURE\$PCB_OFFSET 162
 IMPURE\$PROF_OFFSET 162
 IMPURE\$QUILT_OFFSET 162
 IMPURE\$UART_OFFSET 162
 IMR 265
 include guards 223
 inflate 157
 inflate.c 158
 ini default 51
 IniLoader 69
 init 23
 Initializing 177
 InlineWorkQueue 265
 INQUIRY 265
 install 99
 install target 251
 Installation 6, 42
 installer 42
 interleaving 238
 internal processor register 134
 interrupt 250
 interval timer 198, 200
 IntervalTimer 116
 intrclock_freq 198
 intrFlag 265
 Introduction 6
 invalidateSingle 265
 inventory 57, 254
 INVEXCEPTN 7
 IoCommand 265
 IoResult 265
 IPciDeviceHandler 145
 ipcr 23
 IPINTR 265
 IPR 118, 131, 134, 225
 IPR 0x2d 23, 185
 IPREQ 265
 IRQ_DEV_M 162
 IRQ4 250
 IRQ6 150
 ISA 17, 89
 ISA bridge 145, 148, 149
 ISA table corrupt 230
 isEpochLive 265
 ISO 99, 104, 250
 ISO-9660 94
 ISP 63, 178, 181, 210
 ISP mode 23, 185
 ISP_MODEL 178, 181, 194
 ITB 18, 133, 136
 ITBPROBE 185
 ItFp 265
 ITINTR 265

- J -

JDK 21 245, 249
 JIT 23
 JRN-PLATID-001 71
 JUMP_TO_ARC 241

- K -

KBD 17
 kCchipIntervalTimerCycles 265
 kCchipTimerBit 198
 kCchipTimerMask 200
 KCRCM 203
 kernel executive 241
 kernel stack not valid 212
 kernel.c 161
 keywords 245
 kFaultBusError 265
 kFaultHalt 212, 265
 kFaultUnaligned 265
 kFaultUnimplemented 185, 265
 klicBaseByModel 23
 kill EmulatR 234
 kit 246
 kKeyValue 170
 kNoRegWrite 265
 known gaps 23
 known limitations 234
 kProfileAlphaClockHz 200
 km\$_micro_delay 178
 km\$_reset_toy 265
 km\$_sleep 228
 kseg 118, 133, 136, 225
 KSP 265

- L -

LAB 265
 launch_paths 206
 launch_run 206
 launch_systems 206
 launch_ui 206
 launch_vms_boot 250
 launcher 63
 Launching EmulatR 48
 LBA 16 94
 leaf 125
 leafBase 125
 legacy interconnect 95
 LFU 7, 21, 60, 75, 109, 177, 241
 license 14
 license header 14

List 109
 listener 229
 loadMedia 265
 localhost 60
 LockArbiter 265
 LockMonitor 118, 132, 265
 log subsystem 116
 --log-disable 116
 --log-file 51, 116
 logging 112
 logging subsystems 116
 logical device 235
 --log-only 116
 logs 223
 --log-verbose 116
 loop detection 249
 lowerPciInterrupt 265
 LPT1 6, 17, 235
 LRB 265
 LSR 265
 LUN 265

- M -

M5229 147
 MAC address 153
 Machine 118, 265
 machine check 212
 Machine.cpp 175
 Machine::tryFetch 265
 Make Disk 99
 makeBlockMedia 265
 manifest 259
 Mark Adler 157, 158
 mask 265
 master abort 142
 maxCycles 265
 --max-cycles 50, 51, 77
 maximum memory 238
 MBox 118
 mBox::execFetch 265
 mBox::execLda 265
 mBox::execLdah 265
 mBox::execLdbu 265
 mBox::execLdl 265
 mBox::execLdlL 265
 mBox::execLdq 265
 mBox::execLdqL 265

mBox::execLdqU 265
 mBox::execLdwu 265
 mBox::execStb 265
 mBox::execStl 265
 mBox::execStlC 265
 mBox::execStq 265
 mBox::execStqC 265
 mBox::execStqU 265
 mBox::execStw 265
 MCHK_FROM_PAL 212
 MCR 265
 media 104, 105
 media_kind 99
 MediaStatus 265
 MEDIUM NOT PRESENT 92
 --mem 51, 71, 77, 90
 member 203
 memData 265
 MemDrainer 123, 178, 181, 265
 MEMDSC 23
 memEffect 265
 memory 118
 memory architecture 238
 memory array 90
 memory geometry 71
 memory map 131
 memory motherboard 238
 memory sizing 60, 241
 memory_test 75, 90, 228
 memorySize 71, 90
 memSize 265
 MFPR_VPTB 265
 MHz 198
 mirror log 63
 MISC 265
 mkdisk 96, 251
 mm_stat 265
 mmap 132
 MMIO 132
 mmuLib 262
 MobaXterm 80, 81
 MockBlockMedia 265
 MODE SELECT(10) 265
 MODE SELECT(6) 265
 model 69
 model derivation 71
 module reset 177
 MOUSE 17

MP\$BREAK 162
 MP\$CONTROL_P 162
 MP\$HALT 162
 MP\$RESTART 162
 MP_WORK_REQUEST 241, 265
 MS620 238
 MSI 42
 MSR 150
 MSVC 42, 44, 45, 47, 48
 MSVC 2022 246
 MTPR_EXC_ADDR 241, 265
 multiple instances 80
 multiprocessor 175

- N -

named system 63
 NDS 142
 netcat 80, 81
 netstat 80
 New System 63
 NIC 153
 No Dvsel 142
 --no-autoload 50, 51, 77, 106, 195
 nohup 51, 74
 non-commercial 14
 nt_fw 109
 NVRAM 18, 21, 75, 82, 83, 90, 234
 NVRAM flush 61
 NVRAM slots 91
 NXM 23, 132, 265

- O -

objectives 15
 OCW1 265
 OCW3 265
 oem_string 90
 offset 248
 offset keys 170
 OPCDEC 265
 opcode 127
 Open Console 63
 Open in PlatEd 63
 OpenVMS 6, 7, 23, 99, 250
 OpenVMS boot 110
 operating 62

operator 6
 OPERCRASH 7
 option firmware 104, 241
 option firmware scan 150
 option-firmware scan 228, 236
 options 77
 Oracle 107, 128, 157, 158, 264
 oracle.c 158
 OS boot 21
 osfalpha_defs.mar 262
 out/build 56, 195
 OUT2 265
 outtig 177
 ovly_init () 161

- P -

p_temp 185
 P00 83
 P00>>> 7
 PA-chip 144
 PACKET 265
 PACKET command 92
 PAL 252
 PAL image 248
 PAL\$CALLBACK 162
 PAL\$CONSOLE_BASE 162
 PAL\$CPU0_START_BASE 162
 PAL\$CPU1_START_BASE 162
 PAL\$ENTER_CONSOLE_PTR 162
 PAL\$ENTER_OSF 162
 PAL\$EVMSBUILD_BITMASK 162
 PAL\$EVMSVERSION 162
 PAL\$HALT_SWITCH_IN 162
 PAL\$HWRPB_BASE 162
 PAL\$IMPURE_BASE 162
 PAL\$IMPURE_COMMON_SIZE 162
 PAL\$IMPURE_SPECIFIC_SIZE 162
 PAL\$LOGOUT_BASE 162
 PAL\$LOGOUT_SPECIFIC_SIZE 162
 PAL\$OSFPAL_BASE 162
 PAL\$OSFPAL_SIZE 162
 PAL\$OSFVERSION 162
 PAL\$PAL_BASE 162
 PAL\$PAL_SIZE 162
 PAL\$PRIMARY 162
 PAL\$TEMPS_BASE 162
 PAL\$TEMPS_SPECIFIC_SIZE 162

PAL__CONSOLE_BASE 162
 PAL__ENTER_OSF 162
 PAL__IMPURE_BASE 162
 PAL__IMPURE_COMMON_SIZE 162
 PAL__IMPURE_SPECIFIC_SIZE 162
 PAL__OSFPAL_BASE 162
 PAL__PAL_BASE 162
 PAL__TEMPS_BASE 162
 PAL__TEMPS_SPECIFIC_SIZE 162
 PAL_BASE 122, 134, 157
 PAL_TEMP 134
 palBase 265
 PalBox 118
 palBox::execBpt_tru64 265
 palBox::execBpt_vms 265
 palBox::execCallPalDispatch 265
 palBox::execChme_vms 265
 palBox::execChmk_tru64 265
 palBox::execCserve 265
 palBox::execHalt 265
 palBox::execHwMfpr 265
 palBox::execHwMtptr 265
 palBox::execHwRei 265
 palBox::execLdqp_vms 265
 palBox::execMfprVptb_vms 265
 palBox::execMfprWhami 265
 palBox::execMtptrVptb_vms 265
 palBox::execStqp_vms 265
 palBox::execWtint 265
 palBoxLib 262
 PALcode 6, 15, 18, 75, 83, 118, 185, 225
 PALcode trap delivery 123
 PALcode-visible IPR 134
 PalEntries.cpp 185, 241
 palMode 265
 pc264 148, 149, 178, 181, 265
 pc264_init.c 161
 PCC 198
 PCF8584 18
 Pchip 6, 17, 118, 139, 140, 265
 P-chip 238
 PCHIP 1 23
 Pchip error 142
 Pchip1 265
 PCI 153
 PCI config 93
 PCI device manifest 233
 PCI enumeration 21

- PCI hose 17, 89, 96
- PCI INTx 265
- PCI probe 60, 241
- PCI-to-ISA bridge 6
- PCN 265
- PCSAMPLE 114
- PCTX 265
- peek poke 170
- pendingSoftInt 265
- per-CPU slot 170, 212
- PerCpuSlot 170
- performance 62
- Performance Validator 113, 246
- per-model reference 232
- PERPENDICULAR MODE 150
- PERRMASK 142
- PERROR 142
- PERRSET 142
- personality 125
- phase1_dispatch_gate 251
- physical address 91
- physical address map 134
- Pic8259Pair 118
- PIDENTIFY 265
- PIN 265
- pipeline 131
- PipelineDriver 113, 118, 123, 265
- PipelineSlot 118, 123
- PIRQ 147
- planned platform 237
- platedit_qt 259
- platform appendix 232
- platform editor 99, 259
- platform execution mode 181
- platform identity 71
- platform latch 71
- platform manifest 17, 62, 63, 69, 99, 203, 227, 232, 237, 259
- PLATFORM MISMATCH 71
- platform selection 17
- platform() 178, 181, 194
- platform.json 259
- platform_init1() 161
- PlatformConfig 69
- platformConfigPath 17
- platformMode 206
- Plink 6, 44, 60, 80, 81
- PME 265
- pmem 91, 229, 230, 253
- port 69, 229
- port 10023 48, 79, 80
- port in use 227
- POST_BUILD 249
- power-up 60, 122
- power-up sequence 75
- PPCE 265
- precedence 69
- preset 45
- primary channel 93
- primary table 127
- Processor Support 262
- profile clock 198
- profile directory 63
- profile file 113
- profile_resolve 252
- profileAlphaClockHz 265
- profiler 112, 244, 246
- profiling 246
- prog-IF 181
- Program Files 42
- PT_IMPURE 185
- PT_VPTB 185, 265
- PT0 134
- PTBR 185, 212
- PTE 133
- putChar 265
- PuTTY 6, 18, 44, 60, 80, 81
- PuTTY auto-launch 48, 80
- PuTTY console 195
- puttyPath 69
- PVD 94
- Python 48

- Q -

- QLogic ISP 23
- QSettings 206
- Qt 42, 44, 48
- Qt 6.10.2 246
- Qt Creator 244, 246
- Qt Widgets 259
- qualifiers 77
- Quick Start 6, 50

- R -

RA disk 95
 raisePciInterrupt 265
 raw image 105
 RAW mode 229
 RAW TCP 60, 79, 80, 81
 RAW TCP socket 80
 RAW-TCP client 44
 RBR 265
 rcm_nvram0 265
 READ CAPACITY 265
 READ DATA 150
 READ DELETED DATA 150
 READ ID 150
 read intercept 178, 181
 READ SECTORS 265
 READ TOC 265
 READ(10) 94
 READ10 265
 REAL_HW 178, 181, 194, 210
 reason for halt 212
 RECALIBRATE 150, 265
 ref leaf 262
 ref_decompressed.bin 158
 reference 206
 REFERENCE_INDEX.md 262
 region map 253
 region rollup 252
 register level 131
 registry 206
 regression guard 158
 regWriteldx 265
 reinstall 42
 release 45, 195
 release build 114
 release checkpoint 23
 release notes 7
 Release Stages 39
 relocation 122
 RelWithDebInfo 45, 195, 249
 remote host 80
 REQUEST SENSE 92, 265
 reset re-entry 185
 RESTART 90
 restore_state 185
 restore-except-flash 107

retire 123
 retire trace 249, 250
 retire-PC histogram 113
 RetireProfiler 47, 113, 252
 REV 265
 reverse engineering 244, 245
 roadmap 21
 ROM 44, 69
 rounding mode 128
 roundLog2Nearest 265
 routeMmioOffset 265
 RPCC 198, 200
 RSCC 178
 RSCCWARP 200
 run directory 51, 56, 57, 63, 99
 run scripts 254
 run.log 50
 RUN/STOP 104, 229
 run_<plat>.sh 195
 RUN_DIR 56, 195
 run_fw 252
 run_fw.sh 195
 runbook 247, 254
 runDir 206
 running 62
 runtime layout 57
 RX FIFO 265
 RZ28 94, 96
 RZ29 94
 RZ29L 99

- S -

S_ExcBarrier 265
 S_Locked 265
 S_Mb 265
 S_PalLinux 125
 S_PalTru64 125
 S_PalVms 125
 S_PrefetchOnly 265
 S_TrapBarrier 265
 S_Wmb 265
 SCAN EQUAL 150
 scatter-gather 23
 scb_init() 161
 SCBB 212
 scbd field 134
 schedLib 175, 262

- script 247, 254
- SCRIPTS 265
- SCSI 7, 18, 23, 92, 94, 96
- scsi_poll 94
- scsi_reset 94
- SDA 7
- SDE 265
- SDI 95
- secondary boot 185
- secondary start 241
- secondary table 127
- sector 105, 251
- SEEK 150
- self-location 56
- SemanticFlags.tsv 125
- semFlags 265
- SENSE DRIVE STATUS 150, 265
- SENSE INTERRUPT 150, 265
- separation of concerns 15
- SequentialDriver 251
- set 82, 83, 89, 90
- SGE 142
- show 82, 83, 89, 90
- show config 75, 83, 89, 178, 181
- show dev 23, 83, 178, 181
- show device 75, 89
- show memory 89, 90
- shutdownTimeoutMs 206
- SIA 153
- silicon 63, 178, 194
- silicon ceiling 195
- silicon maximum 69, 71, 238
- silicon mode 23, 83, 181, 185, 210
- SimH 262
- SIO-TRACE 235
- SIRR 265
- six-stage pipeline 123
- smir 229, 250
- SMP 15, 23, 175
- SMP architecture 175
- SmpDrivers.h 175
- SmpHarness.h 175, 241
- snapmap 253
- snapshot 18, 21, 50, 77, 107, 116, 253
- snapshot restore 195
- snapshotMode 206
- snapshot-on-pc 77, 106
- snapshots 62, 63, 75
- snapshots directory 106
- softfloat 48, 128, 265
- SoftFloatBackend 128, 265
- Software Verify 244, 246
- source cross reference 262
- source header 223
- Source References 262
- south bridge 139, 145, 147, 148
- SPAM 118, 133, 225
- SPAMBucket 265
- SPAMShardManager 118, 133, 136, 265
- Sparse I/O 265
- sparse image 96
- sparse paging 132
- spdlog 44, 48, 112, 113, 160
- SPECIFY 150
- SPEC-PLATED-001 259
- SPEC-SCSIH-001 259
- Spinlocks 161
- SQWE 265
- SRE 245
- SRM 6, 18, 42, 82, 118, 122, 225, 252
- SRM boot 235, 241
- SRM commands 89, 235
- SRM console 6, 15, 62, 83
- SRM firmware 245
- SRM firmware image 44
- SRM image 51
- SRM prompt 21, 48, 50, 60, 83
- SRM V7.3 44
- SRM variables 91
- srm_fw 109
- SRMConsole 69
- SRMConsoleDevice 265
- SROM 21, 122, 153, 157, 225
- ST0 265
- ST506 95
- stamp 251
- STARTUP_P1 7
- station address 153
- stderr noise 114
- stem-wins 71
- step 265
- step loop 246
- stepCycle 251
- STO 265
- stop sentinel 61
- StopReason 212

storage 62, 92
 storage fabric 96
 styleclass 245
 subDecode 127
 subsys 116
 subsystem internals 122, 232
 SuperIO 6, 18, 235
 superpage 133, 136
 Supported Systems 17
 switches 51
 SWPCTX 185, 265
 SWR 153
 symbols 252
 sys_cserve 241
 sys_exit_console 185, 241
 sys_reset 185, 241
 sys_reset_init 185, 241
 sys_serial_num 230
 SYSBOOT 7, 99, 110
 system disk 99
 system serial number 230
 systemLib 175, 262
 systemNow 200
 SYSTYPE 203
 SYSVAR 170, 203

- T -

table of contents 245
 task matrix 236
 taskfile 178, 181
 taskkill 61
 TBIA 136, 265
 TBIAP 136, 265
 TBIS 136
 TCP console 6, 18, 50, 60, 229
 tee 51
 telnet 80, 81
 Tera Term 80, 81
 terminal client 44, 79
 test matrix 210
 TEST UNIT READY 265
 THR 265
 THRE 265
 three-platform gate 195
 throttle 116
 tick counter 200
 TIG 17, 23, 140, 177, 229, 250, 253

timeout 229
 timeout loop 228, 236
 timestampMMDDhhmm 265
 Timothy Peer 14
 Titan 139, 144, 238
 Titan 21274 71
 TitanChipset 144
 TLB 18, 118, 131, 133
 TlbEntry::matches 265
 TlbEpoch 118, 133, 136
 toolchain 246
 tooling 247
 tools 42, 56, 247, 254
 tools directory 223, 254
 topic 245
 topiclink 245
 ToyRtc 265
 trace 112
 -trace 51, 77
 TRACE_RETIRE_COMPACT 249
 traced boot 74
 traces 223
 translation 118
 translation buffer 136
 trap mode 128
 troubleshooting 81, 227
 Tru64 23
 Tru64 boot 110
 Tru64 UNIX 6
 tryRead 265
 tryWrite 265
 T-Stage_Version 39
 Tsunami 6, 17, 139, 140, 225, 233, 238, 265
 Tsunami21272_CsrSpec 198
 TsunamiCchip.h 241
 TsunamiChipset 118, 132, 140
 TsunamiTig 265
 tulip 153, 233, 236
 two instances 75
 Typhoon 6, 17, 118, 140, 177, 238, 265

- U -

unassigned IPR 185
 uninstall 42
 unit tests 264
 unregisterIoPortRange 265
 unregisterPciMemRange 265

unzip and run 56
UPD 60, 75, 109
Update 109
update srm 75, 109
upgrade 42
User Guide 6

- V -

V5 23
V5 trajectory 181
VA_CTL 134, 136
VA_FORM 265
validation 259
variantFromModel 265
VAX float 128
vector 0x660 23
Verify 109
VERSION 150
Version Numbering 39
View union 170
virtual 178
virtual CDROM 89, 104
virtual CD-ROM 92
virtual disk 92, 96, 105
virtual mode 210
VirtualAlloc 118, 132
VirtualIsoDevice 94, 265
virtualization 194
VirtualScsiDevice 94
Visual Studio 48
vptb 185, 265
VRT 265
VS Developer prompt 47
V-Stage-Version 39

- W -

W1C_MASK 265
W1S_MASK 265
WARP 7, 178, 181, 200
webui 259
wedge 249
what's new 7
will not start 227
WimC 158
Windows Defender 209

wireDevices 145
WO_MASK 265
word-fold 235
WRITE DATA 150
Writeback 15
WSBA0 265
wwid 91

- X -

x64 47
X-Stage-Version 39